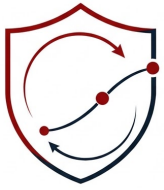
Principles of Robot Autonomy I

SfM, Features, Image convolutions, CNNs, Semantic Perception

Stanford University



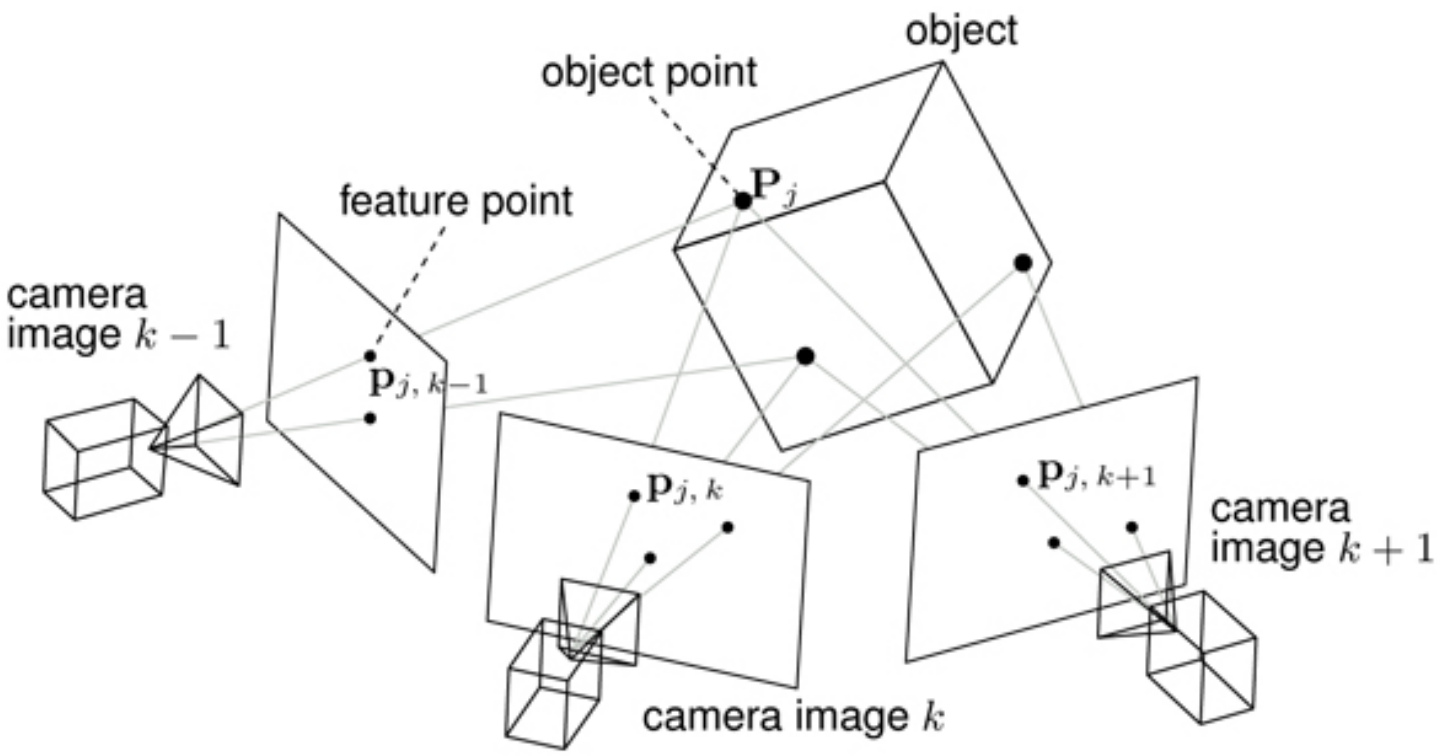
SIA Lab
Safe and Intelligent Autonomy Lab

IPRL

Logistics

- Homework 3: due Sunday, Nov 1
- Homework 4: out Tues Nov 3 (After the Midterm)
- Midterm: Thu, Oct 29, 1:30pm, in-class
  - Lecture 1-8
  - One double-sided cheat sheet allowed
  - No internet, no GenAI, no working with others
- Lecture 11:
  - SfM
  - Features
  - Image convolutions
  - CNNs and learning-based semantic CV

Structure from motion (SFM)



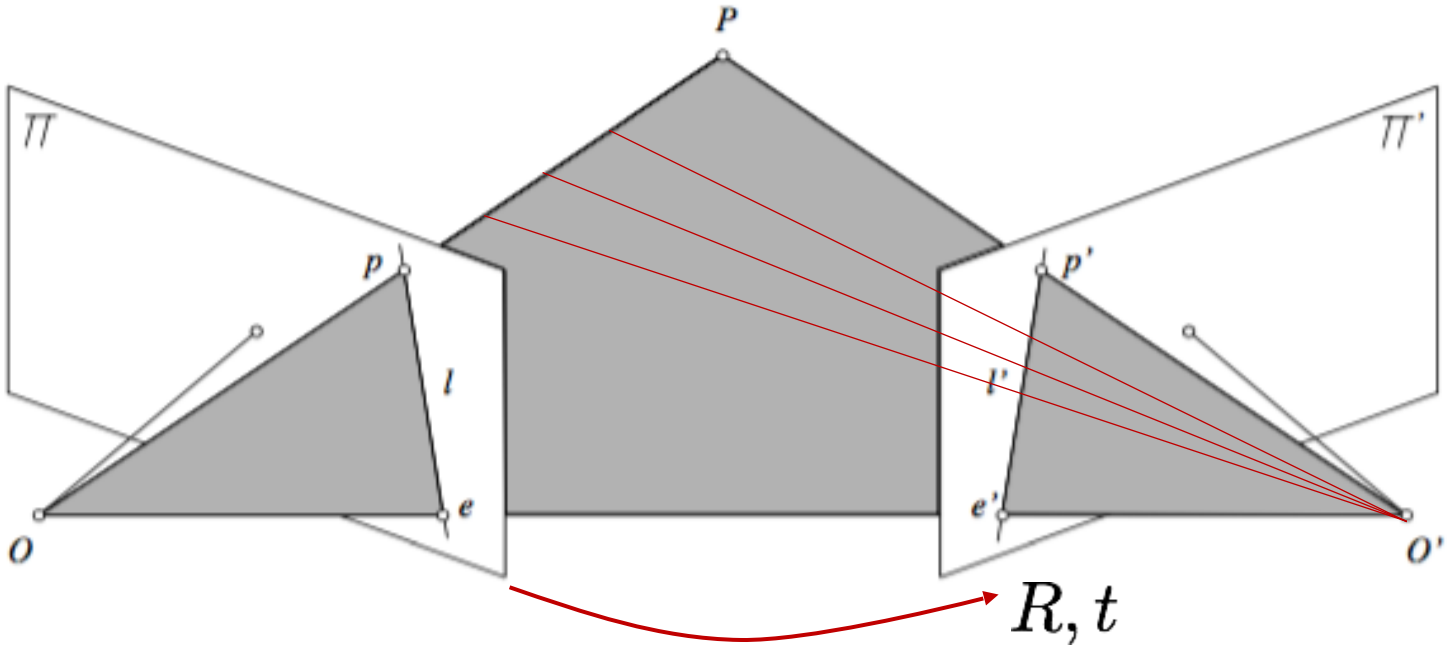
Given *m* images of *n* fixed 3D points

$$p^h_{j,k} = M_k P^h_j$$

Find:

- *m* projection matrices M_k (motion)
- *n* 3D points P_j (structure)

Epipolar constraint derivation

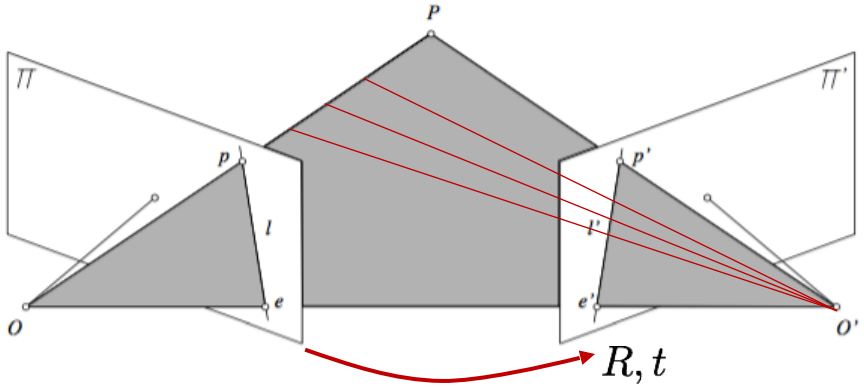


- Epipolar constraint: $\bar{O'p'}$, $\bar{Op}$, and $\bar{O'O}$ (when expressed in that same frame) must be co-planar, or

Epipolar constraint derivation

(more details: CS231a)

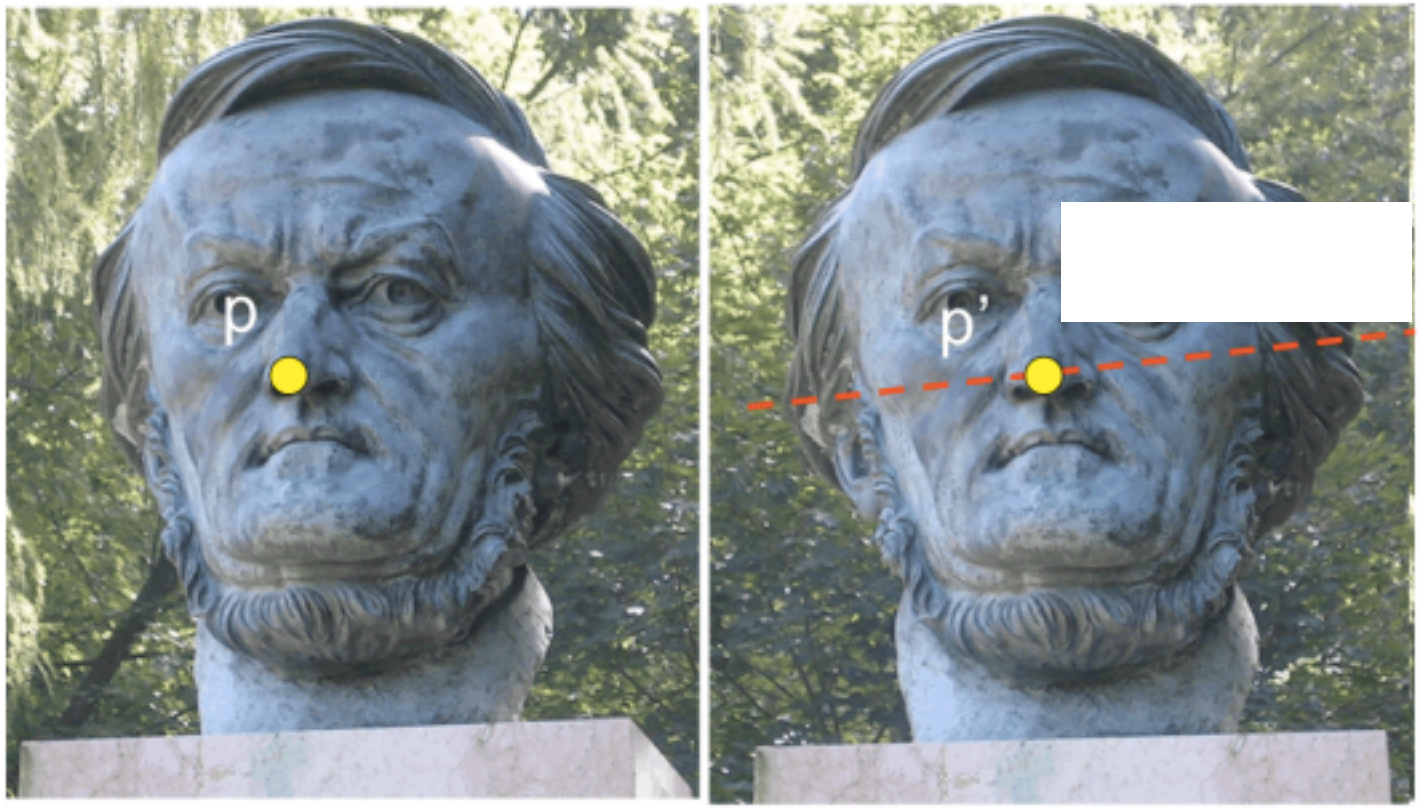
- Translate into vector expressions, all in frame O’



where

Fundamental matrix

Usefulness of fundamental matrix



- Assume *F* is given
- Given a point in image 1, one can compute the corresponding epipolar line in image 2 without any additional information needed!

Estimating the fundamental matrix

Estimating the fundamental matrix

- 8-point algorithm

Estimating the fundamental matrix

- 8-point algorithm

$\Rightarrow$

Estimating the fundamental matrix

- 8-point algorithm

$\Rightarrow$

$\Rightarrow$

Estimating the fundamental matrix

- 8-point algorithm

$\Rightarrow$

$\Rightarrow$

f

Estimating the fundamental matrix

- 8-point algorithm

$\Rightarrow$

$\Rightarrow$

$\Rightarrow \quad Wf = 0$

*n*x9 matrix of known coefficients

f

Estimating the fundamental matrix

- 8-point algorithm

$\Rightarrow$

$\Rightarrow$

*n*x9 matrix of known coefficients

$\Rightarrow \quad Wf = 0$

f

- Given $n \geq 8$ correspondences, one then solves

Estimating the fundamental matrix

- 8-point algorithm

$\Rightarrow$

$\Rightarrow$

$\Rightarrow \quad Wf = 0$

*n*x9 matrix of known coefficients

f

$$\min_{f \in R^9} \quad \|Wf\|^2$$
$$\text{subject to} \quad \|f\|^2 = 1$$

- Given $n \geq 8$ correspondences, one then solves

Estimating the fundamental matrix

- 8-point algorithm

$\Rightarrow$

$\Rightarrow$

*n*x9 matrix of known coefficients

$\Rightarrow \quad Wf = 0$

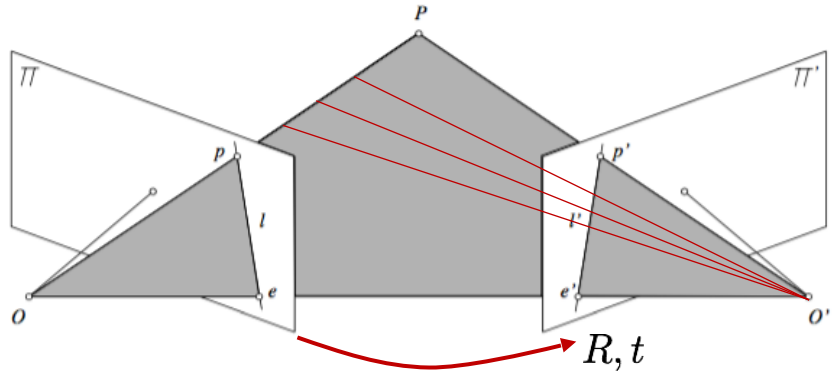
f

$$\min_{f \in R^9} \quad \|Wf\|^2 \quad \text{subject to} \quad \|f\|^2 = 1 \Rightarrow \tilde{F}$$

- Given $n \geq 8$ correspondences, one then solves

Obtaining relative camera pose

- Recall:



- To get we compute

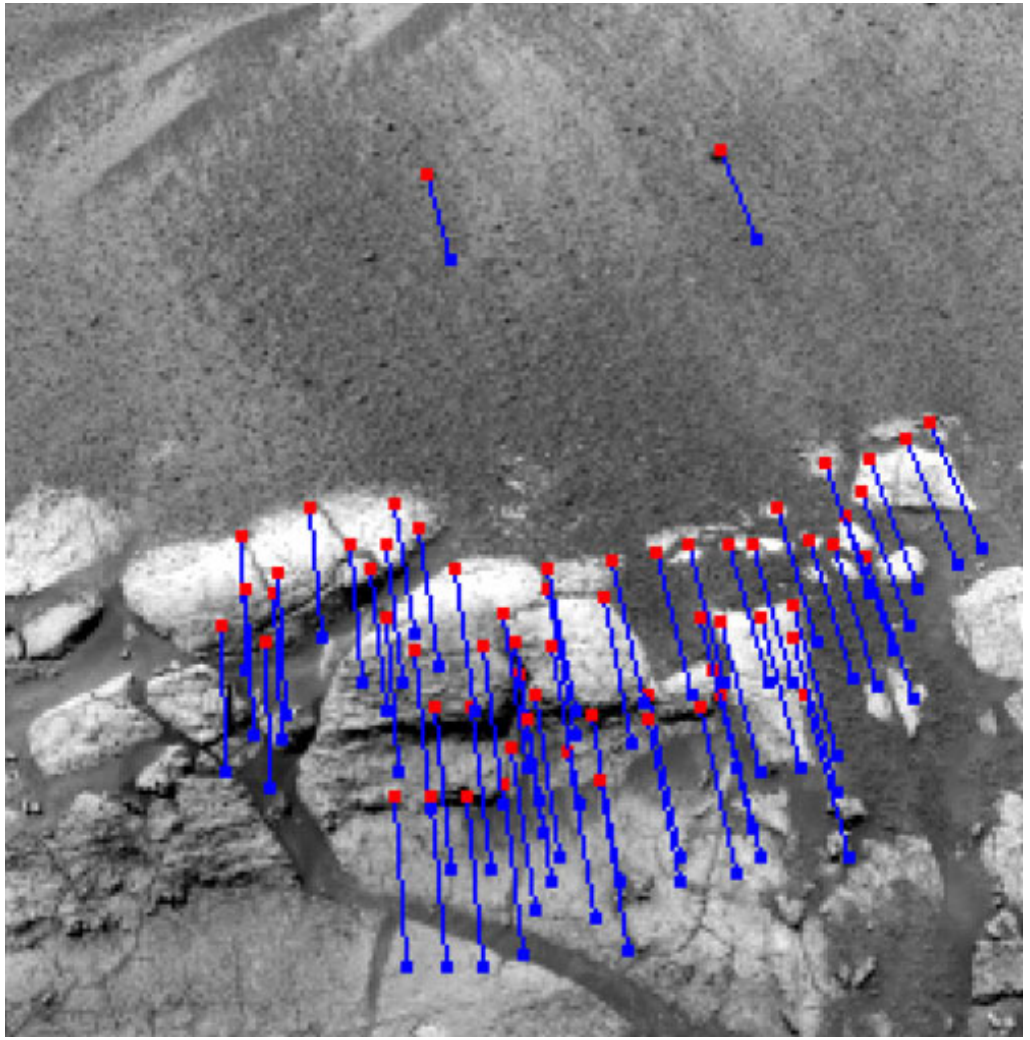
- Use RQ decomposition to get

Solution to SFM problem (high-level)

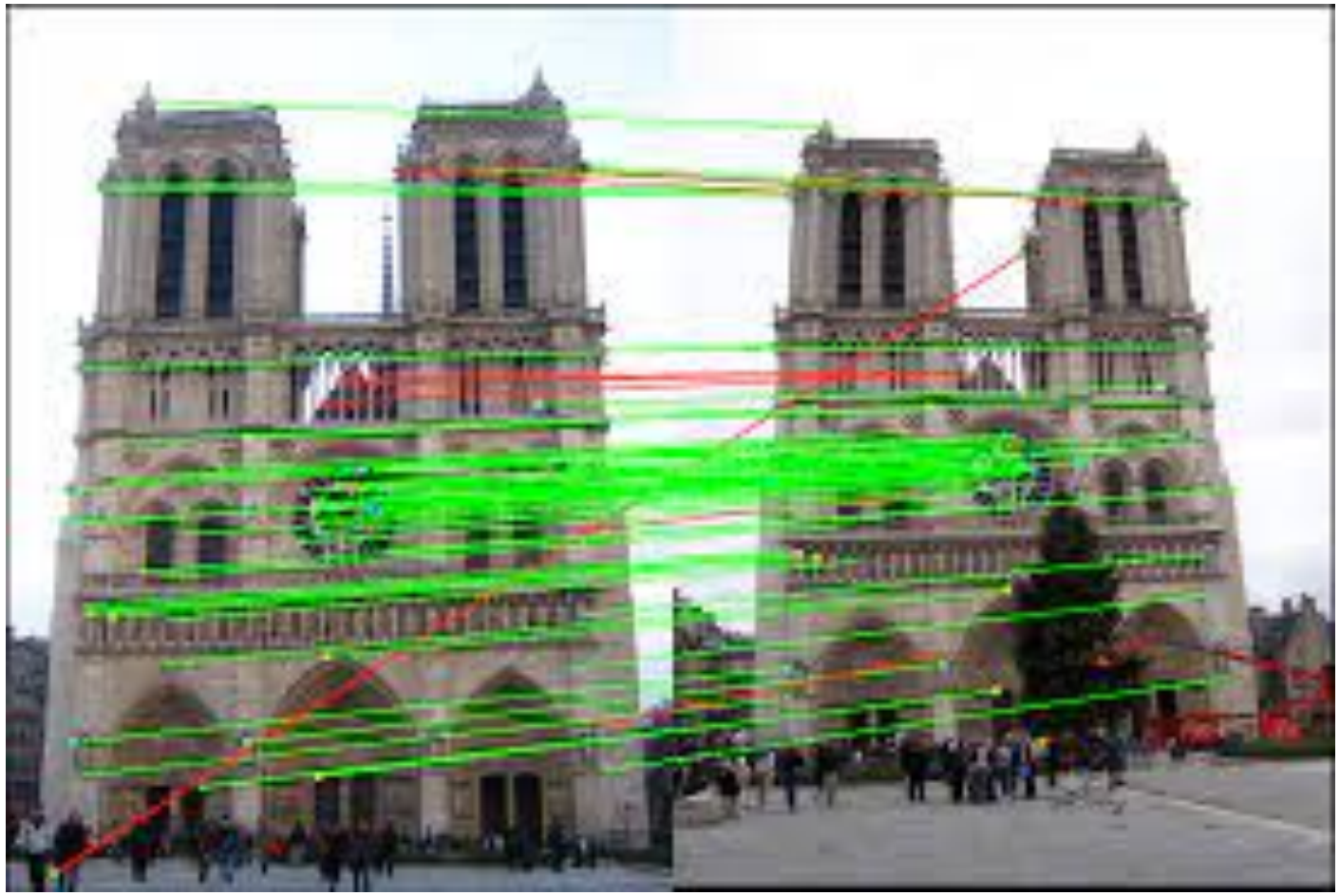
- Several approaches available:
  - Algebraic approach (by fundamental matrix)
  - Bundle adjustment (large-scale non-convex optimization problem to find [R t] camera poses for each image, and 3D points (i.e., structure) in common world frame
- Algebraic approach (2-views) – gives relative poses between two successive views, and 3D points relative to first view
  1. Compute fundamental matrix *F* (e.g., via 8-point algorithm)
  2. Use *F* to estimate camera projection matrices
  3. Use camera projection matrices for triangulation
- Used as ”front end” in visual SLAM, to obtain pose graph for PGO back end

Application of SFM: visual odometry

- SLAM/Visual Odometry: estimate the motion of the robot by using visual input (and possibly additional information)
  - Single camera: absolute scale must be estimated in other ways
  - Stereo camera: measurements are directly provided in absolute scale

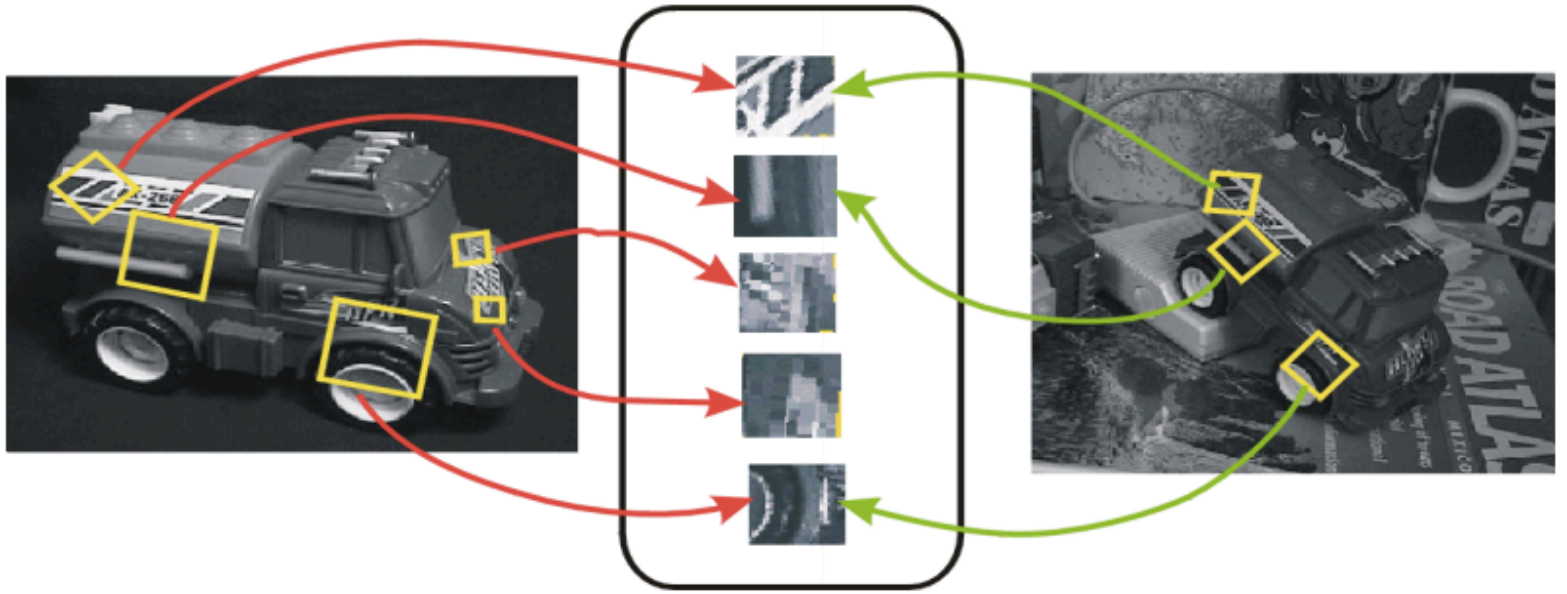


Elephant in the room: how to find correspondences for PnP, stereo, or SfM?



Keypoint Features and their Descriptors

- Goal: *describe* keypoints so that we can compare them across images or use them for object detection or matching
- Desired properties:
  - Invariance with respect to pose, scale, illumination, etc.
  - Distinctiviness

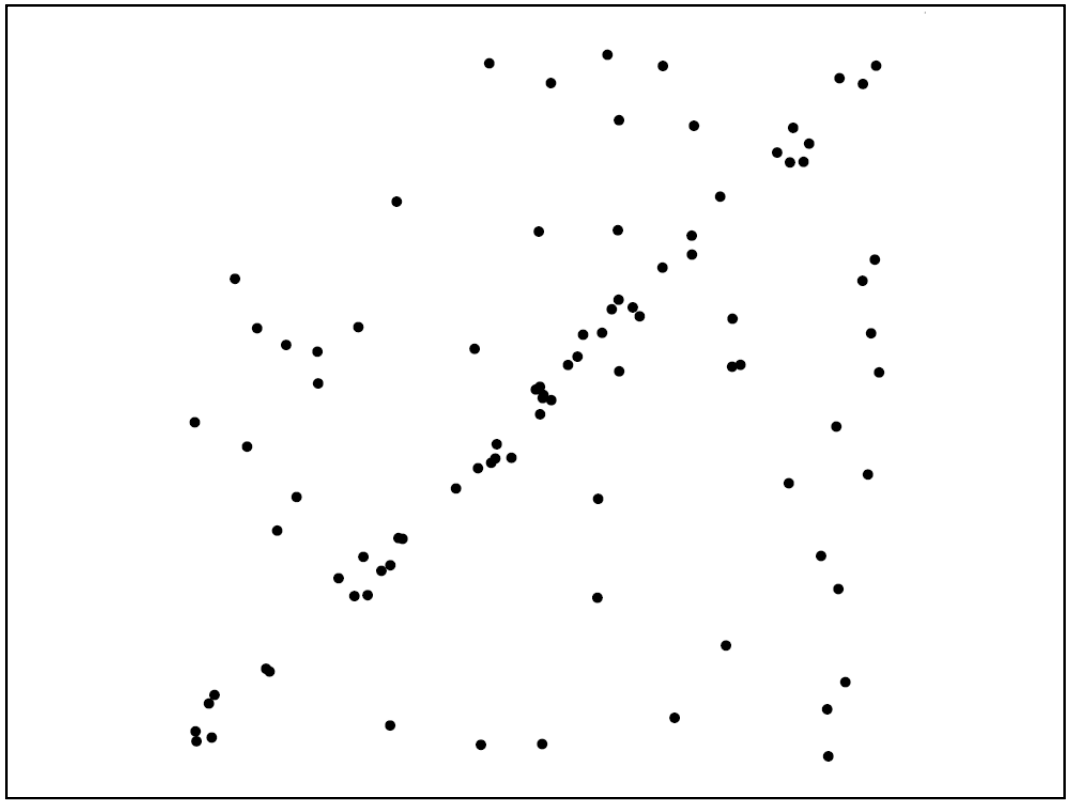


RANSAC iterations

- In principle, one would need to check all possible combinations of 2 points in dataset
- If $\left|S\right| = N$, number of combinations is $\frac{N(N-1)}{2}$ → too many
- However, if we have a rough estimate of the percentage of inliers, we do not need to check all combinations…

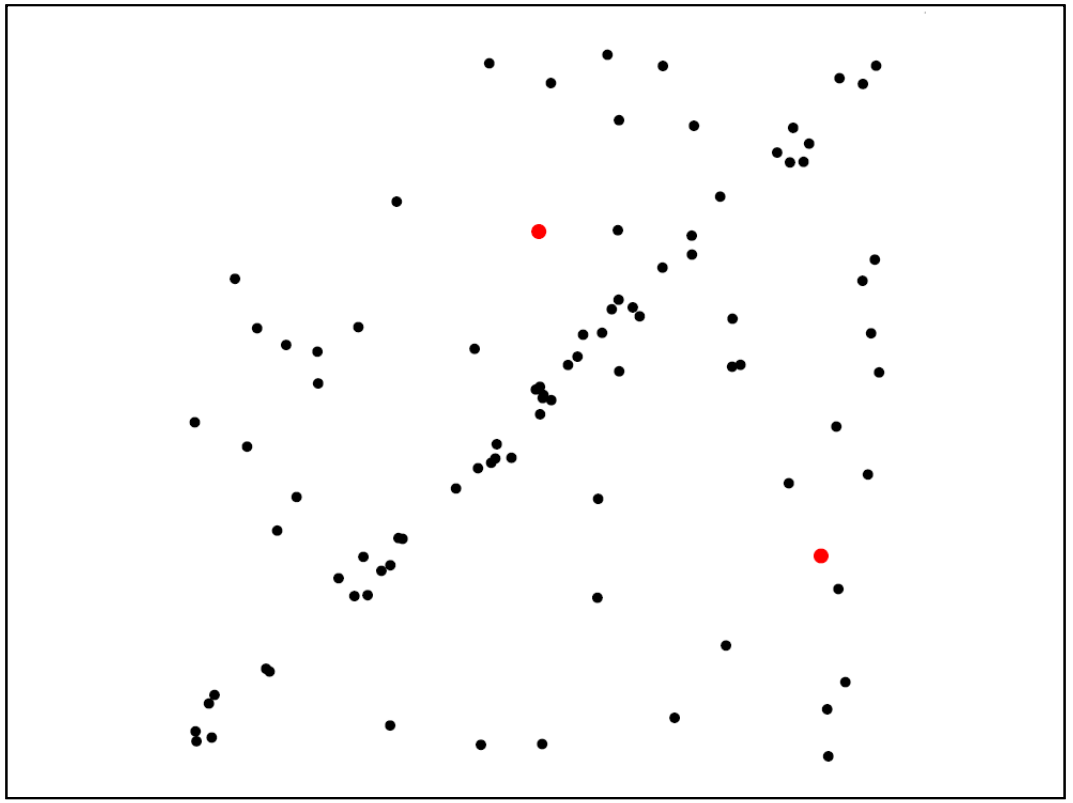
E.g., RANSAC for line fitting (for simplicity)

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



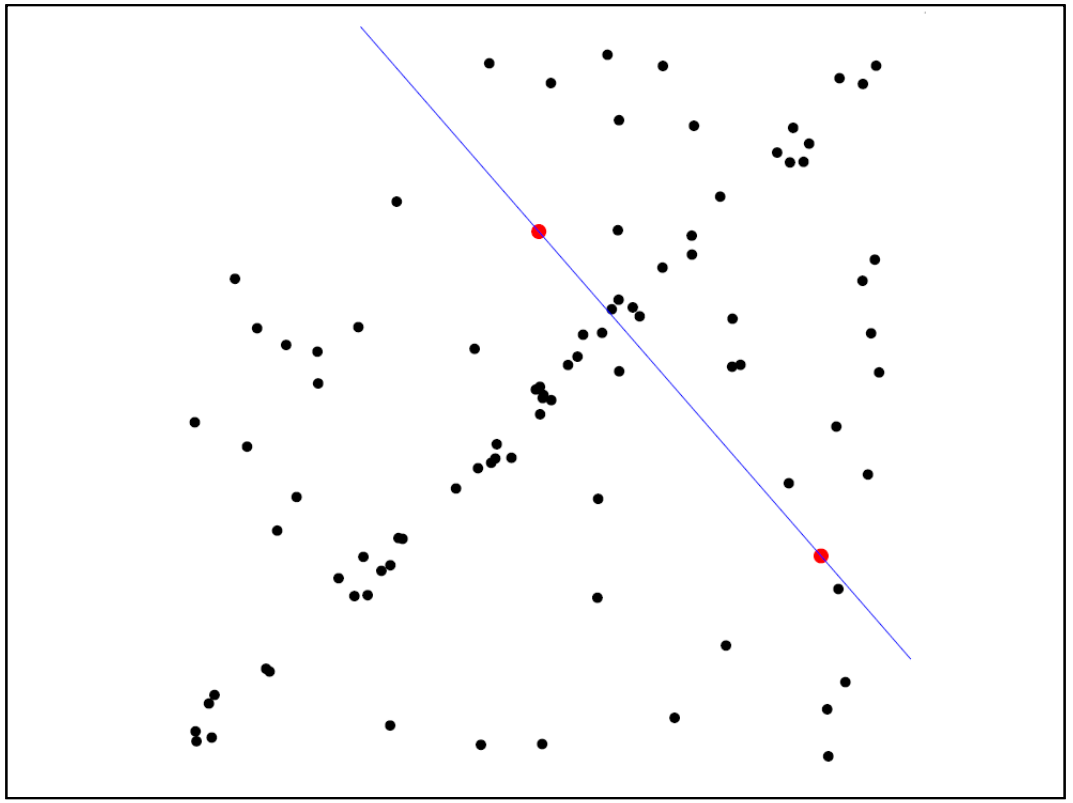
RANSAC

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



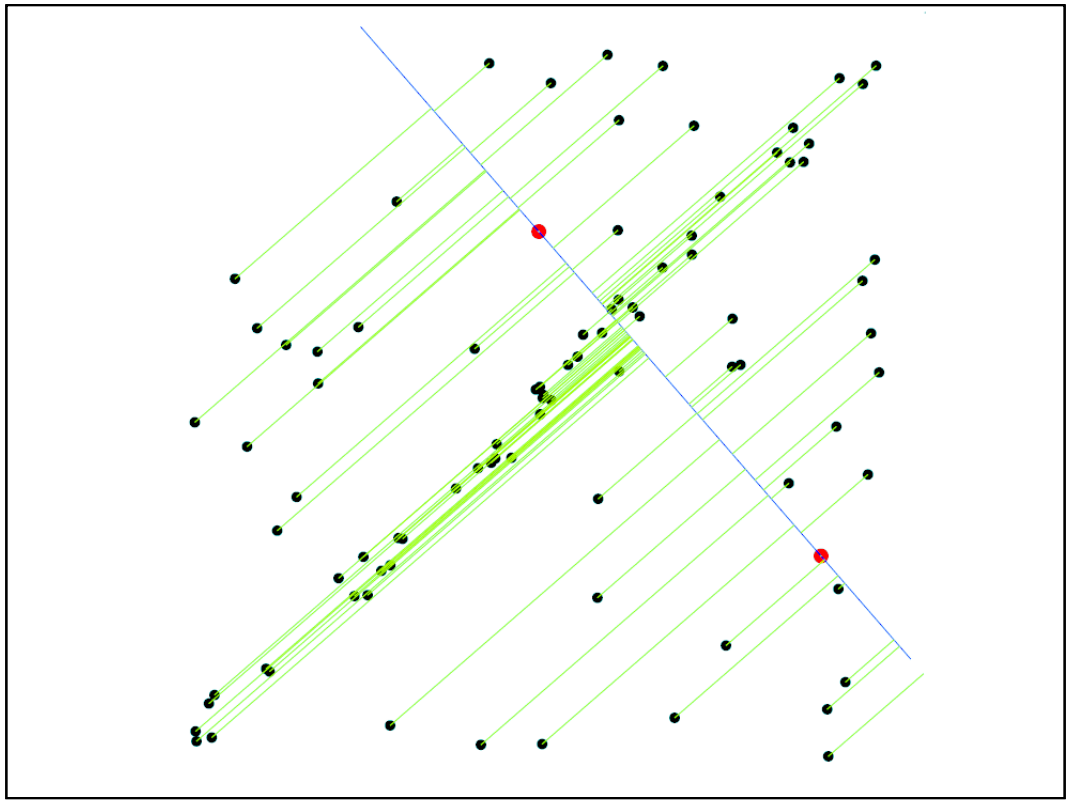
RANSAC

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



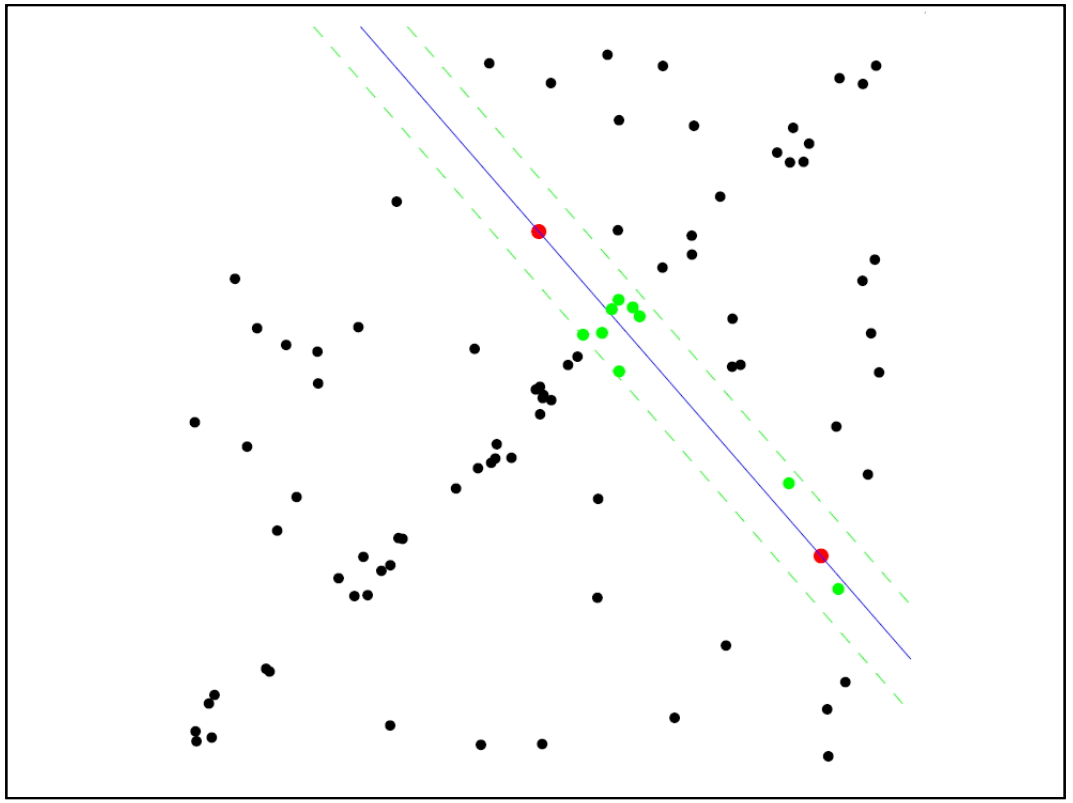
RANSAC

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



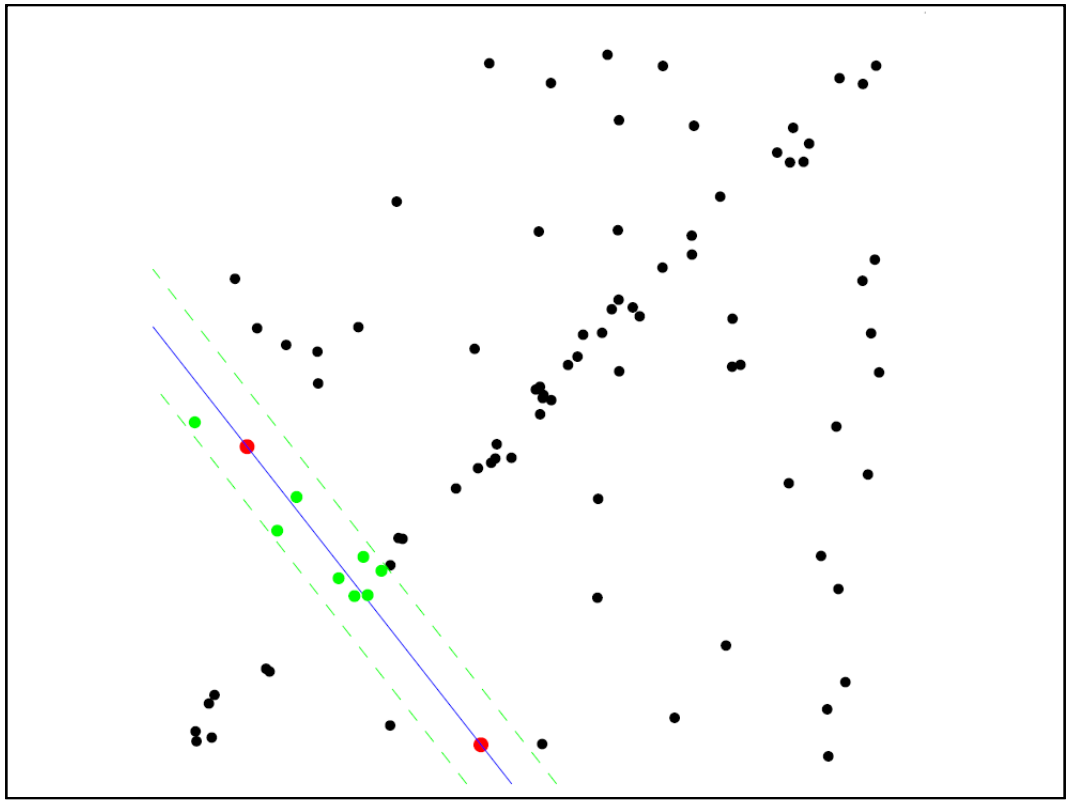
RANSAC

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



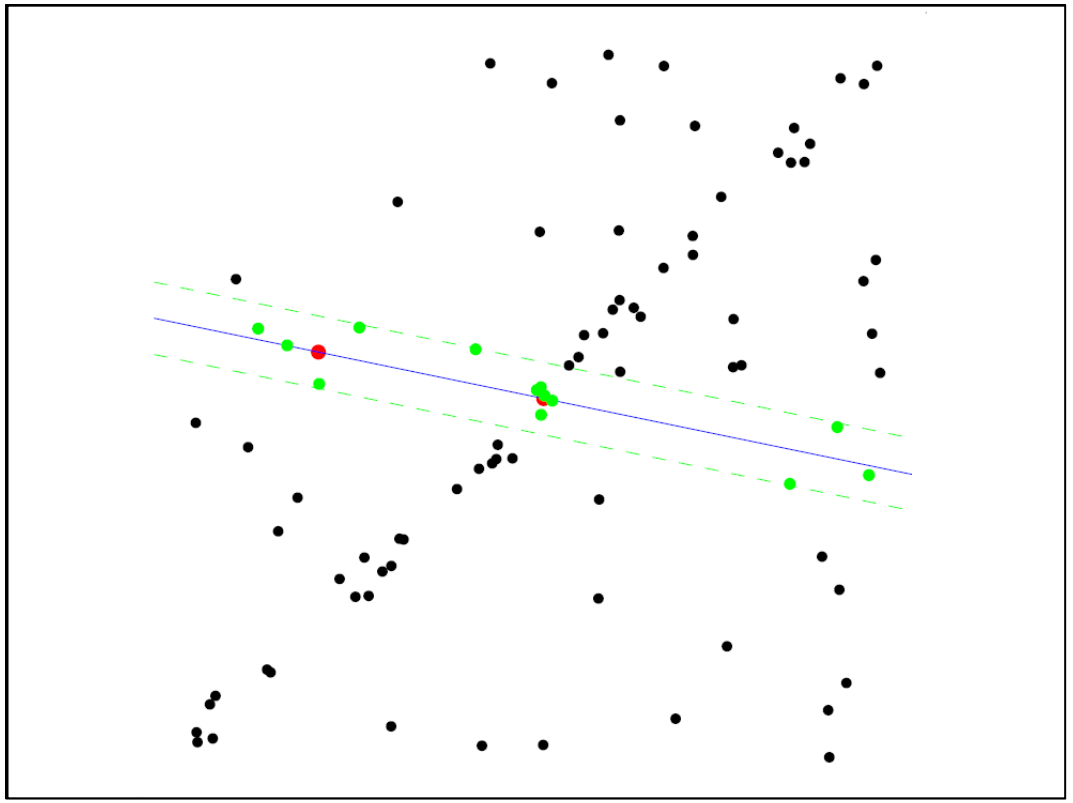
RANSAC

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



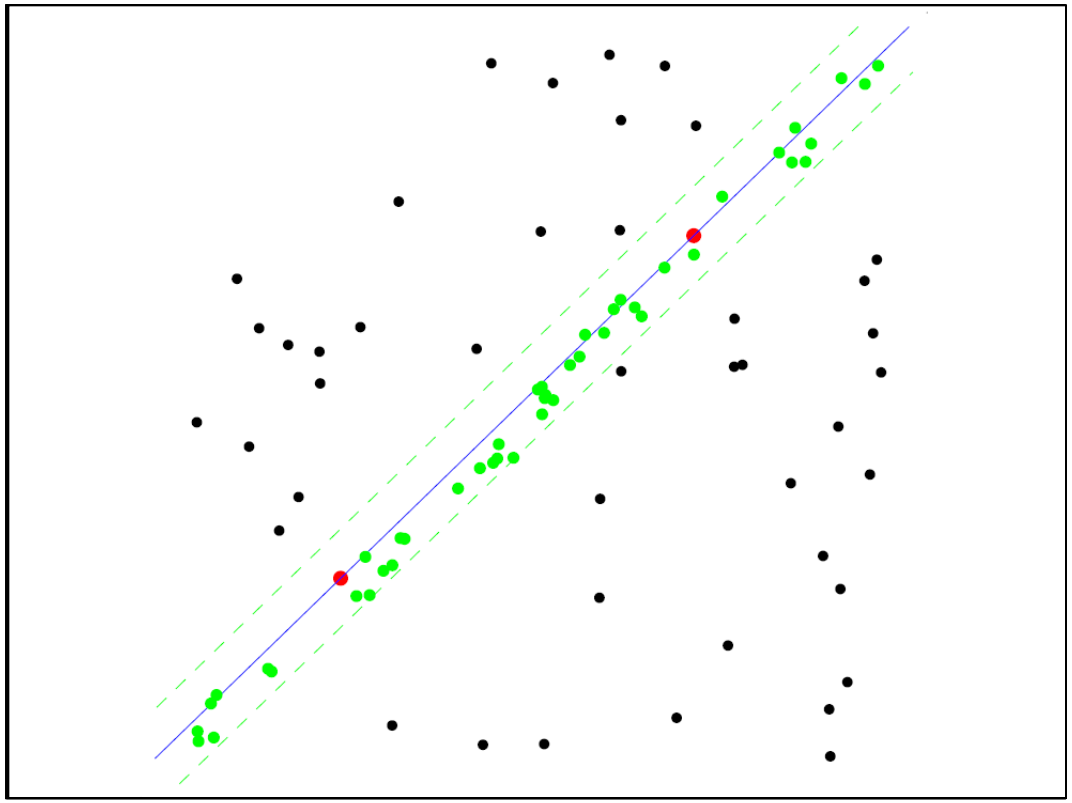
RANSAC

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



RANSAC

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



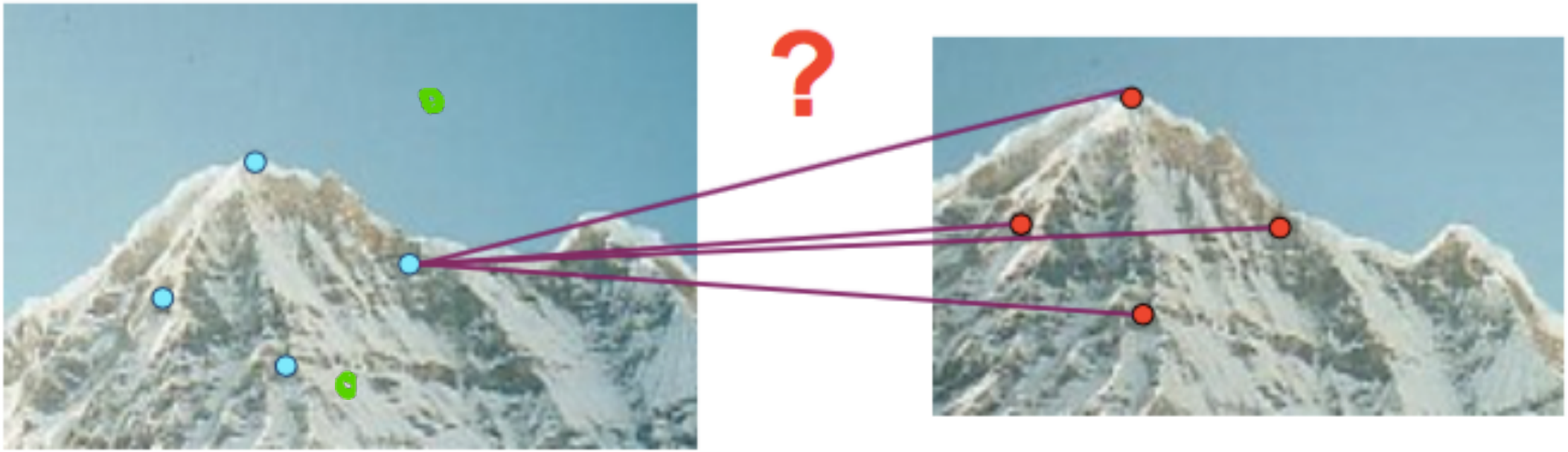
Core “feature based” CV pipeline

- Input two images (stereo or SfM), or image and 3D model (PnP)
- Feature detection, feature matching -> N correspondences
- Problem: in practice, we get loads of bad feature matches
- Use Random Sample Consensus (RANSAC) to reject outliers
- In RANSAC, we
  - Sample a small subset n of matched features
  - Solve geometric CV problem with these correspondences (PnP, stereo, SfM)
  - Check solution fit with remaining (N-n) feature matches – record fit score
  - Repeat
  - Return the fit with the highest score

Repeatability

Without repeatability, matching is impossible

Distinctiveness

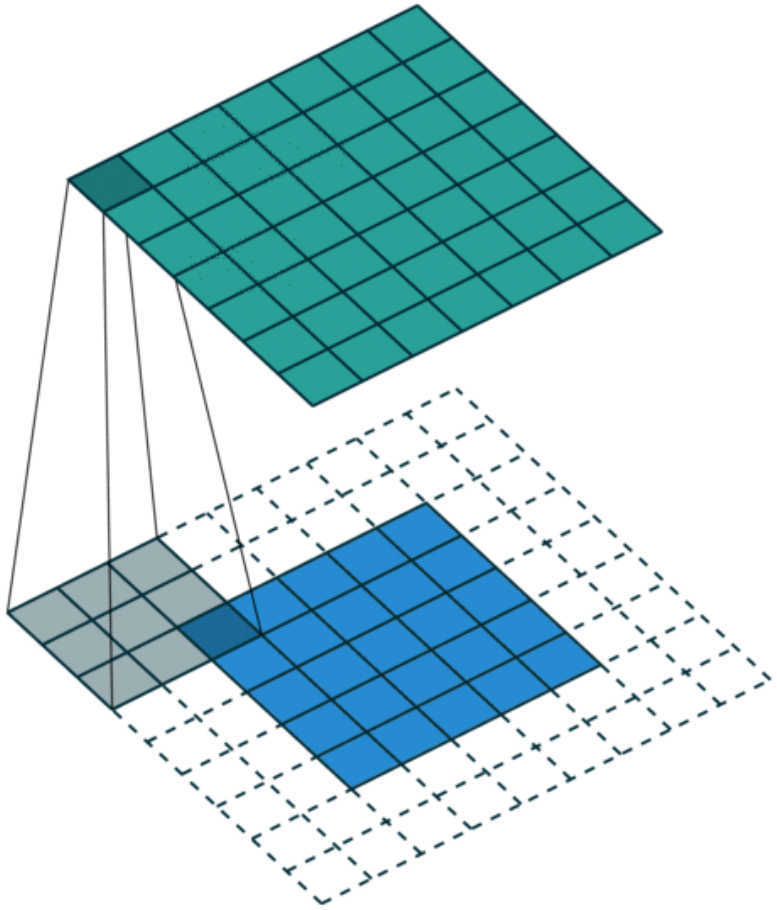


Without distinctiveness, it is not possible to establish reliable correspondences; distinctiveness is key for having a useful descriptor

Filter/Convolution Visualization

Source: Sanja Fidler

Filter/Convolution Visualization



Source: Sanja Fidler

Image Convolutional Filters

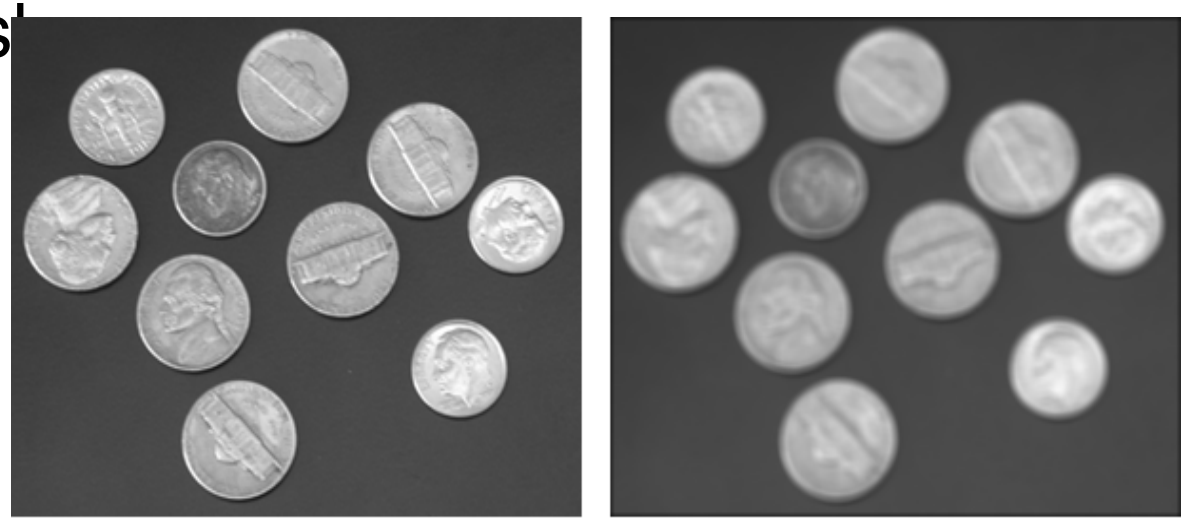
- Filters can be linear or non-linear
- We will focus on linear spatial filters

- Filter F (of size $(2n+1) \times (2m+1)$) is usually called a mask, kernel, or window
- Dealing with boundaries: e.g., pad, crop, extend, or wrap

Filter example #1: moving average

- The moving average filter returns the average of the pixels in the mask
- Achieves a smoothing effect (removes sharp features)
- E.g., for a *normalized* 3×3 mas

$$F = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



Generated with a 5x5 mask

Filter example #2: Gaussian smoothing

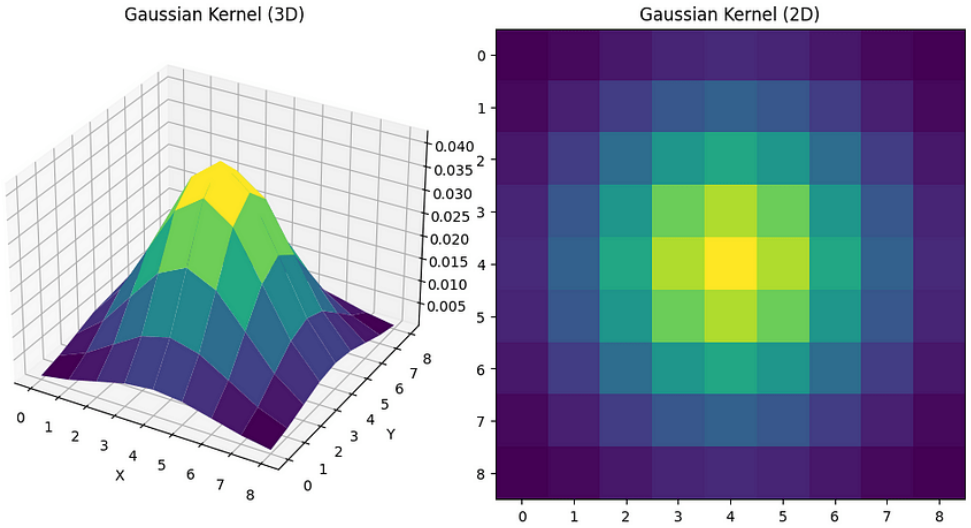
- Gaussian function

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

- To obtain the mask, sample the function about its center
- E.g., for a *normalized* 3×3 mask with $\sigma = 0.85$

$$G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

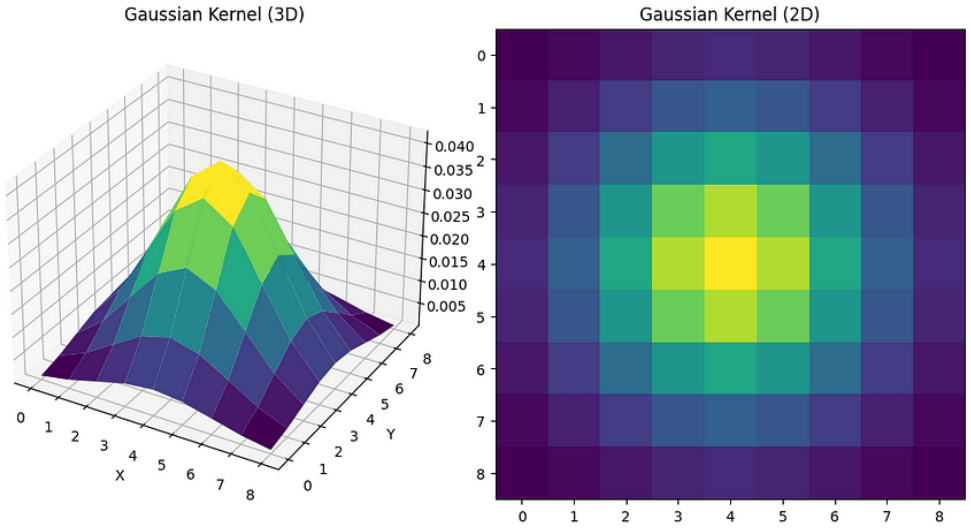
Filter example #2: Gaussian smoothing

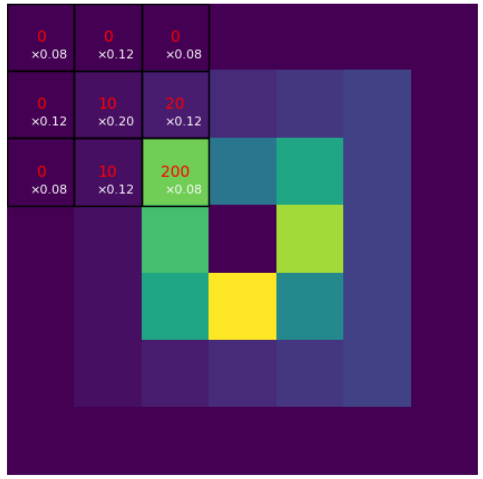


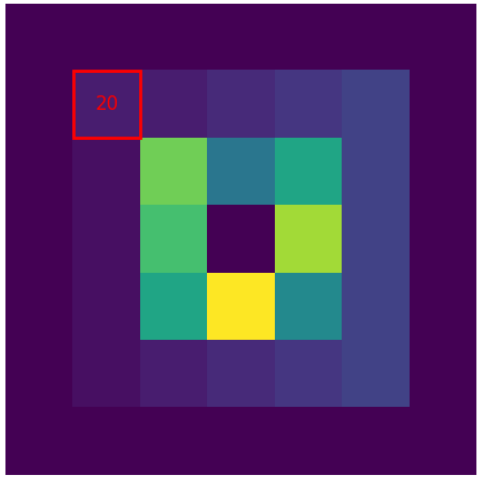
$$G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Visualizations by Vijay Vignesh

Filter example #2: Gaussian smoothing



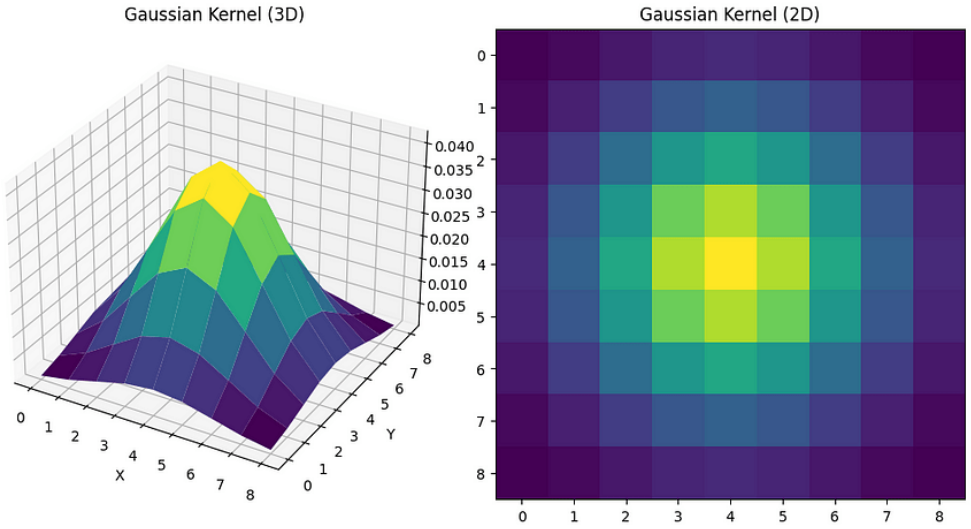


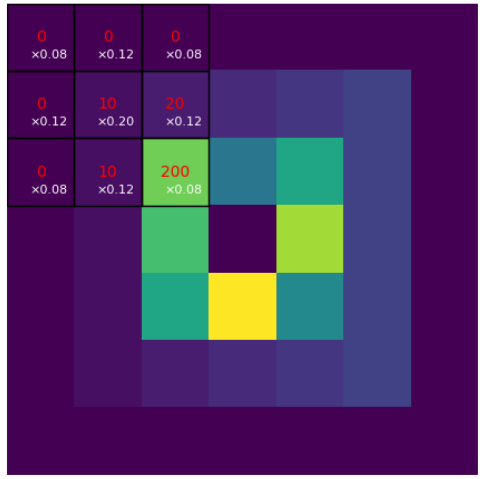


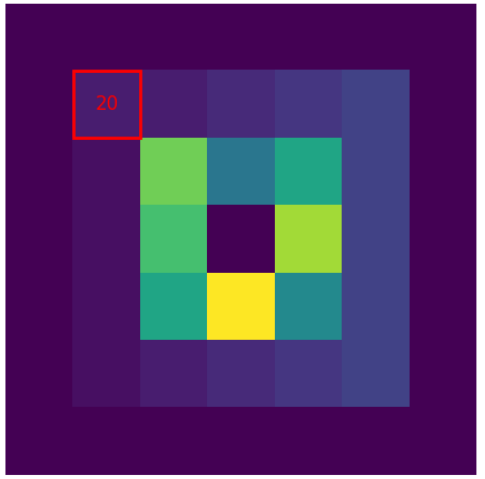
$$G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Visualizations by Vijay Vignesh

Filter example #2: Gaussian smoothing







$$G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Visualizations by Vijay Vignesh

Differentiation

- Derivative of discrete function (centered difference)

$$\frac{\partial I}{\partial x} = I(x+1, y) - I(x-1, y) \qquad F_x = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$$

$$\frac{\partial I}{\partial y} = I(x, y+1) - I(x, y-1) \qquad F_y = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

- Derivative as a convolution operation; e.g., Sobel masks:

Along *x* direction

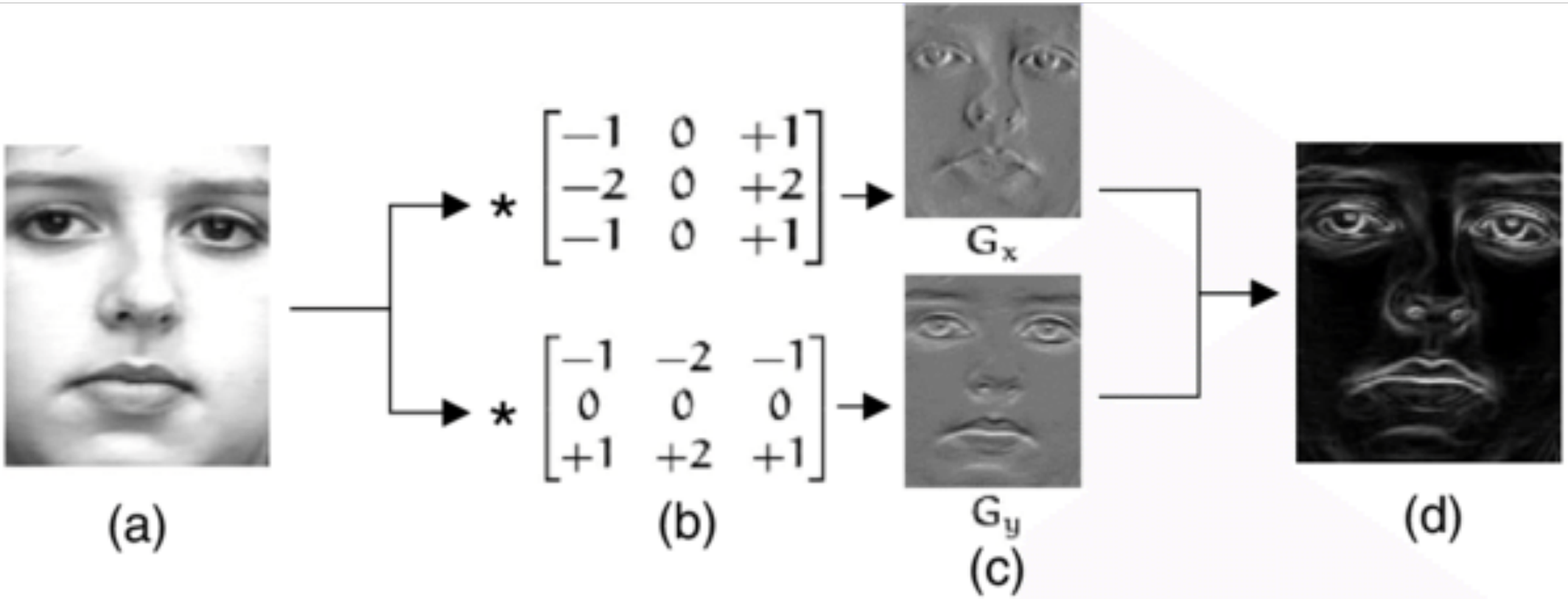
$$S_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

Along *y* direction

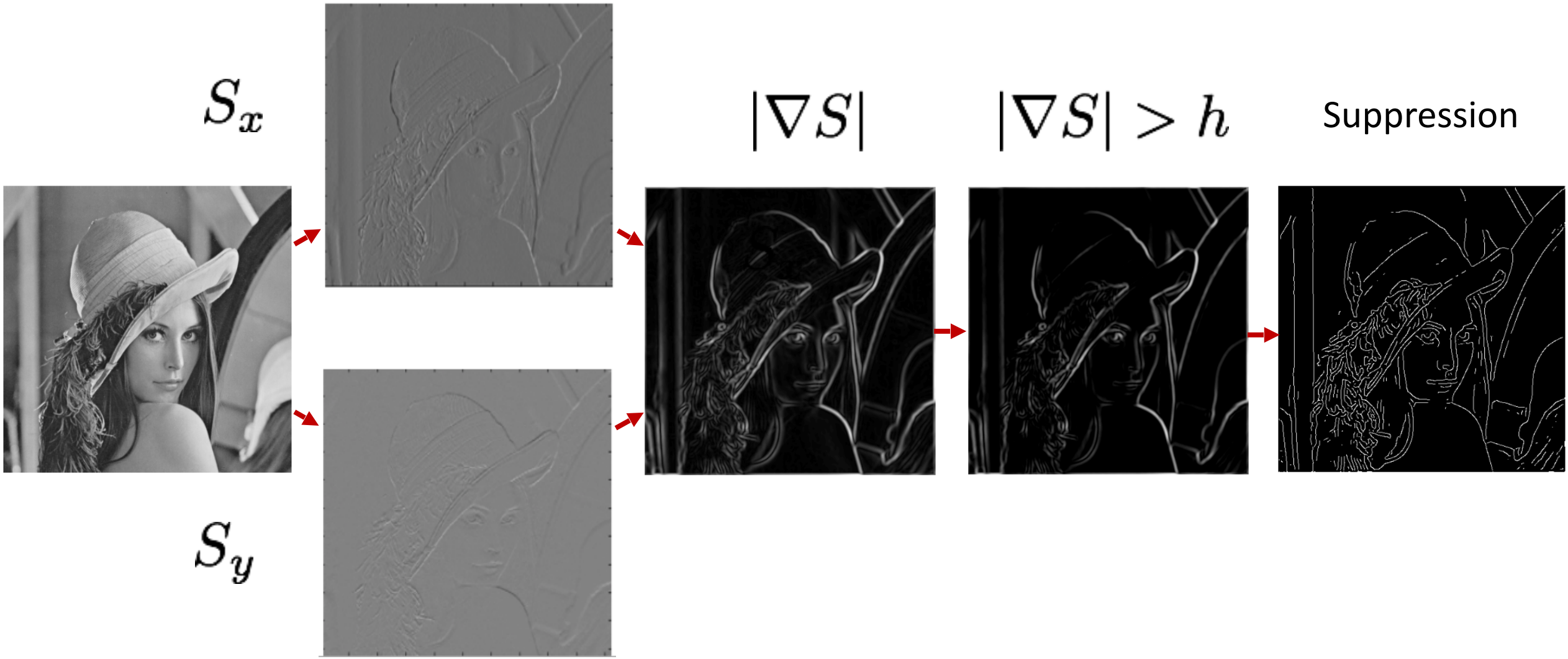
$$S_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Note: masks are mirrored
In convolution

Visualization of Sobel Masks



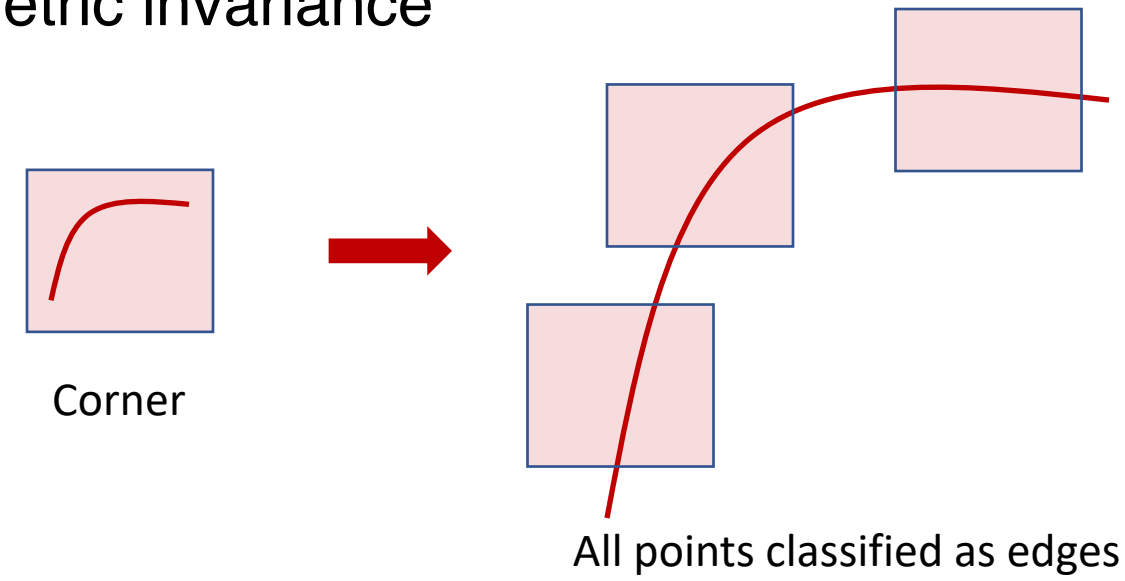
Canny edge detector



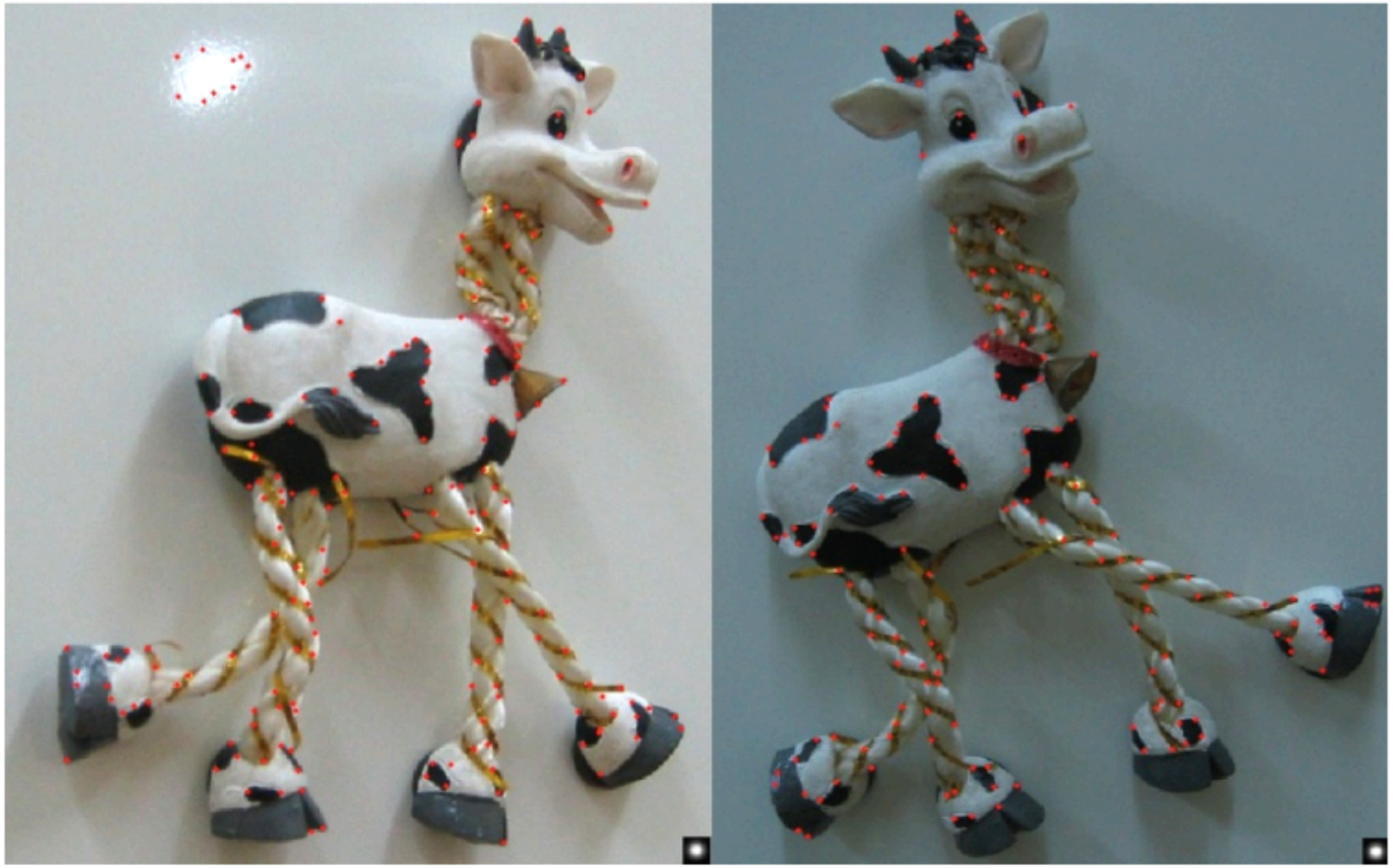
Properties of Harris detectors

- Widely used
- Detection is invariant to
  - Rotation -> geometric invariance
  - Linear intensity changes -> photometric invariance

- Detection is not invariant to
  - Scale changes
  - Geometric affine changes



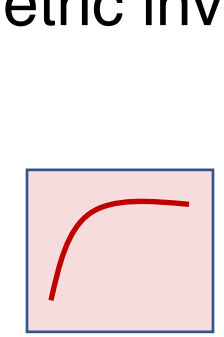
Harris detector: example



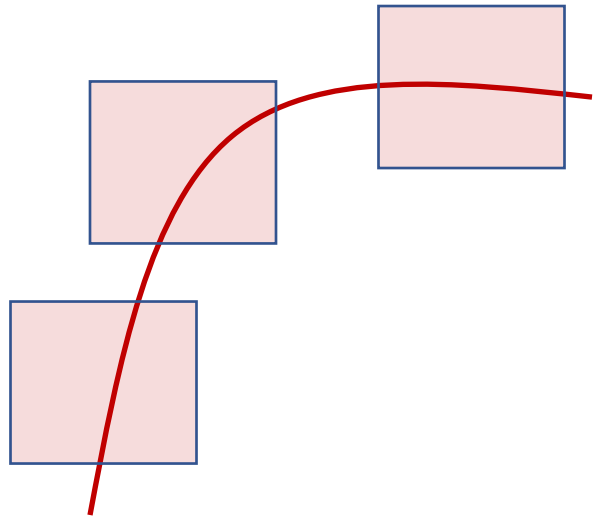
Properties of Harris detectors

- Widely used
- Detection is invariant to
  - Rotation -> geometric invariance
  - Linear intensity changes -> photometric invariance

- Detection is not invariant to
  - Scale changes
  - Geometric affine changes



Corner



All points classified as edges!

→ Scale-invariant detection, such as

1. Harris-Laplacian
2. in SIFT (specifically, Difference of Gaussians (DoG))

Popular detectors / descriptors

- SIFT (Scale-Invariant Feature Transformation)
  - Invariant to rotation and scale, but computationally demanding
  - SIFT descriptor is a 128-dimensional vector!
- SURF
- FAST
- BRIEF
- ORB
- BRISK
- LIFT

Classical vs Learning based Features

- Hand engineered features/filter masks (SIFT, ORB, etc)
- SVM for learning based image classification

2012: AlexNet 2012

- Feature masks learned from examples (CNNs)
- Multi-layer CNNs (deep networks)
- For specific classification/regression tasks
- small scale training data

YOLO: 2015

Transformer: 2017
"Attn is all you need"

2018: BERT (Google), GPT-1 (OpenAI)

2021: CLIP 2021 (OpenAI)

- Foundation models
- Very large transformers
- Pre- trained on internet scale data
- Fine-tuned (post-trained) to accomplish specific tasks

2024/2025: SAMv2 (Meta), DINO v3 (Meta), ChatGPT-5 (OpenAI), Imagen 4 (Google), Sora (OpenAI), …

Learning Image Features

Task: Classify whether an image has a cat

Labeled data:

(, 1)

(, 0)

Learning Image Features

Task: Classify whether an image has a cat

Labeled data:

Model Architecture:

Learning Image Features

Task: Classify whether an image has a cat

Labeled data:

Model Architecture:

Model

Free parameters (NN weights)

Learning Image Features

Task: Classify whether an image has a cat

Labeled data:

Model Architecture:

Model

Free parameters (NN weights)

Filter mask parameters

Learning Image Features

Task: Classify whether an image has a cat

Labeled data:

Model Architecture:

Model

Free parameters (NN weights)

Filter mask parameters

Learning Image Features

Task: Classify whether an image has a cat

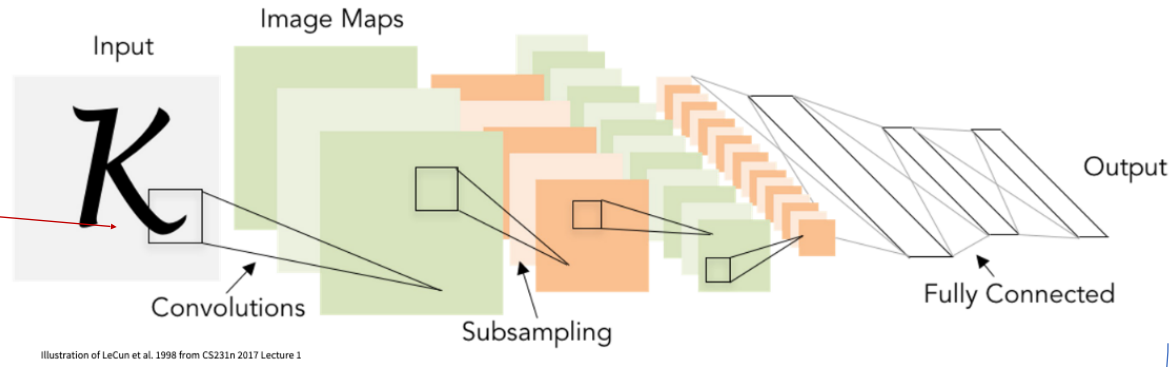
Labeled data:

Model Architecture:

Model

Free parameters (NN weights)

Filter mask parameters



Free parameters

Learning Image Features

Task: Classify whether an image has a cat

Labeled data:

Model Architecture:

Training loss function:

Learning Image Features

Task: Classify whether an image has a cat

Labeled data:

Model Architecture:

Training loss function:

E.g., cross entropy loss (classification):

Learning Image Features

Task: Classify whether an image has a cat

Labeled data:

Model Architecture:

Training loss function:

E.g., cross entropy loss (classification):

Learning Image Features

Task: Classify whether an image has a cat

Labeled data:

Model Architecture:

Training loss function:

E.g., cross entropy loss (classification):

E.g., l2 loss (regression):

Learning Image Features

Task: Classify whether an image has a cat

Labeled data:

Model Architecture:

Training loss function:

Optimization problem:

Learning Image Features

Task: Classify whether an image has a cat

Labeled data:

Model Architecture:

Training loss function:

Optimization problem:

Solve to get: e.g., with stoch. grad. descent (e.g. Adam) in Pytorch

Learning Image Features

Task: Classify whether an image has a cat

Labeled data:

Model Architecture:

Training loss function:

Optimization problem:

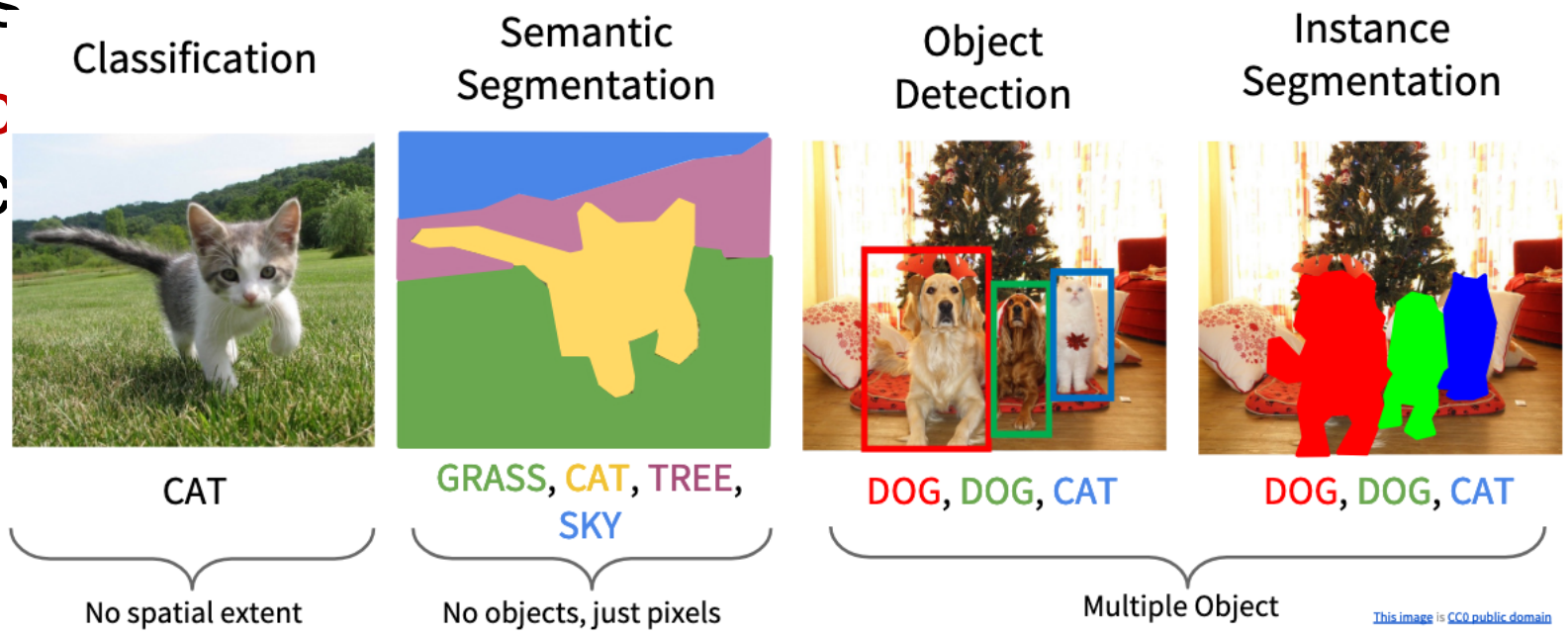
Solve to get: e.g., with stoch. grad. descent (e.g. Adam) in Pytorch

Learned model:

Prob. that a new image has a cat

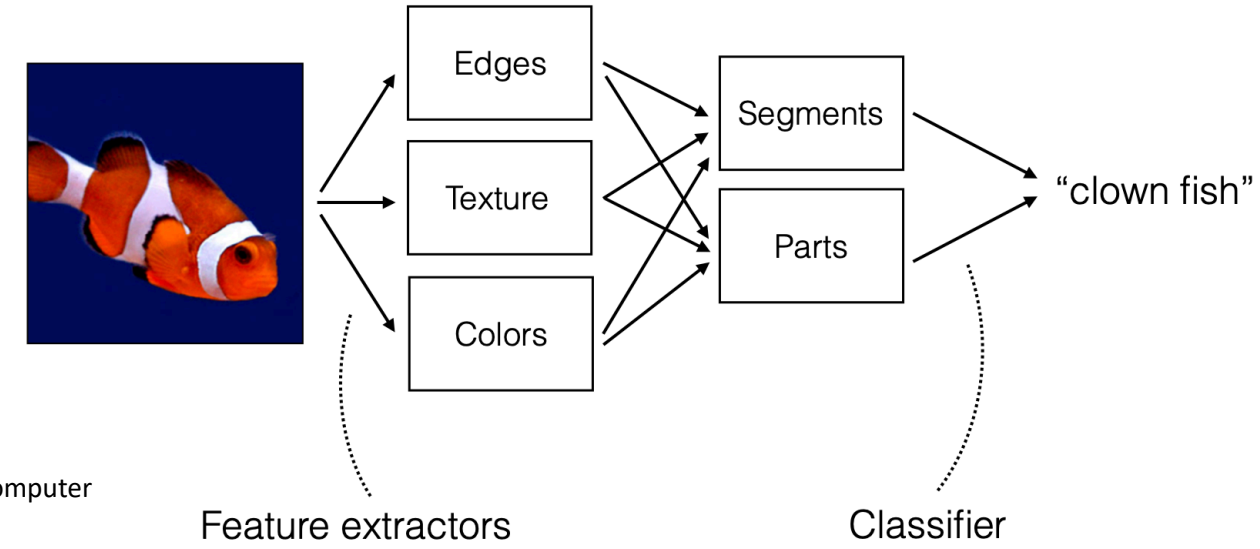
Classic Vision Problems: Semantic

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc



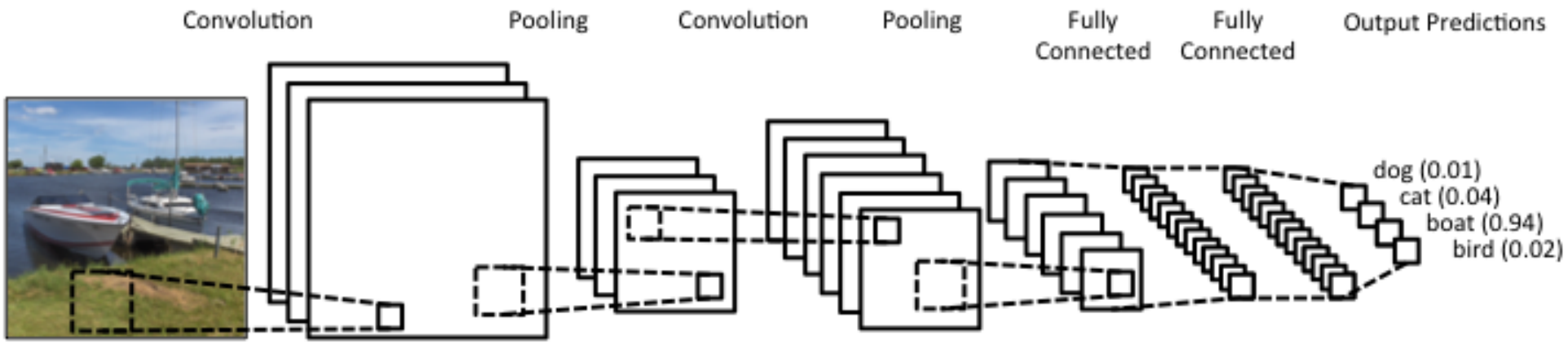
Learned Computer Vision Pipeline: Multiple layers of “feature extractors”

- Multiple layers of “feature extractors”
- Features of features of features – hierarchy of abstractions
- All learned from labeled examples

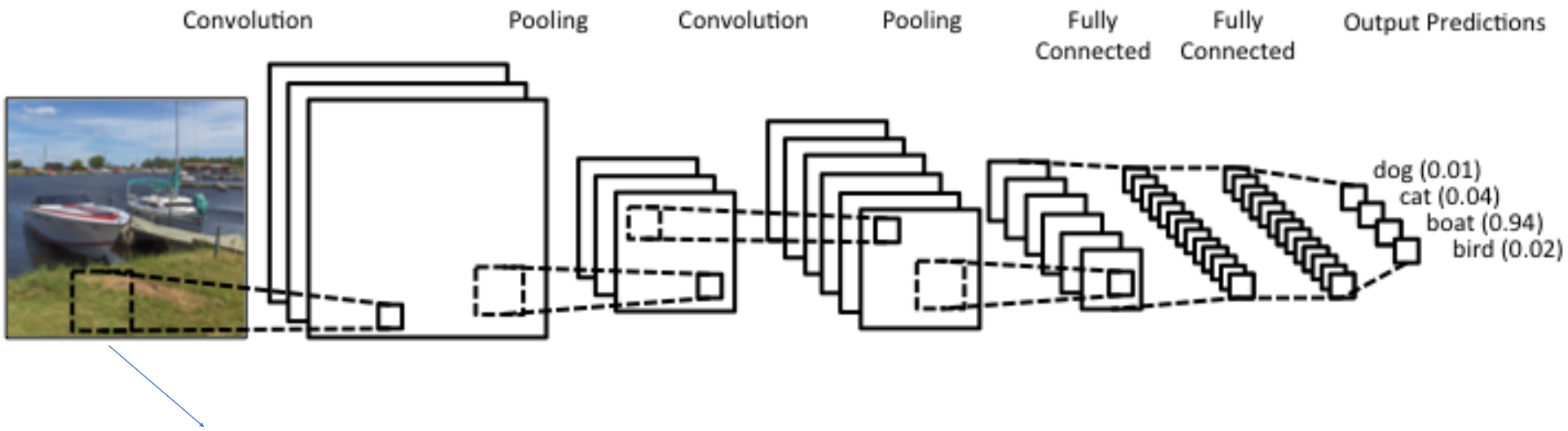


Example from Advances in Computer Vision – MIT – 6.869/6.819

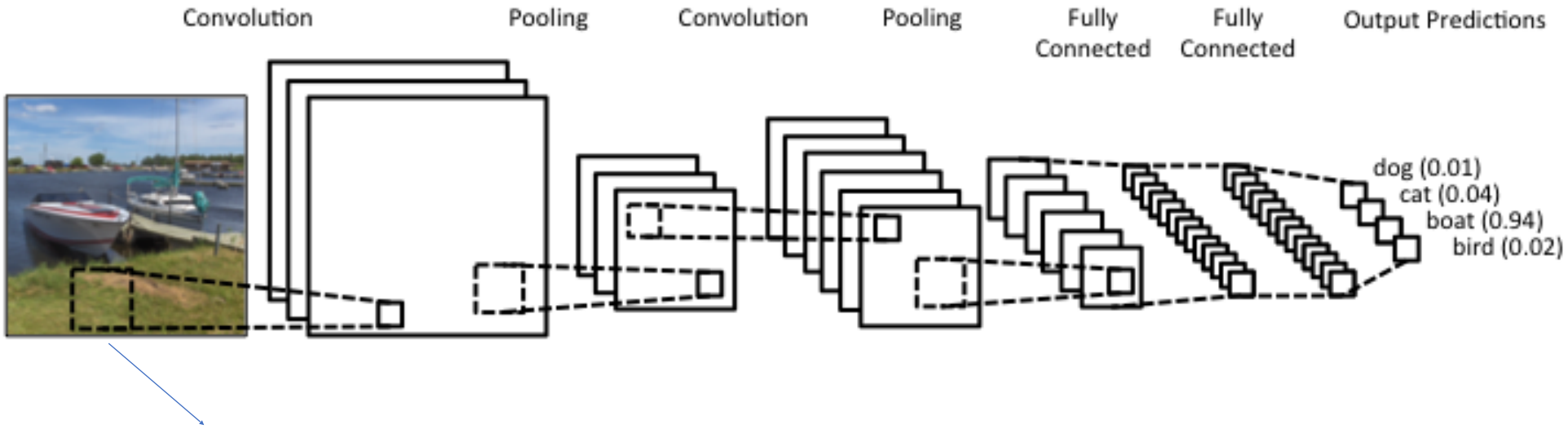
What are these parameters θ?



What are these parameters θ?



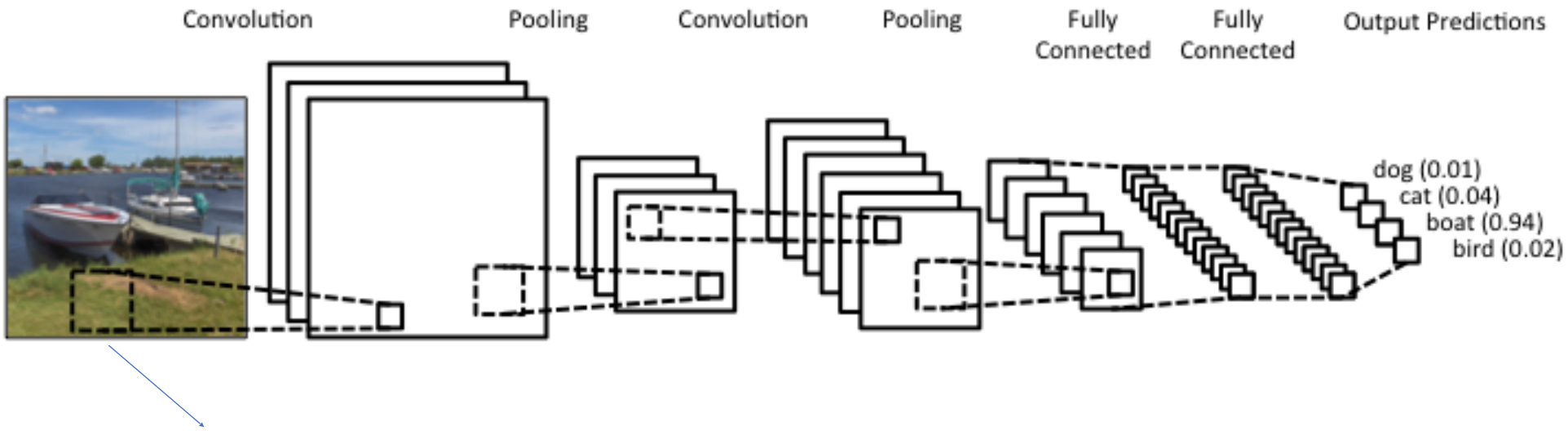
What are these parameters θ?



Hand-designed kernels:

$$F = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \quad S_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

What are these parameters θ?



Hand-designed kernels:

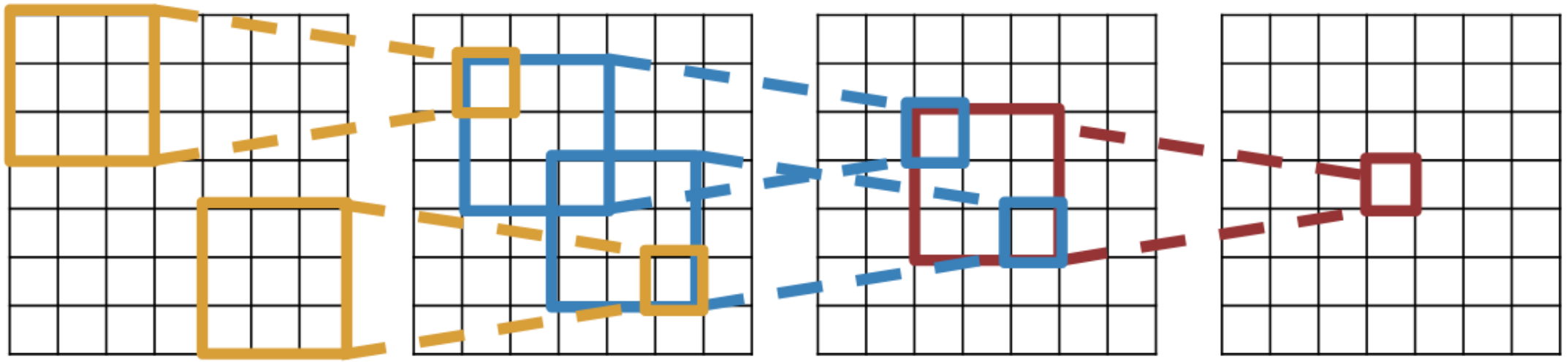
$$F = \frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix} \qquad G = \frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix} \qquad S_x = \begin{bmatrix}1 & 0 & -1\\ 2 & 0 & -2\\ 1 & 0 & -1\end{bmatrix}$$

Learned kernels:

$$F = \begin{matrix}\theta_{0,0} & \theta_{0,1} & \theta_{0,2}\\ \theta_{1,0} & \theta_{1,1} & \theta_{2,1}\\ \theta_{2,0} & \theta_{2,1} & \theta_{2,2}\end{matrix}$$

Receptive Fields

Each successive convolution adds K – 1 to the receptive field size
With L layers the receptive field size is 1 + L * (K – 1)



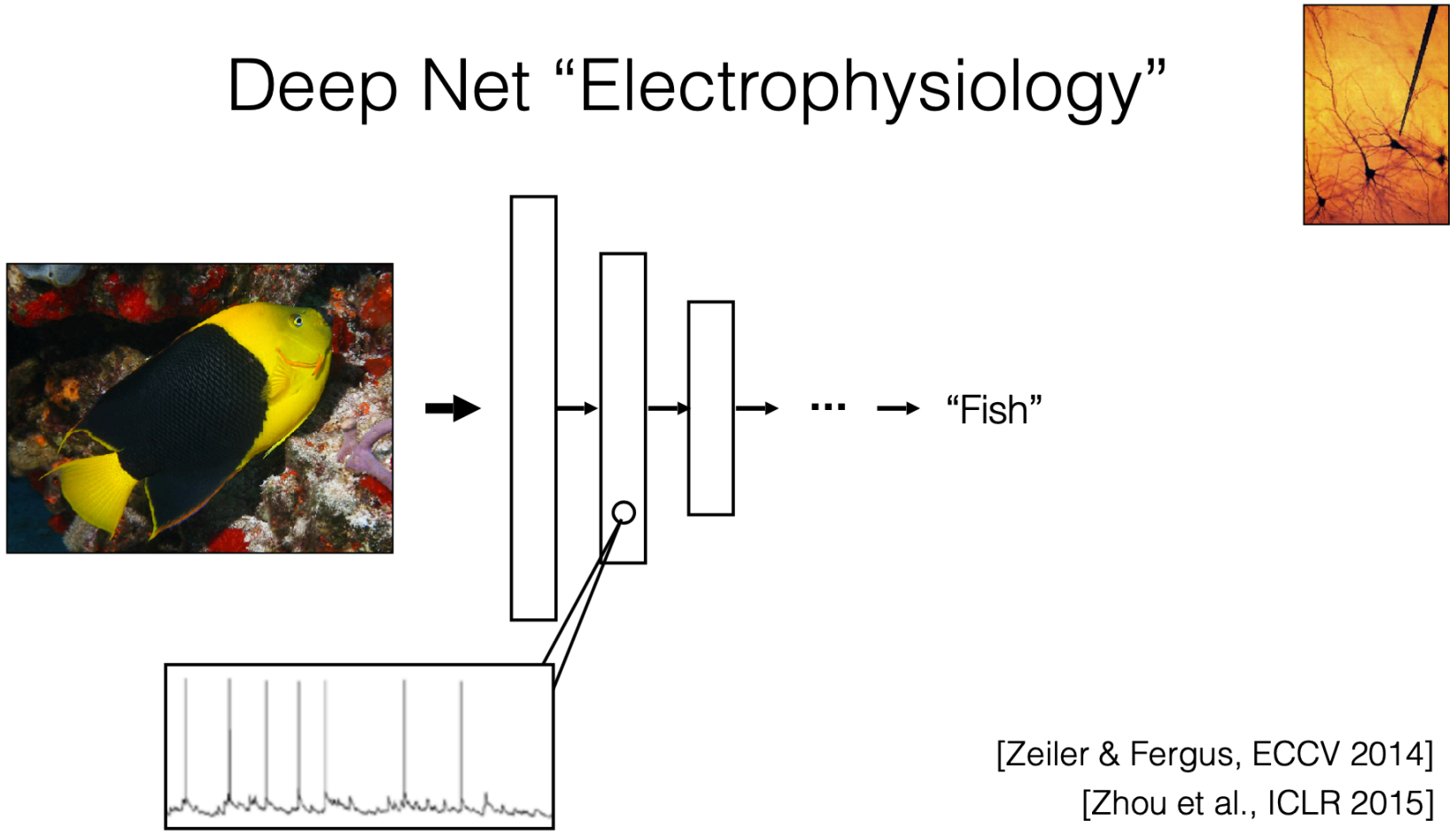
Be careful – “receptive field in the input” vs. “receptive field in the previous layer”

Slide inspiration: Justin Johnson
From Lecture 5 – CS231n

How do you interpret what the network has learned?

Example from Advances in Computer Vision – MIT – 6.869/6.819

How do you interpret what the network has learned?



[Zeiler & Fergus, ECCV 2014]
[Zhou et al., ICLR 2015]

Visualizing and Understanding CNNs

[Zeiler and Fergus, 2014]

Gabor-like filters learned by **layer 1**

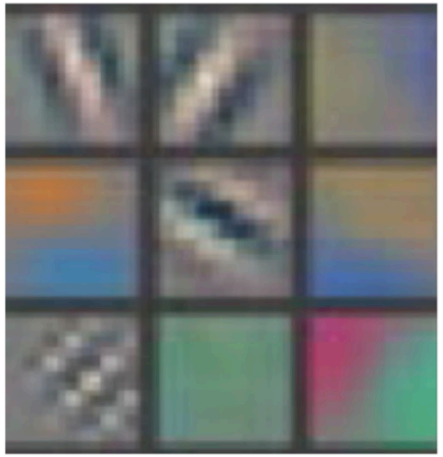
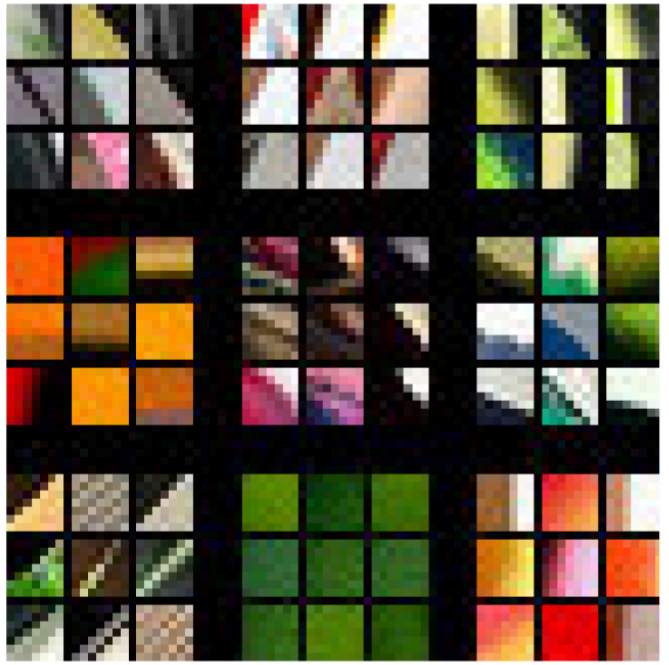


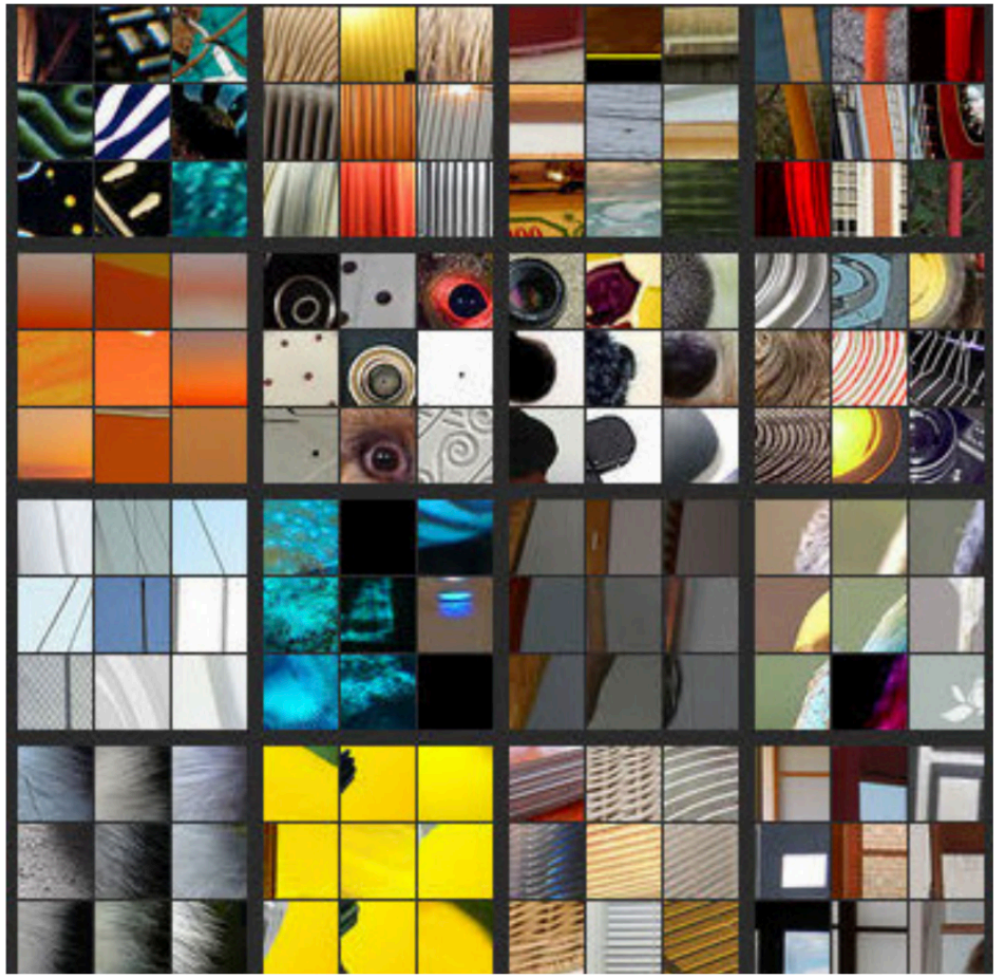
Image patches that activate each of the **layer 1** filters most strongly



Visualizing and Understanding CNNs

[Zeiler and Fergus, 2014]

Image patches that activate each of the **layer 2** neurons most strongly



Example from Advances in Computer Vision – MIT – 6.869/6.819

Visualizing and Understanding CNNs

[Zeiler and Fergus, 2014]

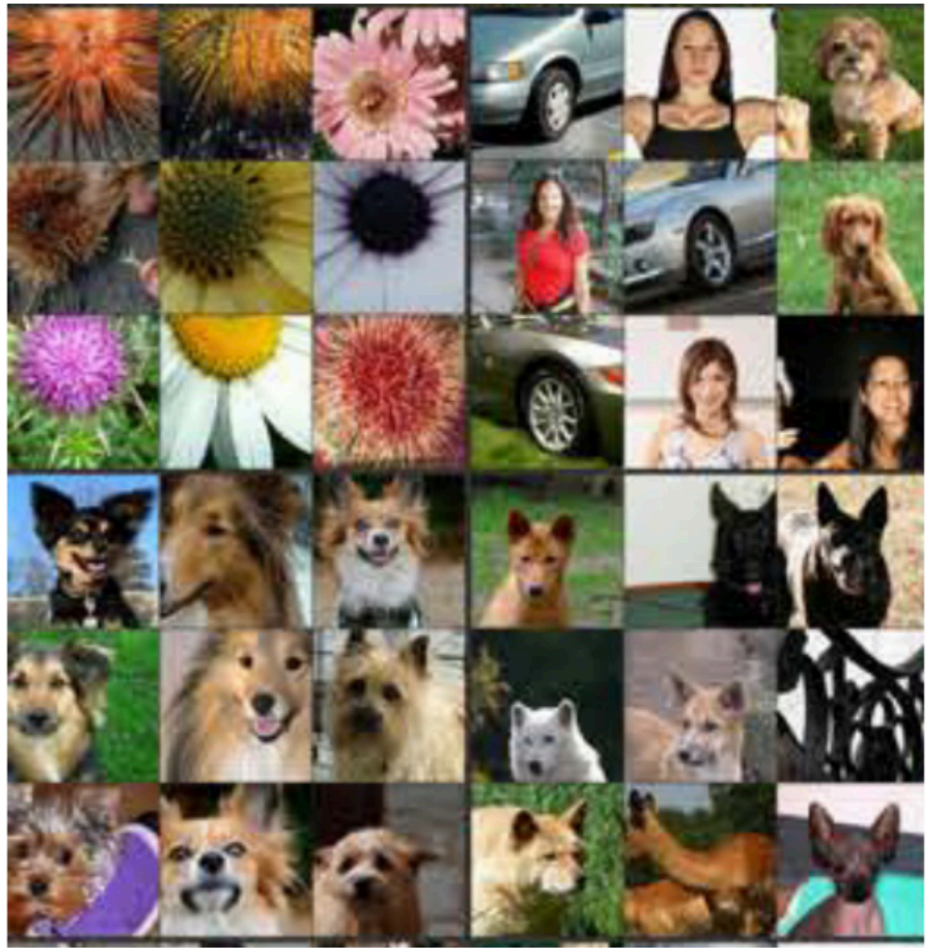
Image patches that activate each of the **layer 4** neurons most strongly

Example from Advances in Computer Vision – MIT – 6.869/6.819

Visualizing and Understanding CNNs

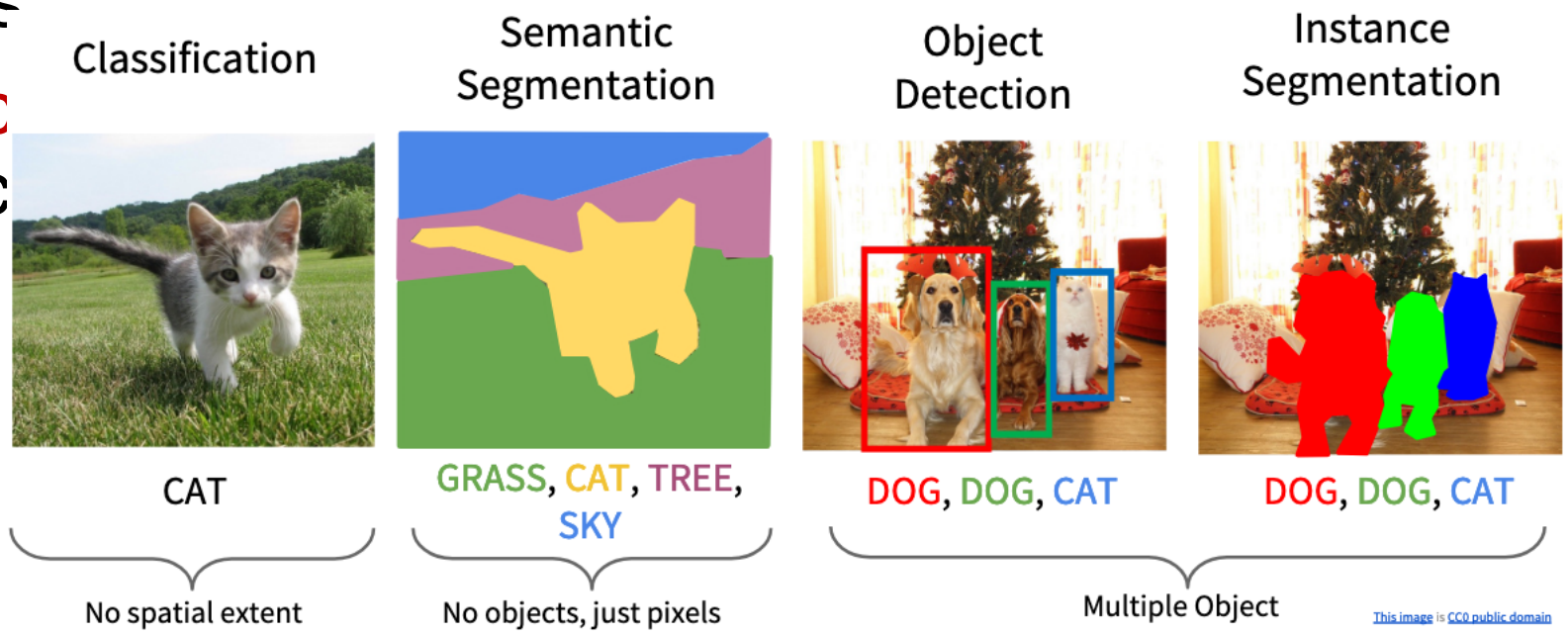
[Zeiler and Fergus, 2014]

Image patches that activate each of the **layer 5** neurons most strongly



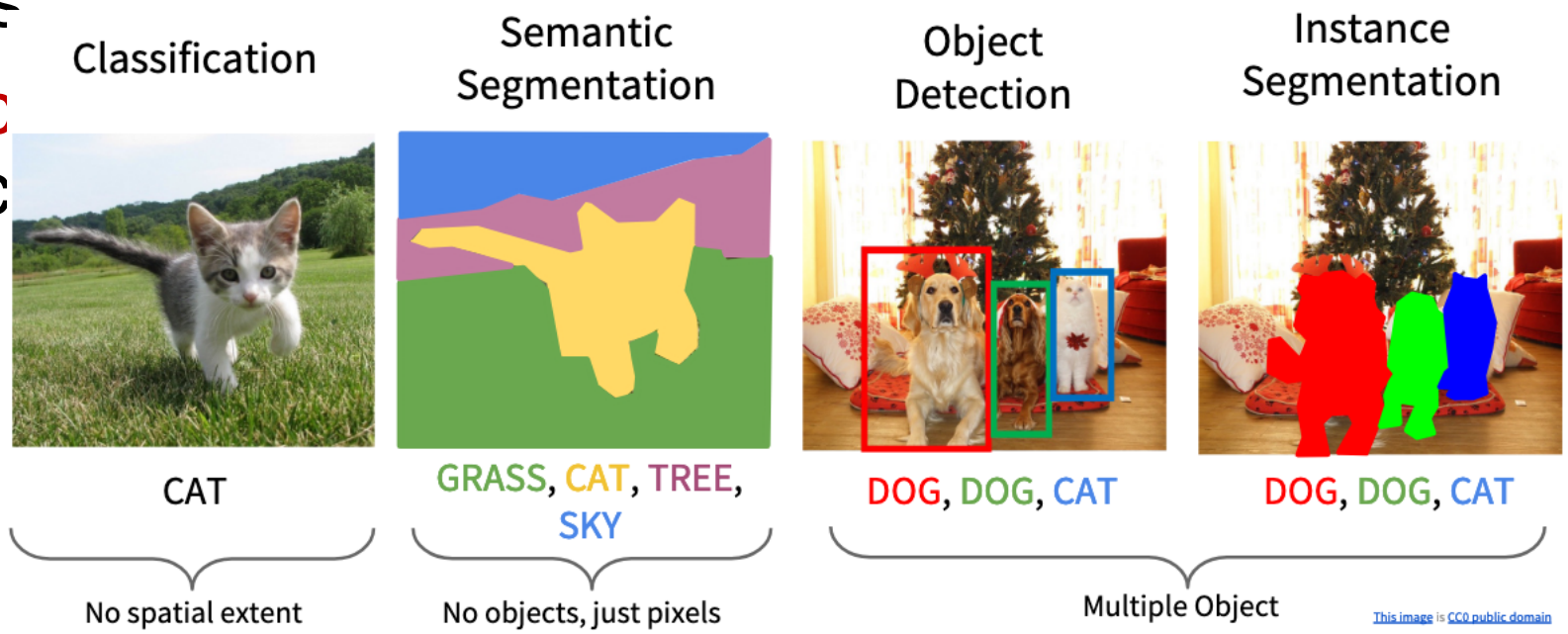
Semantic Perception

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc

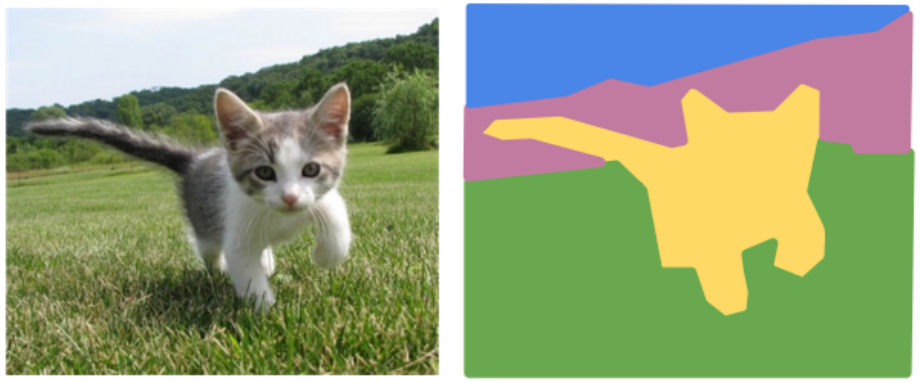


Semantic Perception

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc

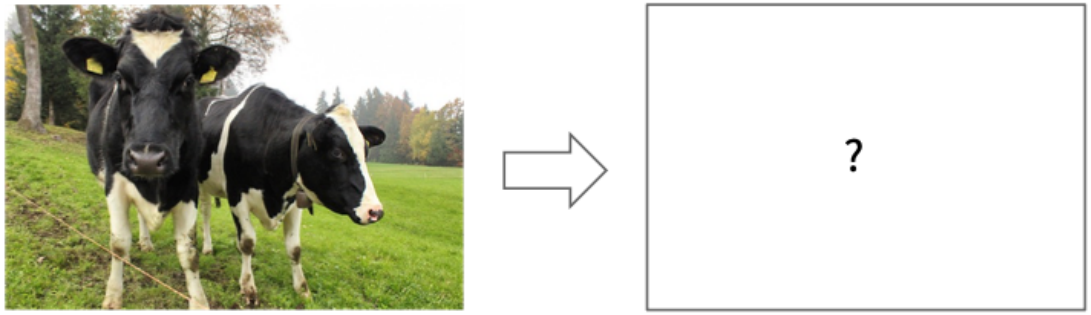


Semantic Segmentation: The Problem



GRASS, CAT, TREE,
SKY, ...

Paired training data: for each training image, each pixel is labeled with a semantic category.



At test time, classify each pixel of a new image.

Semantic Segmentation Idea: Sliding Window

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

Semantic Segmentation Idea: Sliding Window

Impossible to Classify without Context!

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

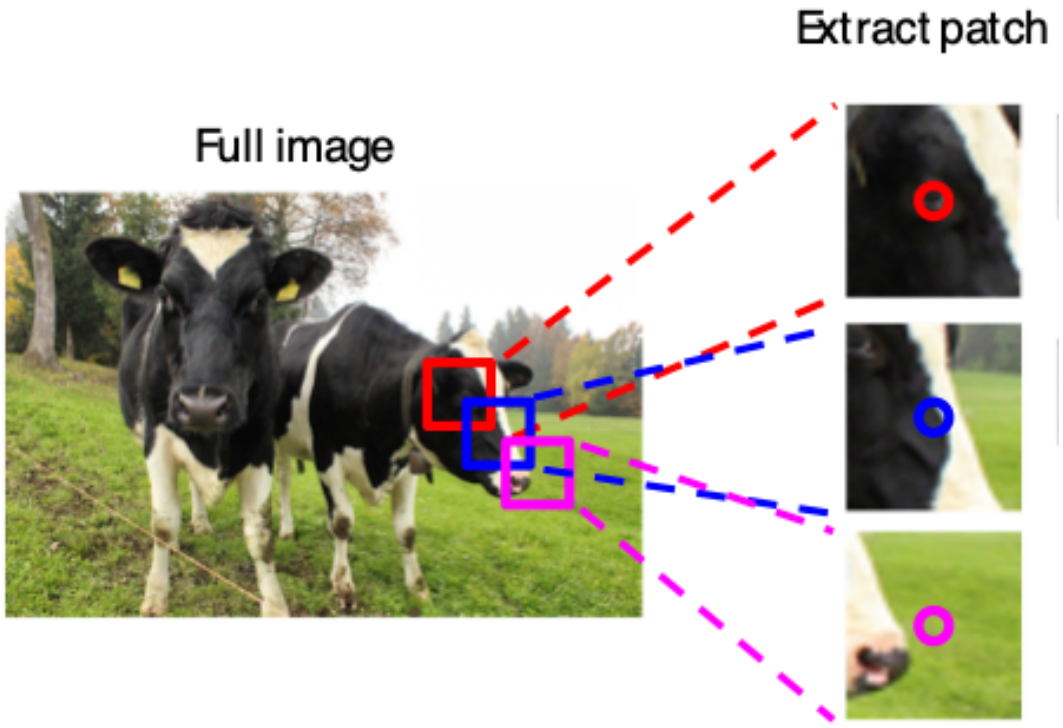
Semantic Segmentation Idea: Sliding Window

Impossible to Classify without Context!

Q: How do we include Context?

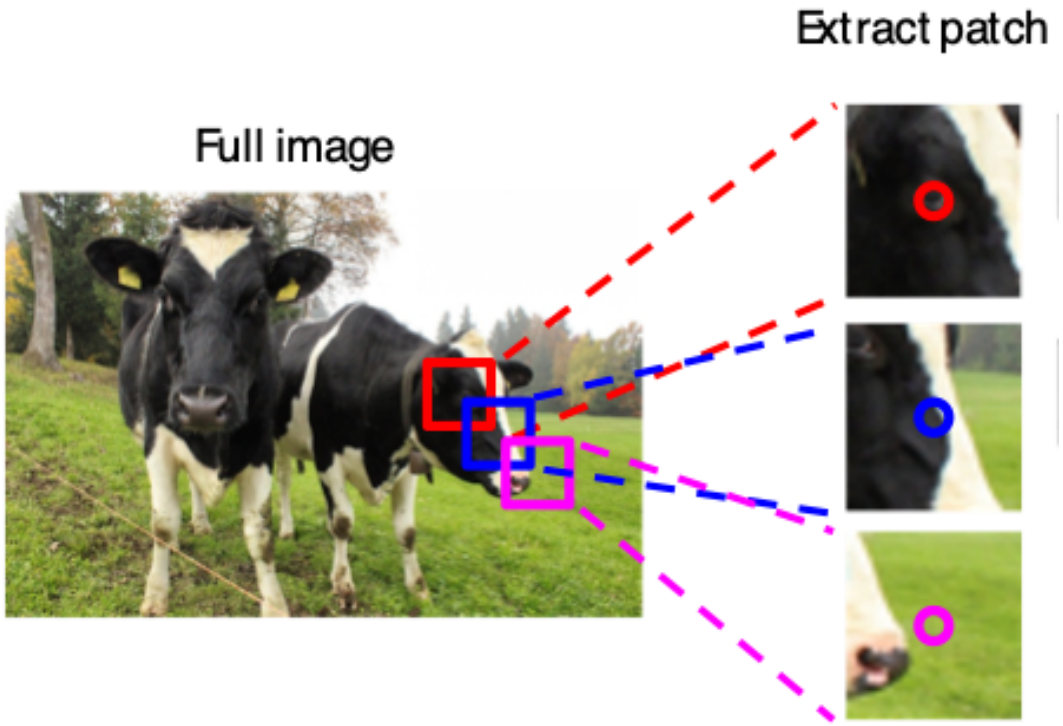
Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

Semantic Segmentation Idea: Sliding Window



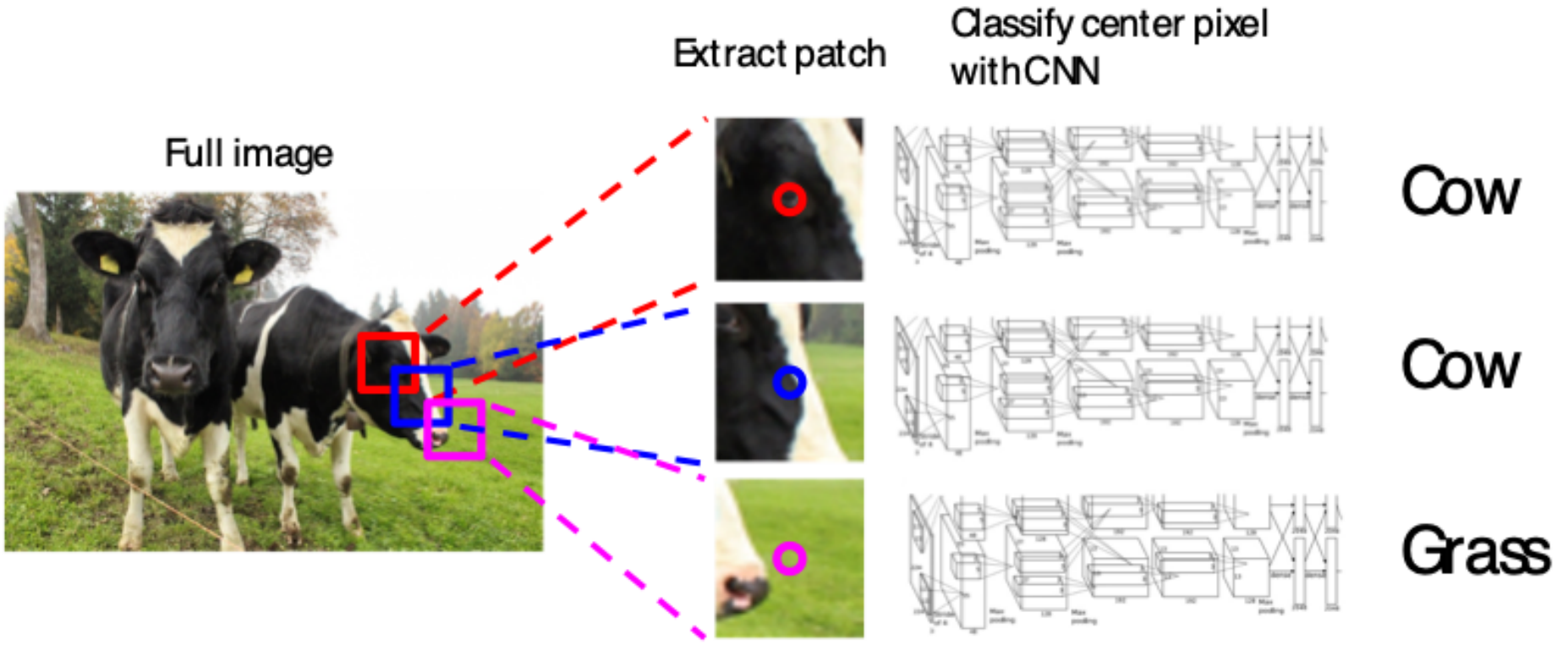
Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

Semantic Segmentation Idea: Sliding Window



Q: How do we model this?

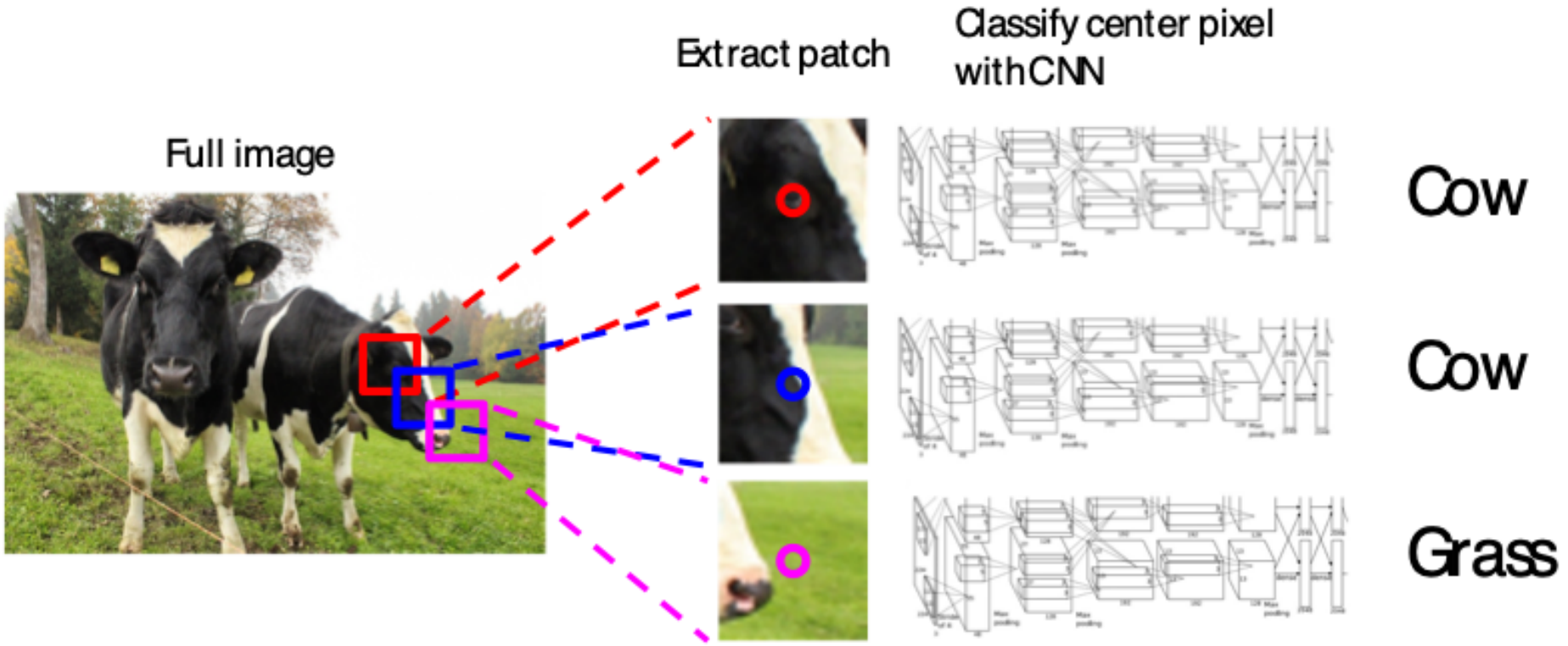
Semantic Segmentation Idea: Sliding Window



Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013
Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

Semantic Segmentation Idea: Sliding Window



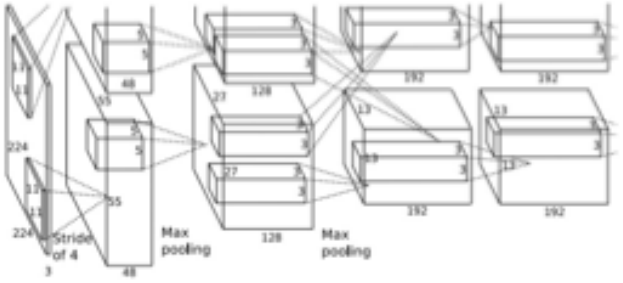
Problem: Very Inefficient! Not reusing shared features between overlapping patches

Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013
Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

Semantic Segmentation Idea: Convolution

Full image



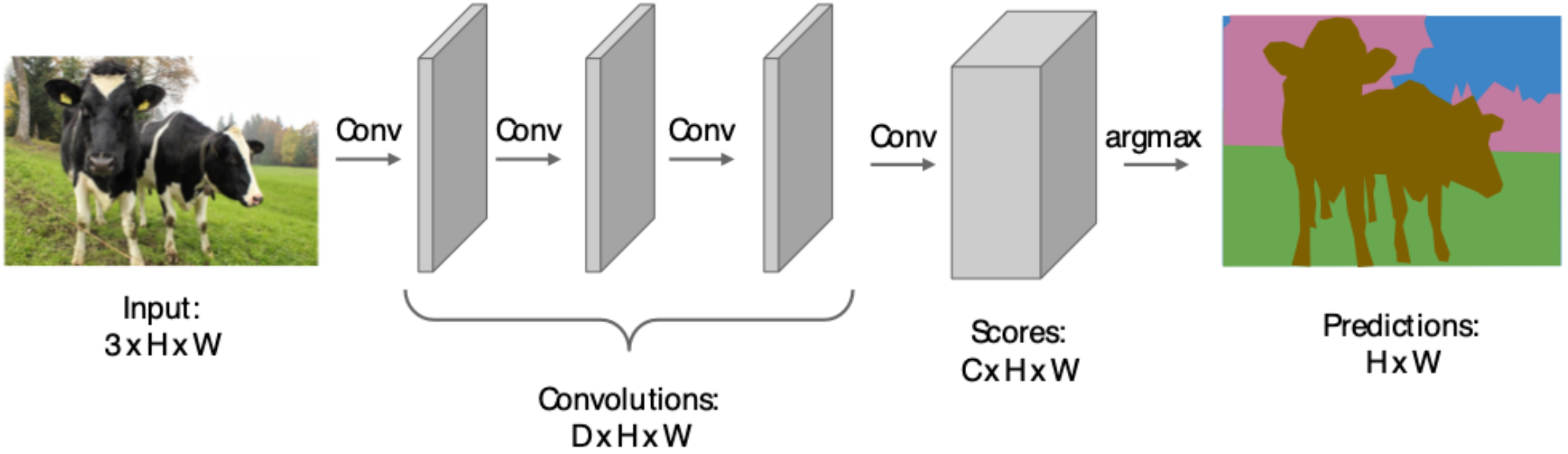
An intuitive idea: encode the entire image with conv net, and do semantic segmentation on top.

Problem: classification architectures often reduce feature spatial sizes to go deeper, but semantic segmentation requires the output size to be the same as input size.

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

Semantic Segmentation Idea: Fully Convolutional

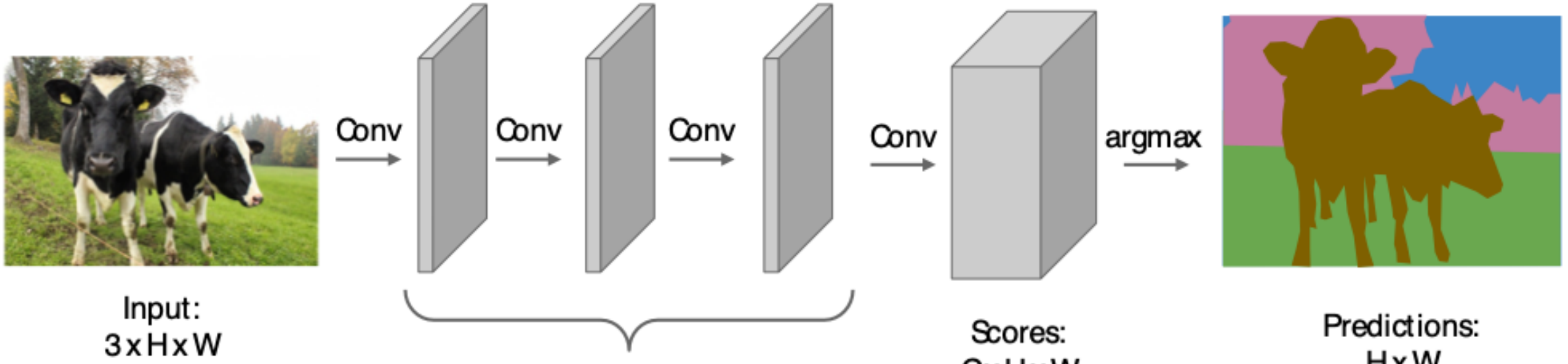
Design a network with only convolutional layers without downsampling operators to make predictions for pixels all at once!



Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

Semantic Segmentation Idea: Fully Convolutional

Design a network with only convolutional layers without downsampling operators to make predictions for pixels all at once!

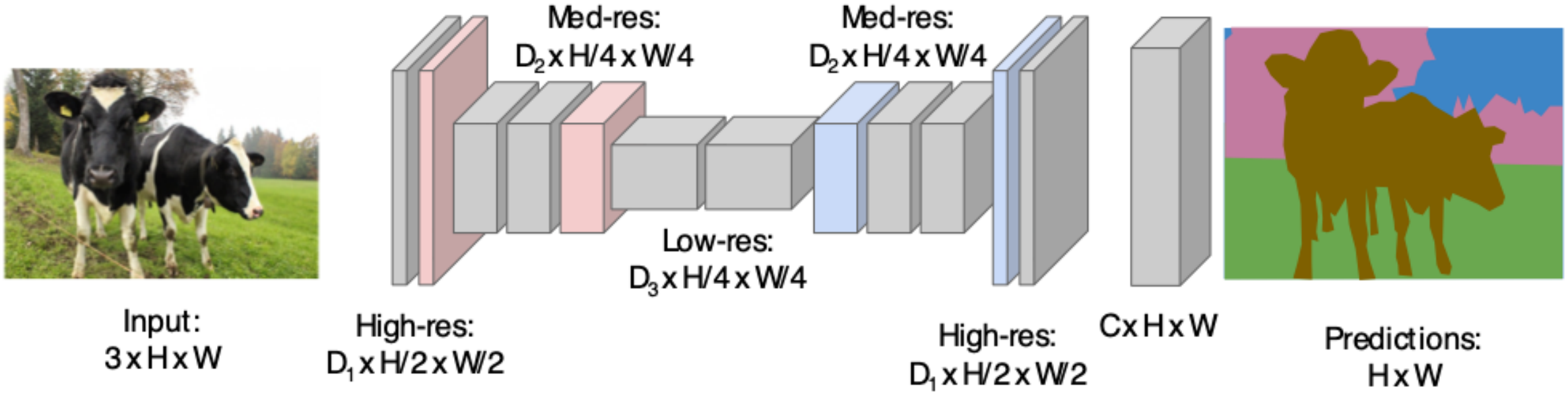


Problem: Convolutions at original image resolution will be very expensive

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

Semantic Segmentation Idea: Fully Convolutional

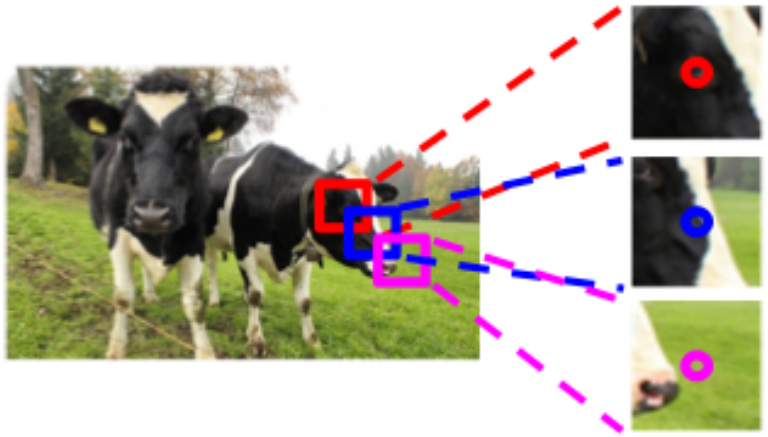
Design network as a bunch of convolutional layers, with downsampling and upsampling inside the network!

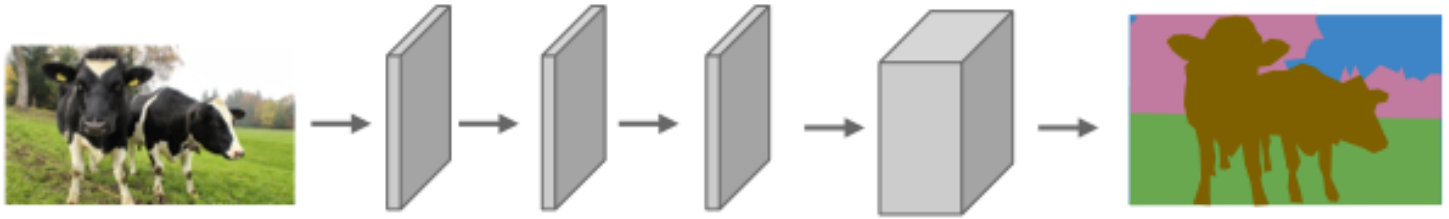


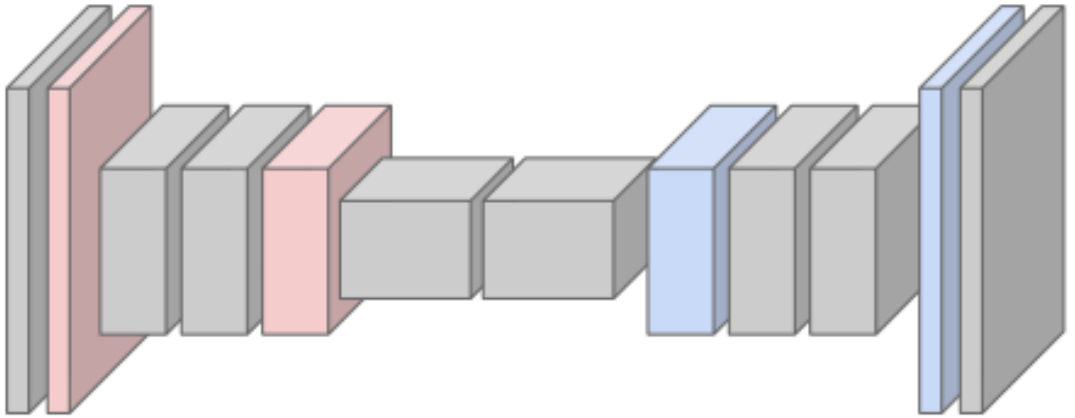
Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

SemanticSegmentation:Summary



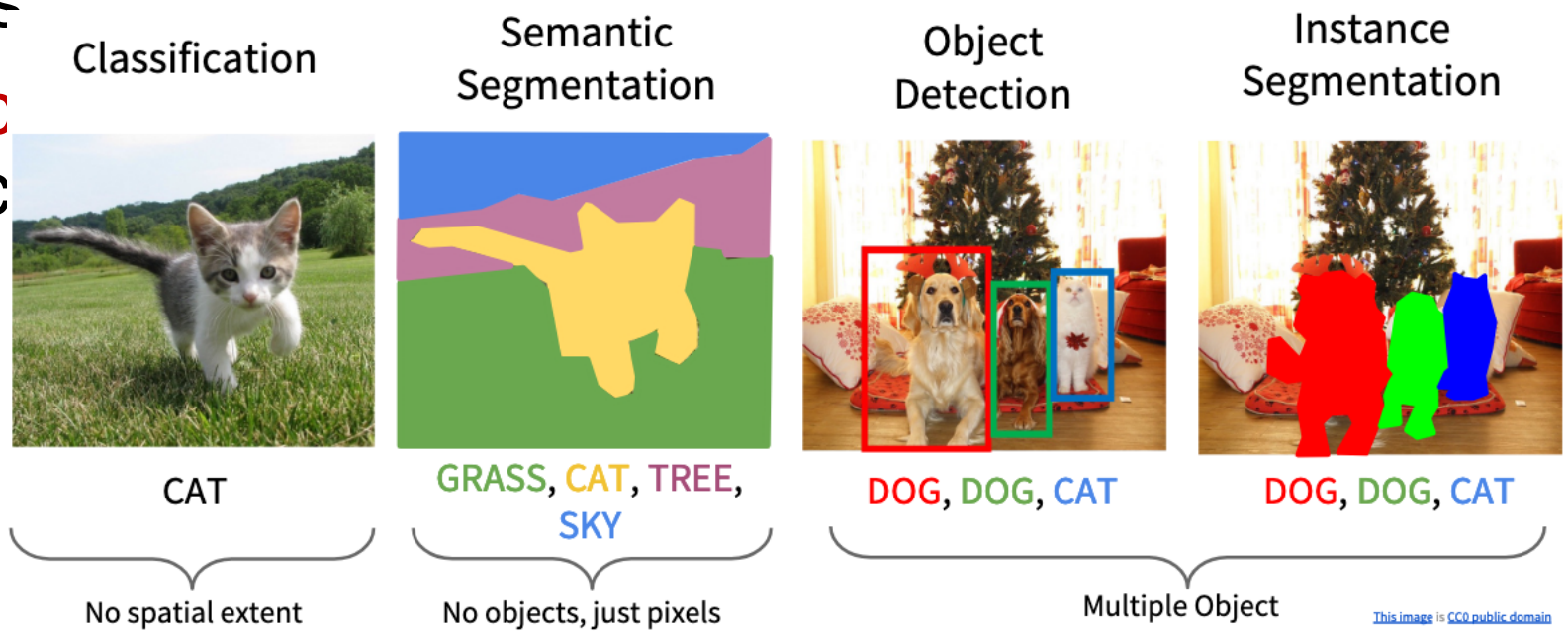




Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

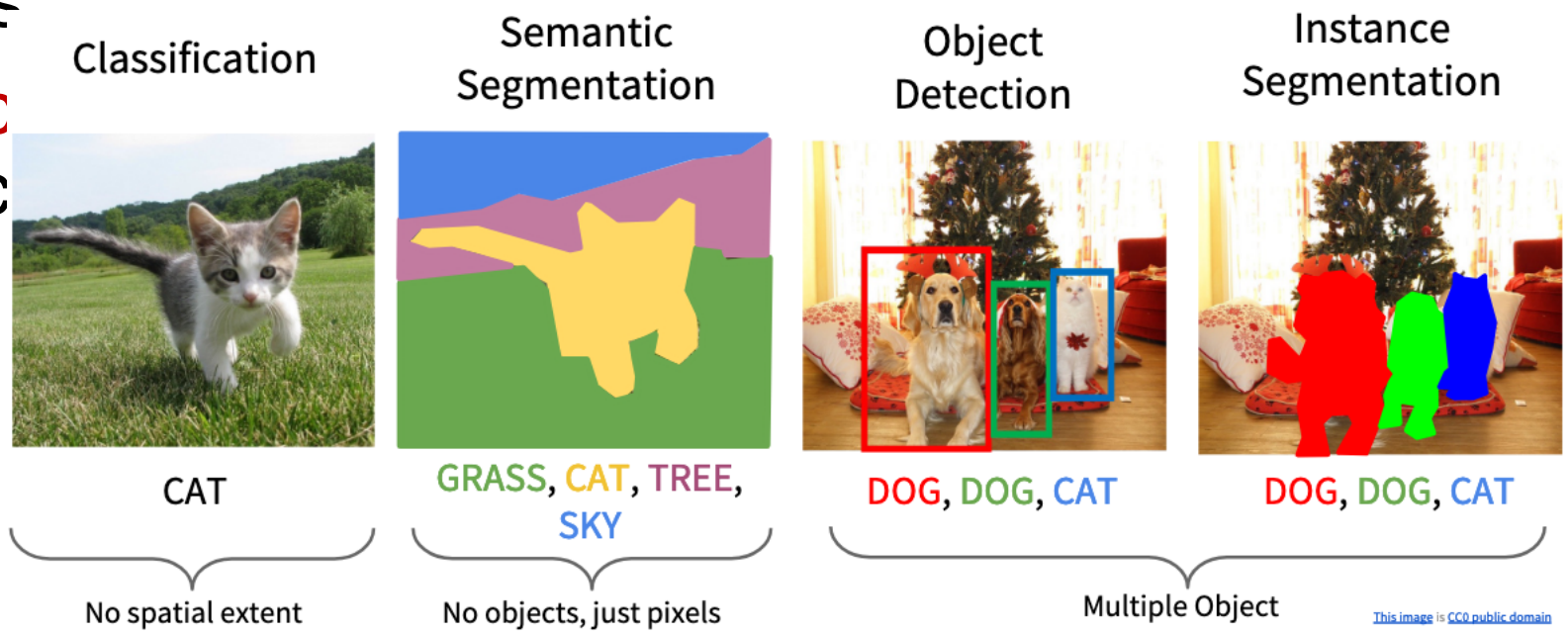
Semantic Perception

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc



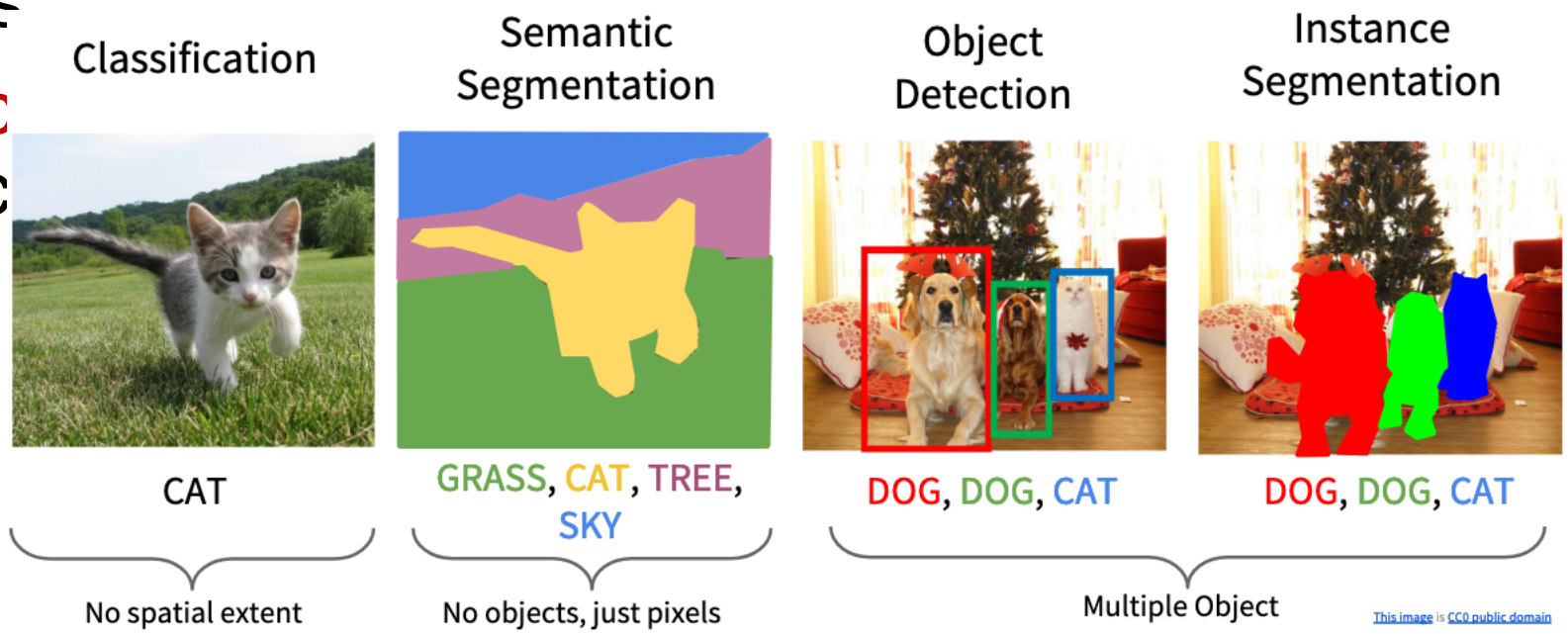
Semantic Perception

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc



Semantic Perception

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc



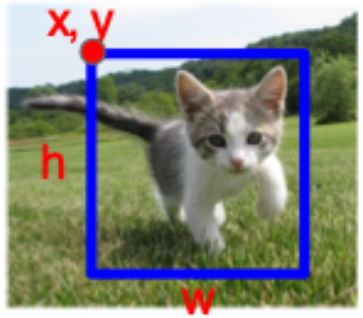
Semantic Perception

Take cs231n for more details and more advanced techniques!

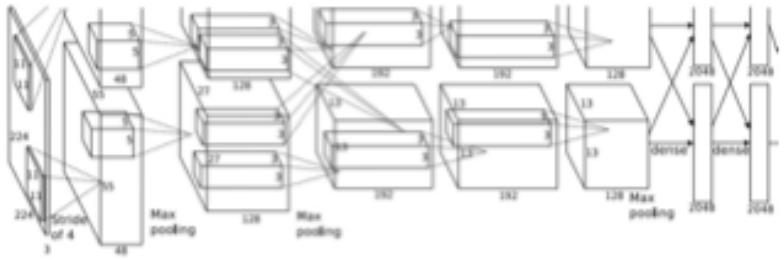
Extra slides from past years

Object Detection: Single Object

(Classification + Localization)



This image is CC0 public domain



Fully Connected: 4096 to 1000

Class Scores
Cat: 0.9
Dog: 0.05
Car: 0.01
...

Vector: 4096

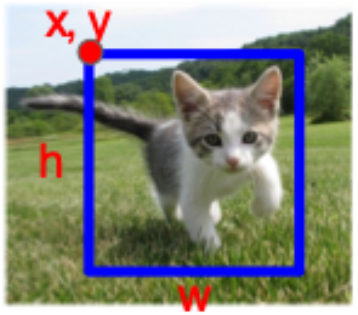
Fully Connected: 4096 to 4

Box Coordinates
(x, y, w, h)

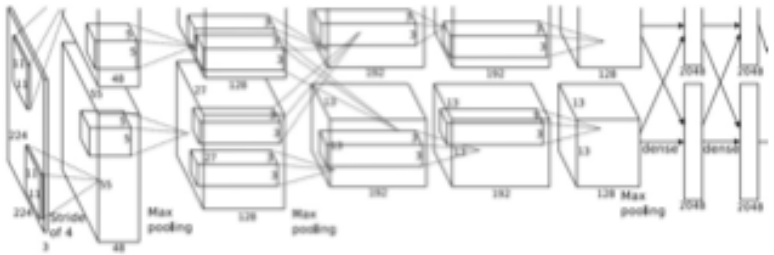
Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

Object Detection: Single Object

(Classification + Localization)



This image is CC0 public domain



Vector: 4096

Fully Connected: 4096 to 1000

Class Scores
Cat: 0.9
Dog: 0.05
Car: 0.01
...

Softmax Loss

Correct label: Cat

Fully Connected: 4096 to 4

Box Coordinates (x, y, w, h)

L2 Loss

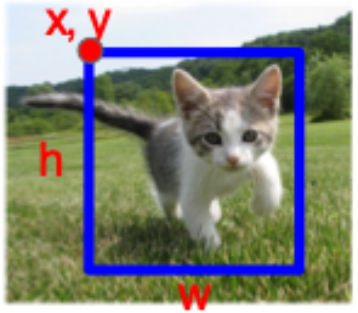
Correct box: (x', y', w', h')

Treat localization as a regression problem!

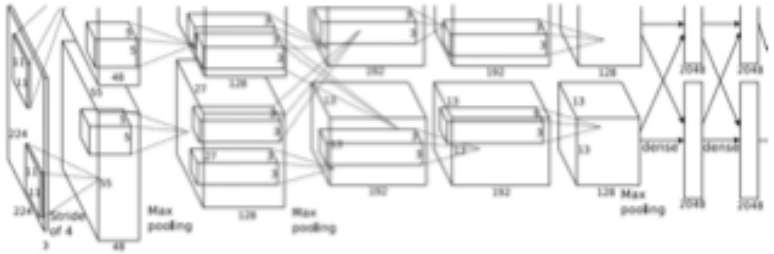
Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

Object Detection: Single Object

(Classification + Localization)



This image is CC0 public domain



Vector: 4096

Fully Connected: 4096 to 1000

Class Scores
Cat: 0.9
Dog: 0.05
Car: 0.01
...

Softmax Loss

Correct label: Cat

Fully Connected: 4096 to 4

Box Coordinates (x, y, w, h)

L2 Loss

Correct box: (x', y', w', h')

Multitask Loss

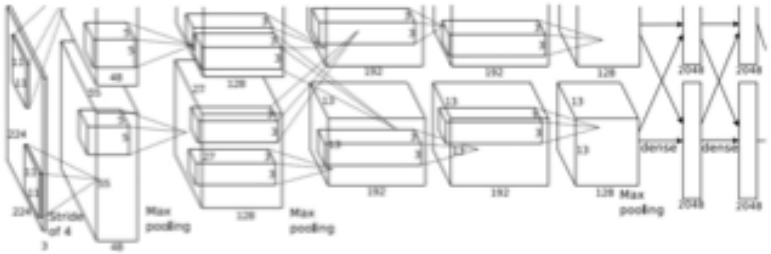
+

Loss

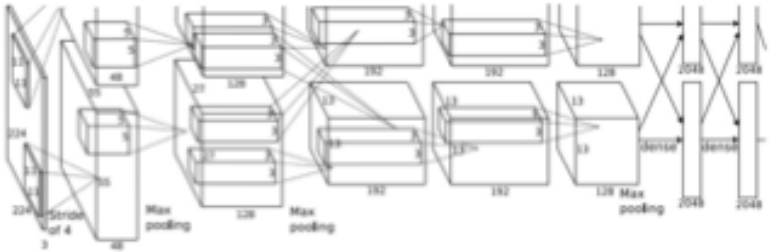
Treat localization as a regression problem!

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

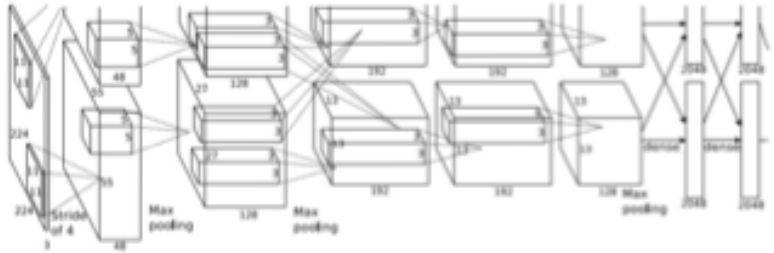
Object Detection: Multiple Objects



CAT: (x, y, w, h)



DOG: (x, y, w, h)
DOG: (x, y, w, h)
CAT: (x, y, w, h)

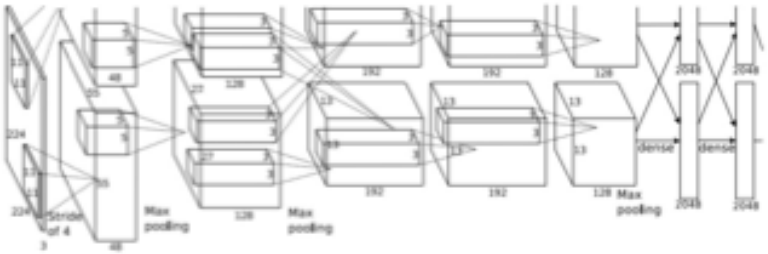


DUCK: (x, y, w, h)
DUCK: (x, y, w, h)
….

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

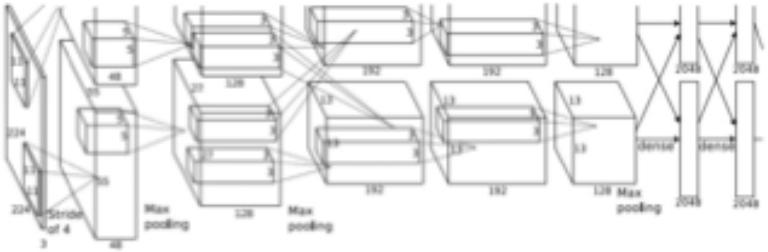
Object Detection: Multiple Objects

Each image needs a different number of outputs!



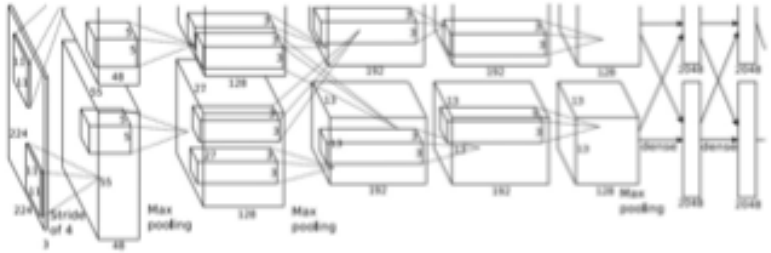
CAT: (x, y, w, h)

4 numbers



DOG: (x, y, w, h)
DOG: (x, y, w, h)
CAT: (x, y, w, h)

12 numbers



DUCK: (x, y, w, h)
DUCK: (x, y, w, h)
….

Many numbers!

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

Region Proposals: Selective Search

- Find "blobby" image regions that are likely to contain objects
- Relatively fast to run; e.g. Selective Search gives 2000 region proposals in a few seconds on CPU

Alexe et al, "Measuring the objectness of image windows", TPAMI 2012
Uijlings et al, "Selective Search for Object Recognition", IJCV 2013
Cheng et al, "BING: Binarized normed gradients for objectness estimation at 300fps", CVPR 2014
Zitnick and Dollar, "Edge boxes: Locating object proposals from edges", ECCV 2014

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

R-CNN

Input image

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

R-CNN

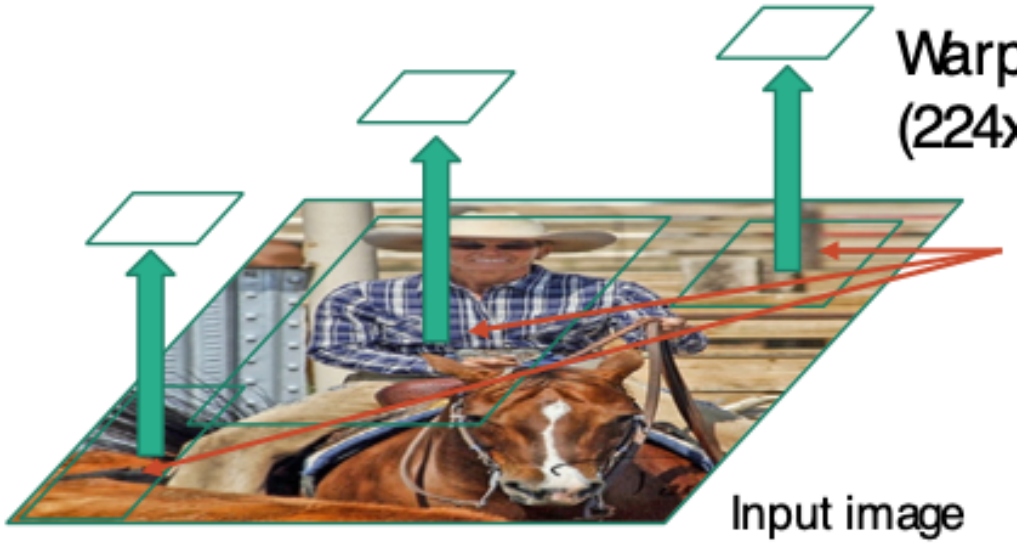
Input image

Regions of Interest (RoI) from a proposal method (~2k)

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

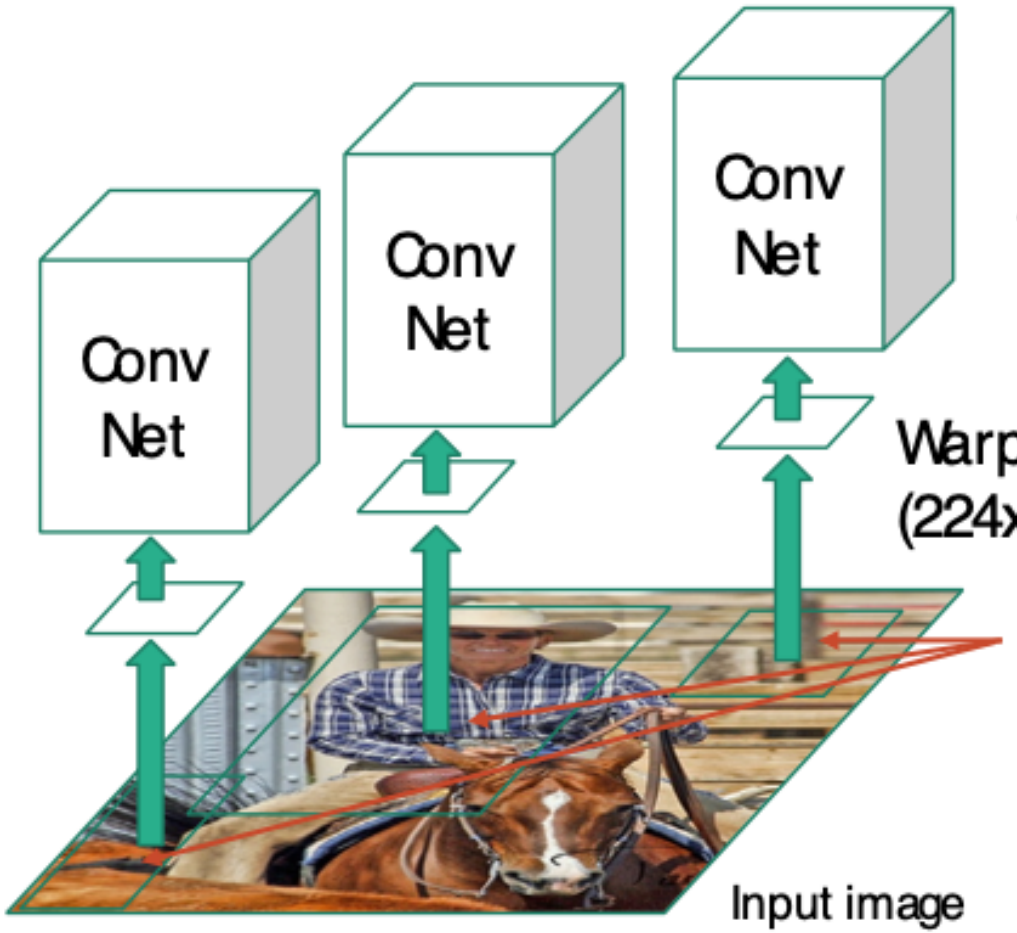
R-CNN



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

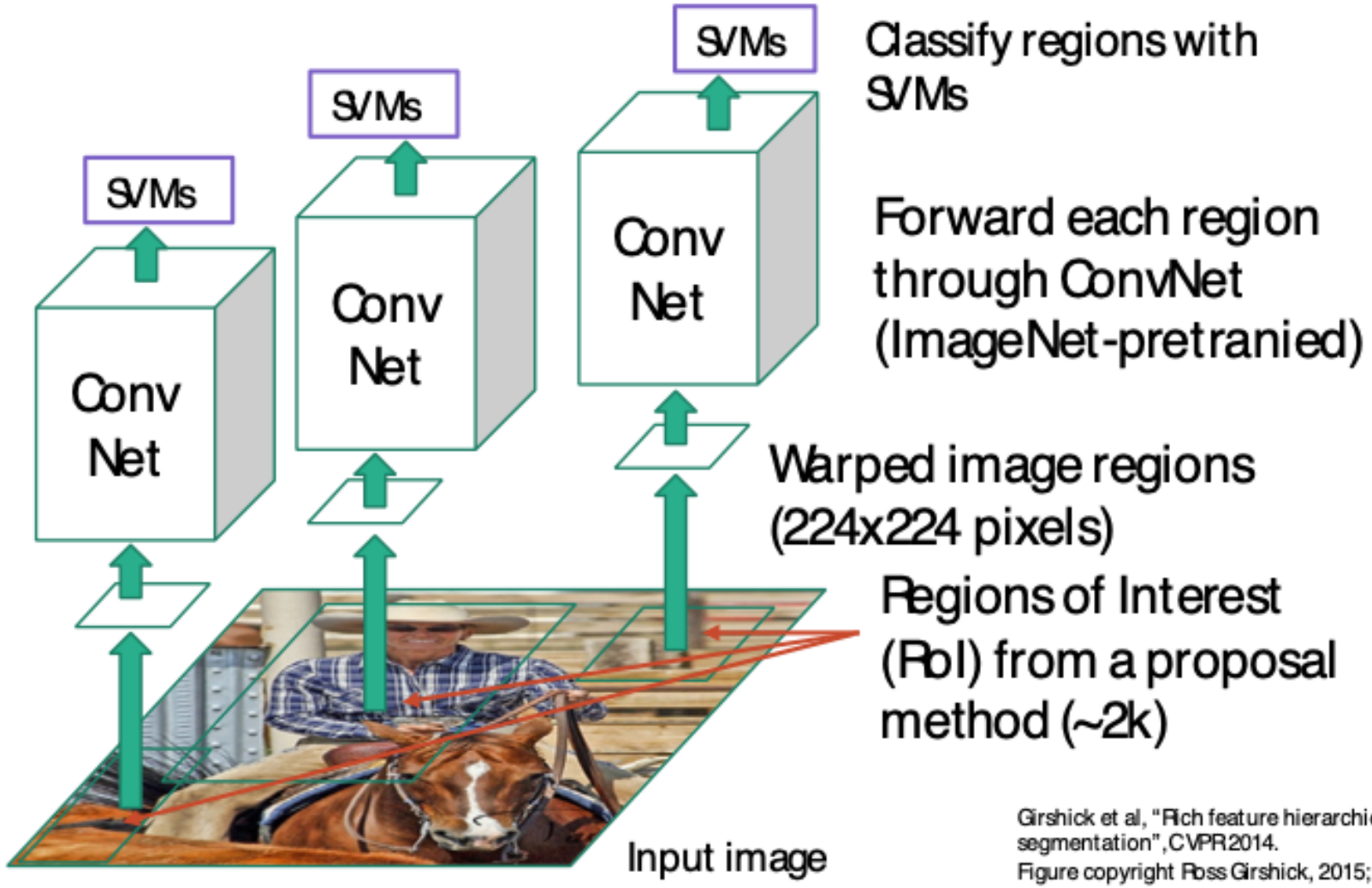
R-CNN



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

R-CNN

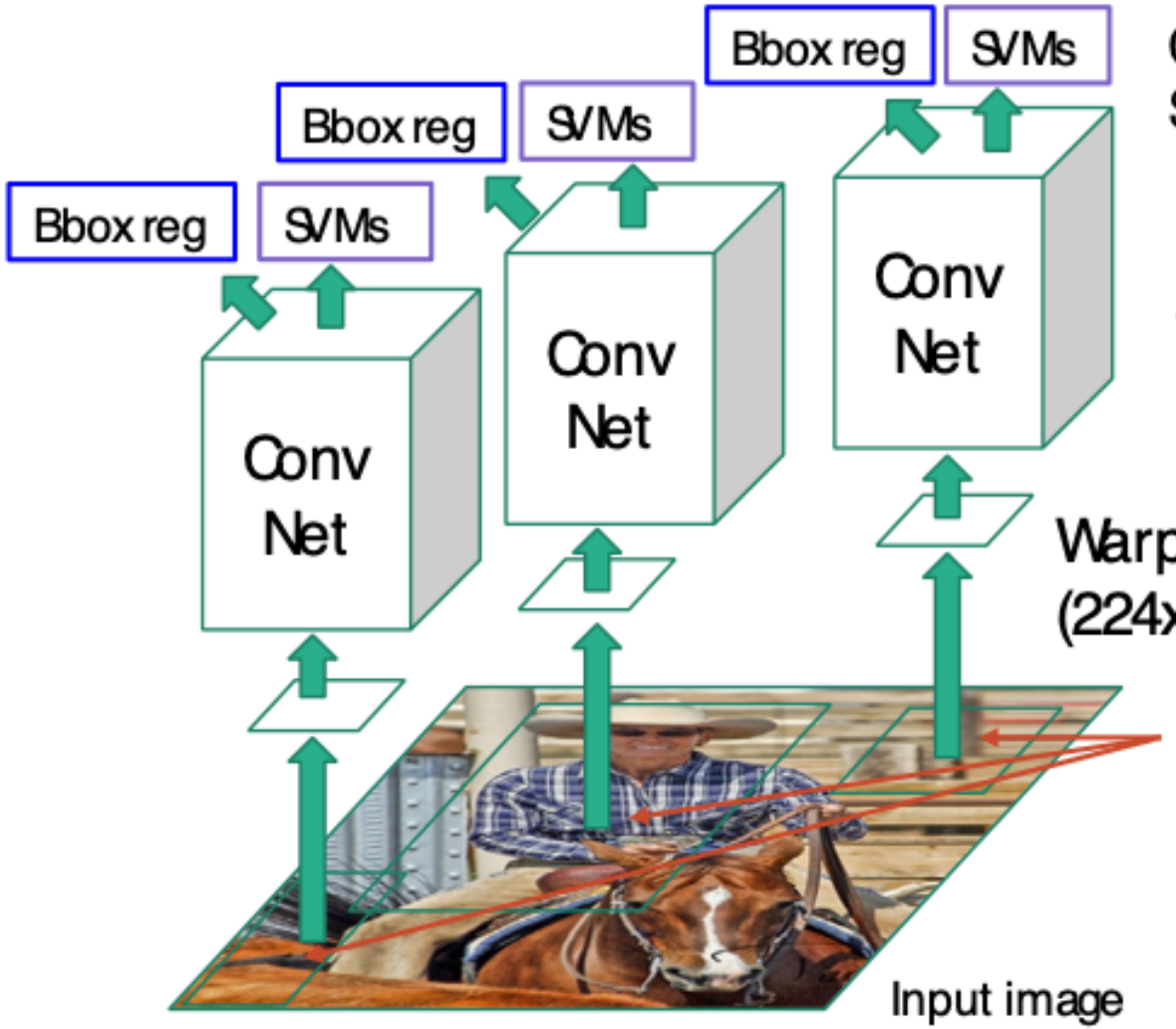


Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

R-CNN

Predict "corrections" to the RoI: 4 numbers: (dx, dy, dw, dh)



Classify regions with SVMs

Forward each region through ConvNet (ImageNet-pretranied)

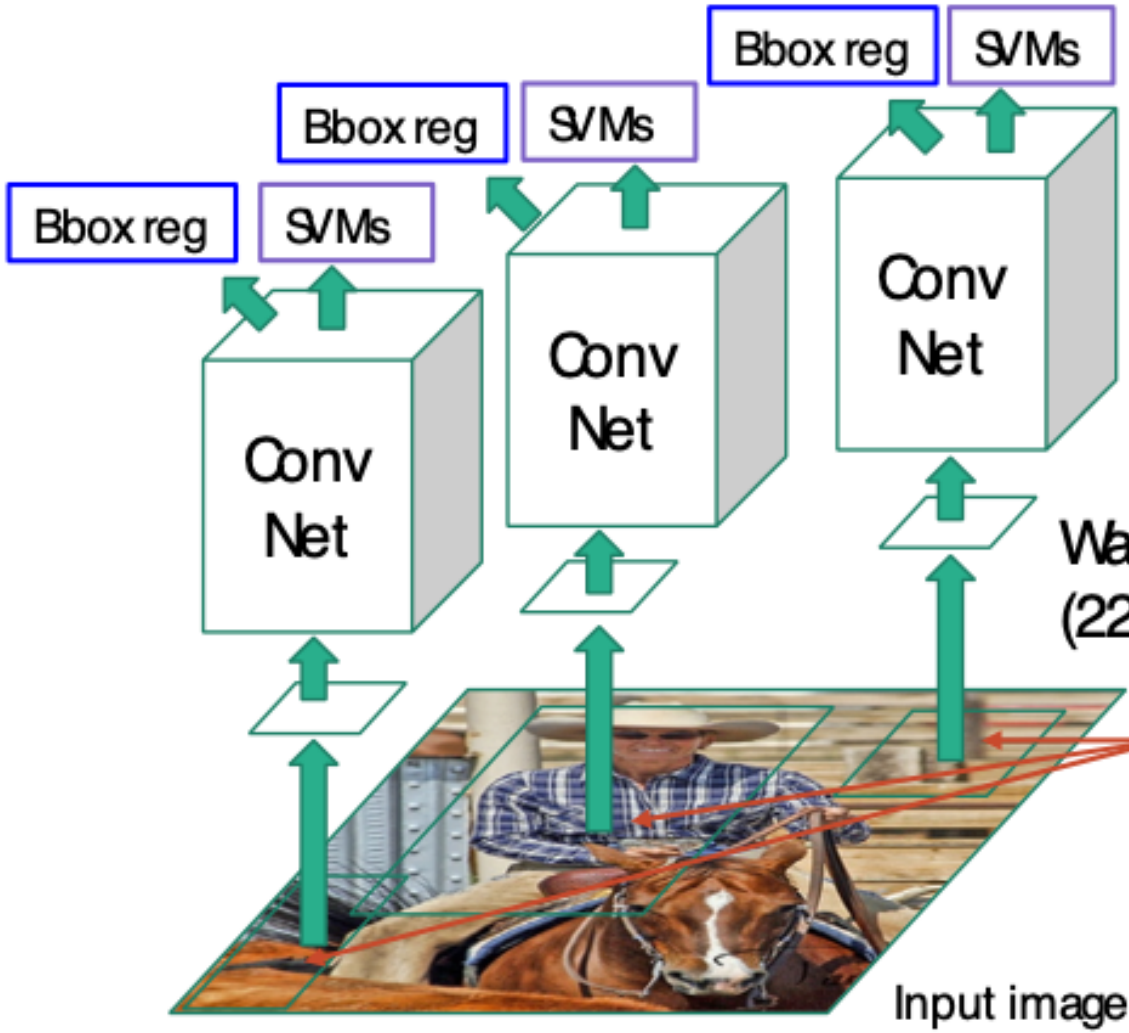
Warped image regions (224x224 pixels)

Regions of Interest (RoI) from a proposal method (~2k)

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

R-CNN

Predict "corrections" to the RoI: 4 numbers: (dx, dy, dw, dh)



Classify regions with SVMs

Forward each region through ConvNet

Warped image regions (224x224 pixels)

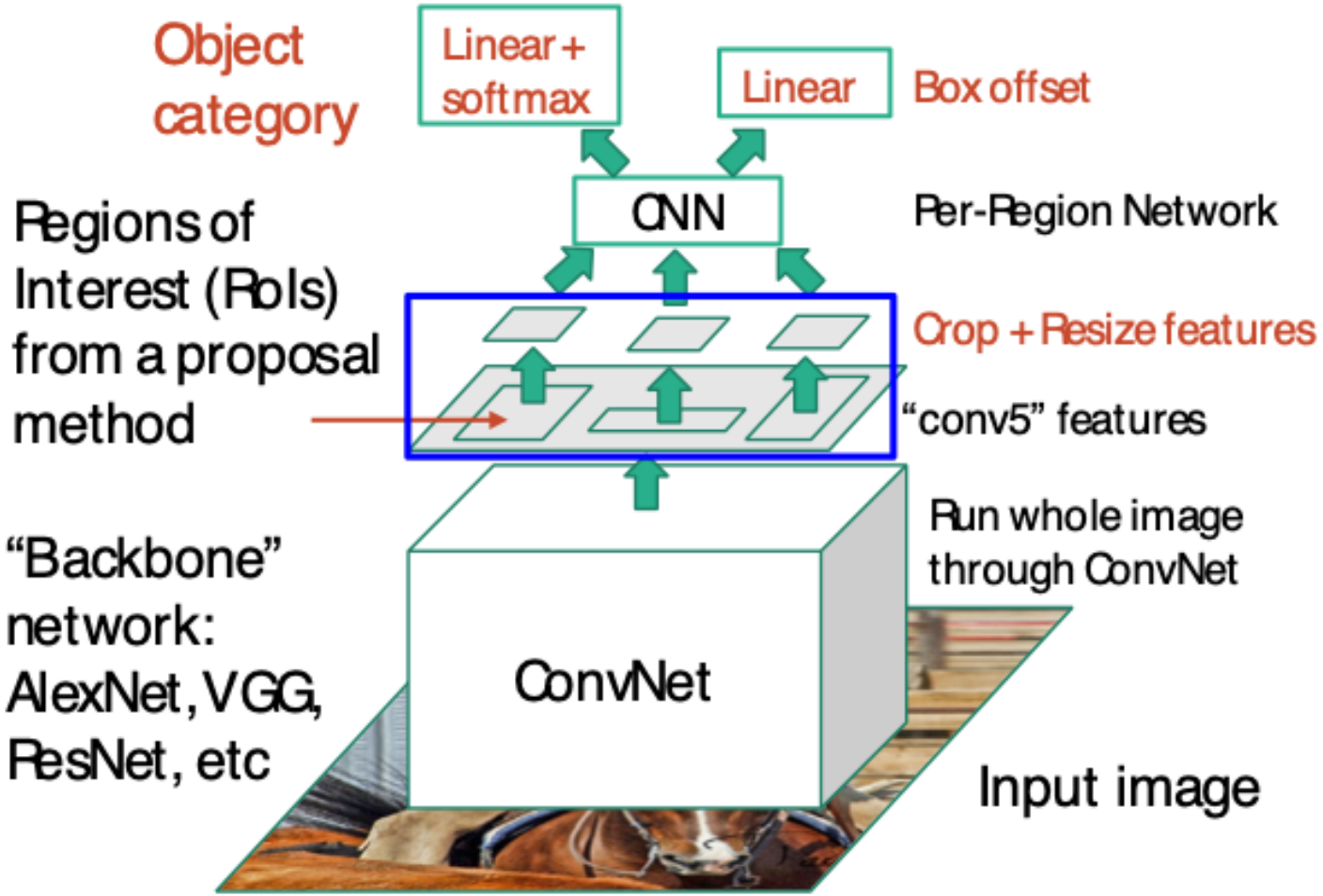
Regions of Interest (RoI) from a proposal method (~2k)

Problem: Very slow! Need to do ~2k independent forward passes for each image!

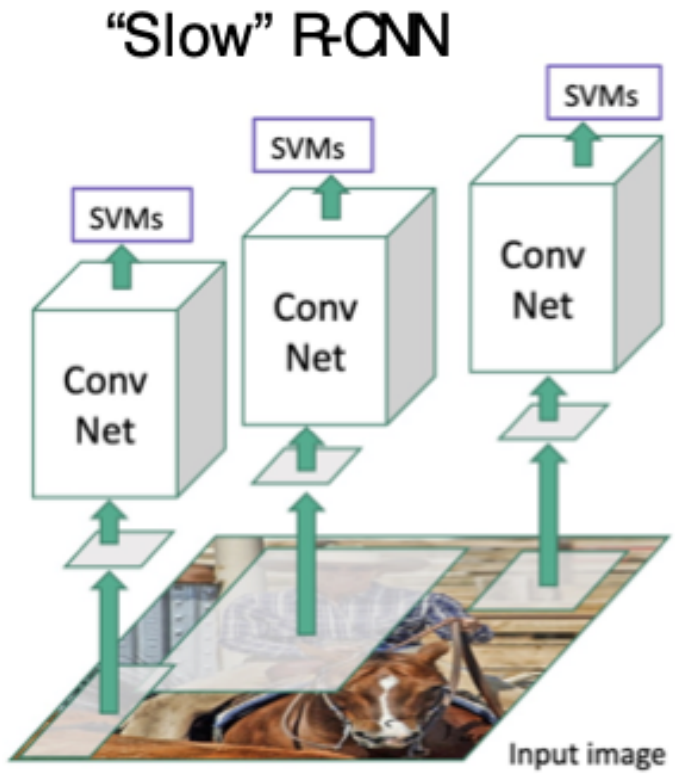
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

Fast R-CNN



Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; source. Reproduced with permission.



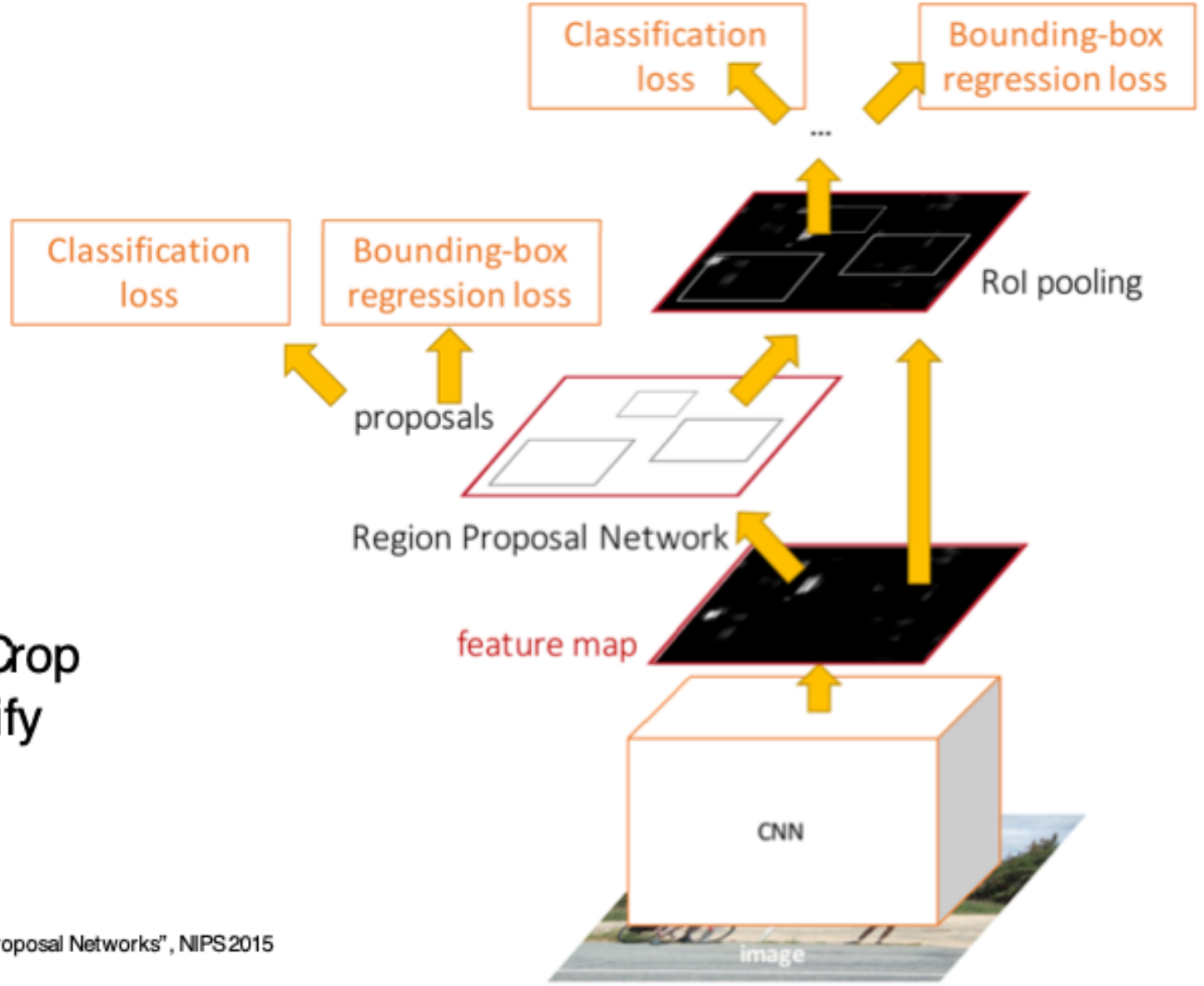
Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

Faster R-CNN:

Make CNN do proposals!

Insert Region Proposal Network (RPN) to predict proposals from features

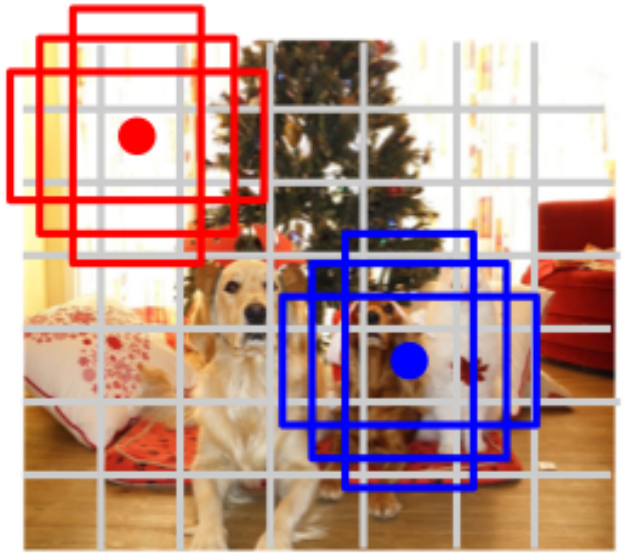
Otherwise same as Fast R-CNN: Crop features for each proposal, classify each one



Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015
Figure copyright 2015, Ross Girshick; reproduced with permission

Single-Stage Object Detectors: YOLO / SSD / RetinaNet

Input image
3 x H x W



Divide image into grid
7 x 7

Image a set of base boxes centered at each grid cell
Here B = 3

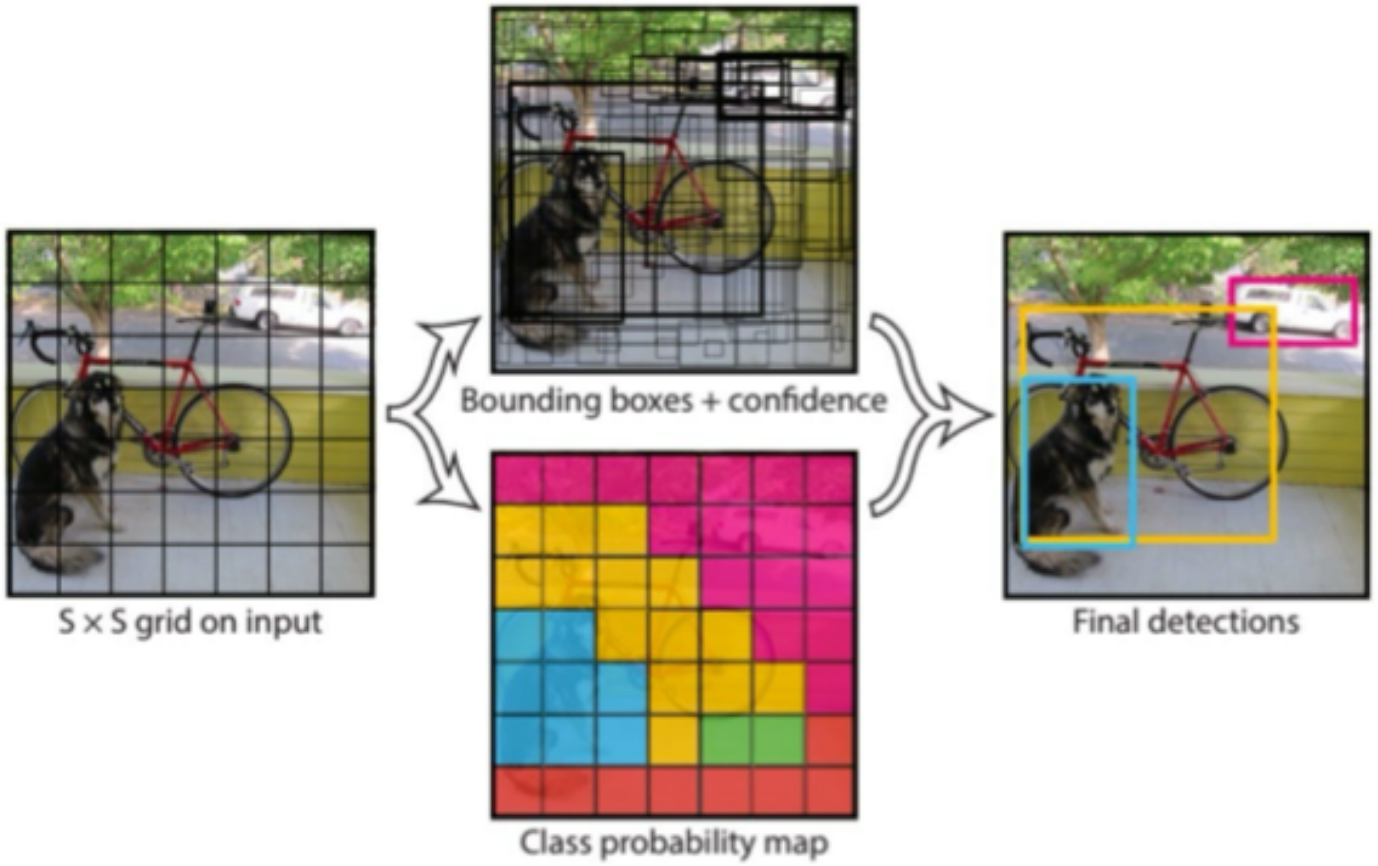
Within each grid cell:

- Regress from each of the B base boxes to a final box with 5 numbers: (dx, dy, dh, dw, confidence)
- Predict scores for each of C classes (including background as a class)
- Looks a lot like RPN, but category-specific!

Output:
7 x 7 x (5 * B + C)

Redmon et al, "You Only Look Once: Unified, Real-Time Object Detection", CVPR 2016
Liu et al, "SSD: Single-Shot MultiBox Detector", ECCV 2016
Lin et al, "Focal Loss for Dense Object Detection", ICCV 2017

YOLO– real-time object detection

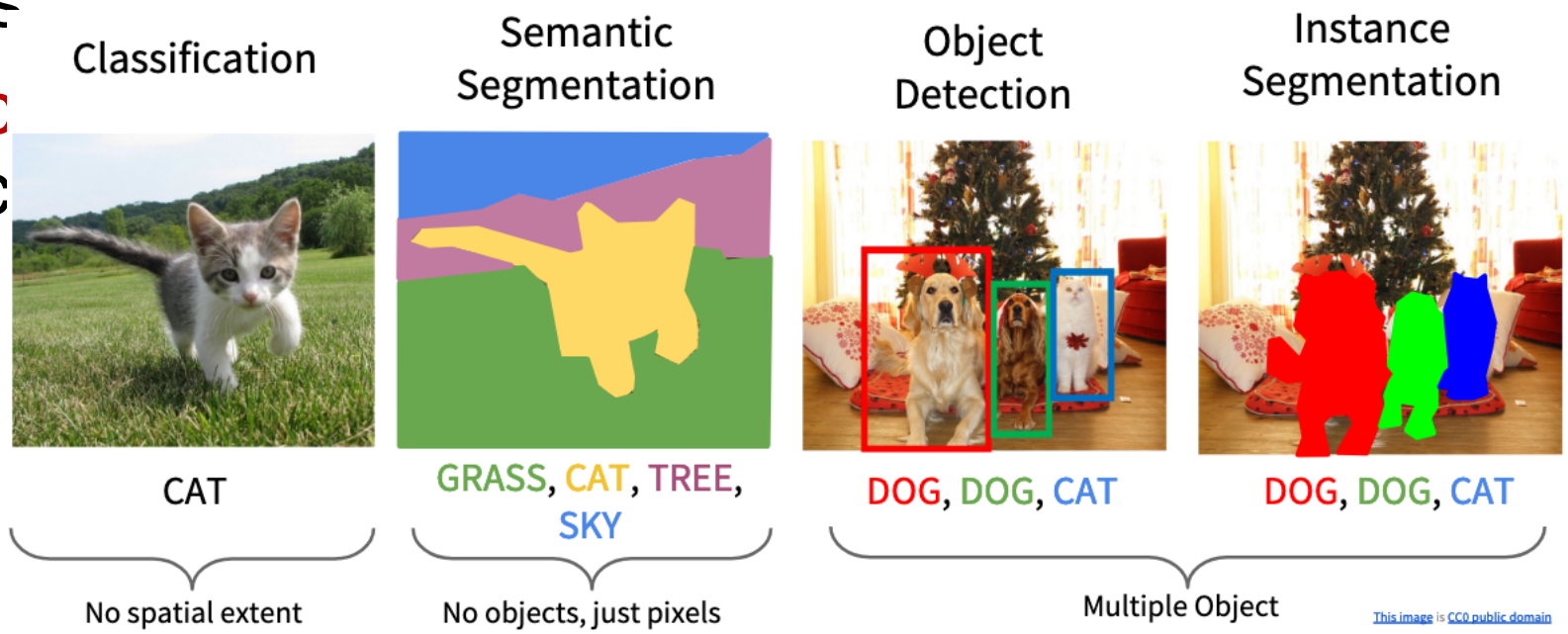


Redmon et al. "You only look once: unified, real-time object detection (2015)."

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

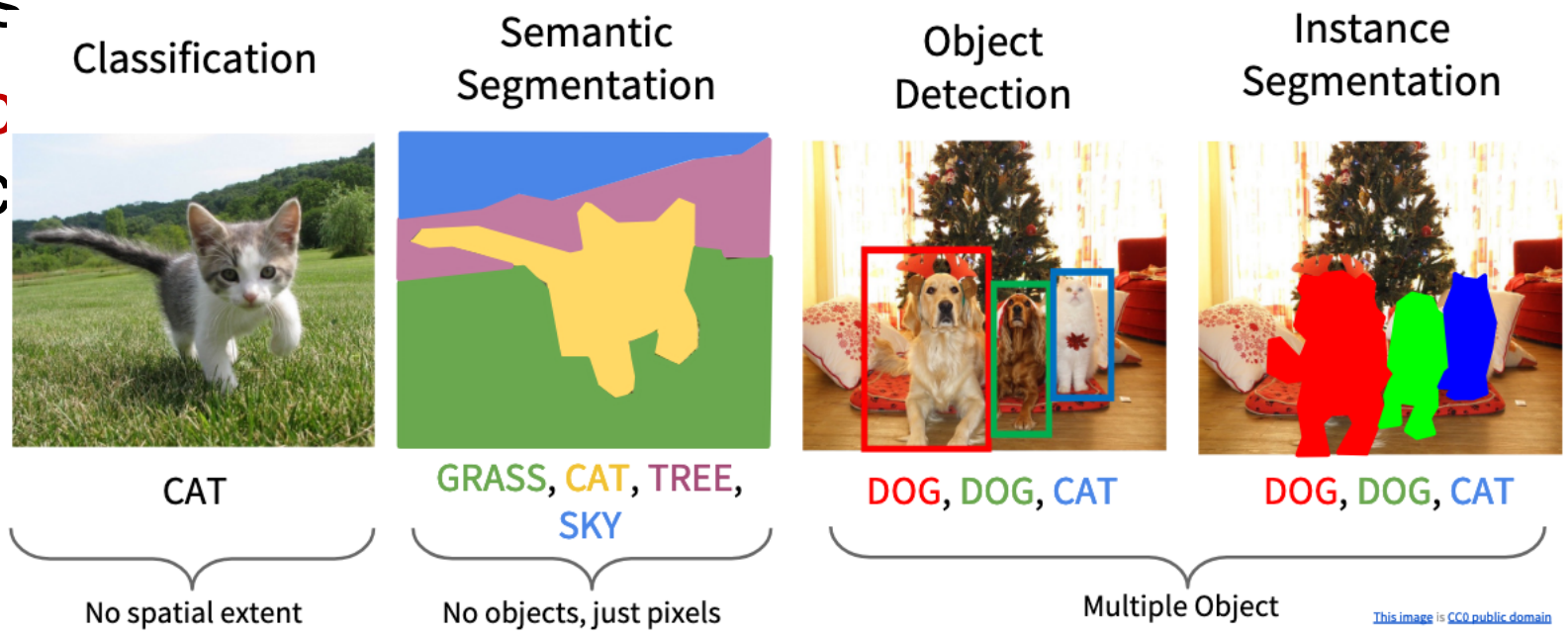
Semantic Perception

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc



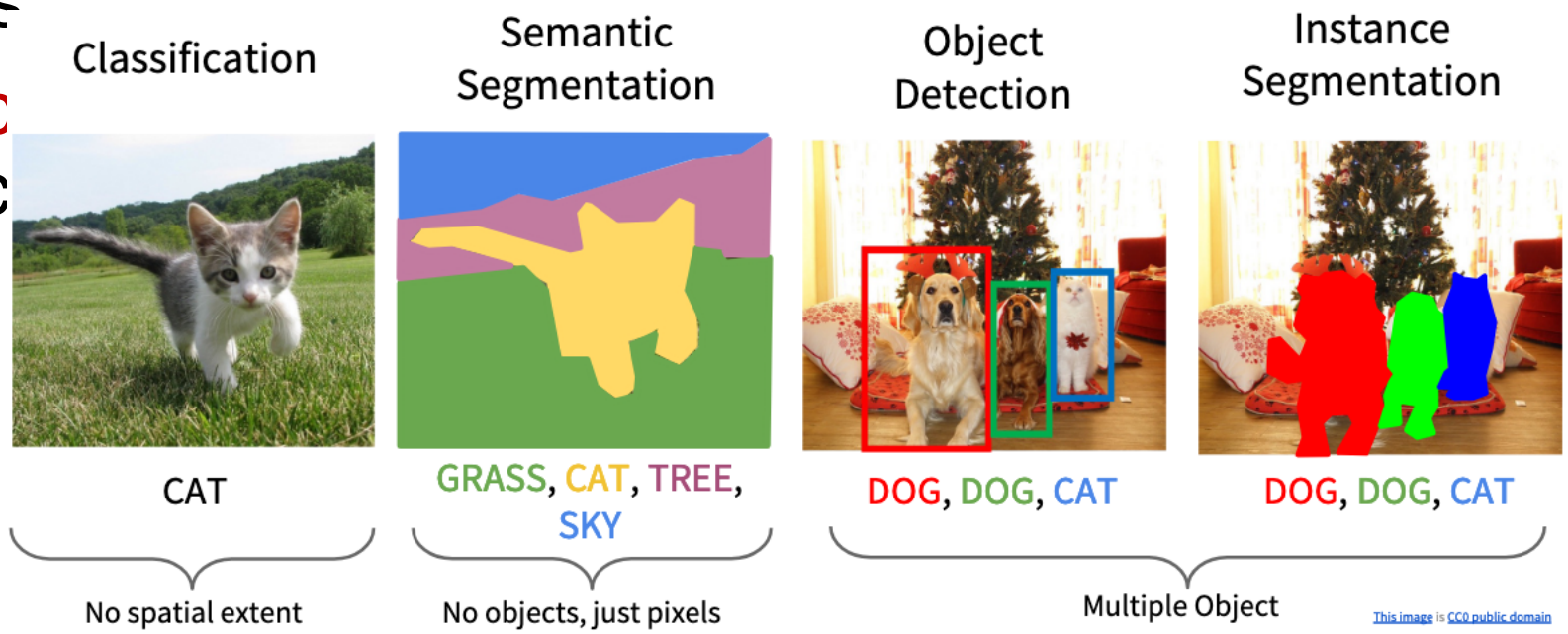
Semantic Perception

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc



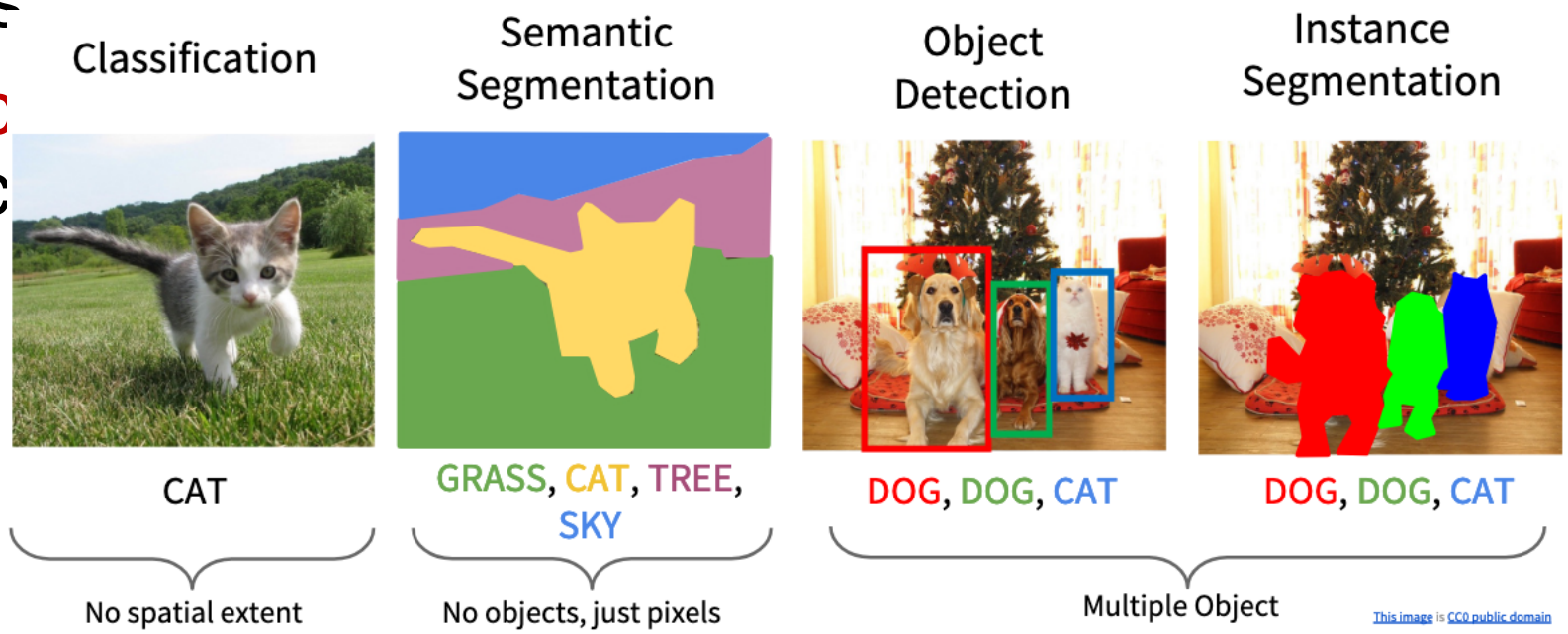
Semantic Perception

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc

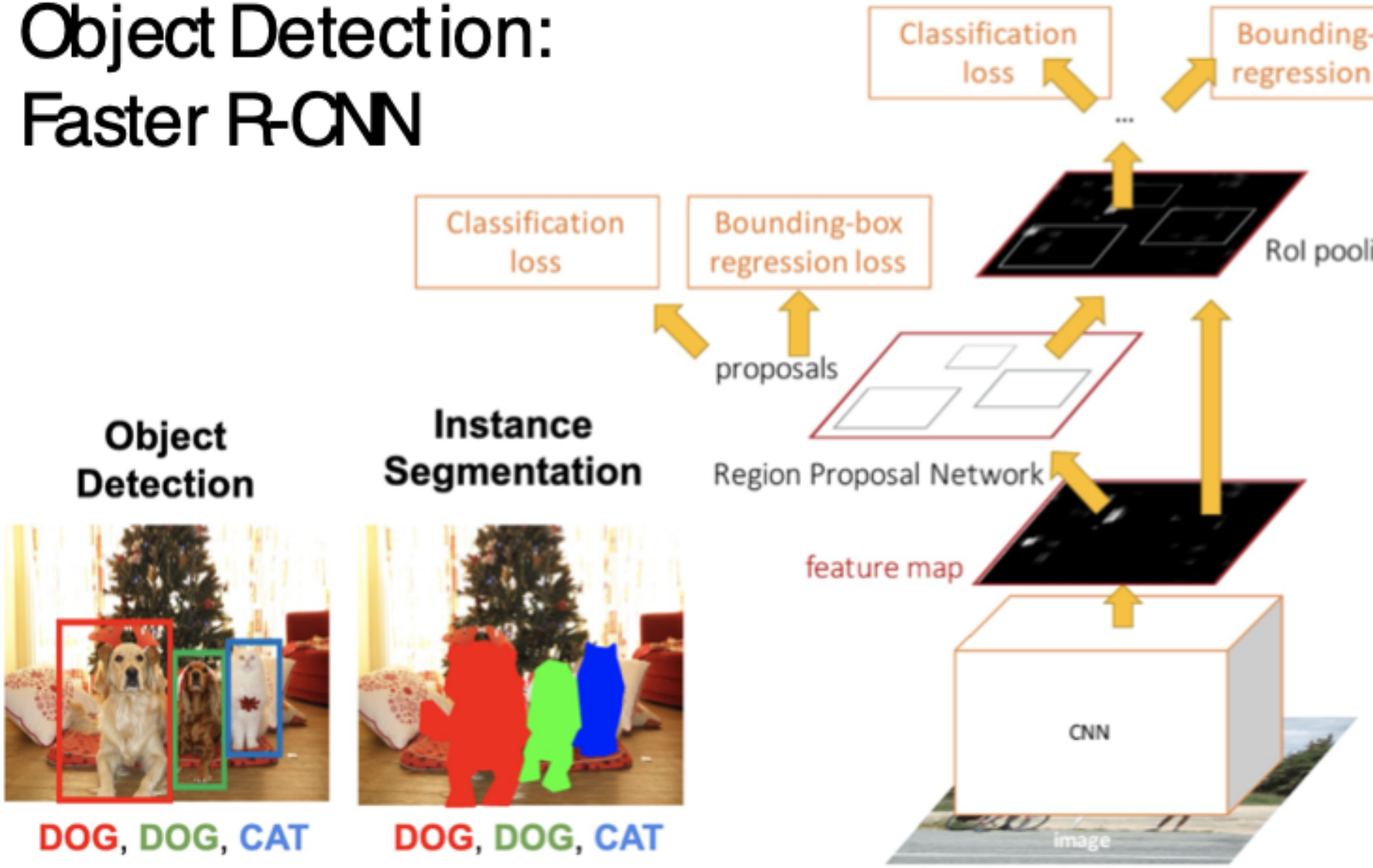


Semantic Perception

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc

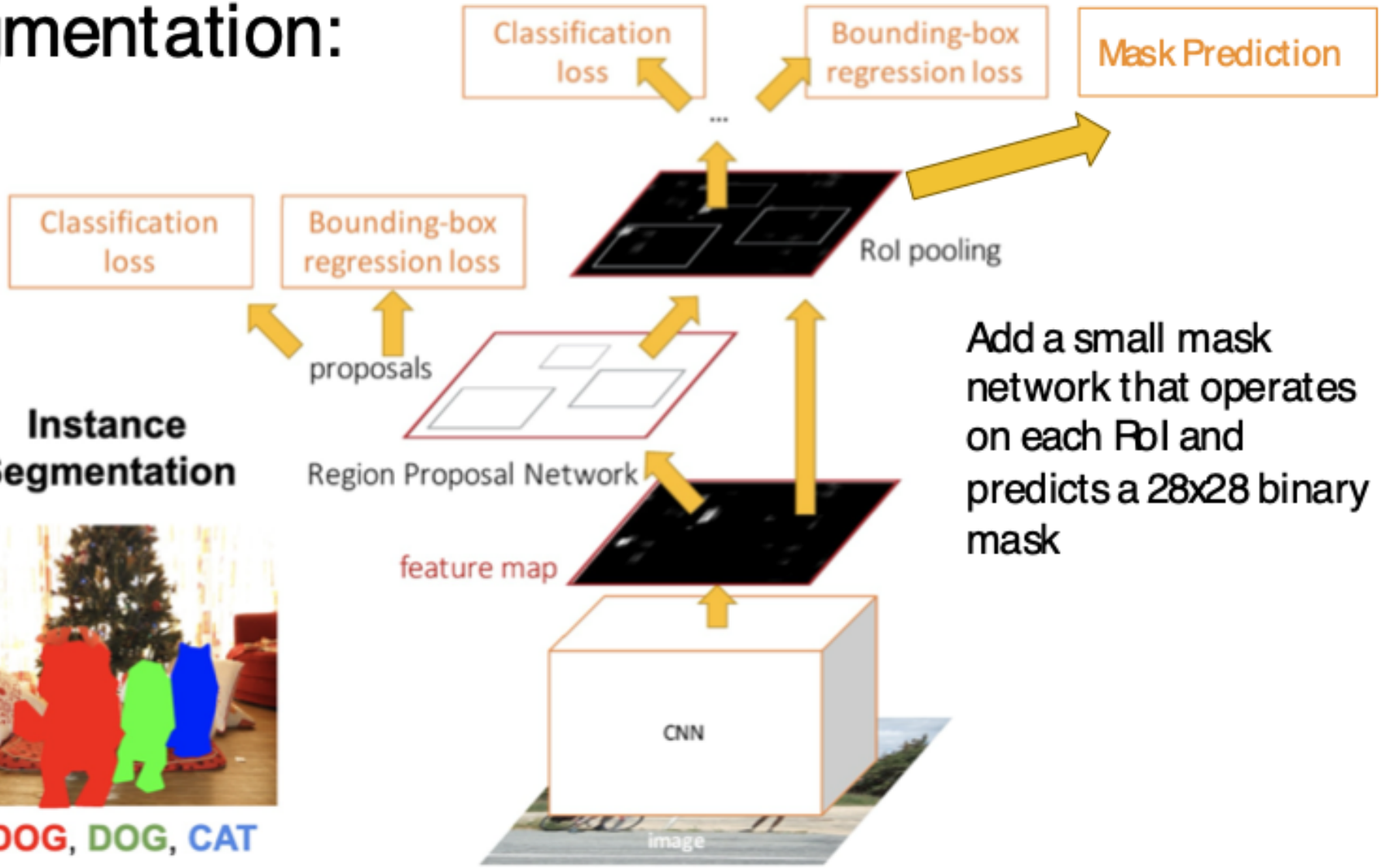


Object Detection: Faster R-CNN



Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

Instance Segmentation: Mask R-CNN



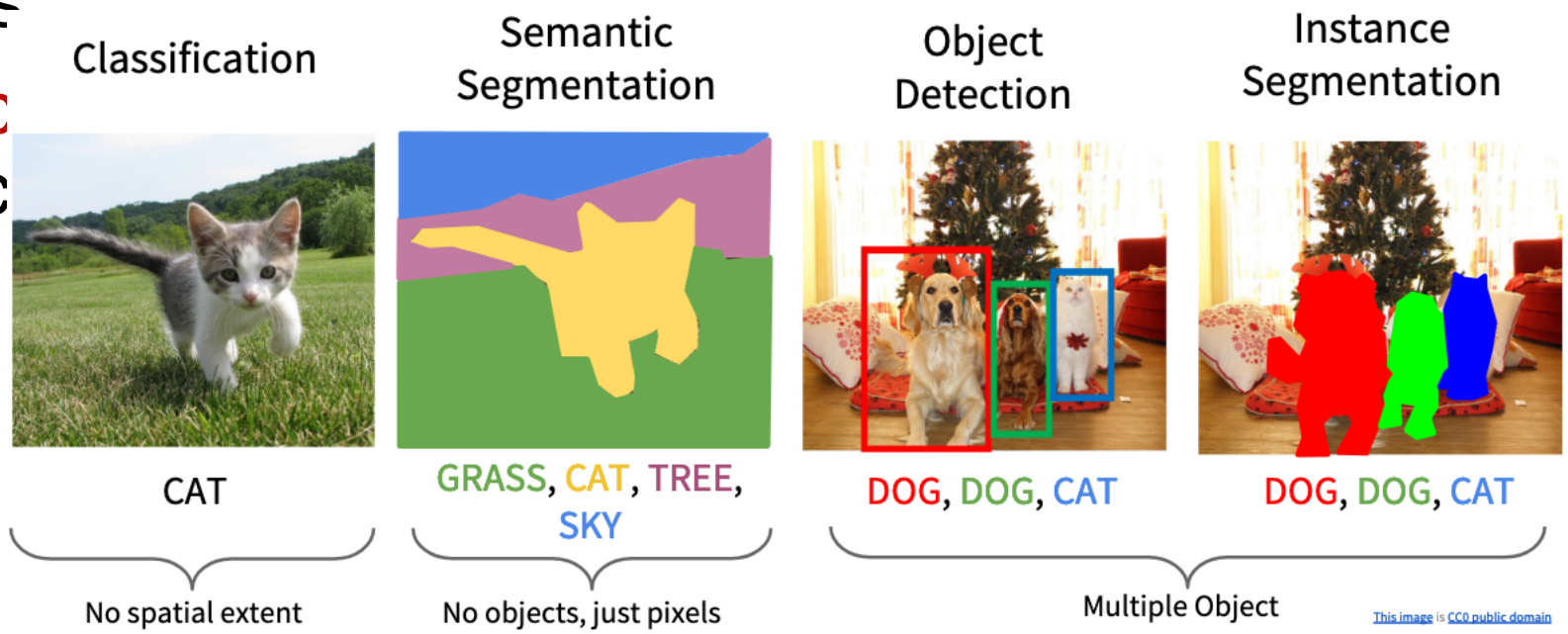
Add a small mask network that operates on each RoI and predicts a 28x28 binary mask

He et al, "Mask R-CNN", ICCV 2017

Adapted from Lecture 9 – CS231n. Fei-Fei Li, Ehsan Adeli

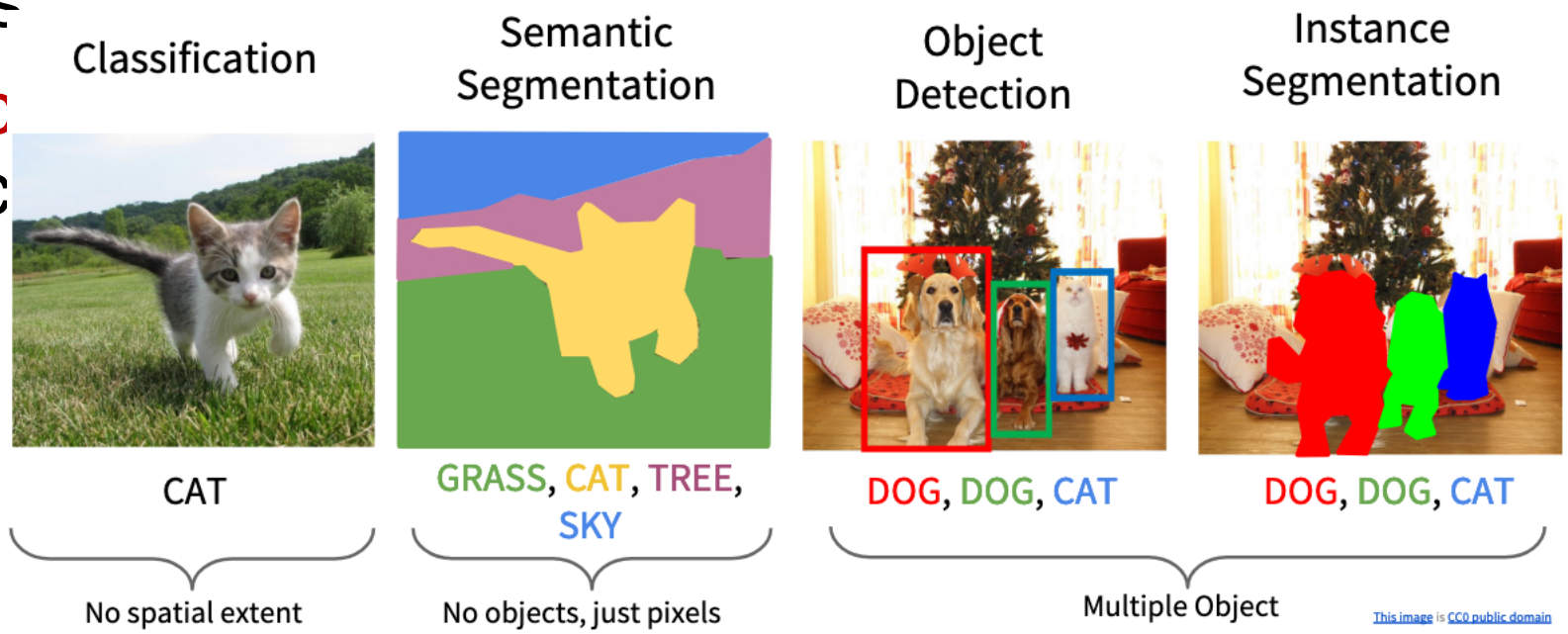
Semantic Perception – Take CS231n for Deep Dive

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc



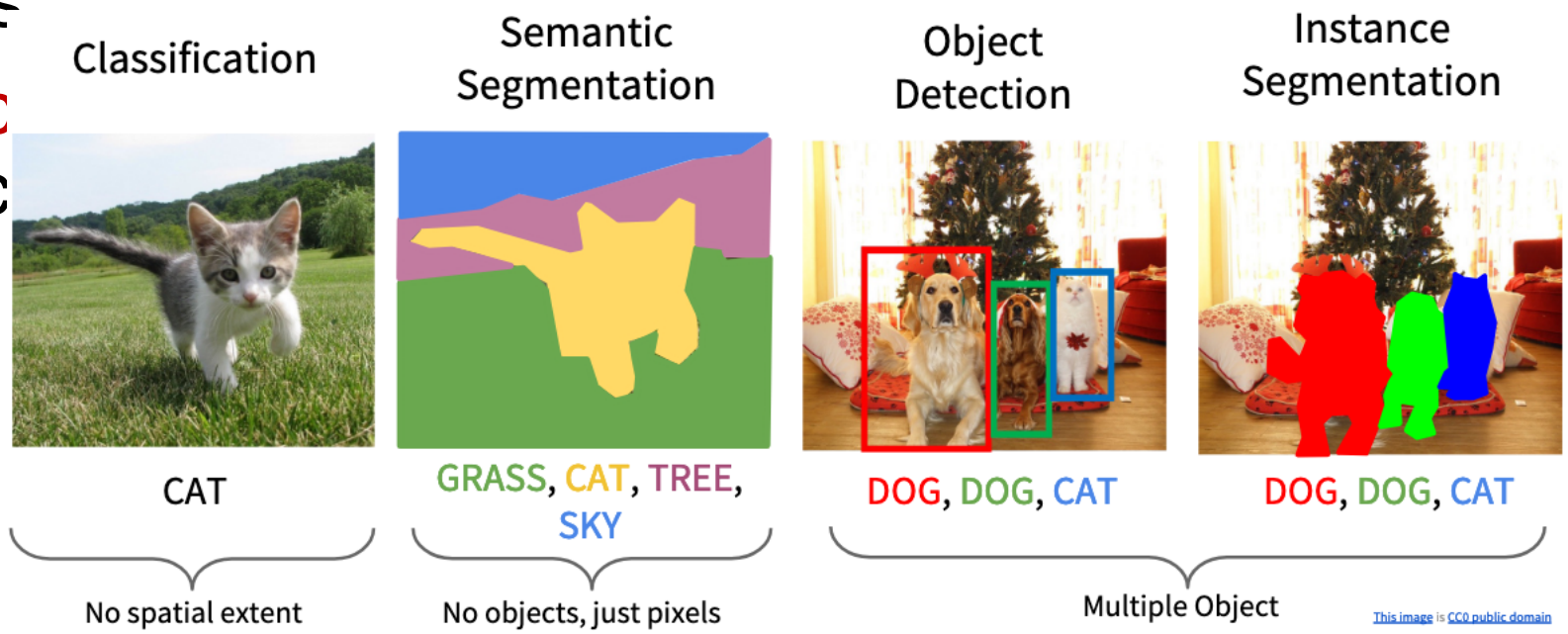
Semantic Perception – Take CS231n for Deep Dive

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc



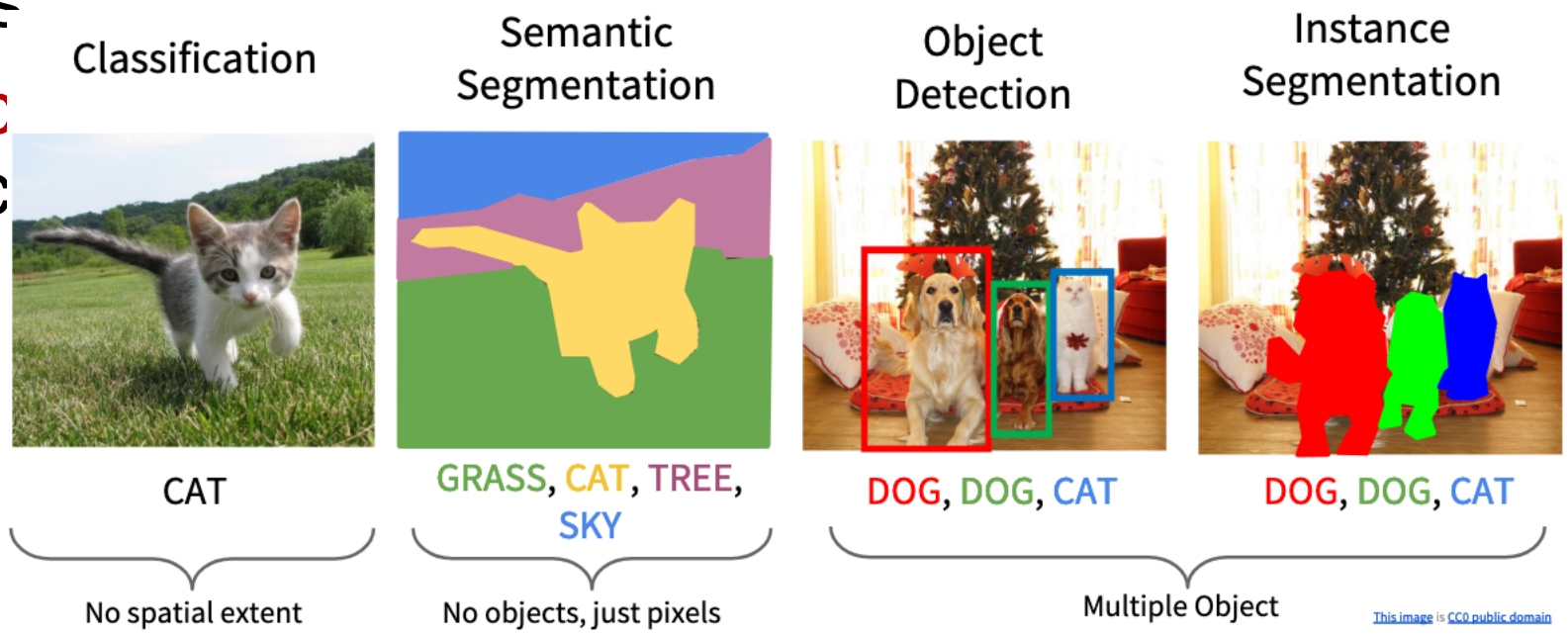
Semantic Perception – Take CS231n for Deep Dive

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc



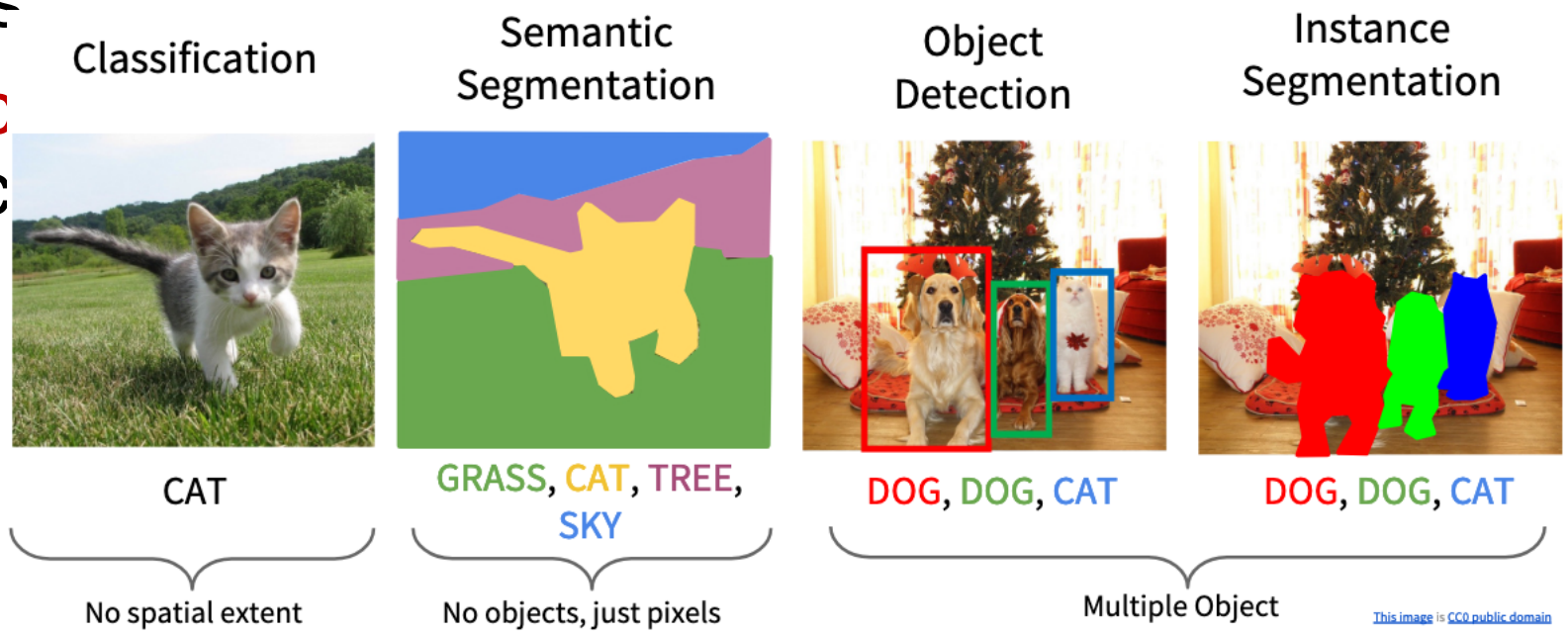
Semantic Perception – Take CS231n for Deep Dive

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc



Semantic Perception – Take CS231n for Deep Dive

- Image Classification: What's in the image?
- Object Detection: Draw bounding boxes around objects with labels
- Semantic Segmentation: Identify pixels belonging to object classes
- Instanc ect instanc



Extra slides from previous years

Parallel image planes

- Assume image planes are parallel
- Epipolar lines are horizontal
- *v* coordinates are equal
  - Easier triangulation
  - Easier correspondence problem
- Is it possible to warp images to simulate a parallel image plane?

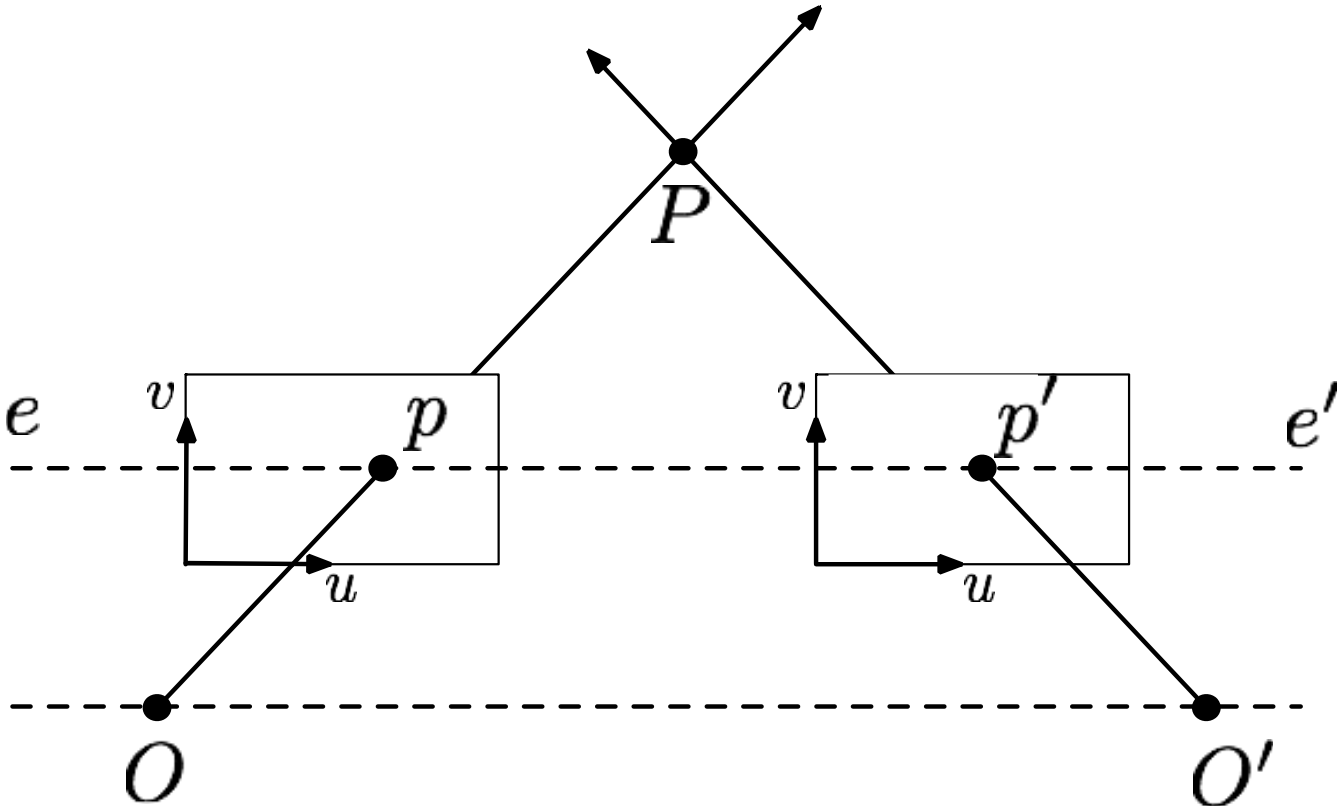
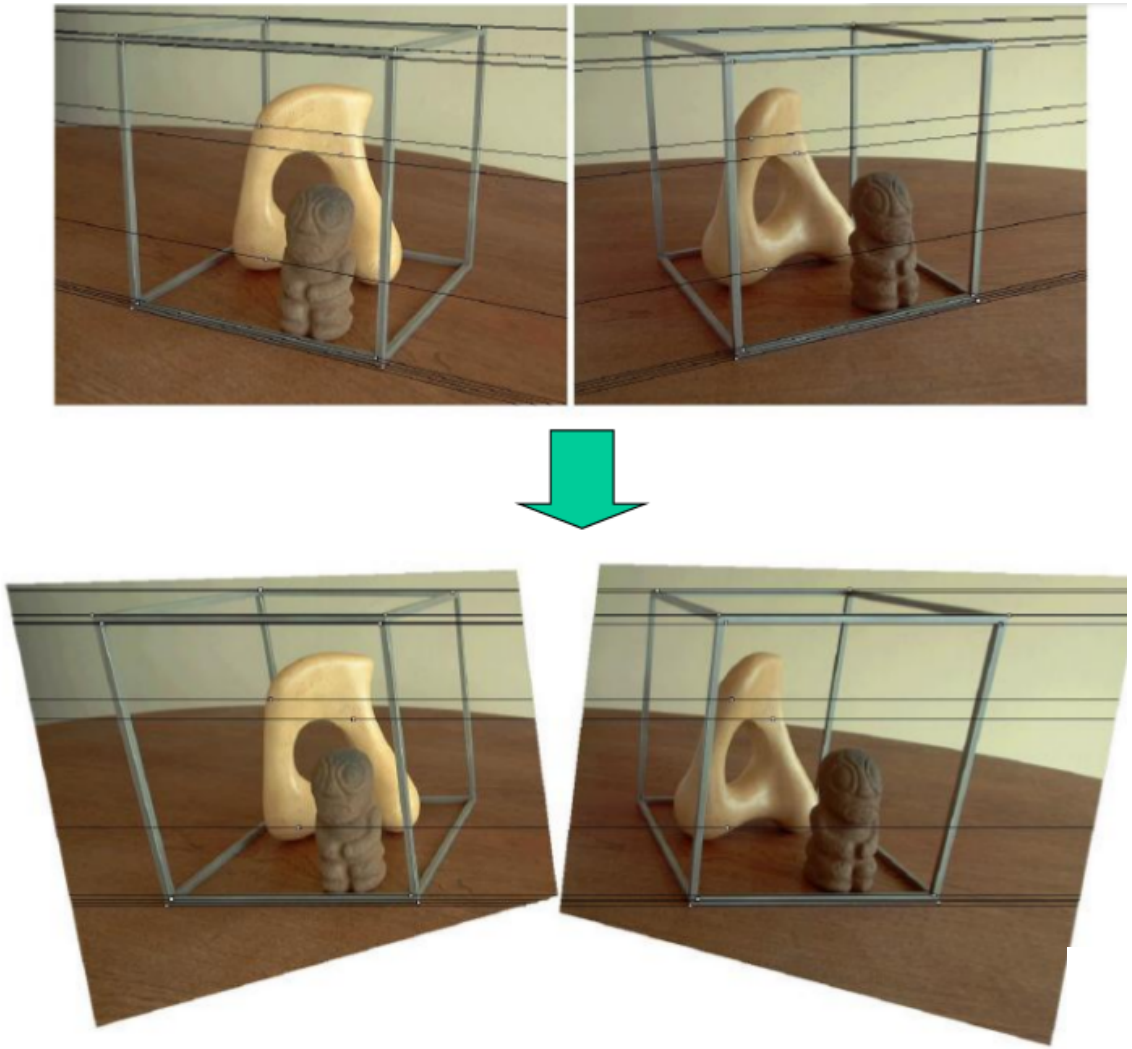


Image rectification



- Achieved by applying an appropriate projective transformation
- Several algorithms exist
- From now on, we assume rectified image pairs

Back to stereo vision process

Back to stereo vision process

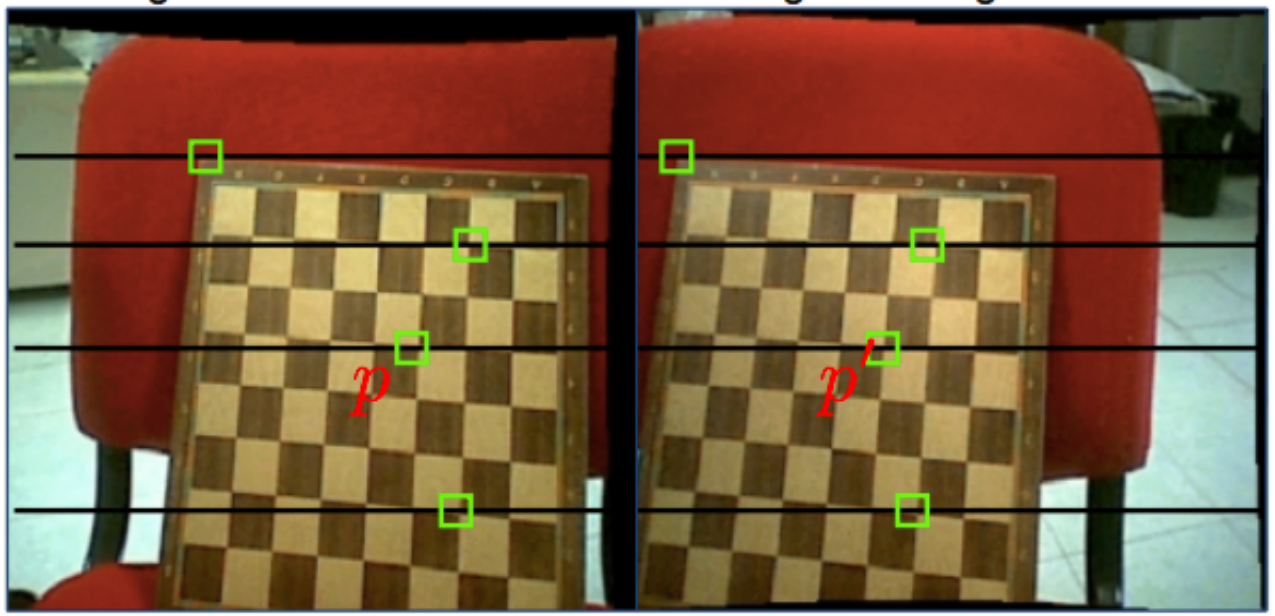
- Recall that stereo vision consists of two steps:
  1. *fusion* of features observed by two (or more) cameras (correspondence)
  2. *reconstruction* of their three-dimensional preimages (triangulation)

Back to stereo vision process

- Recall that stereo vision consists of two steps:
  1. *fusion* of features observed by two (or more) cameras (correspondence)
  2. *reconstruction* of their three-dimensional preimages (triangulation)
- Correspondence problem

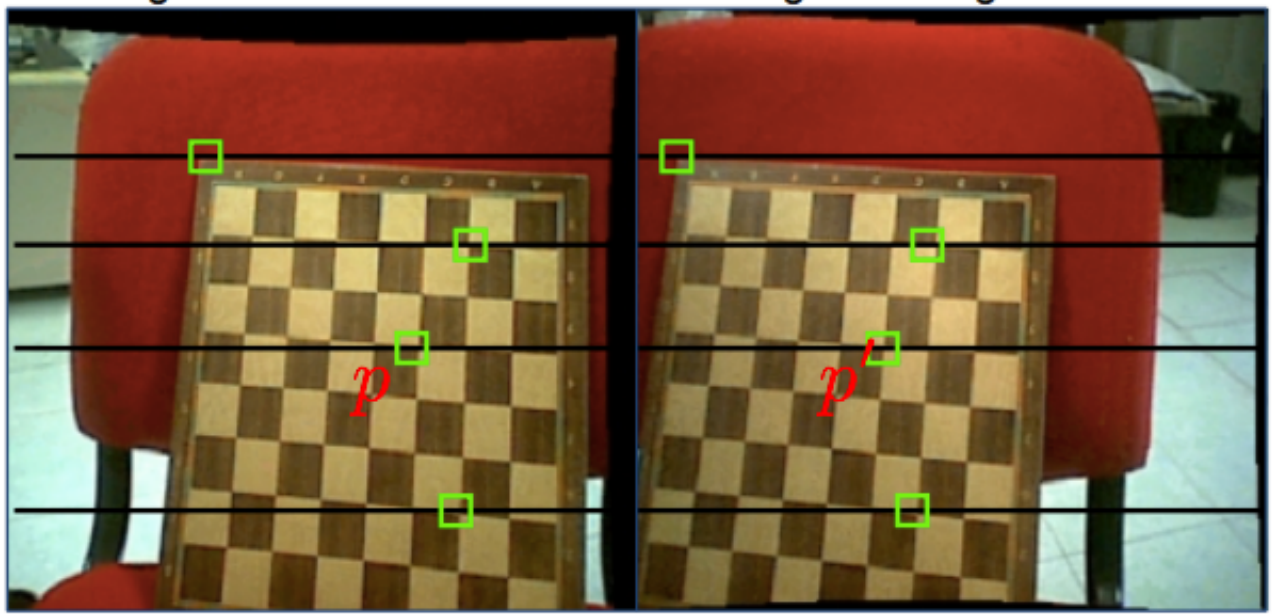
Back to stereo vision process

- Recall that stereo vision consists of two steps:
  1. *fusion* of features observed by two (or more) cameras (correspondence)
  2. *reconstruction* of their three-dimensional preimages (triangulation)
- Correspondence problem



Back to stereo vision process

- Recall that stereo vision consists of two steps:
  1. *fusion* of features observed by two (or more) cameras (correspondence)
  2. *reconstruction* of their three-dimensional preimages (triangulation)
- Correspondence problem



Goal: find corresponding observations p and p'

Exploits epipolar constraints

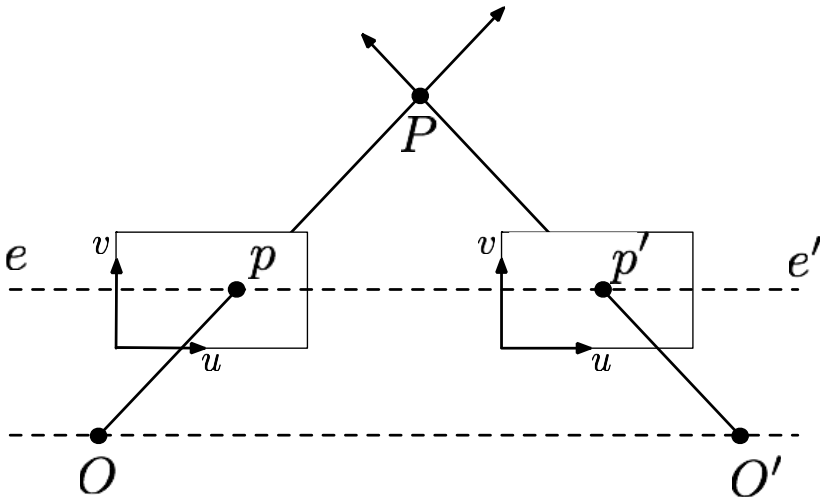
Two classes of algos: *area-based*

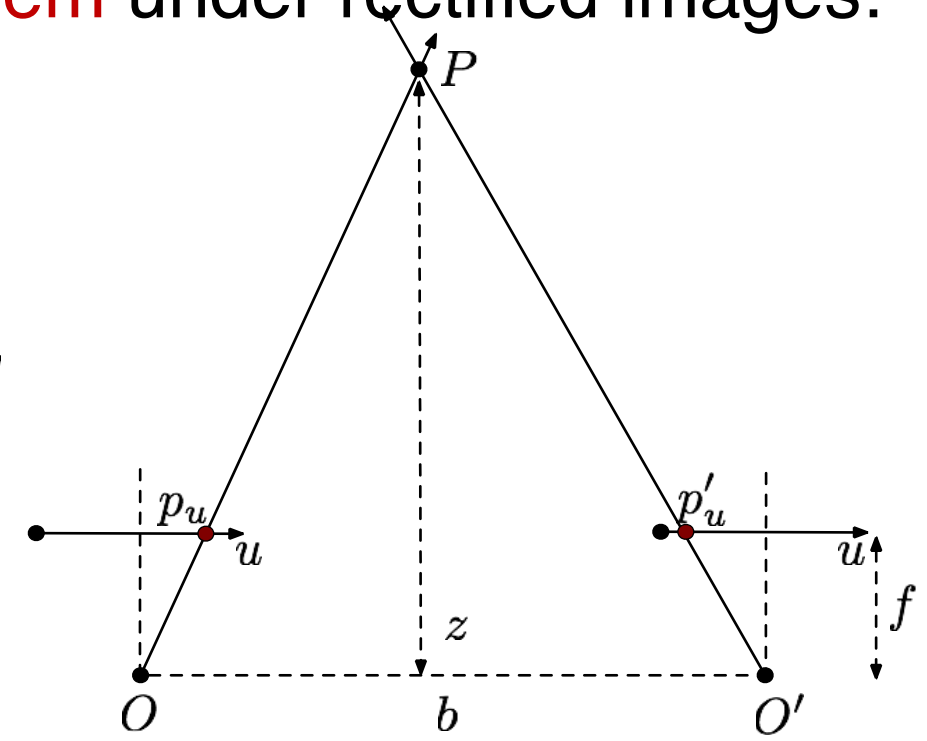
and *feature-based*

Hard problem: occlusions, repetitive patterns, etc.; more on this later

Triangulation under rectified images

- We already saw how to triangulate correspondences in the general case
- Triangulation problem under rectified images:





From similar triangles:

$$z = \frac{b\,f}{p_u - p'_u}$$

disparity

Large baseline: Object might be visible from one camera, but not the other

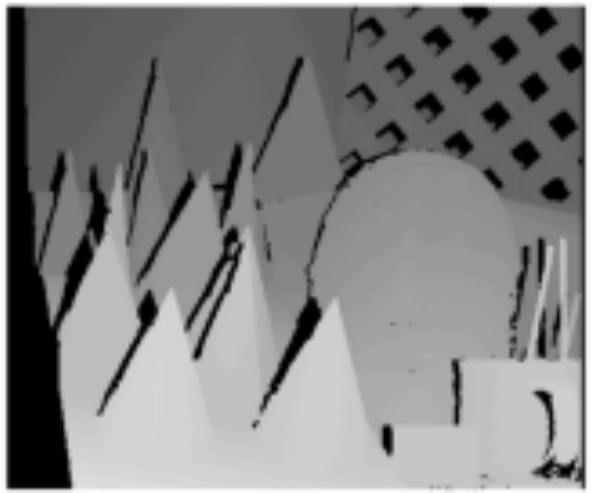
Small baseline: large depth error

Disparity map

- Disparity: pixel displacement between corresponding points
- Disparity map: holds the disparity values for every pixel
- Nearby objects experience largest disparity

Stereo pair

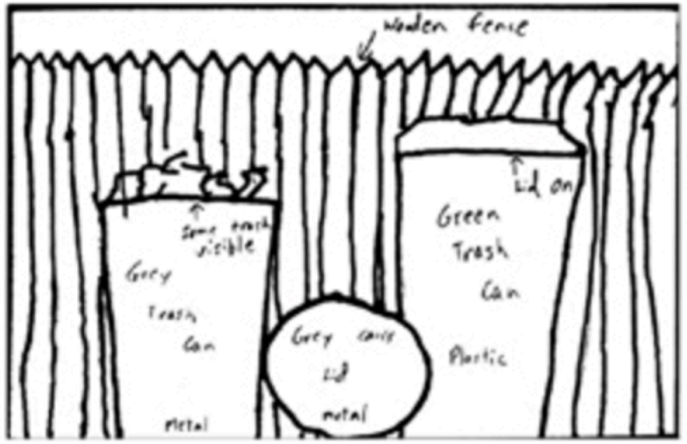
Disparity map



Representations in Computer Vision

Observed image

Drawn from memory

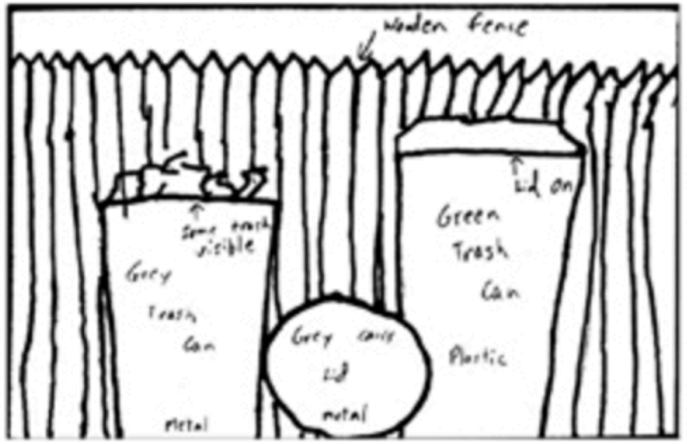


[Bartlett, 1932]
[Intraub & Richardson, 1989]

Representations in Computer Vision

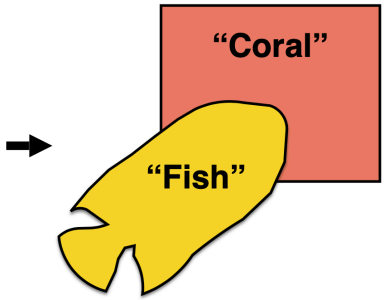
Observed image

Drawn from memory



[Bartlett, 1932]
[Intraub & Richardson, 1989]

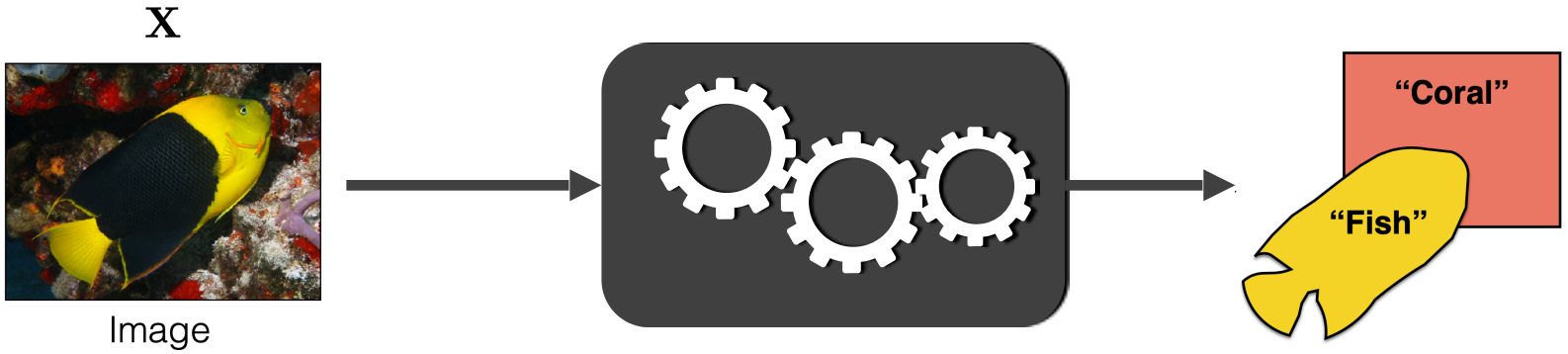
Image

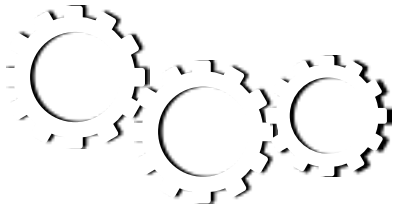


Compact mental representation

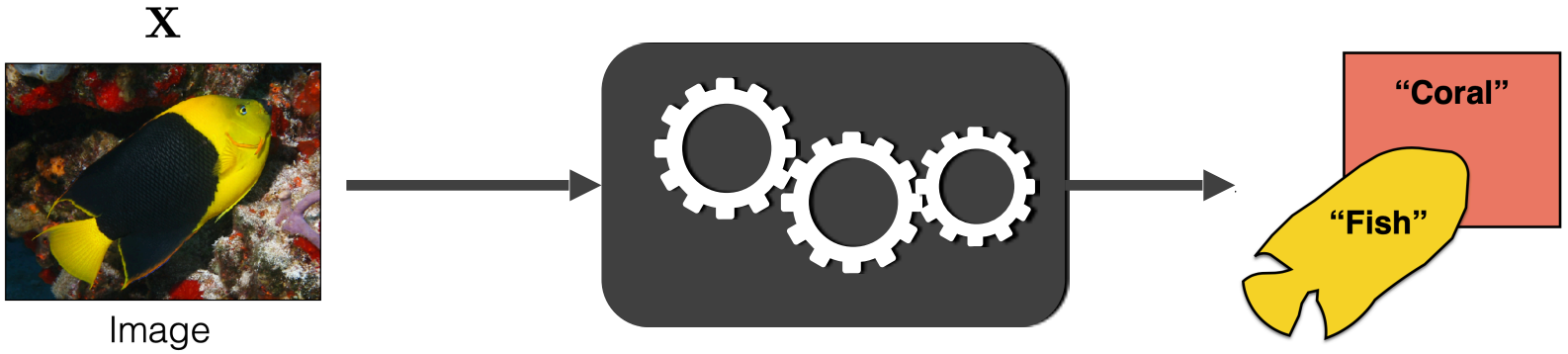
Example from Advances in Computer Vision – MIT – 6.869/6.819

Typical CV Pipeline



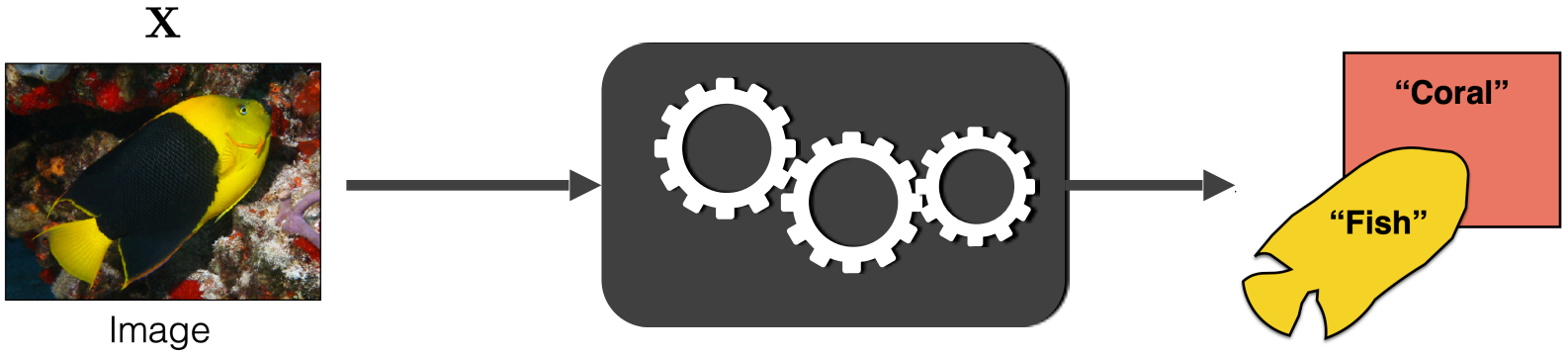


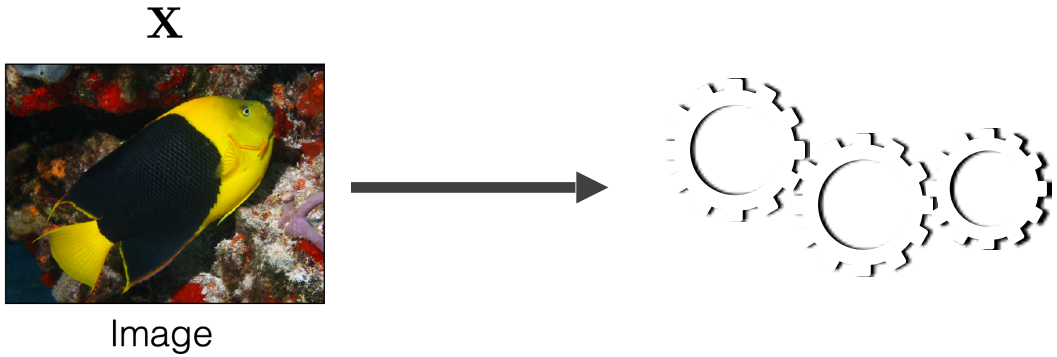
Typical CV Pipeline



Input Image

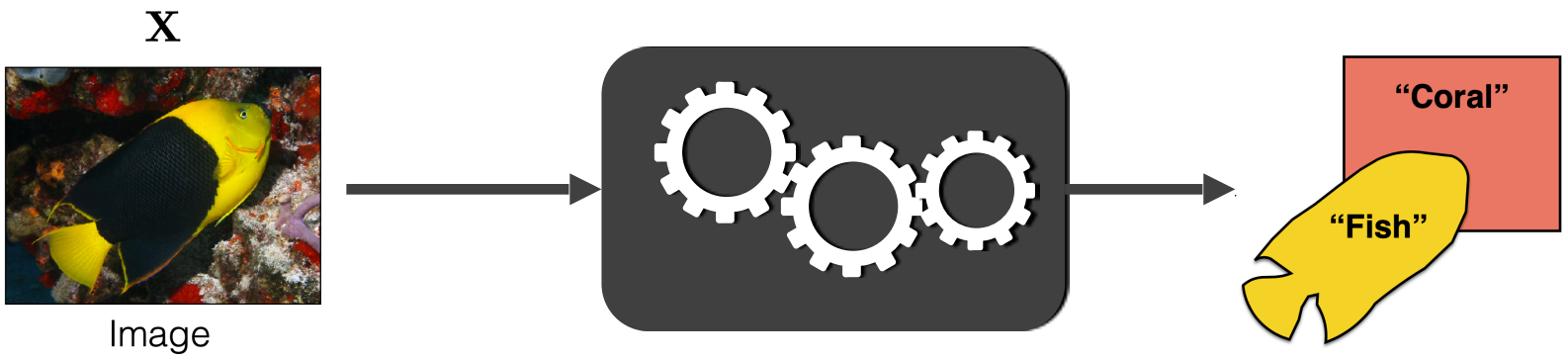
Typical CV Pipeline

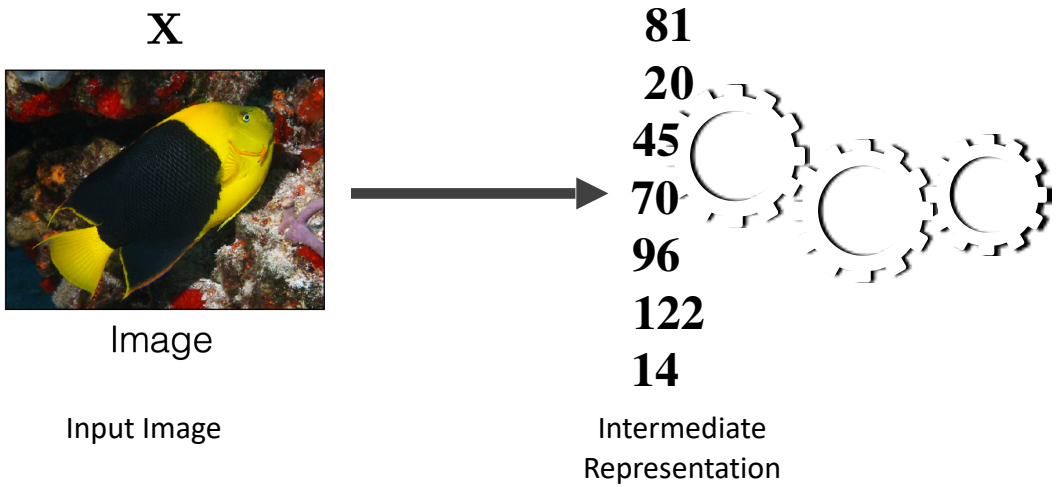




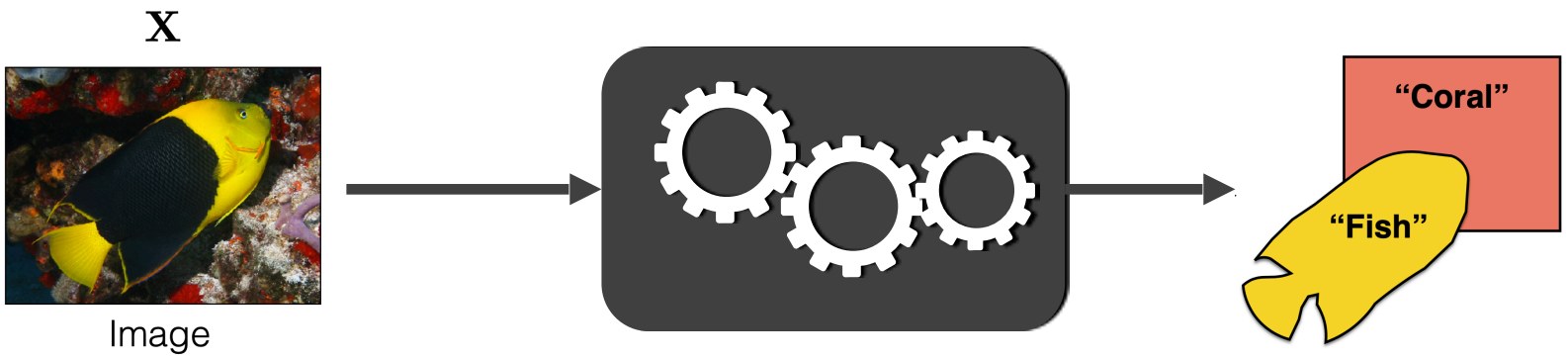
Input Image

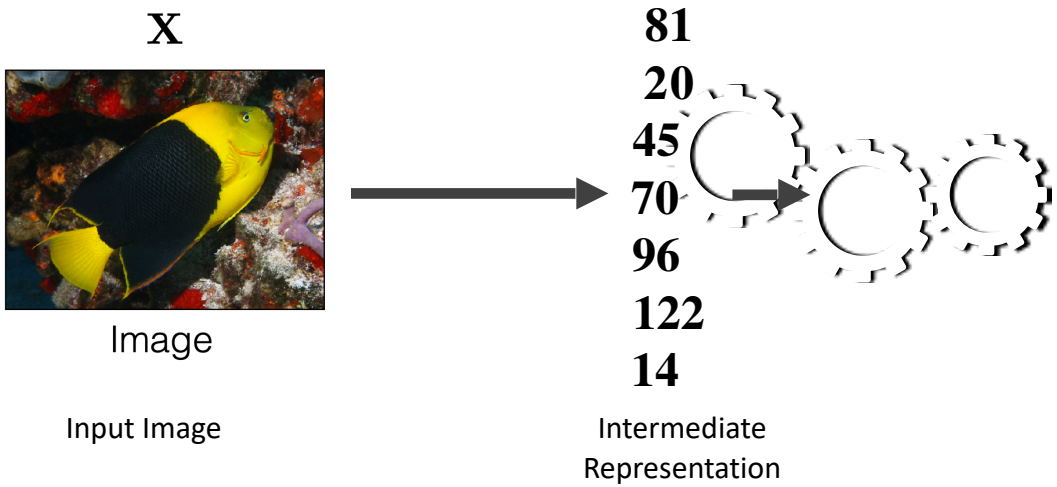
Typical CV Pipeline



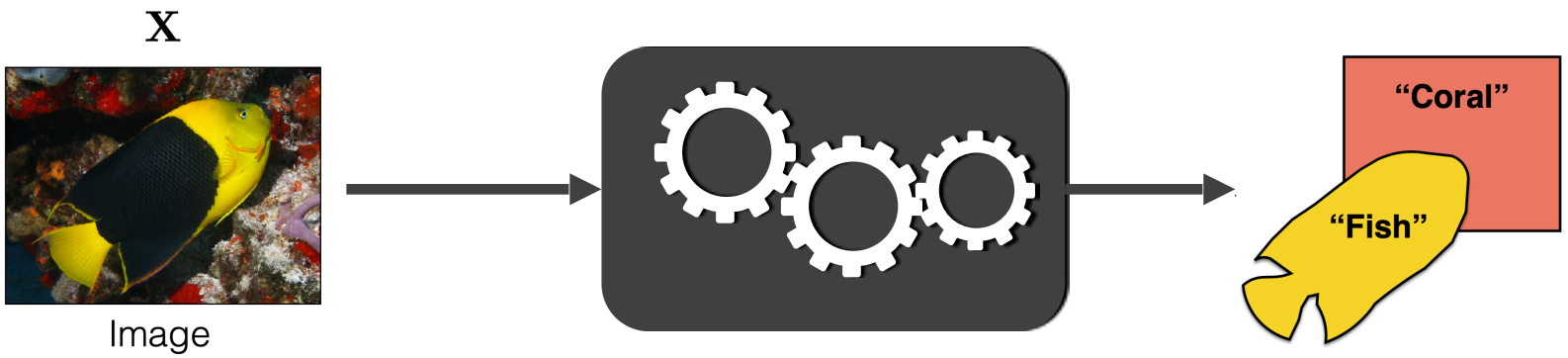


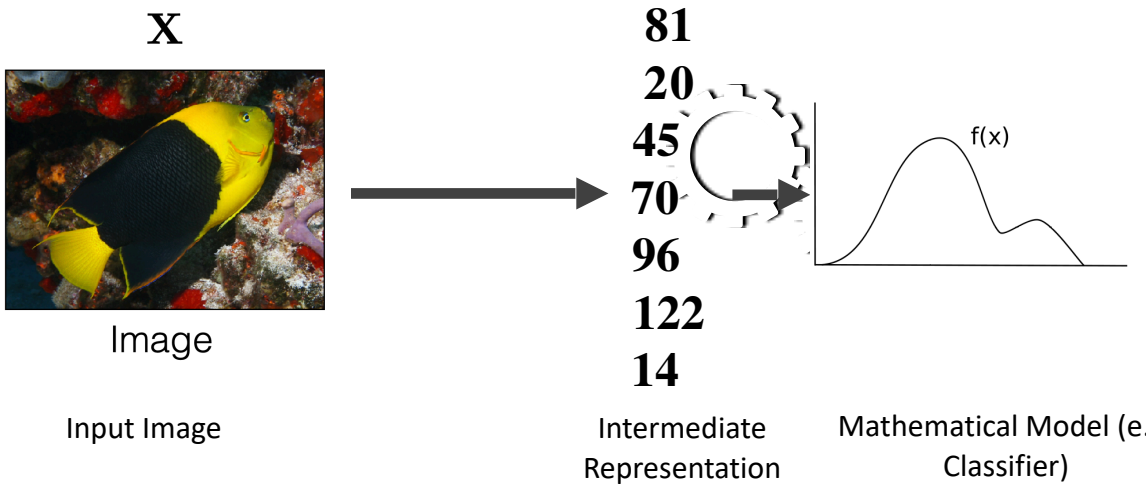
Typical CV Pipeline



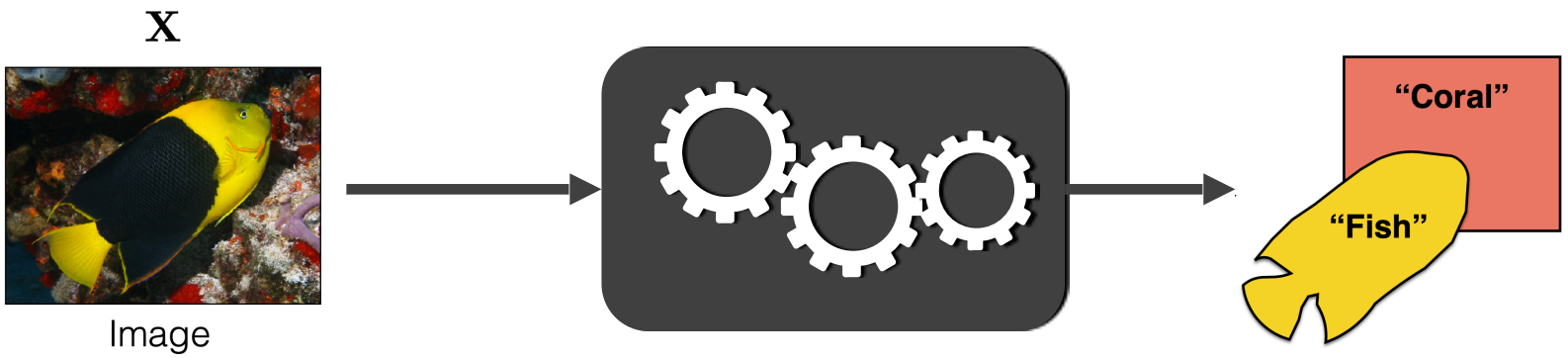


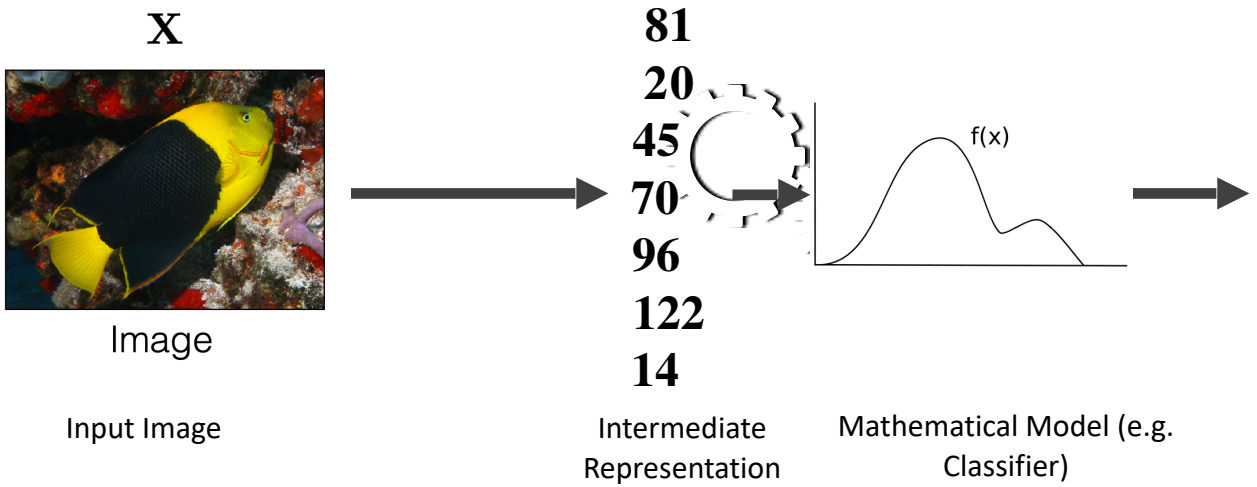
Typical CV Pipeline



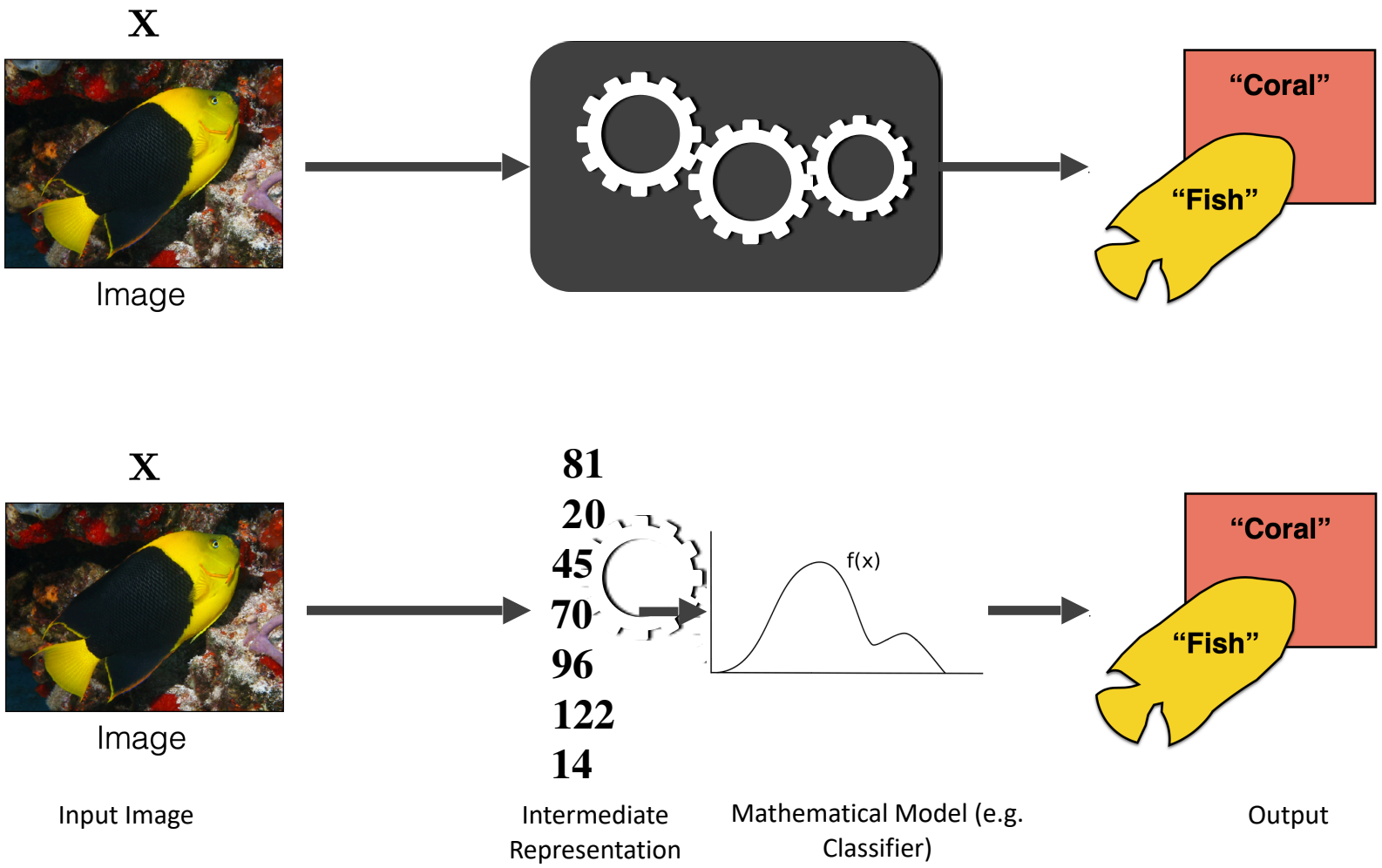


Typical CV Pipeline

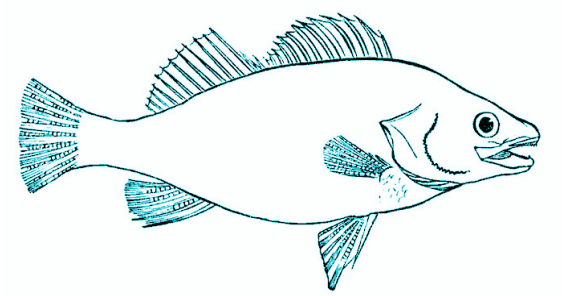


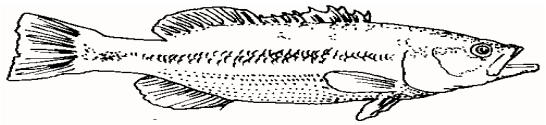


Typical CV Pipeline



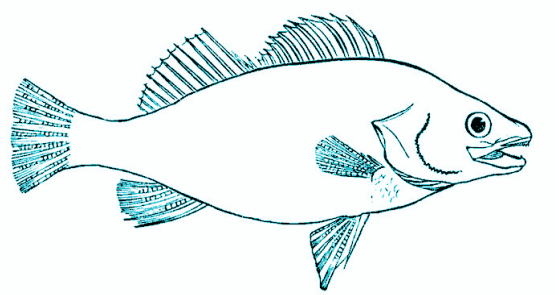
Example



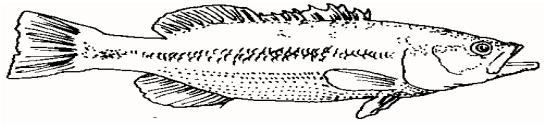


Example from CS331B: Representation Learning in Computer Vision

Example



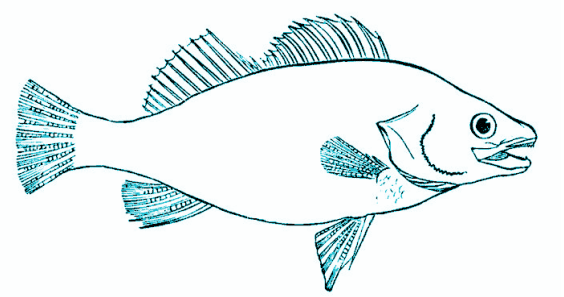
~12 lbs



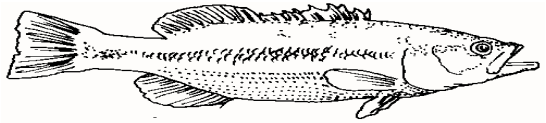
~8 lbs

Example from CS331B: Representation Learning in Computer Vision

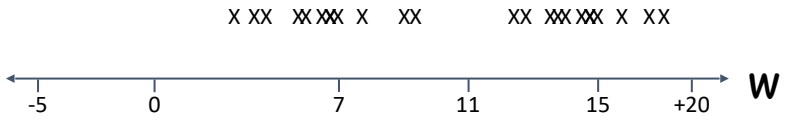
Example



~12 lbs

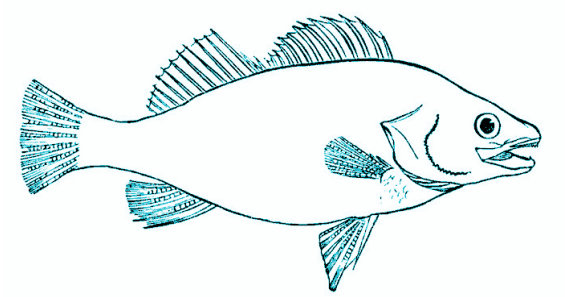


~8 lbs

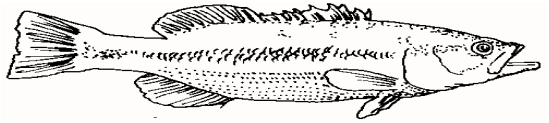


Example from CS331B: Representation Learning in Computer Vision

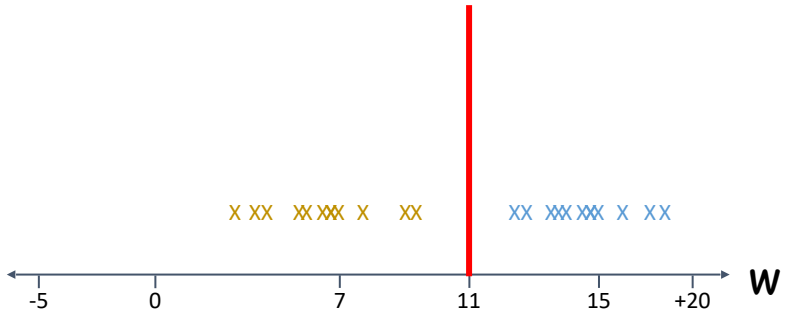
Example



~12 lbs



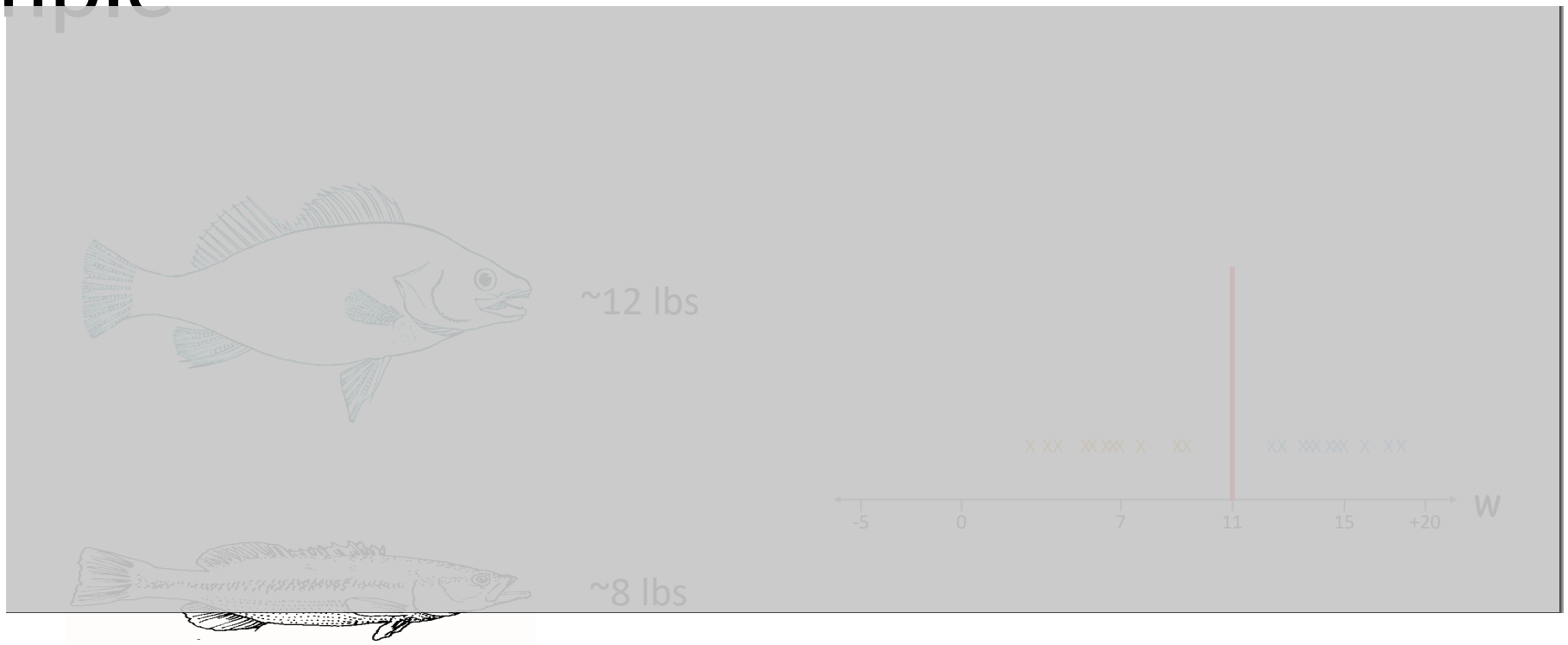
~8 lbs

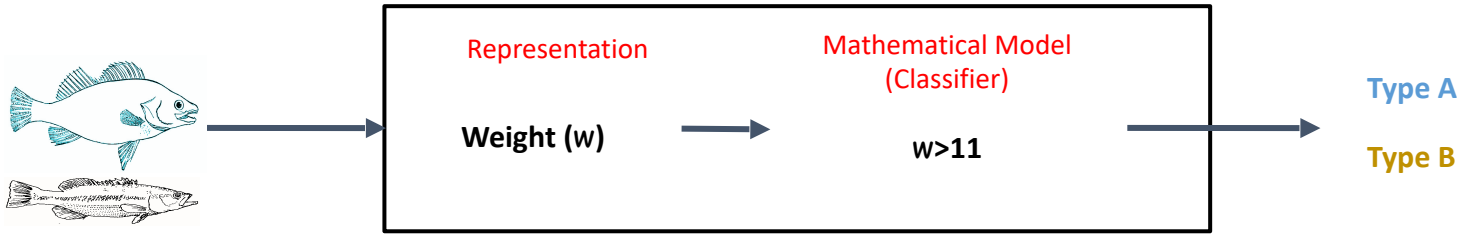


Example from CS331B: Representation Learning in Computer Vision

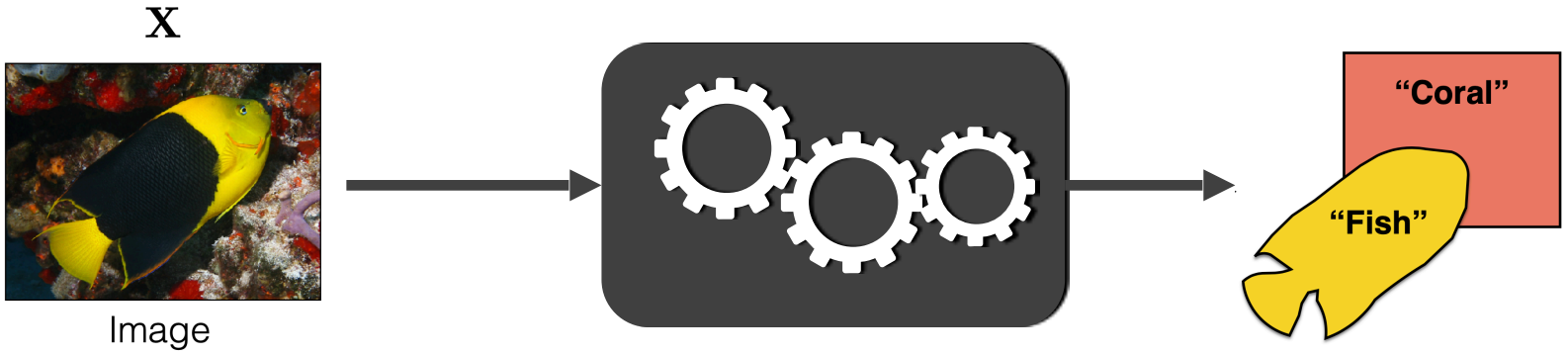
Example from CS331B: Representation Learning in Computer Vision

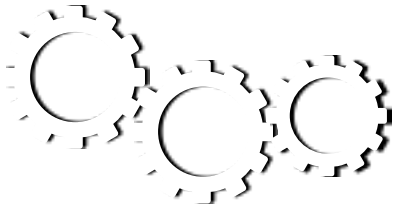
Example



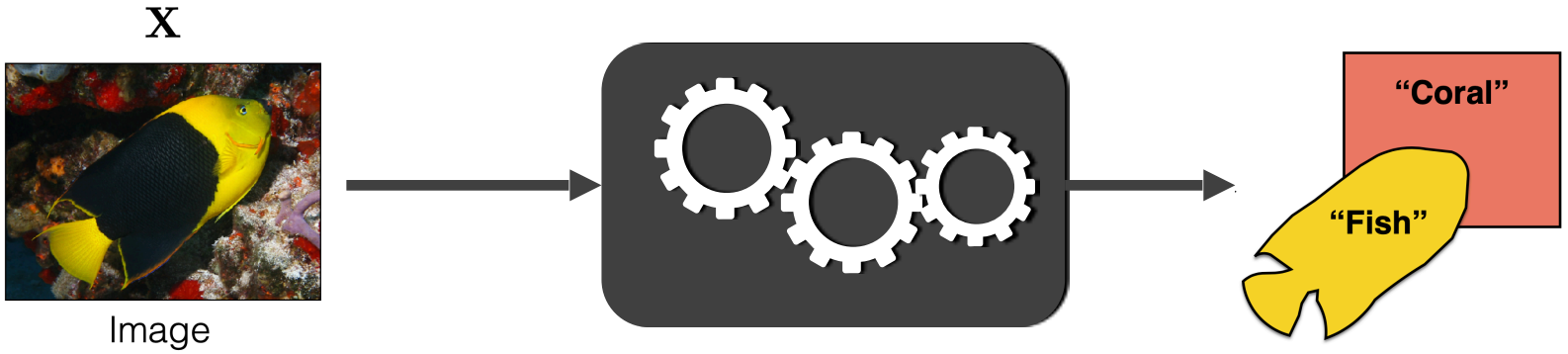


Typical CV Pipeline



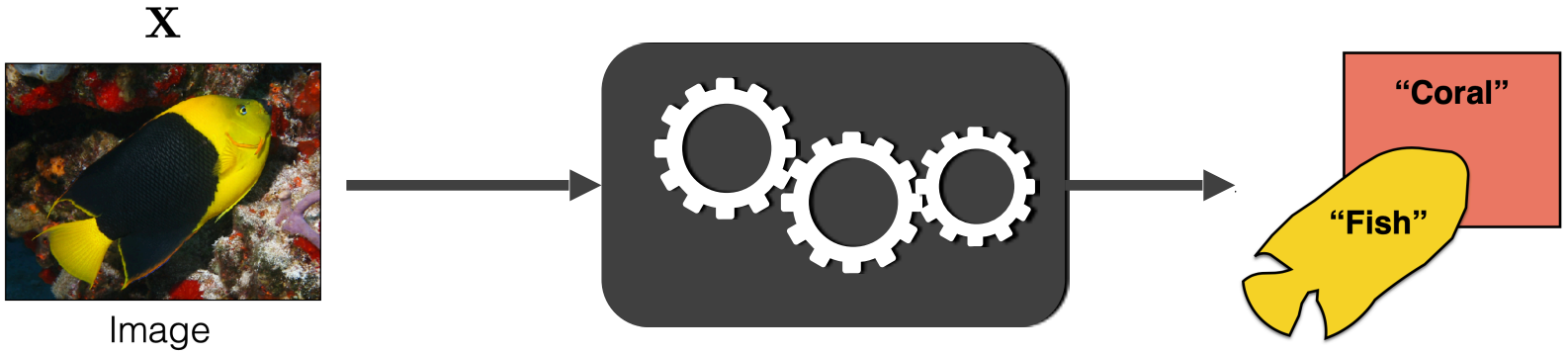


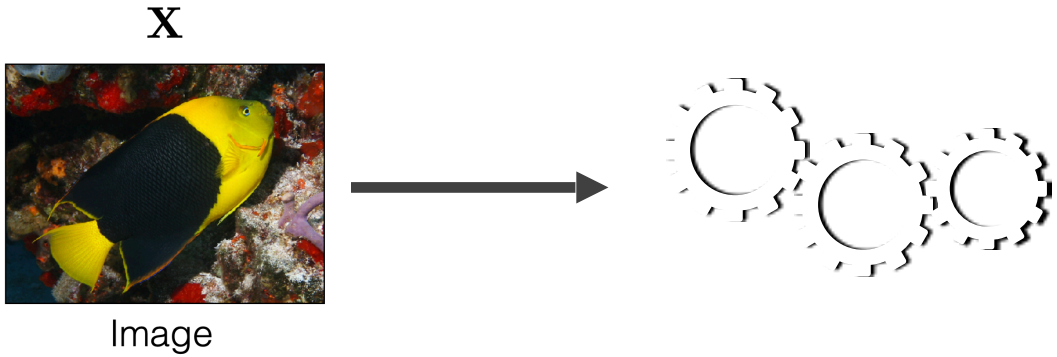
Typical CV Pipeline



Input Image

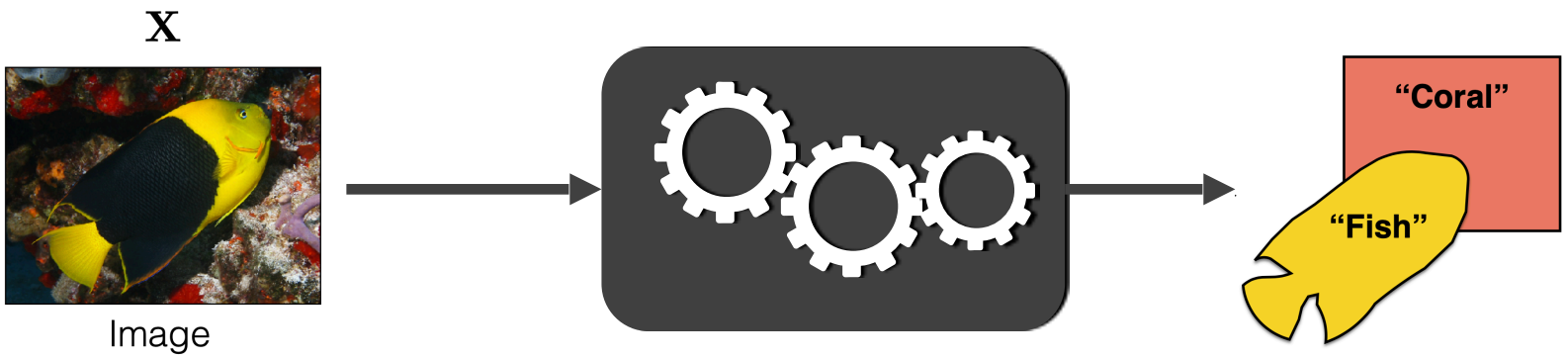
Typical CV Pipeline

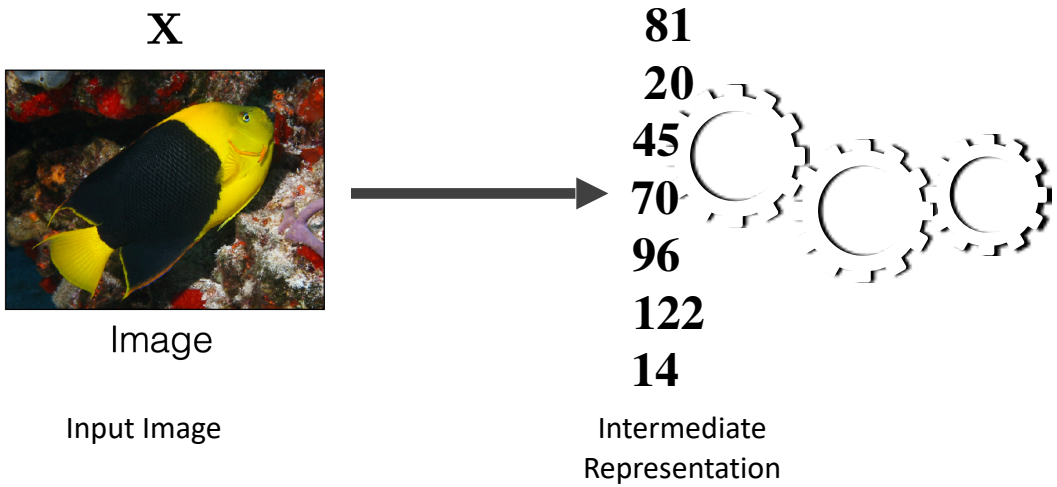




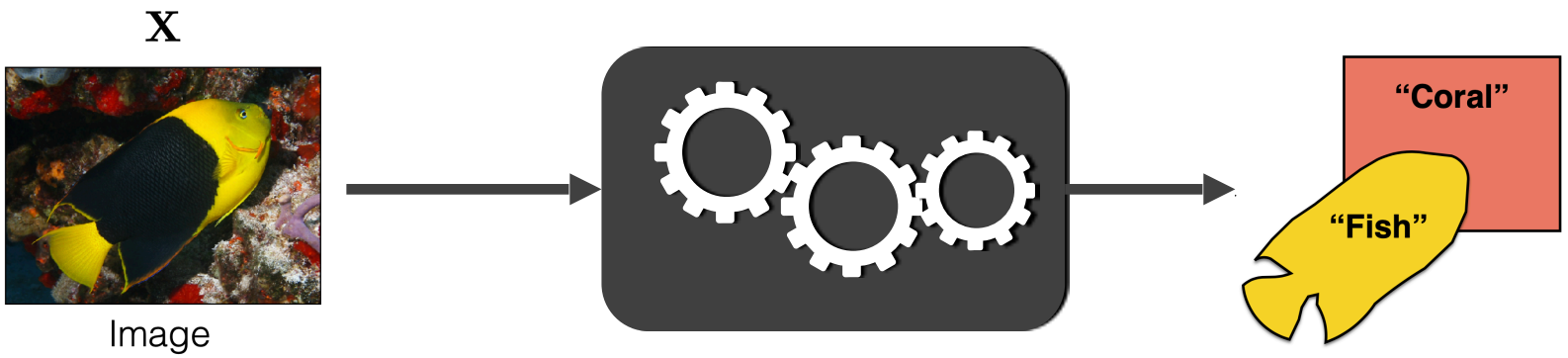
Input Image

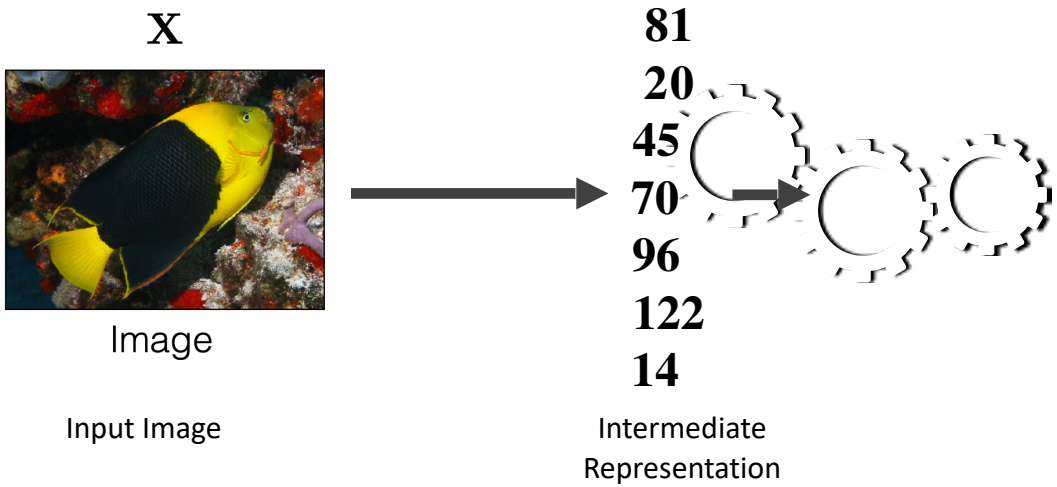
Typical CV Pipeline



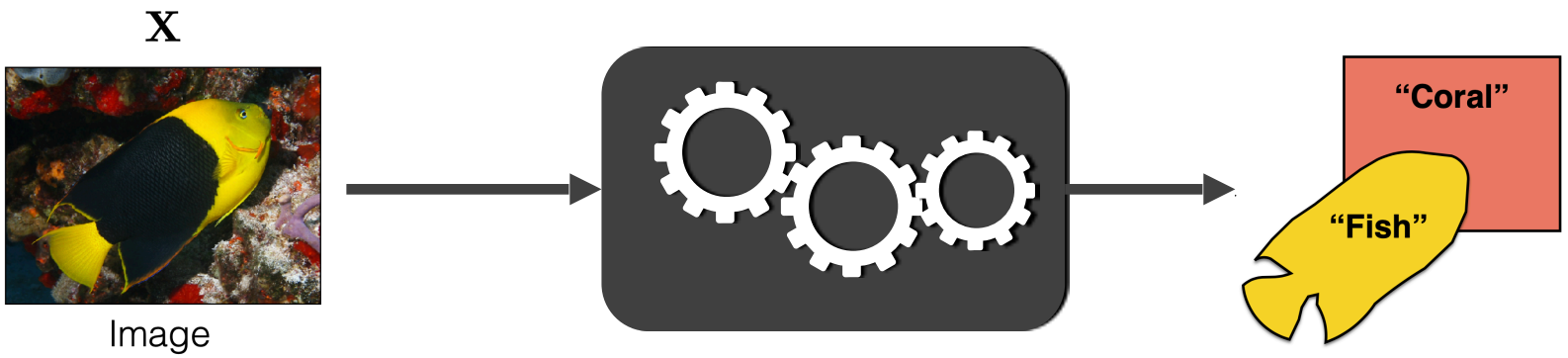


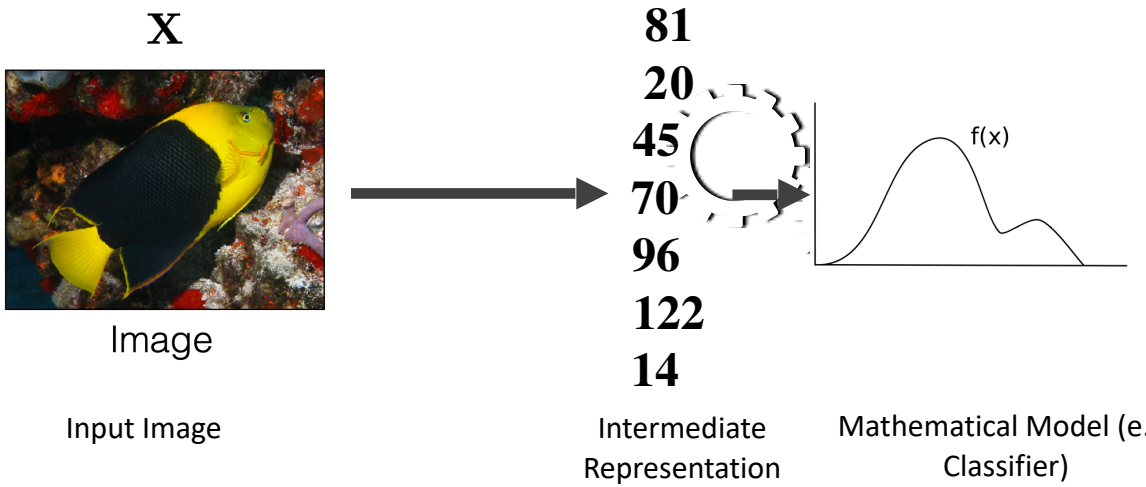
Typical CV Pipeline



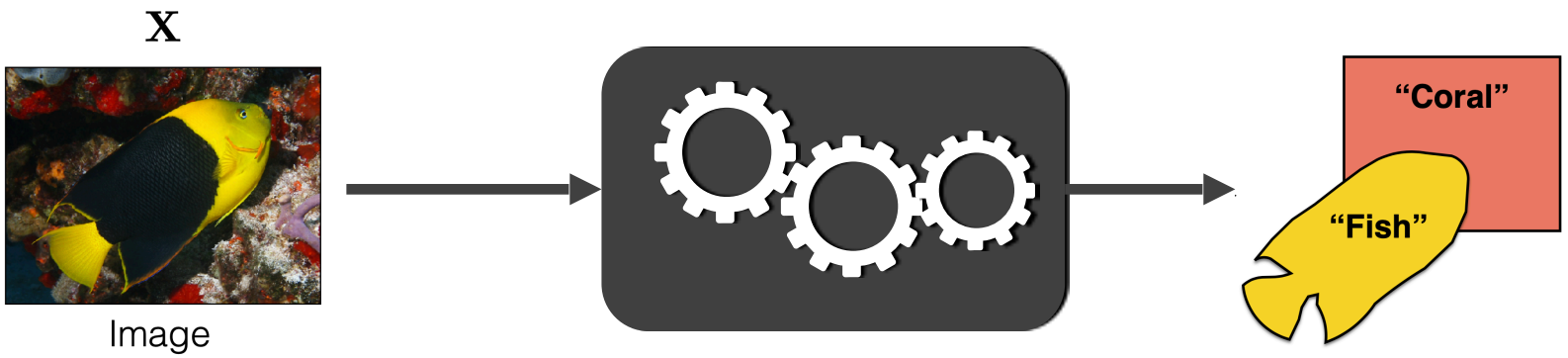


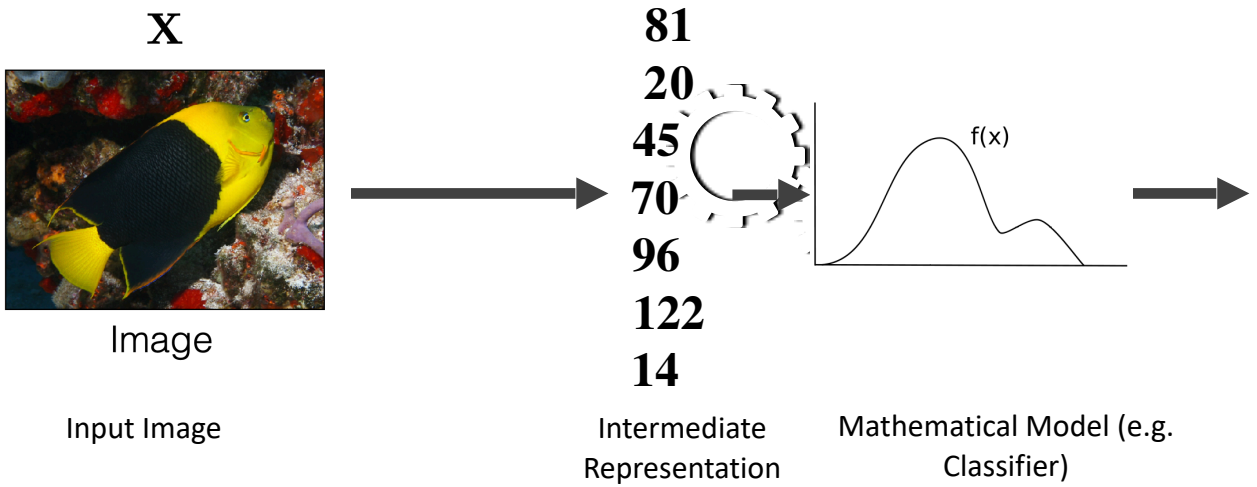
Typical CV Pipeline



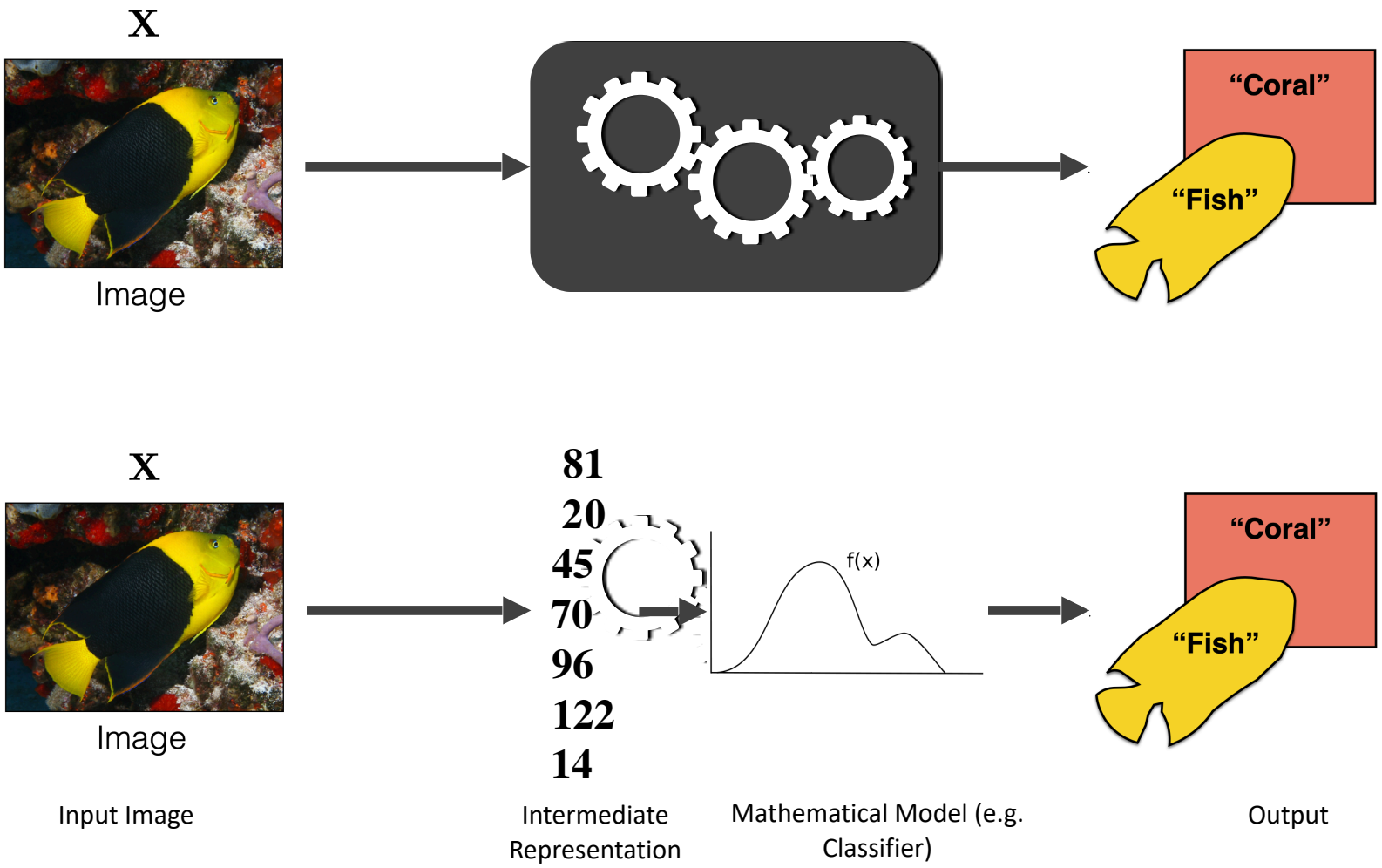


Typical CV Pipeline

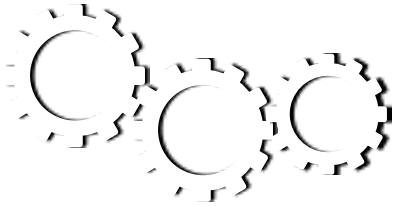




Typical CV Pipeline



Traditional CV Pipeline

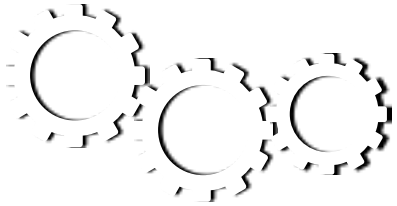


Traditional CV Pipeline

$\mathbf{x}$

Image

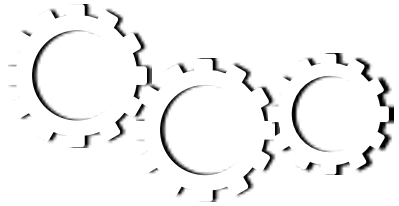
Input Image



Traditional CV Pipeline

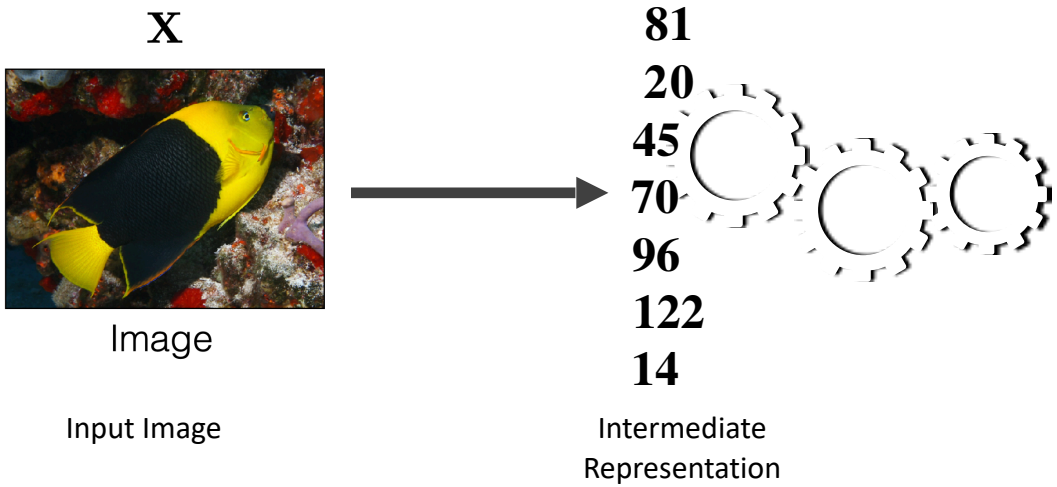
$\mathbf{x}$

Image



Input Image

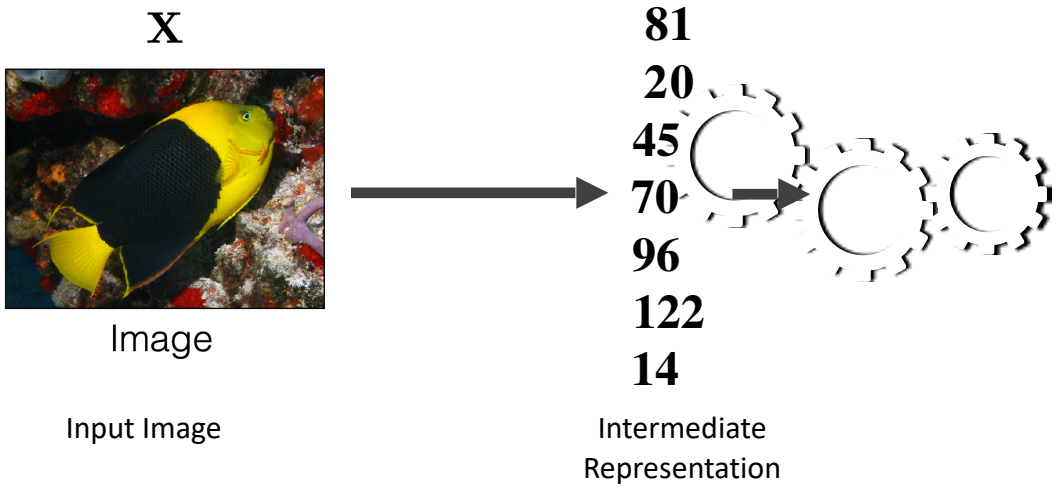
Traditional CV Pipeline



Input Image

Intermediate Representation

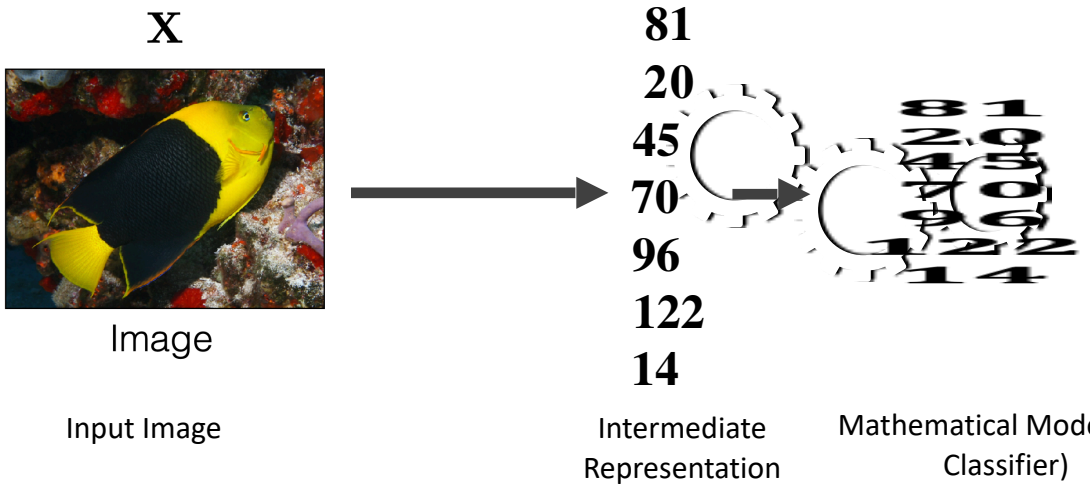
Traditional CV Pipeline



Input Image

Intermediate Representation

Traditional CV Pipeline

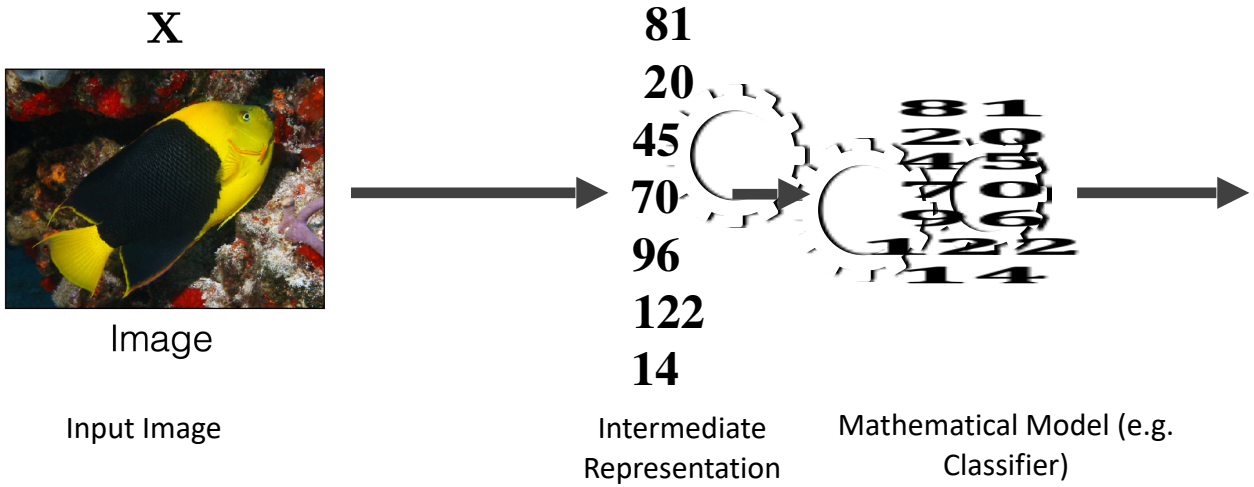


Input Image

Intermediate Representation

Mathematical Model (e.g. Classifier)

Traditional CV Pipeline

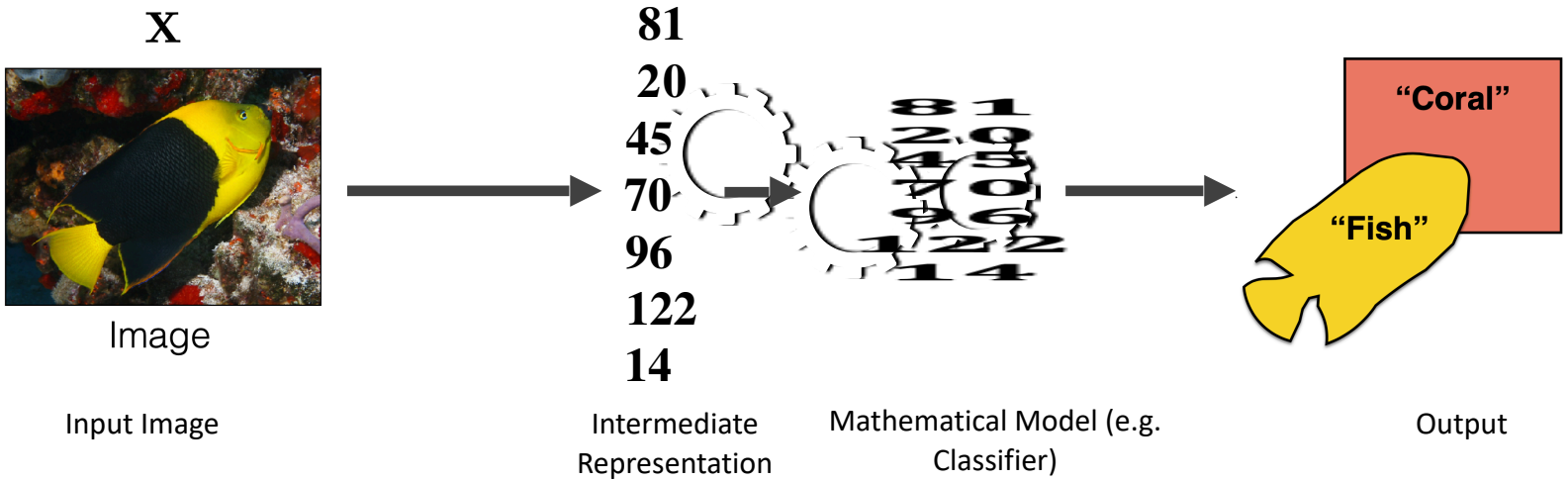


Input Image

Intermediate Representation

Mathematical Model (e.g. Classifier)

Traditional CV Pipeline



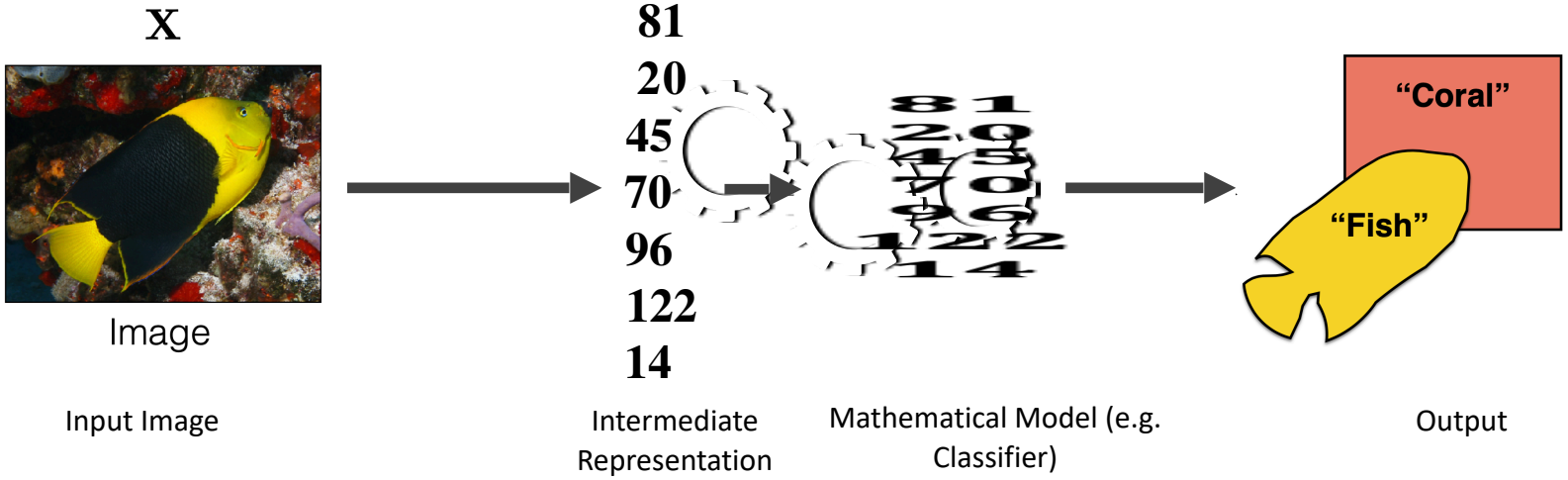
Input Image

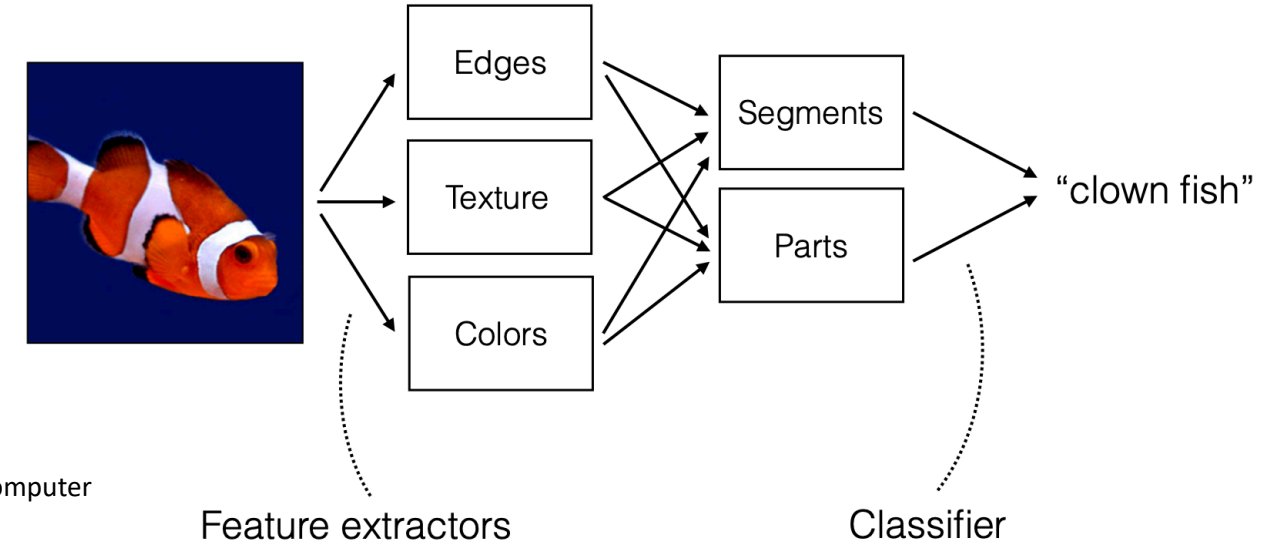
Intermediate Representation

Mathematical Model (e.g. Classifier)

Output

Traditional CV Pipeline





Example from Advances in Computer Vision – MIT – 6.869/6.819

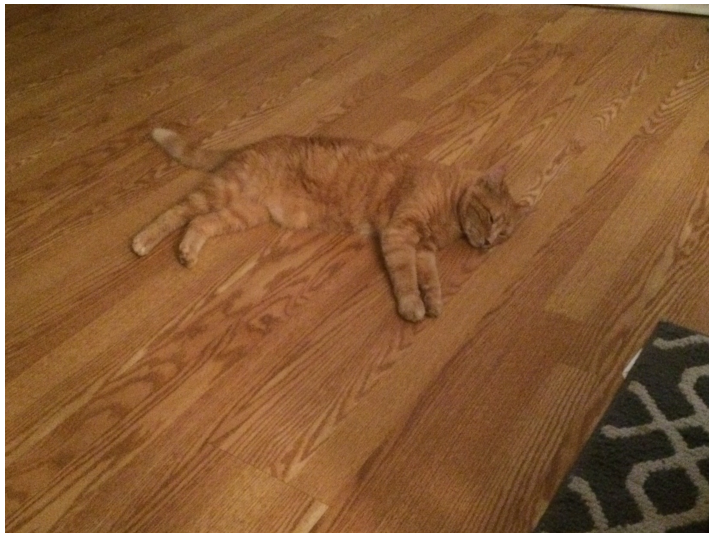
Represent these cats with a cat detector!

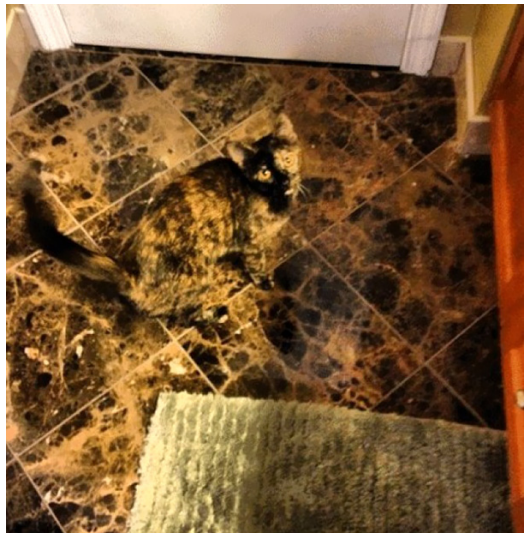
Example from CS331B: Representation Learning in Computer Vision

Represent these cats with a cat detector!

Example from CS331B: Representation Learning in Computer Vision

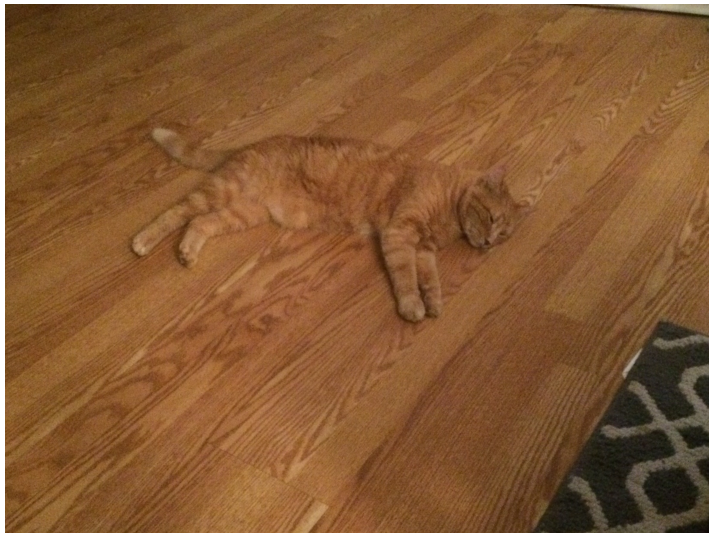
Represent these cats with a cat detector! (II)

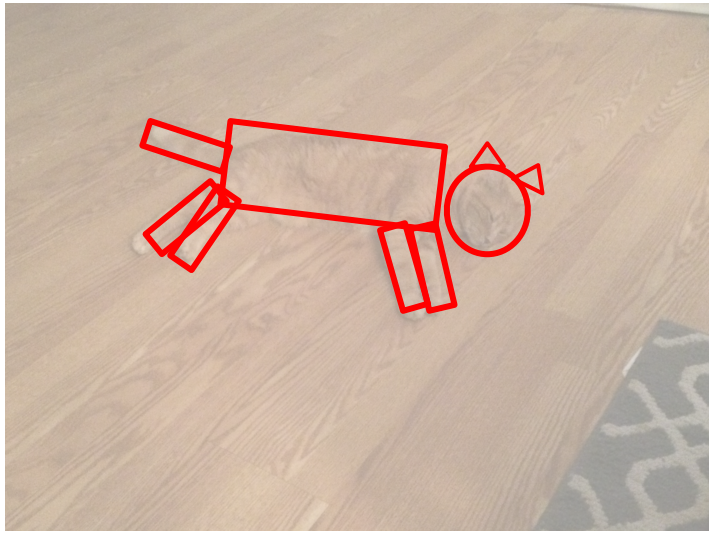




Example from CS331B: Representation Learning in Computer Vision

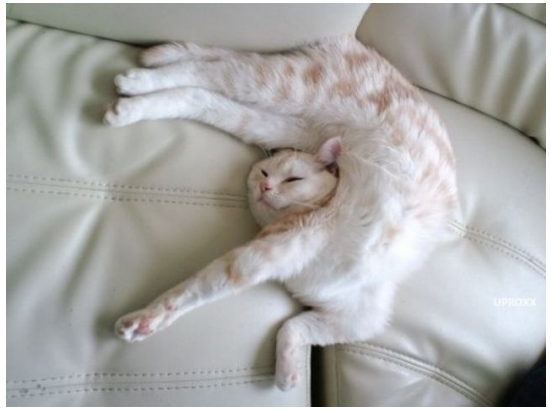
Represent these cats with a cat detector! (II)





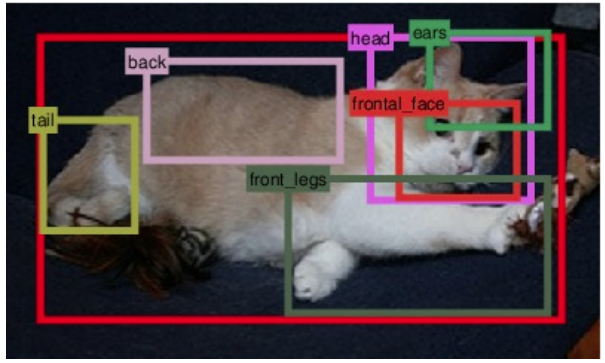
Example from CS331B: Representation Learning in Computer Vision

Represent these cats with a cat detector! (III)



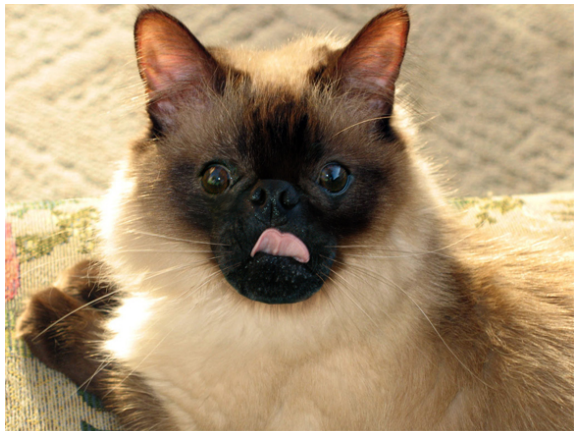
Example from CS331B: Representation Learning in Computer Vision

Represent these cats with a cat detector! (III)



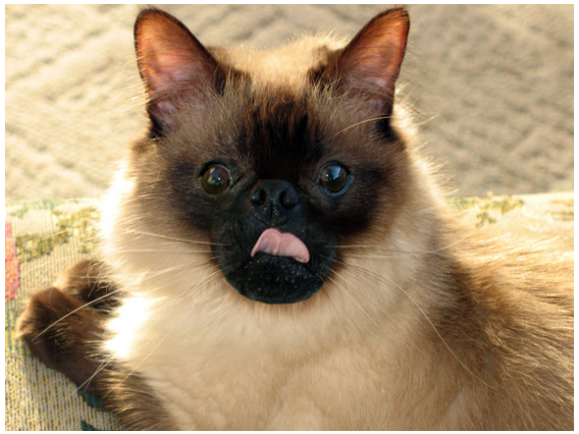
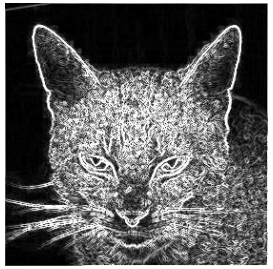
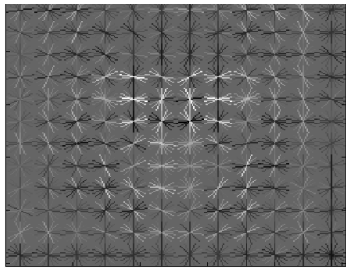
Example from CS331B: Representation Learning in Computer Vision

Represent these cats with a cat detector! (IV)



Example from CS331B: Representation Learning in Computer Vision

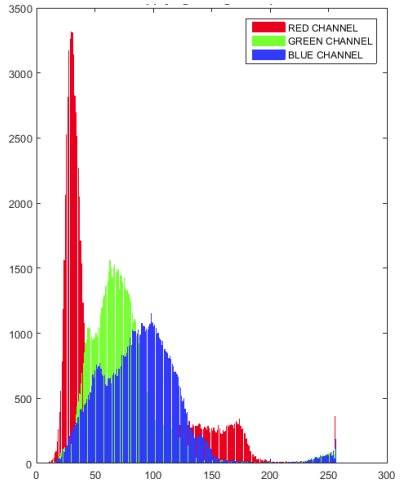
Represent these cats with a cat detector! (IV)

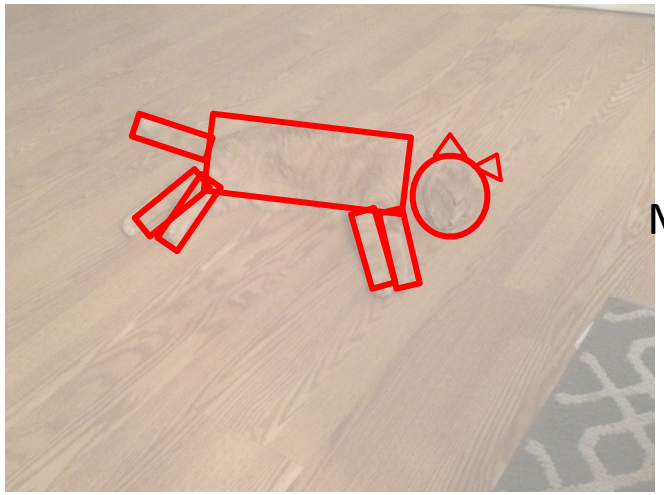




Example from CS331B: Representation Learning in Computer Vision

Summary of Traditional Components

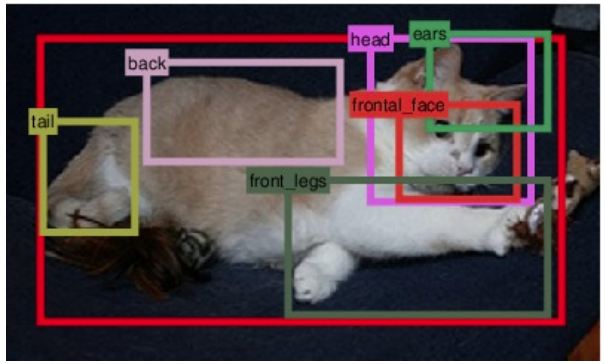
Color Histograms

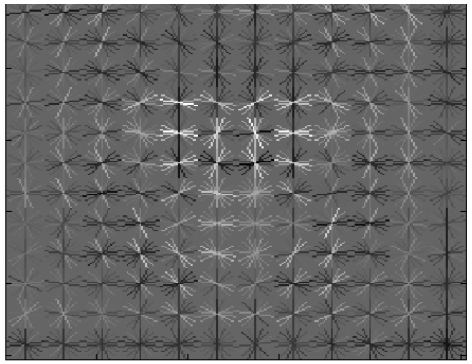




Model based Shapes

Deformable Part based Models (DPM)





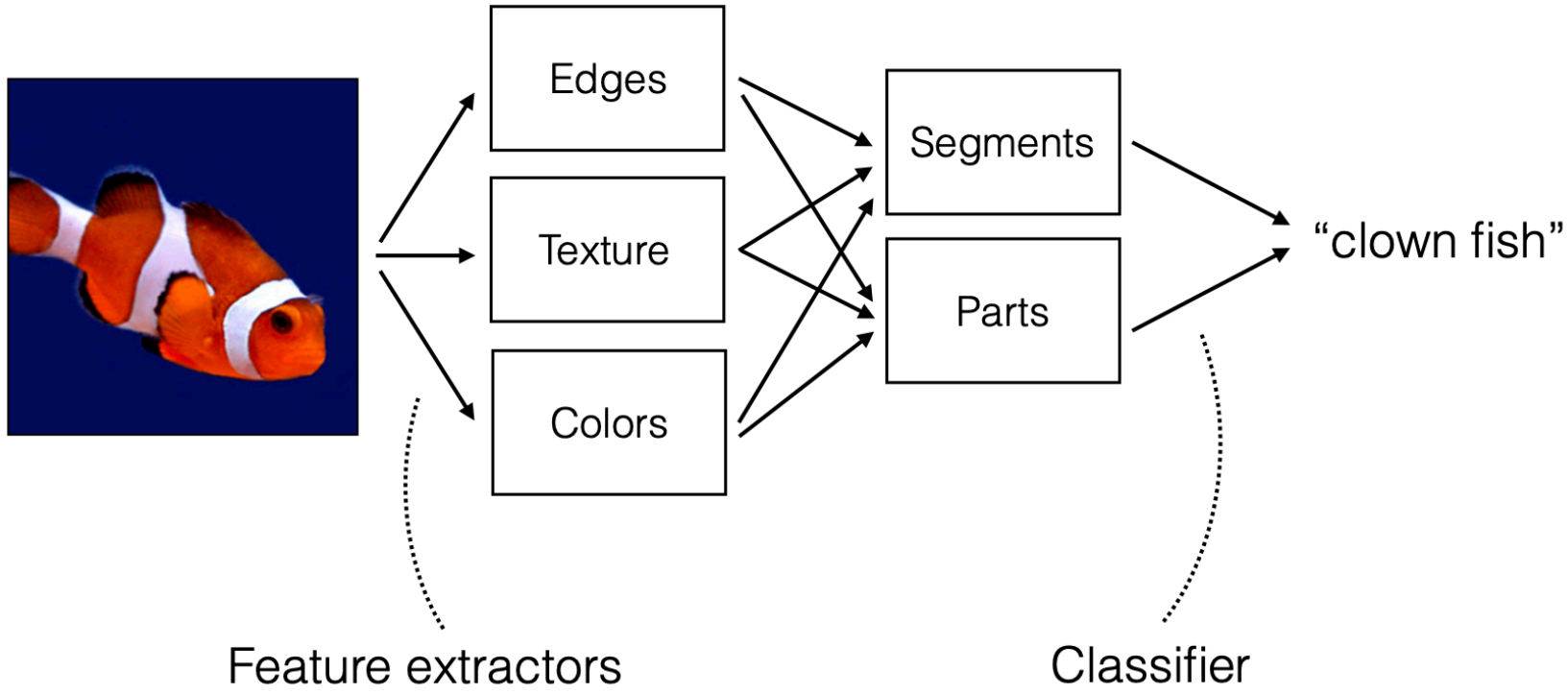
Histogram of Gradients (HOG)

Felzenszwalb et al. 2010.
Dalal and Triggs, 2005.
Beis and Lowe, 1997.

Example from CS331B: Representation Learning in Computer Vision

Traditional CV Pipeline

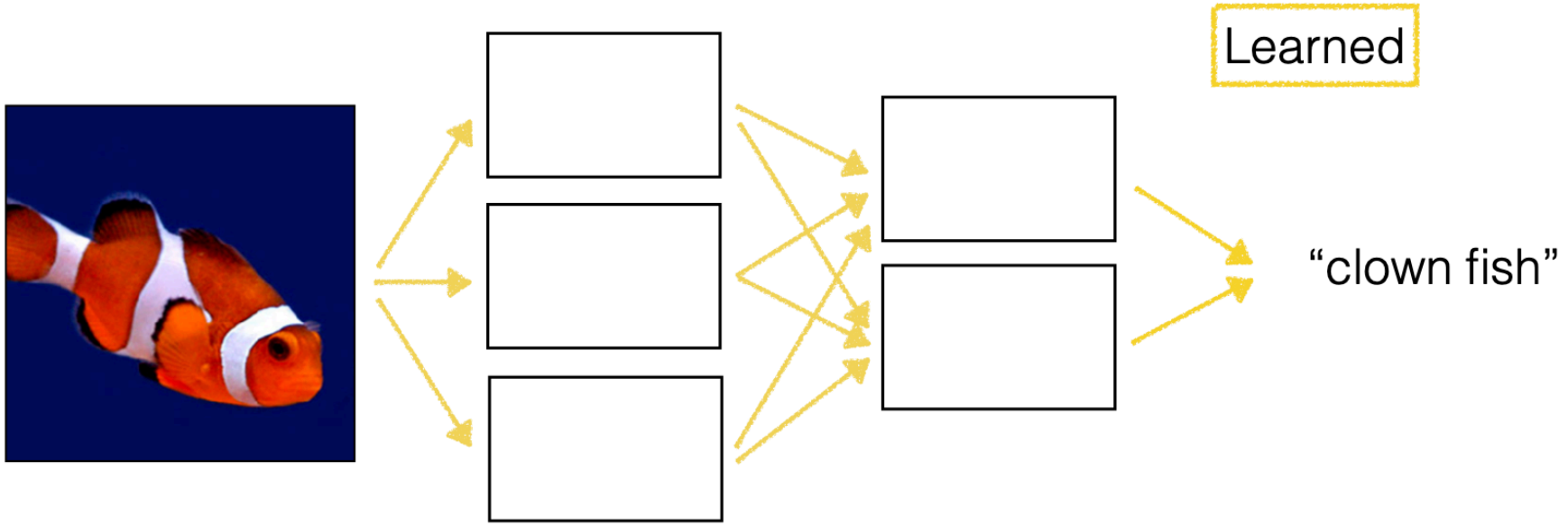
Traditional CV Pipeline



Example from Advances in Computer Vision – MIT – 6.869/6.819

Traditional CV Pipeline

CS 231a → 3D
CS 231n → CNN

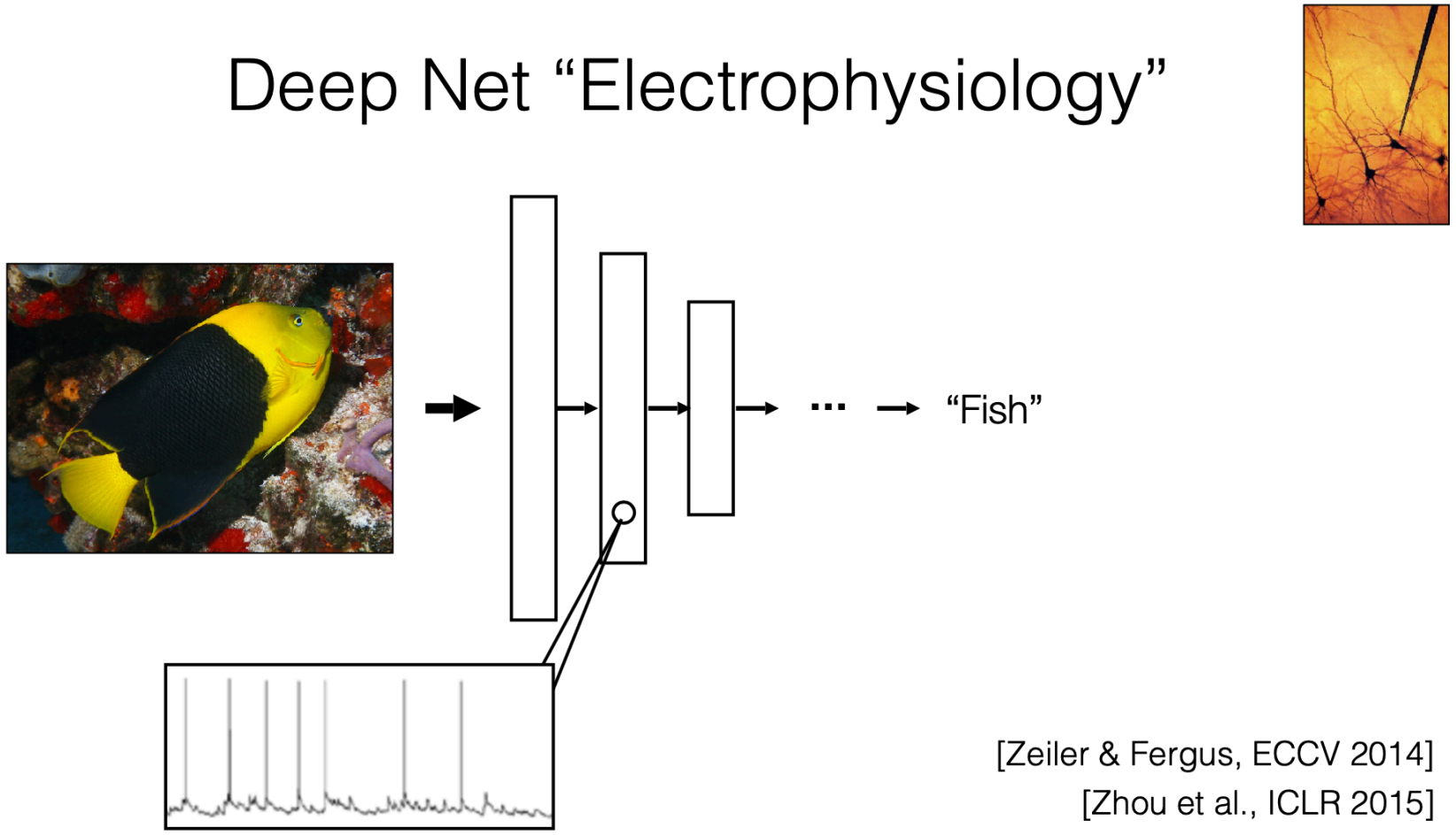


Example from Advances in Computer Vision – MIT – 6.869/6.819

How do you interpret what the network has learned?

Example from Advances in Computer Vision – MIT – 6.869/6.819

How do you interpret what the network has learned?



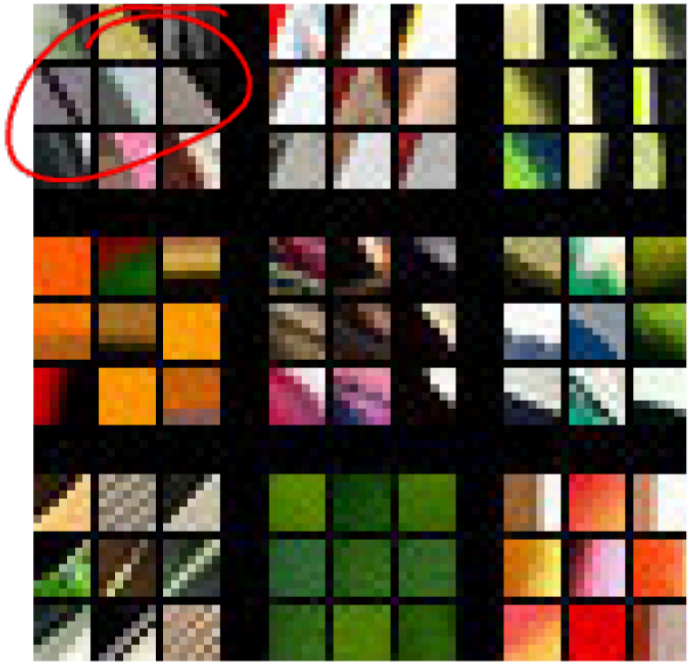
[Zeiler & Fergus, ECCV 2014]
[Zhou et al., ICLR 2015]

Visualizing and Understanding CNNs

[Zeiler and Fergus, 2014]

Gabor-like filters learned by **layer 1**

Image patches that activate each of the **layer 1** filters most strongly

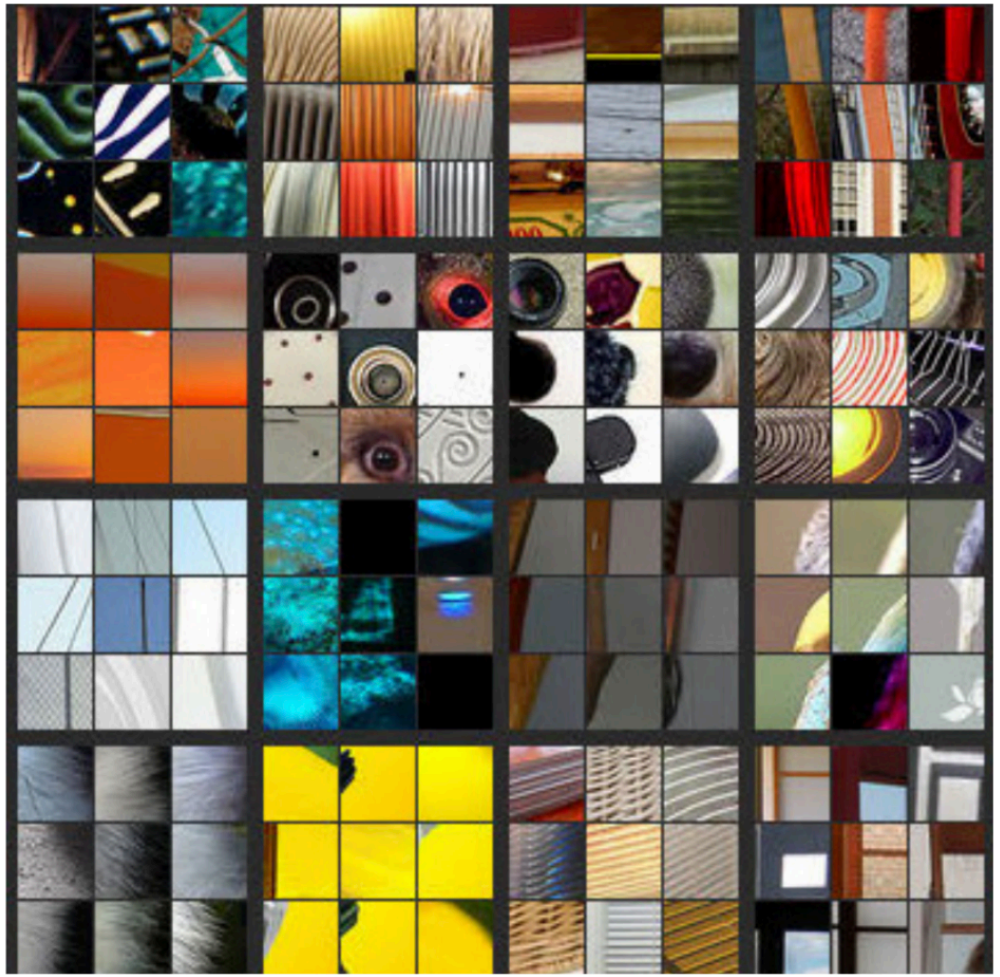


Example from Advances in Computer Vision – MIT – 6.869/6.819

Visualizing and Understanding CNNs

[Zeiler and Fergus, 2014]

Image patches that activate each of the **layer 2** neurons most strongly



Example from Advances in Computer Vision – MIT – 6.869/6.819

Visualizing and Understanding CNNs

[Zeiler and Fergus, 2014]

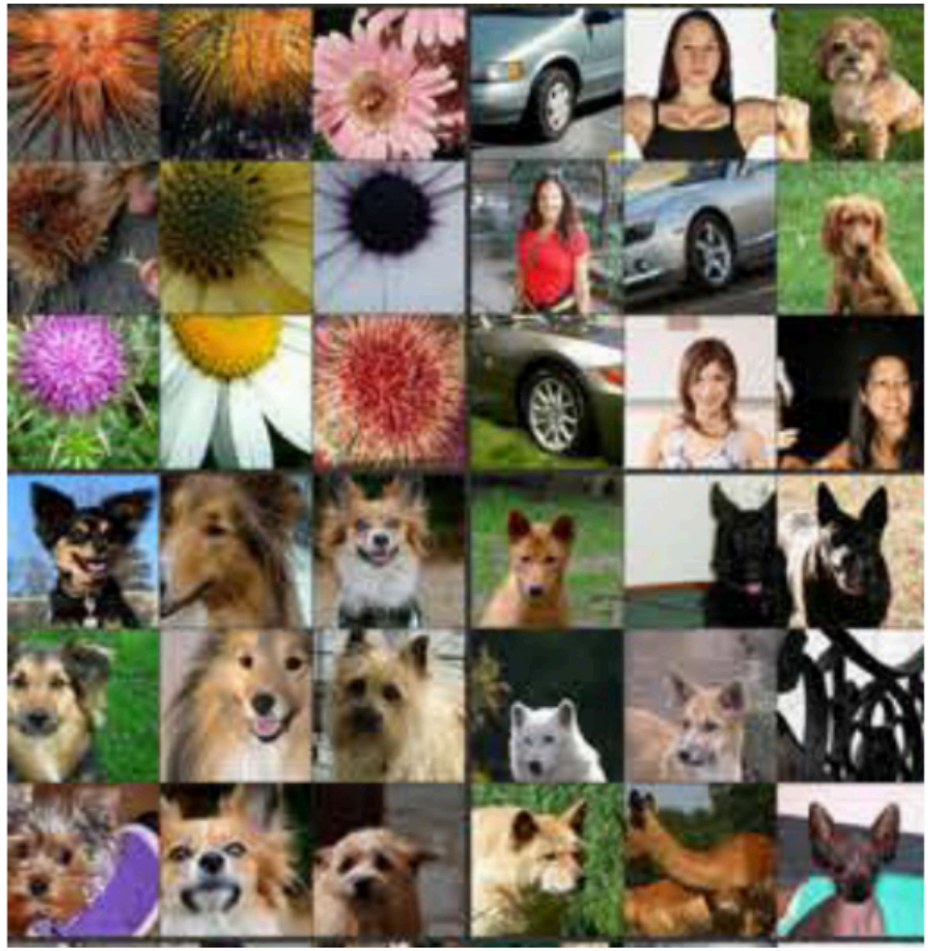
Image patches that activate each of the **layer 4** neurons most strongly

Example from Advances in Computer Vision – MIT – 6.869/6.819

Visualizing and Understanding CNNs

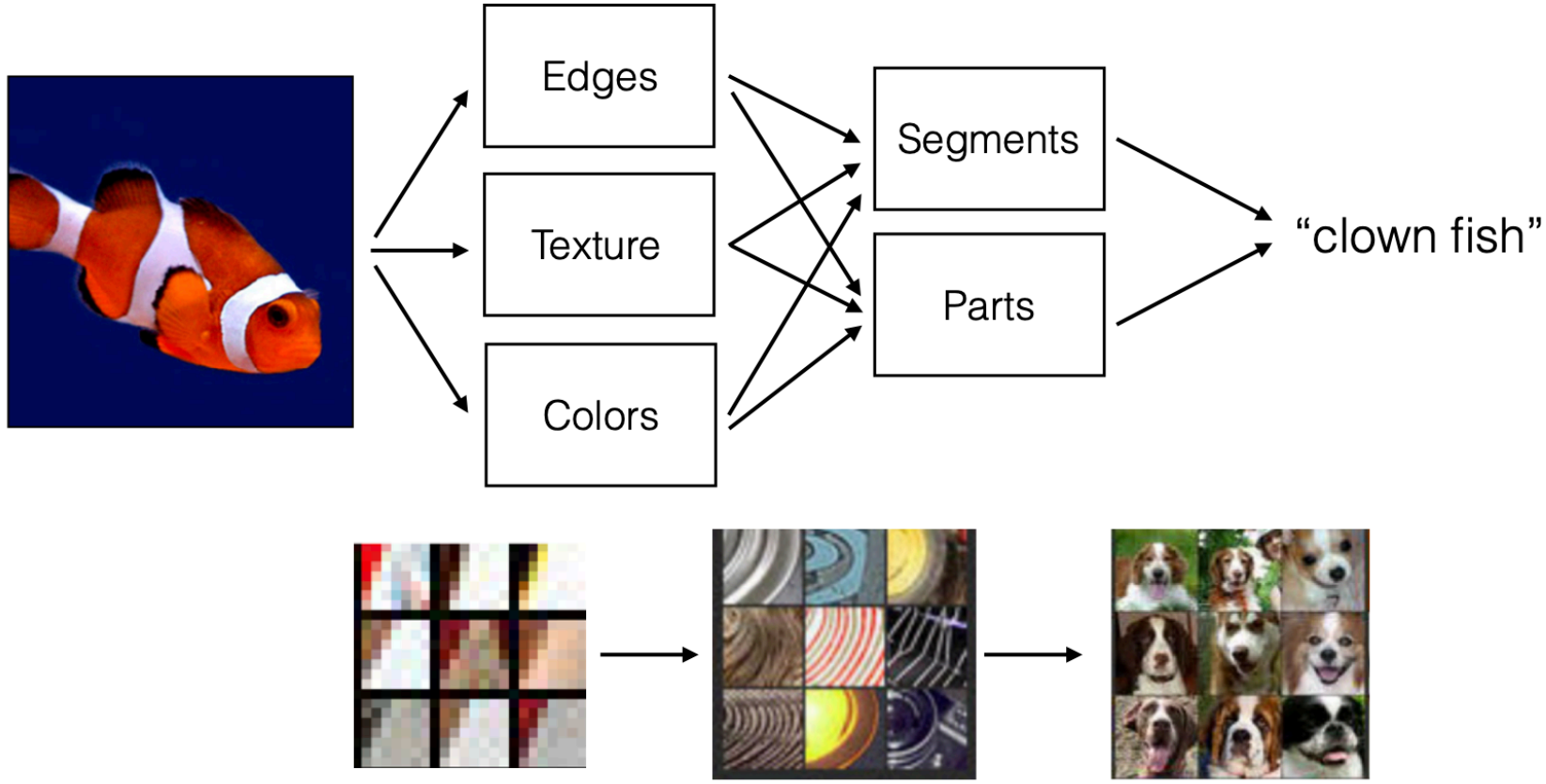
[Zeiler and Fergus, 2014]

Image patches that activate each of the **layer 5** neurons most strongly



Visualizing and Understanding CNNs

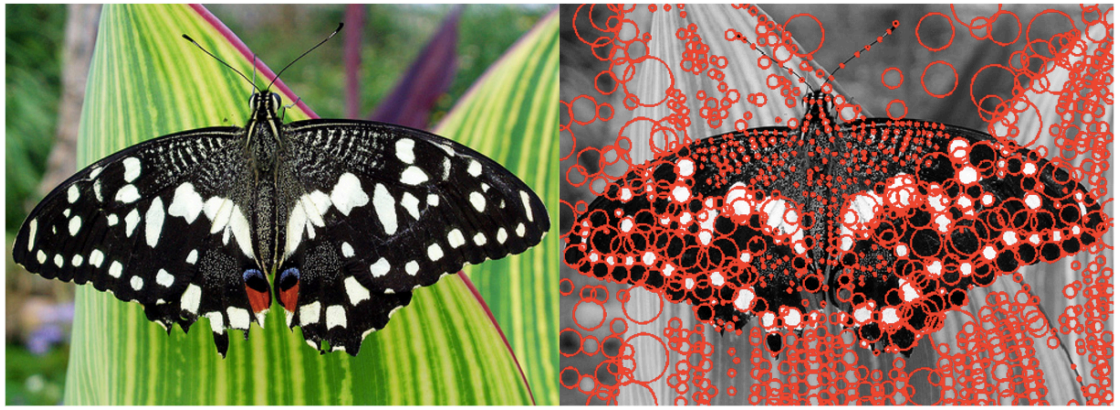
CNNs *learned* the classical visual recognition pipeline!



Example from Advances in Computer Vision – MIT – 6.869/6.819

How to represent images?

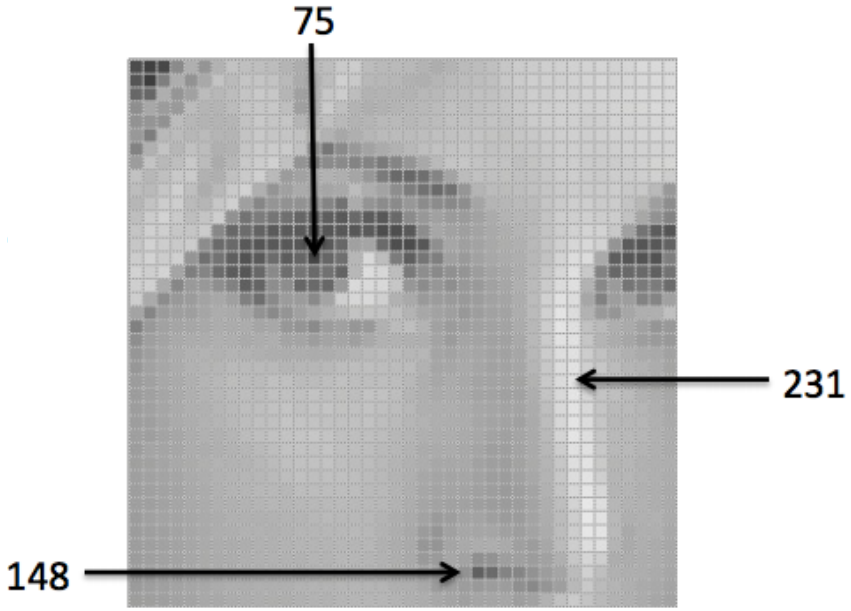
Image processing pipeline



1. Signal treatment / filtering
2. Feature detection (e.g., DoG)
3. Feature description (e.g., SIFT)
4. Higher-level processing

Image filtering

- Filtering: process of accepting / rejecting certain frequency components
- Starting point is to view images as functions $I: [a, b] \times [c, d] \rightarrow [0, L]$, where $I(x, y)$ represents intensity at position (x, y)



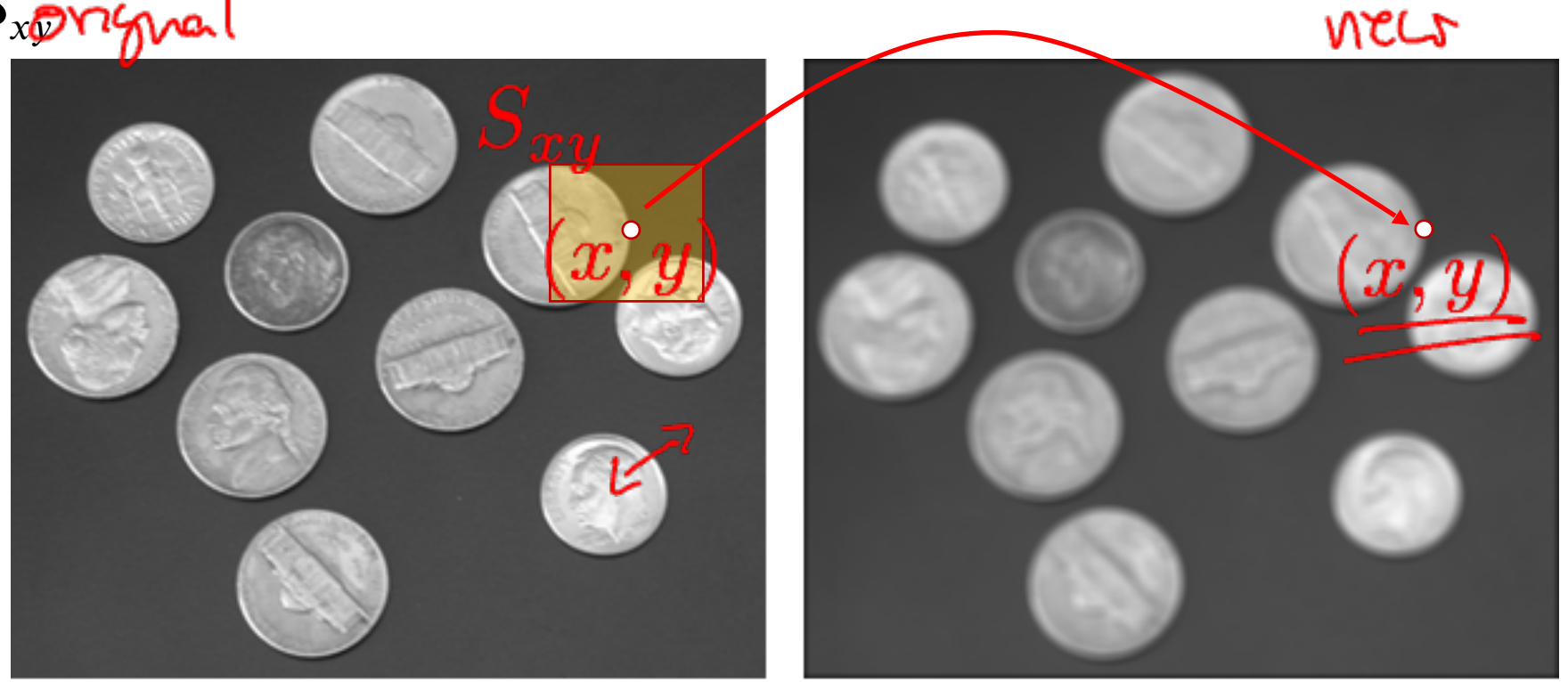
give rise to a vector function with 3

Represented as a matrix

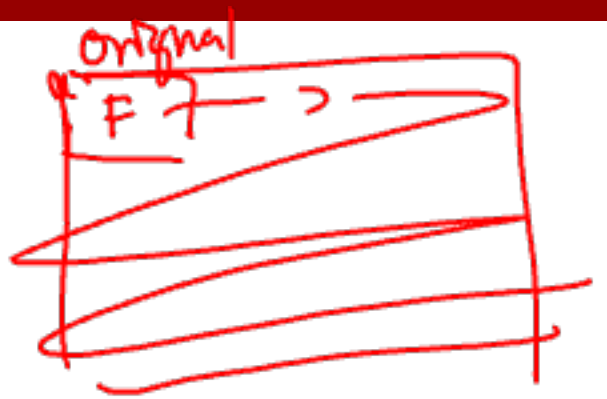
88	82	84	88	85	83	80	93	102
88	80	78	80	80	78	73	94	100
85	79	80	78	77	74	65	91	99
38	35	40	35	39	74	77	70	65
20	25	23	28	37	69	64	60	57
22	26	22	28	40	65	64	59	34
24	28	24	30	37	60	58	56	66
21	22	23	27	38	60	67	65	67
23	22	22	25	38	59	64	67	66

Spatial filters

- A spatial filter consists of
  1. A neighborhood S_{xy} of pixels around the point (x, y) under examination
  2. A predefined operation *F* that is performed on the image pixels within S_{xy}



Linear spatial filters



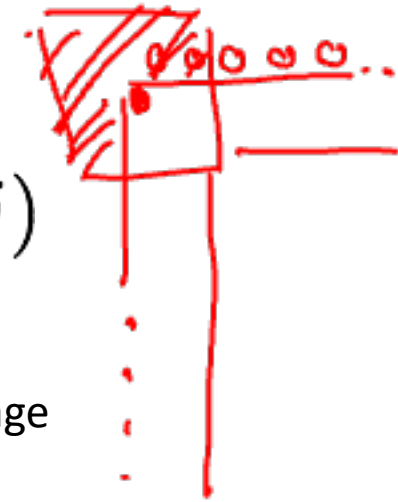
- Filters can be linear or non-linear
- We will focus on linear spatial filters

$$I'(x,y) = F \circ I = \sum_{i=-N}^{N} \sum_{j=-M}^{M} F(i,j) I(x+i, y+j)$$

Filtered image

Filter mask

Original image

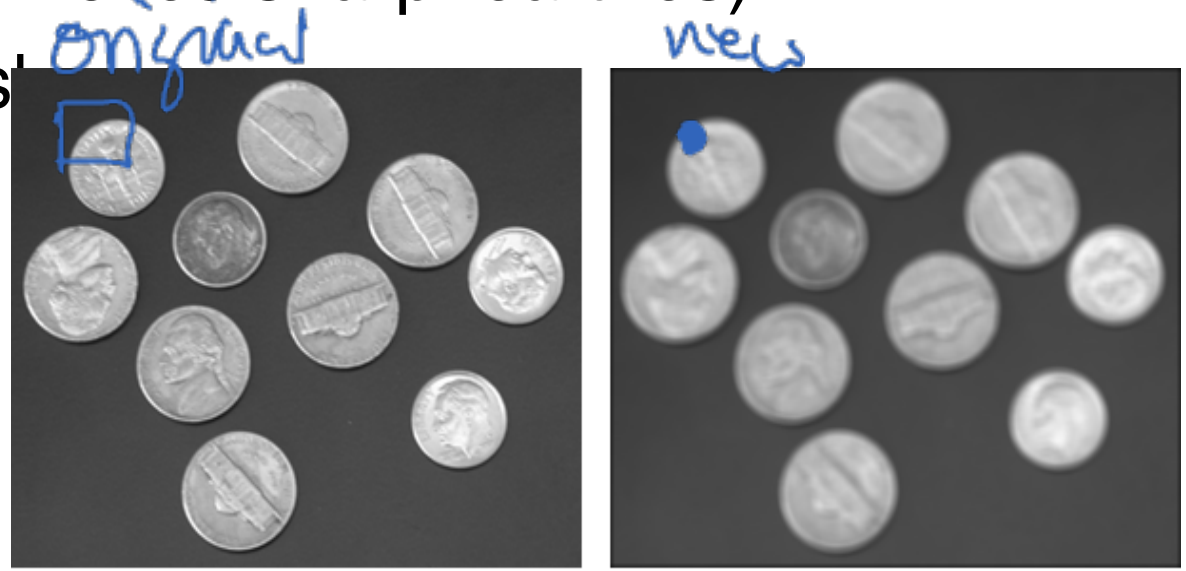


- Filter F (of size $(2N+1) \times (2M+1)$) is usually called a mask, kernel, or window
- Dealing with boundaries: e.g., pad, crop, extend, or wrap

Filter example #1: moving average

- The moving average filter returns the average of the pixels in the mask
- Achieves a smoothing effect (removes sharp features)
- E.g., for a *normalized* 3×3 mask

$$F = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

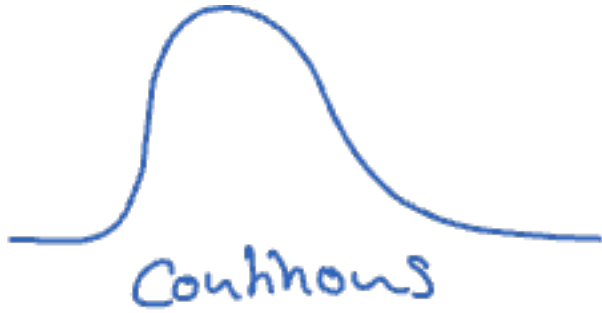


Generated with a 5x5 mask

Filter example #2: Gaussian smoothing

- Gaussian function

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$



- To obtain the mask, sample the function about its center
- E.g., for a *normalized* 3×3 mask with $\sigma = 0.85$

$$G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

normalization

Convolution

- Still a linear filter, defined as

$$I'(x,y) = F * I = \sum_{i=-N}^{N} \sum_{j=-M}^{M} F(i,j) I(x-i, y-j)$$

- Same as correlation, but with negative signs for the filter indices
- Correlation and convolution are identical when the filter is symmetric
- Convolution enjoys the associativity property

$$F * (G * I) = (F * G) * I$$

- Example: smooth image & take derivative = convolve derivative filter with Gaussian filter & convolve the resulting filter with the image

Separability of masks

- A mask is separable if it can be broken down into the convolution of two kernels

$$F = F_1 * F_2$$

- If a mask is separable into "smaller" masks, then it is often cheaper to apply F_1 followed by F_2, rather than F directly
- Special case: mask representable as outer product of two vectors (equivalent to two-dimensional convolution of those two vectors)
- If mask is $M \times M$, and image has size $w \times h$, then complexity is
  - $O(M^2wh)$ with no separability
  - $O(2Mwh)$ with separability into outer product of two vectors

Example of separable masks

- Moving average

$$F = \frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix} = \frac{1}{9}\begin{bmatrix}1\\ 1\\ 1\end{bmatrix}\begin{bmatrix}1 & 1 & 1\end{bmatrix}$$

- Gaussian smoothing

$$\begin{aligned} G_\sigma(x,y) &= \frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right) \\ &= \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{x^2}{2\sigma^2}\right)\frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{y^2}{2\sigma^2}\right) \\ &= g_\sigma(x)\cdot g_\sigma(y) \end{aligned}$$

Differentiation

- Derivative of discrete function (centered difference)

$$\frac{\partial I}{\partial x} = I(x+1, y) - I(x-1, y)$$

$$\frac{\partial I}{\partial y} = I(x, y+1) - I(x, y-1)$$

$$F_x = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$$

$$F_y = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

- Derivative as a convolution operation; e.g., Sobel masks:

Along *x* direction

$$S_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

Along *y* direction

$$S_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

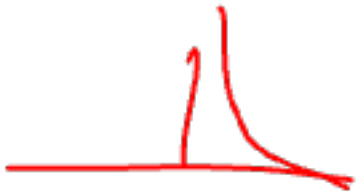
Note: masks are mirrored
In convolution

Similarity measures

- Filtering can also be used to determine similarity across images (e.g., to detect correspondences)

$$SAD = \sum_{i=-n}^{n} \sum_{j=-m}^{m} |I_1(x+i, y+j) - I_2(x'+i, y'+j)|$$

Sum of absolute differences



$$SSD = \sum_{i=-n}^{n} \sum_{j=-m}^{m} [I_1(x+i, y+j) - I_2(x'+i, y'+j)]^2$$

Sum of squared differences

stereo matching

Detectors

- Goal: detect local features, i.e., image patterns that differ from immediate neighborhood in terms of intensity, color, or texture

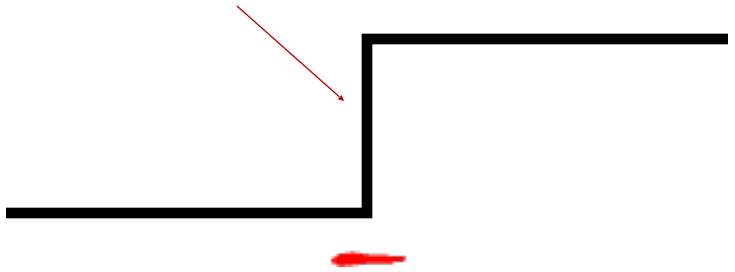
- We will focus on
  - Edge detectors
  - Corner detectors

Edge detectors

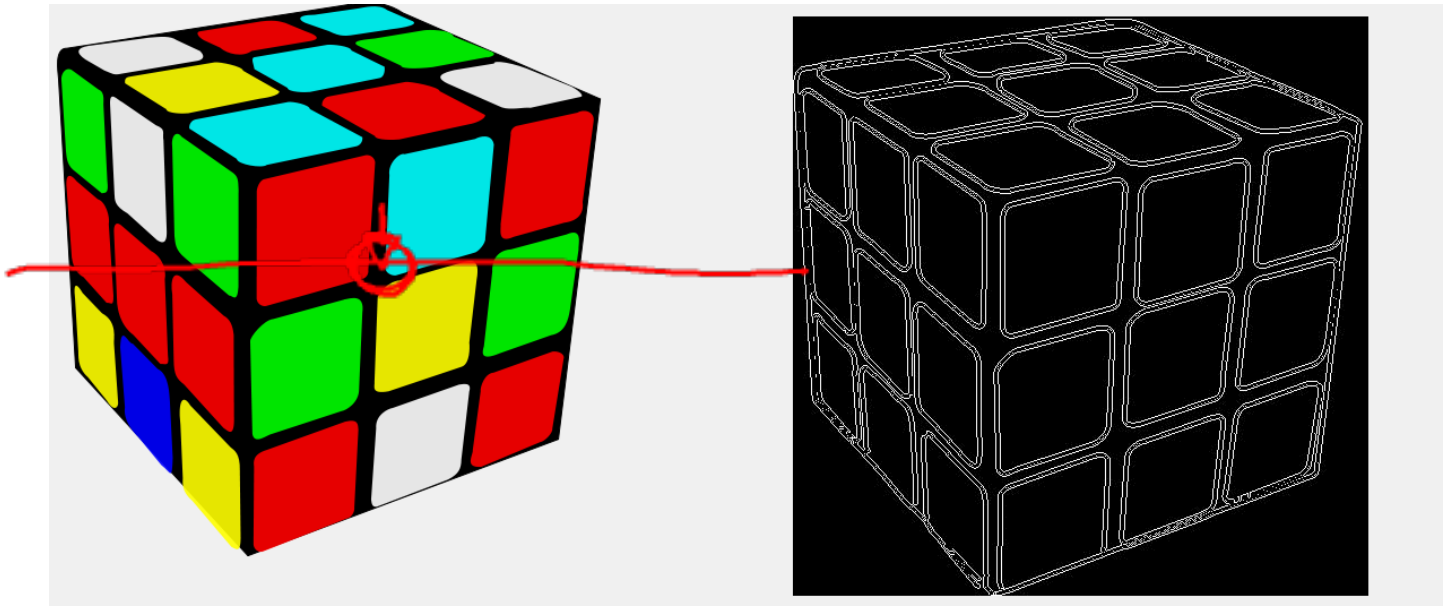
- Edge: region in an image where there is a *significant* change in intensity values along one direction, and *negligible* change along the orthogonal direction

In 1D

Magnitude of 1st order derivative is large,
2nd order derivative is equal to zero



In 2D



Criteria for “good” edge detection

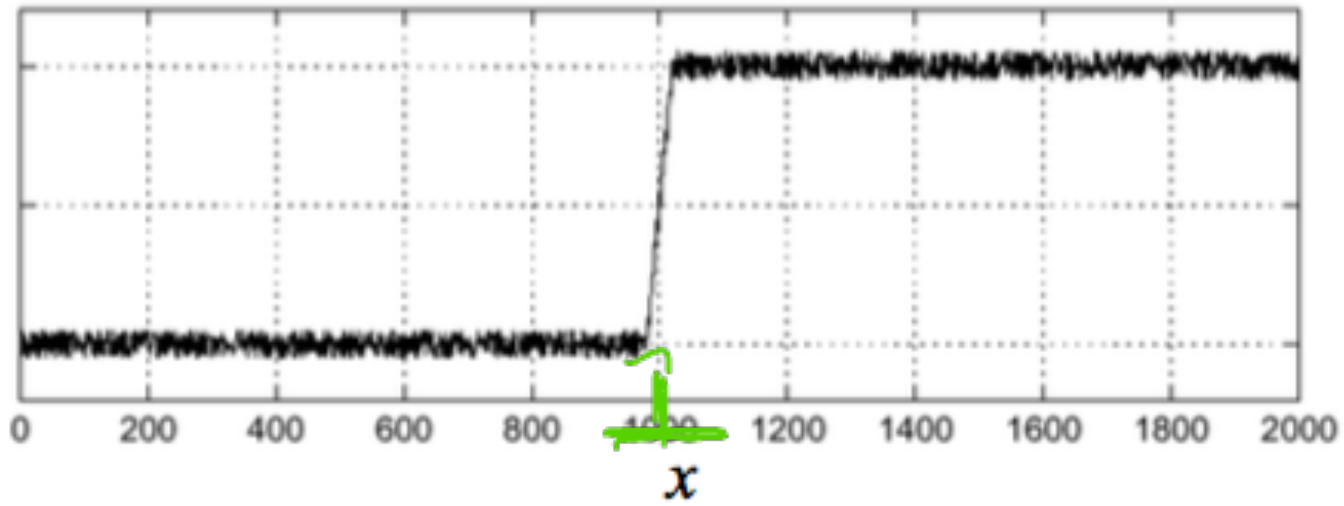
- Accuracy: minimize false positives and negatives
- Localization: edges must be detected as close as possible to the true edges
- Single response: detect one edge per real edge in the image

Strategy to design an edge detector

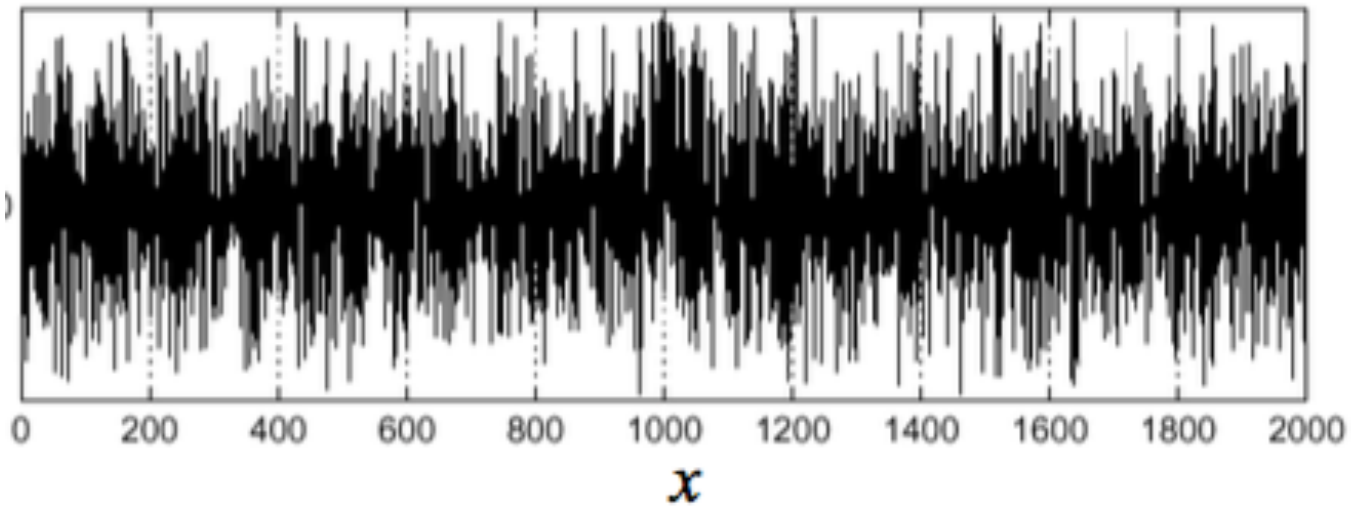
- Two steps:
  1. Smoothing: smooth the image to reduce noise prior to differentiation (step 2)
  2. Differentiation: take derivatives along *x* and *y* directions to find locations with high gradients

1D case: differentiation without smoothing

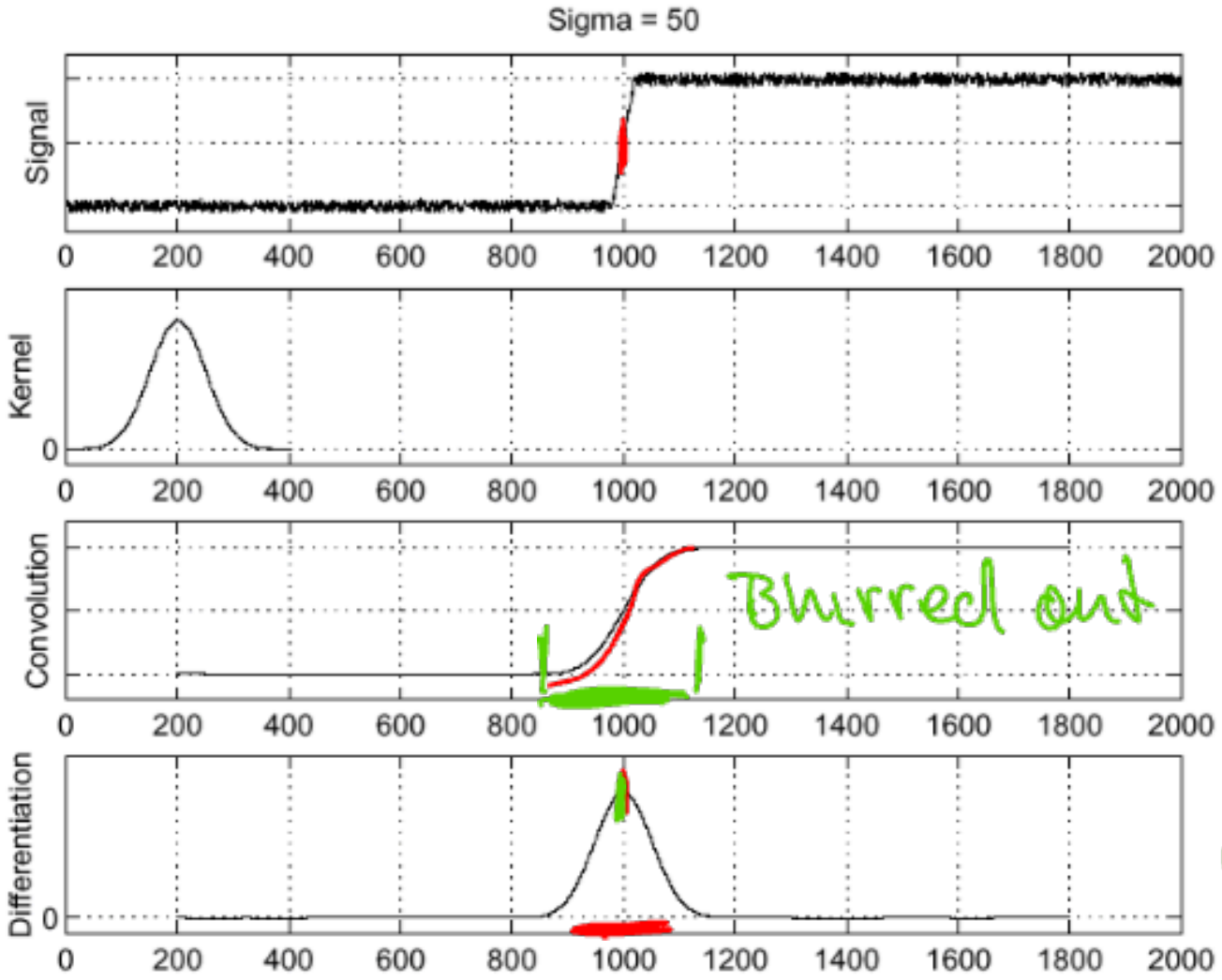
$I(x)$



$\frac{dI(x)}{dx}$



1D case: differentiation with smoothing



$I(x)$

$g_\sigma(x)$

Edges occur at maxima or minima of $s'(x)$

$$s(x) = g_\sigma(x) * I(x)$$

convolution

$$s'(x) = \frac{d}{dx} * s(x)$$

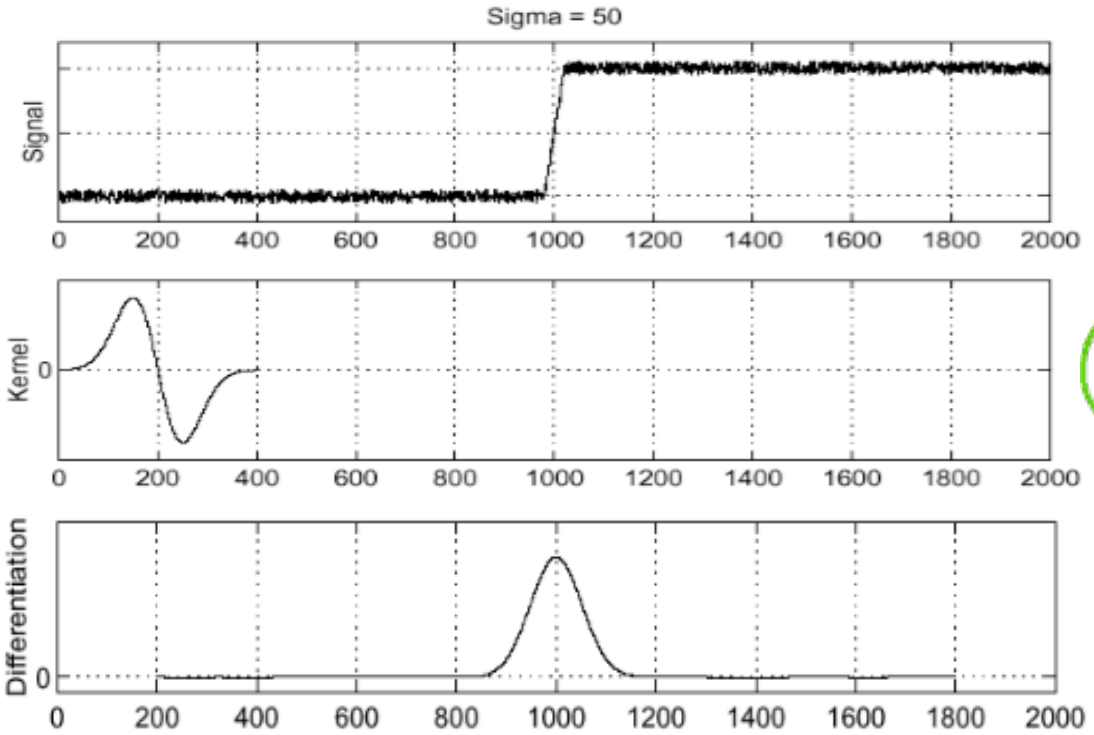
convolute

A better implementation

- Convolution theorem:

$$s'(x) = \frac{d}{dx} * (g_\sigma(x) * I(x)) = \left(\frac{d}{dx} * g_\sigma(x)\right) * I(x)$$

$g'_\sigma(x)$



$I(x)$

$g'_\sigma(x)$ ← just one convolution

$$s'(x) = g'_\sigma(x) * I(x)$$

Edge detection in 2D

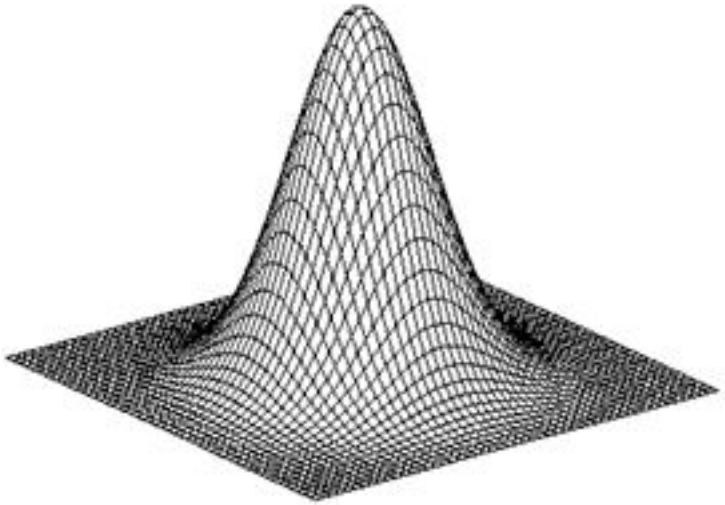
1. Find the gradient of smoothed image in both directions

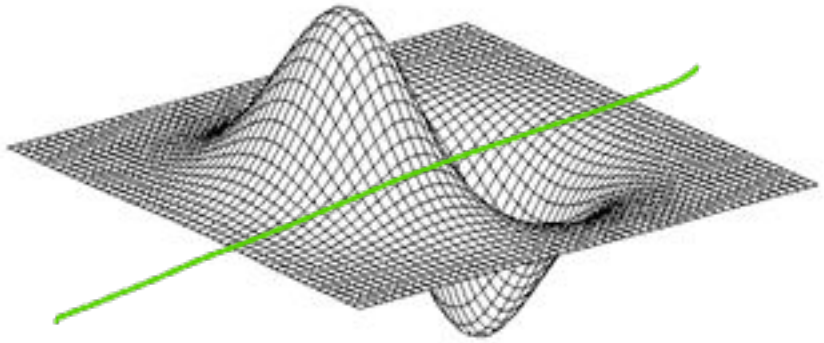
$$\nabla S := \begin{bmatrix} \frac{\partial}{\partial x} * (G_\sigma * I) \\ \frac{\partial}{\partial y} * (G_\sigma * I) \end{bmatrix} = \begin{bmatrix} \left(\frac{\partial}{\partial x} * G_\sigma\right) * I \\ \left(\frac{\partial}{\partial y} * G_\sigma\right) * I \end{bmatrix} = \begin{bmatrix} G_{\sigma,x} * I \\ G_{\sigma,y} * I \end{bmatrix} := \begin{bmatrix} S_x \\ S_y \end{bmatrix}$$

2. Compute the magnitude $|\nabla S| = \sqrt{S_x^2 + S_y^2}$ and discard pixels below a certain threshold
3. Non-maximum suppression: identify local maxima of $|\nabla S|$

Derivative of Gaussian filter

smoothing / low-pass filter / blurring



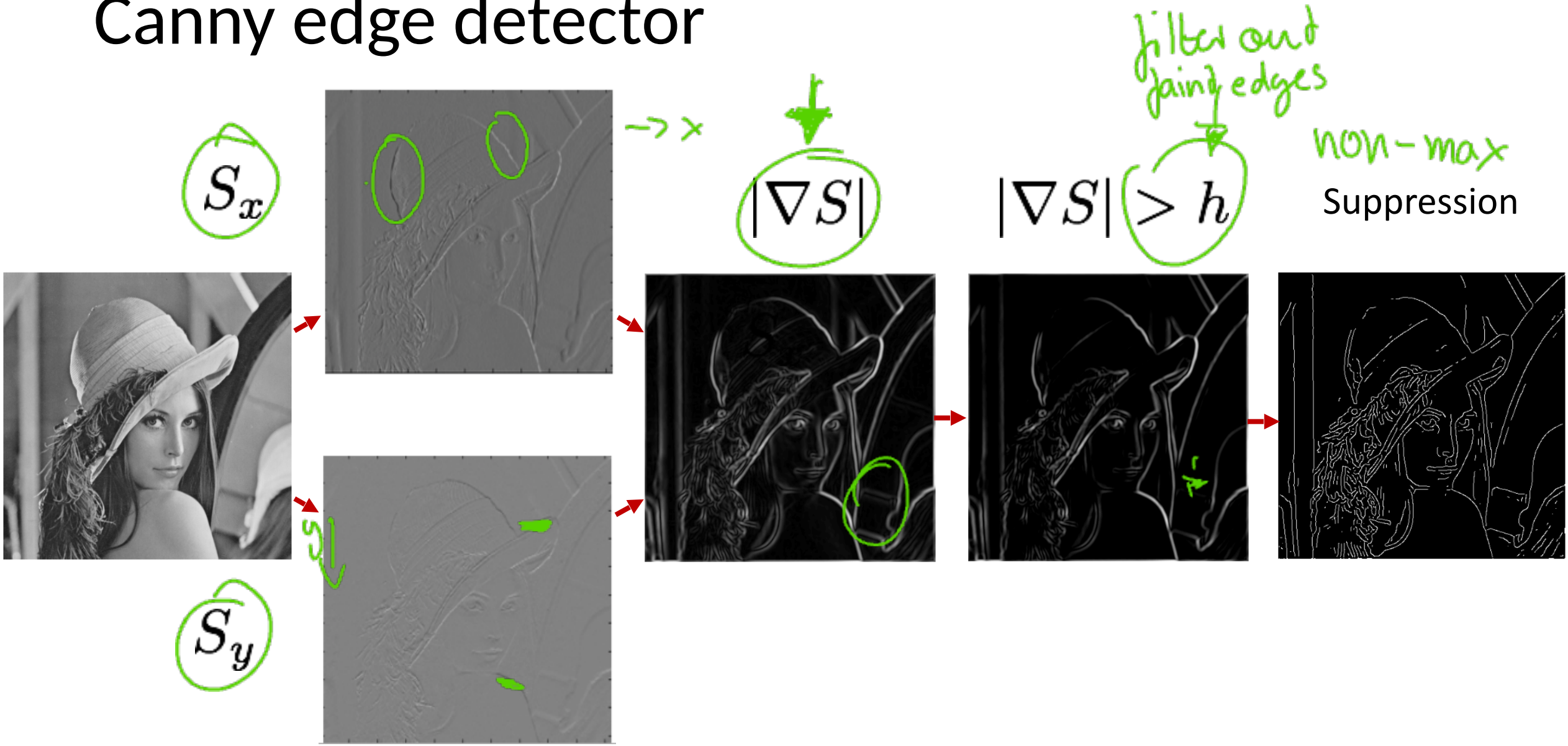


$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$$

$$\frac{\partial G_\sigma(x,y)}{\partial x} \quad S_x$$

$$\frac{\partial G_\sigma(x,y)}{\partial y} \quad S_y$$

Canny edge detector



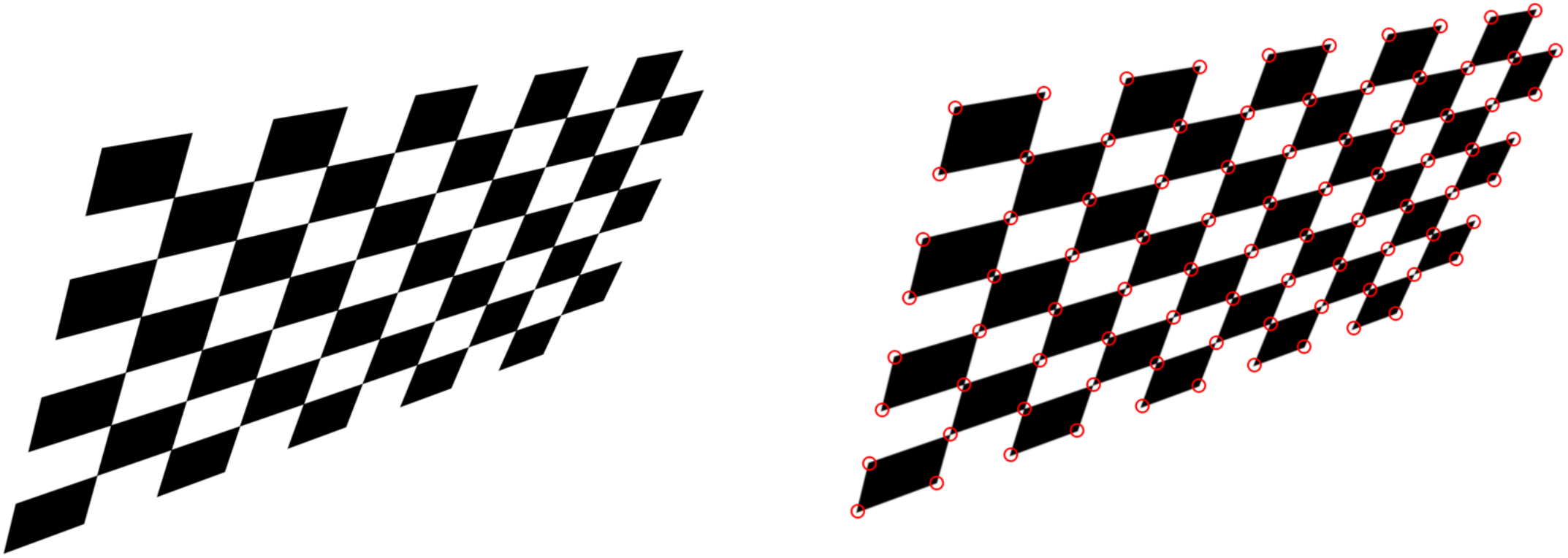
Corner detectors

Key criteria for “good” corner detectors

1. Repeatability: same feature can be found in multiple images despite geometric and photometric transformations

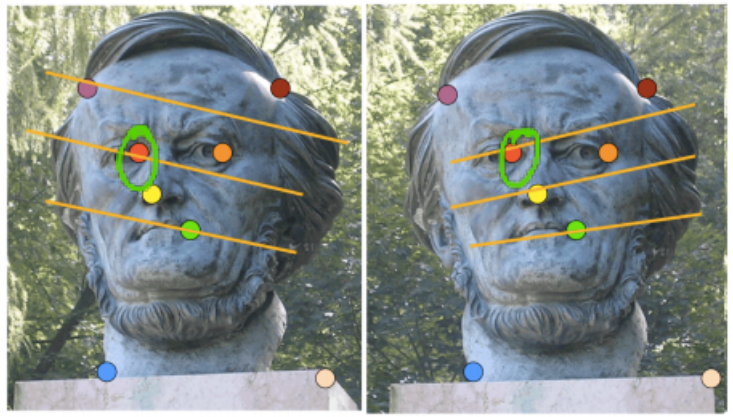
2. Distinctiveness: information carried by the patch surrounding the feature should be as distinctive as possible

Example Application of Corner Detector



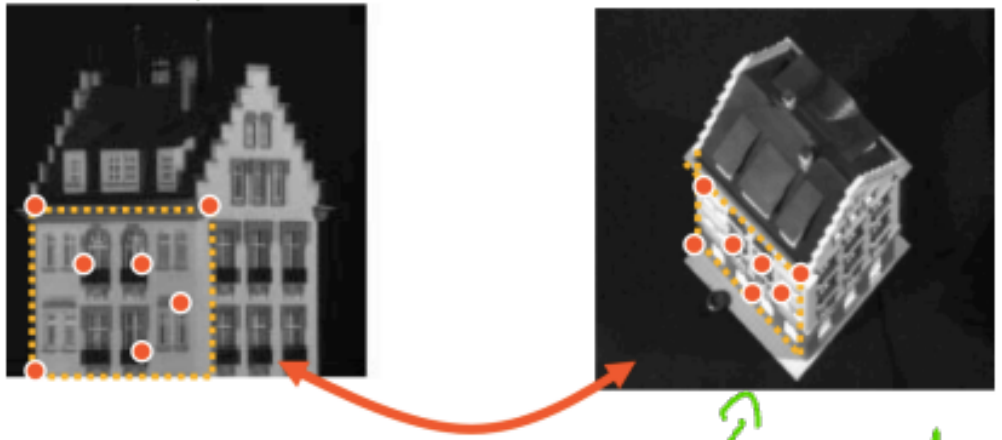
Use of detectors/descriptors: examples

Stereo reconstruction

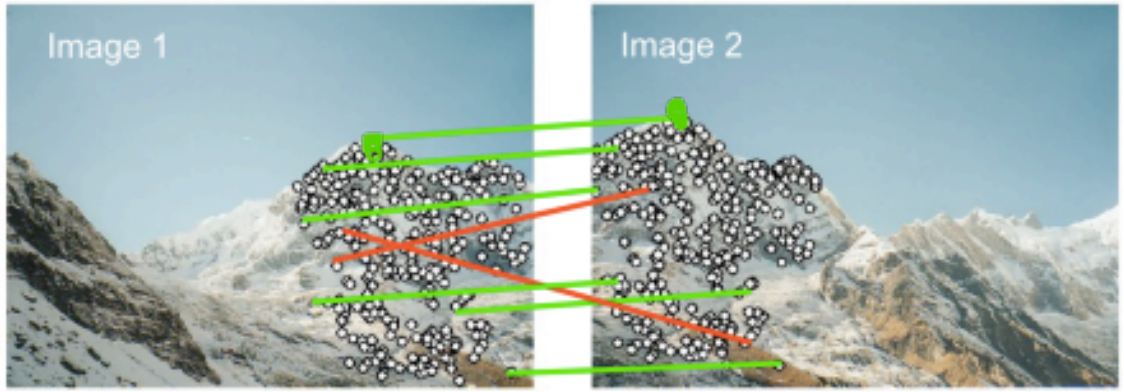


Estimating homographic transformations

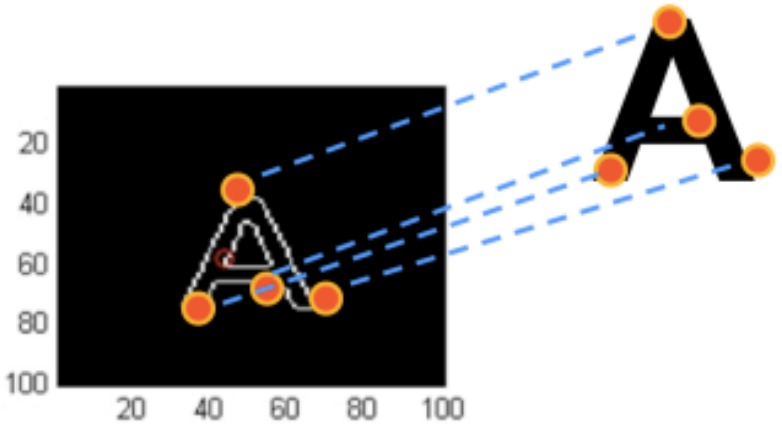
exploits that orange points are on a plane



Panorama stitching



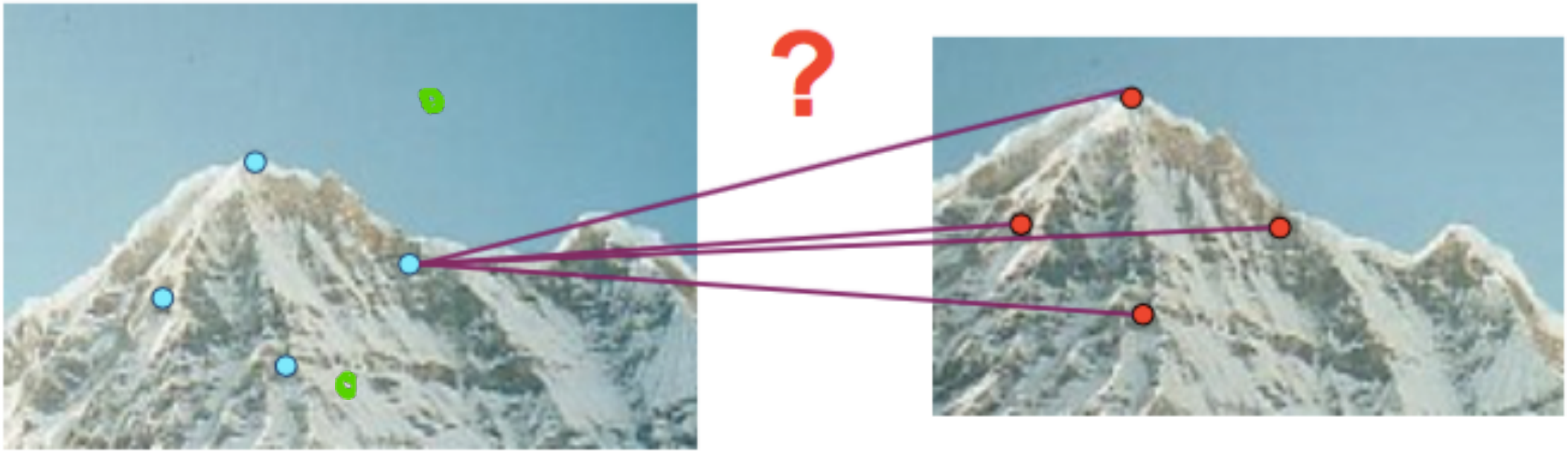
Object detection



Repeatability

Without repeatability, matching is impossible

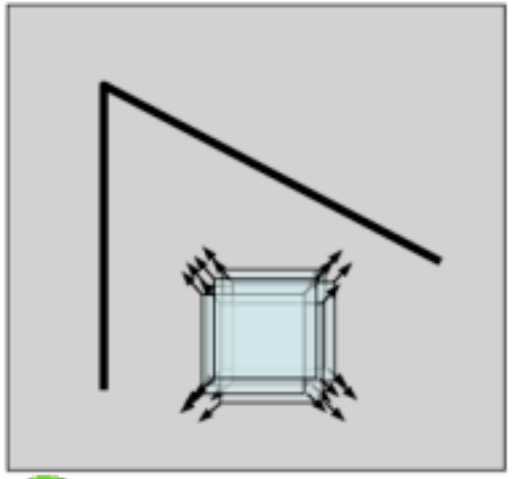
Distinctiveness



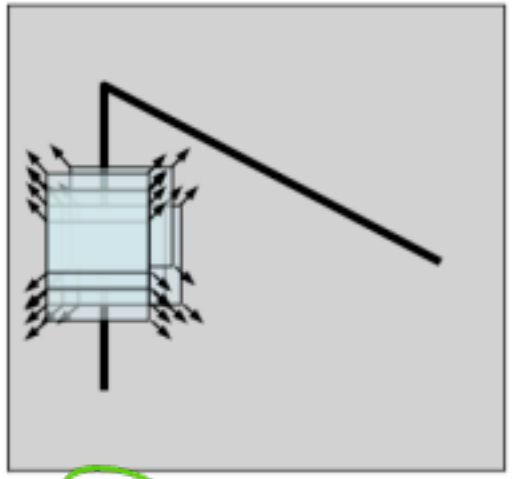
Without distinctiveness, it is not possible to establish reliable correspondences; distinctiveness is key for having a useful descriptor

Finding corners

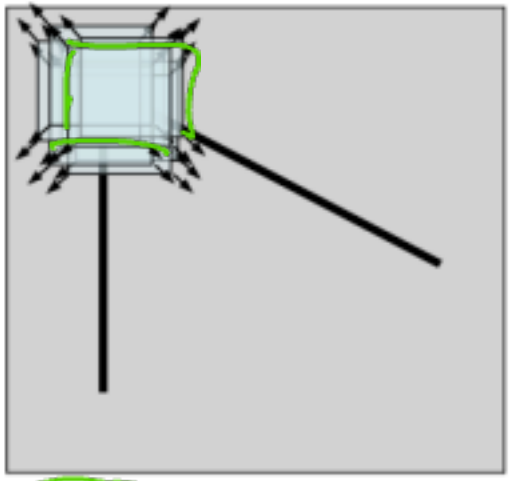
- Corner: intersection of two or more edges
- Geometric intuition for corner detection: explore how intensity changes as we shift a window



Flat: no changes in any direction



Edge: no change along the edge direction



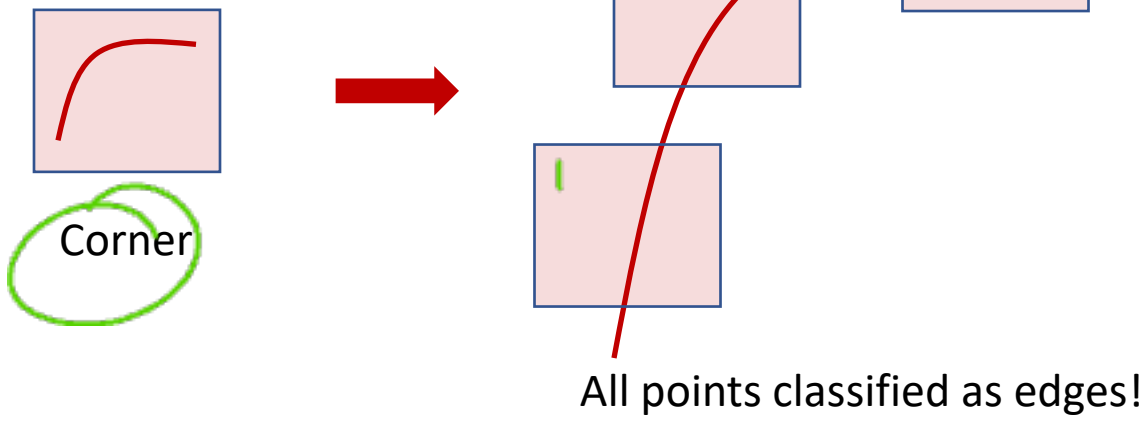
Corner: changes in all directions

Harris detector: example

Properties of Harris detectors

- Widely used
- Detection is invariant to
  - Rotation -> geometric invariance
  - Linear intensity changes -> photometric invariance

- Detection is not invariant to
  - Scale changes
  - Geometric affine changes

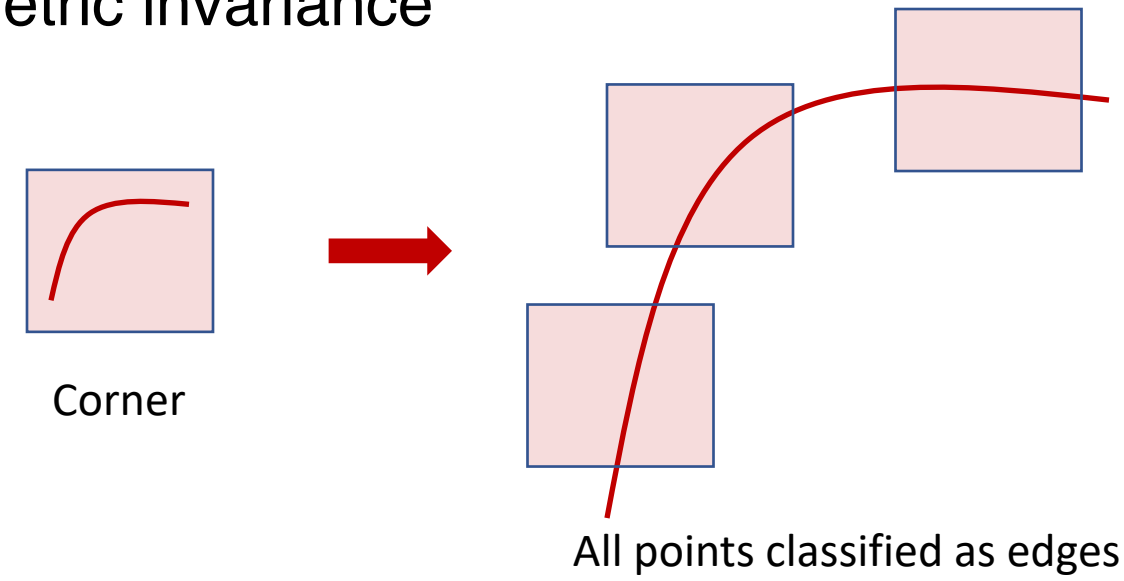


Properties of Harris detectors

SIFT – Scale Invariant Feature Transform

- Widely used
- Detection is invariant to
  - Rotation -> geometric invariance
  - Linear intensity changes -> photometric invariance

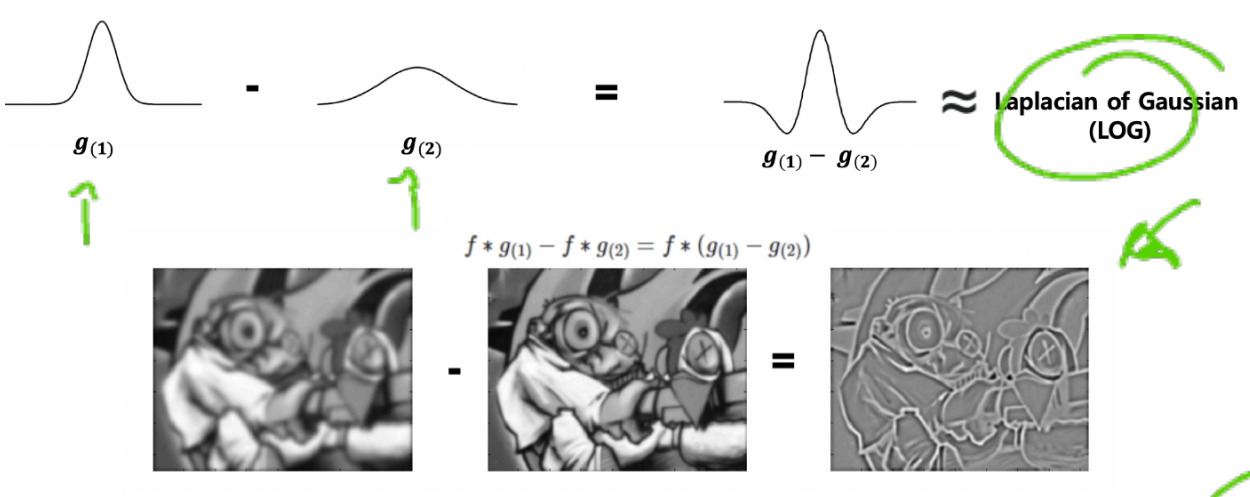
- Detection is not invariant to
  - Scale changes
  - Geometric affine changes



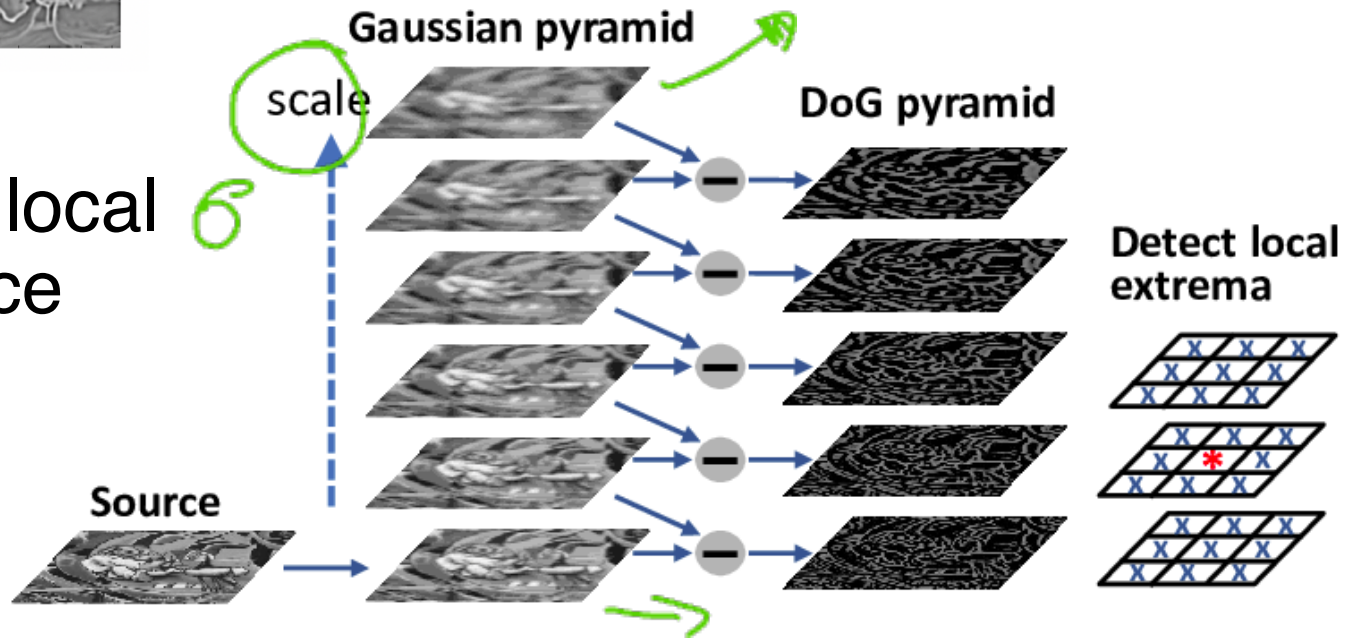
→ Scale-invariant detection, such as

1. Harris-Laplacian
2. in SIFT (specifically, Difference of Gaussians (DoG))

Difference of Gaussians (DoG)



- Features are detected as local extrema in scale and space

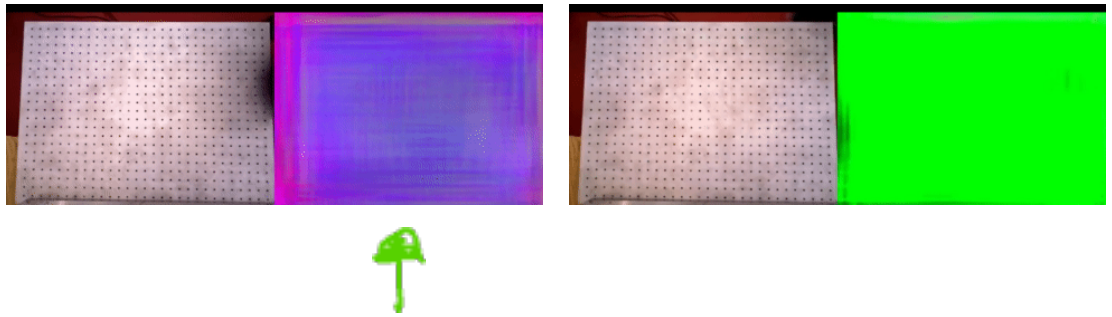


A case study for learning-based Descriptors Dense Object Nets

Learning *Dense* Visual Object *Descriptors*

By and *For* Robotic Manipulation. CORL 2018

Peter R. Florence, Lucas Manuelli, Russ Tedrake



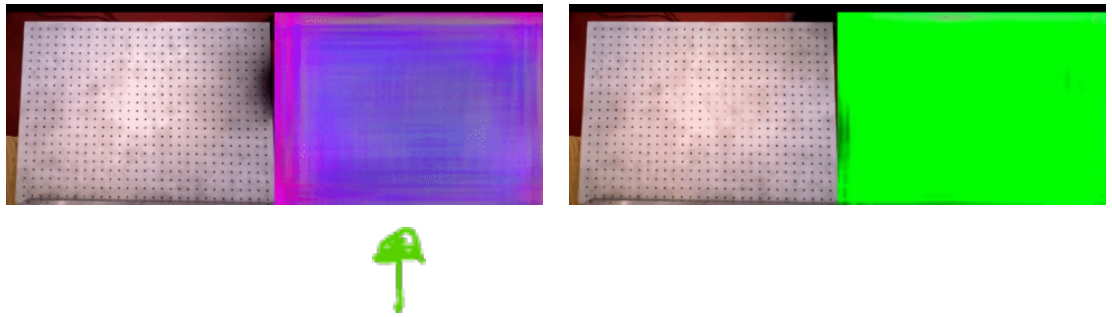
Slides adapted from CS326 by Kevin Zakka and Sriram Somasundaram

A case study for learning-based Descriptors Dense Object Nets

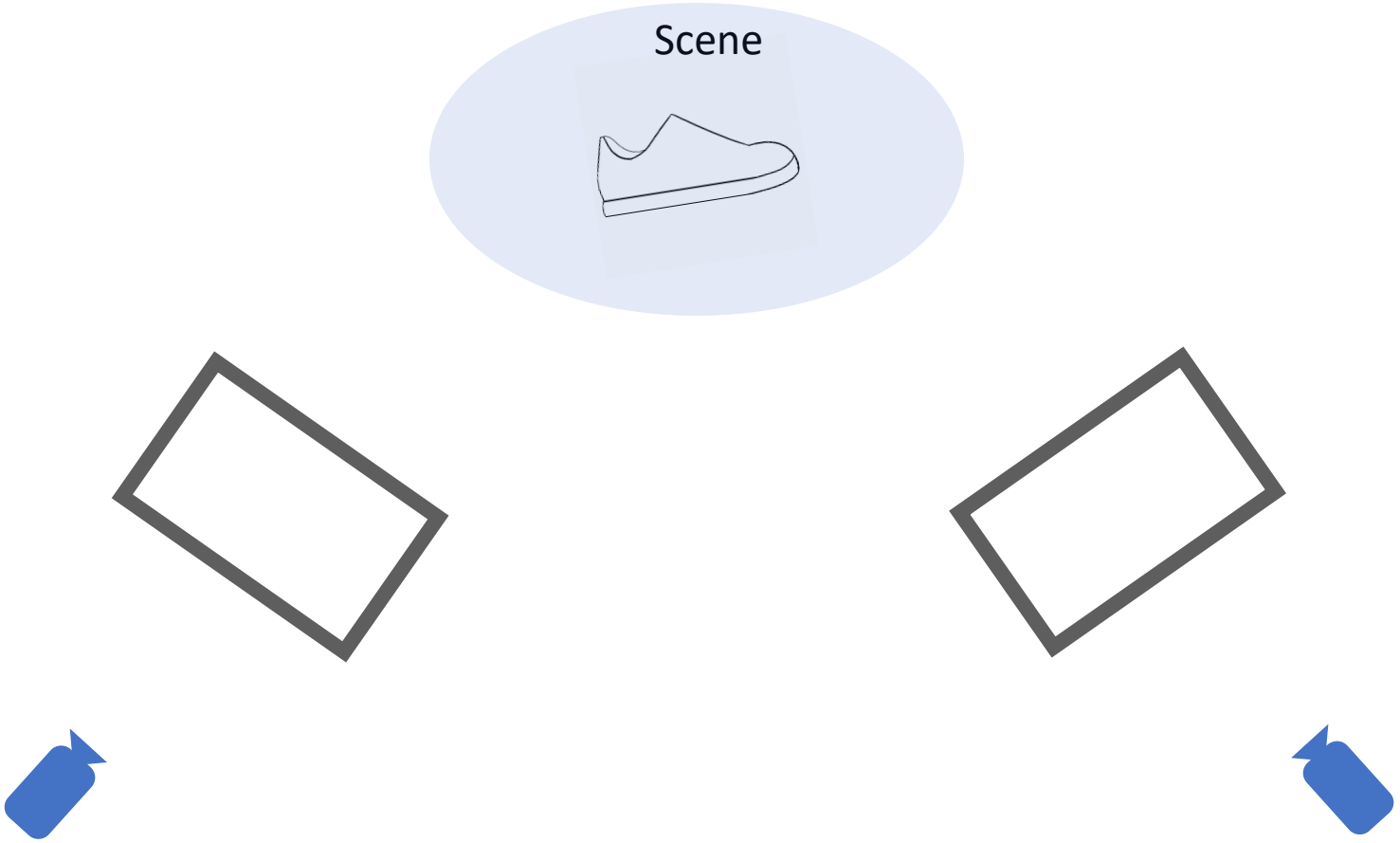
Learning *Dense* Visual Object *Descriptors*

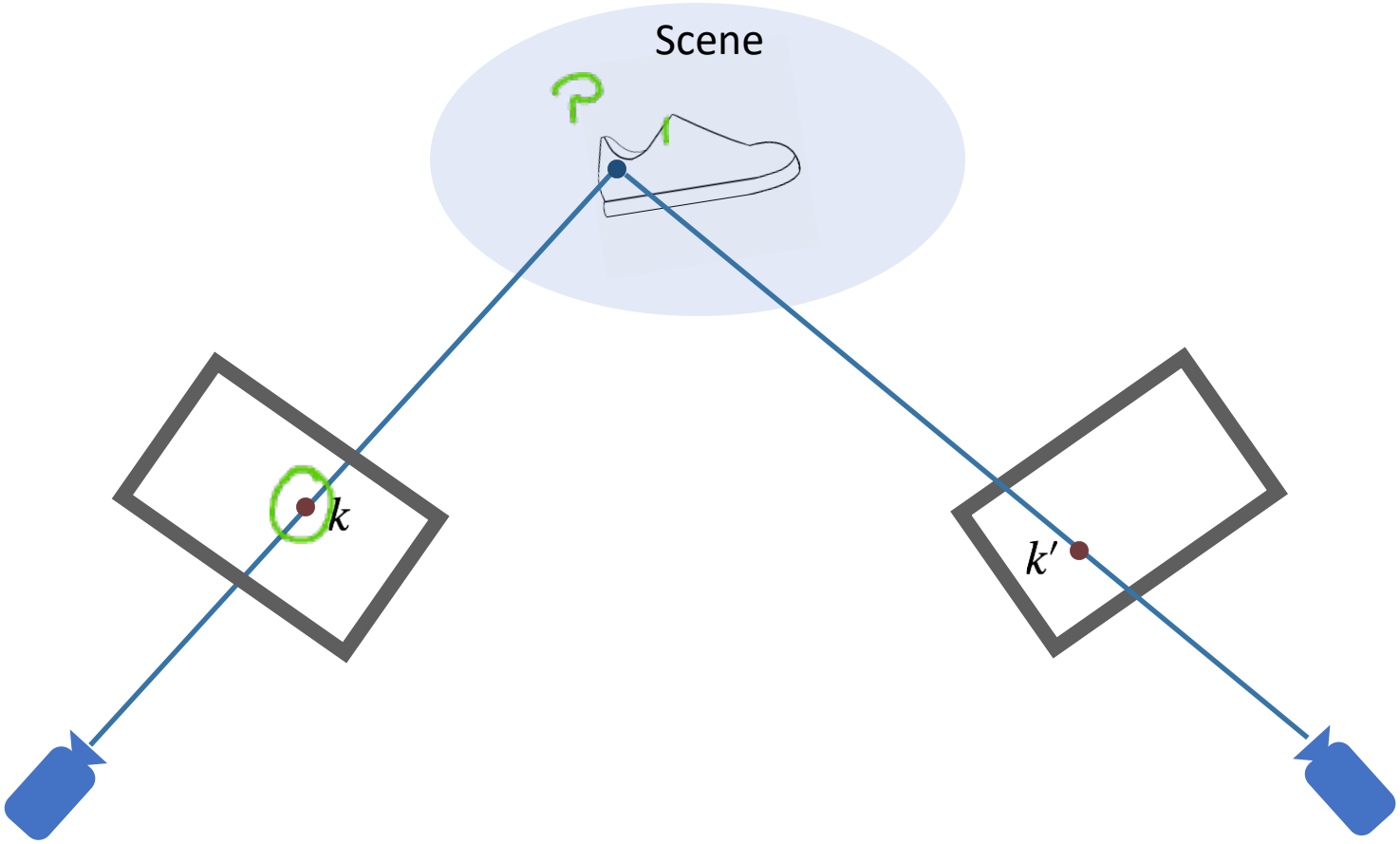
By and *For* Robotic Manipulation. CORL 2018

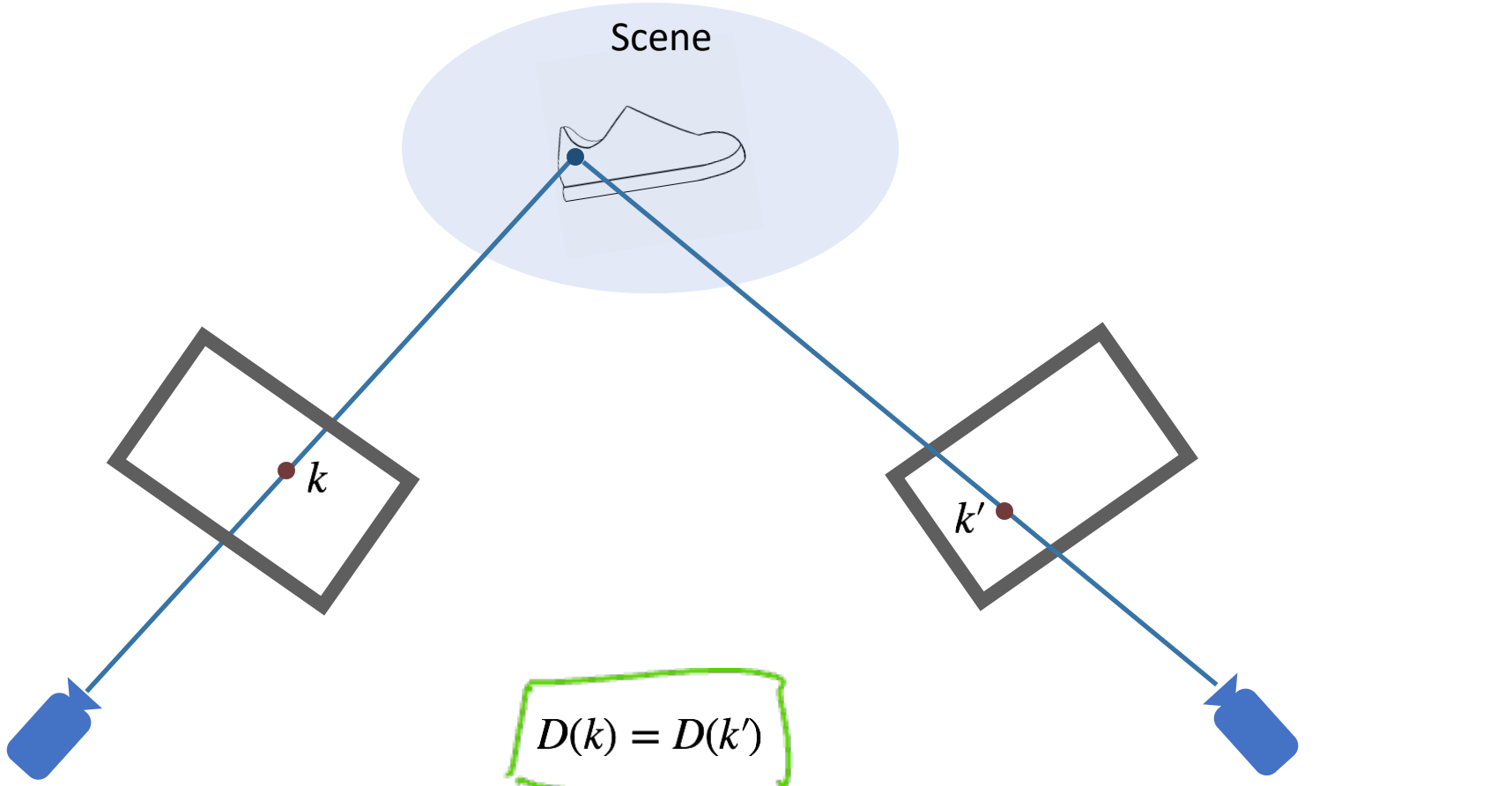
Peter R. Florence, Lucas Manuelli, Russ Tedrake



Slides adapted from CS326 by Kevin Zakka and Sriram Somasundaram


Scene

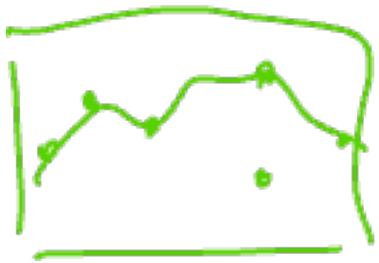

Scene
k
k′



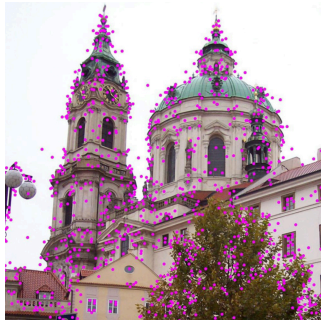
$$D(k) = D(k')$$

D = Descriptor(k) = D(k')

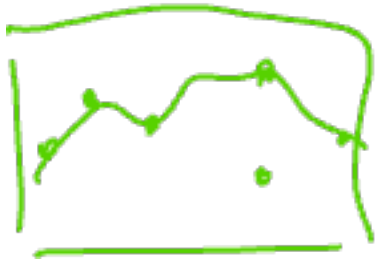
A Brief History



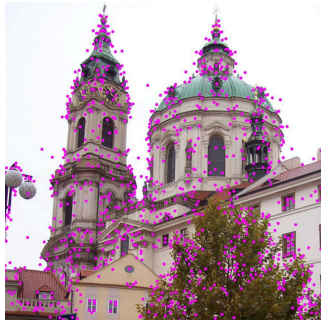
A Brief History



Sparse Engineered: SIFT

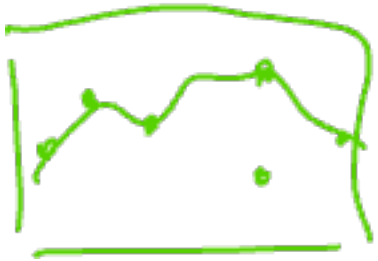


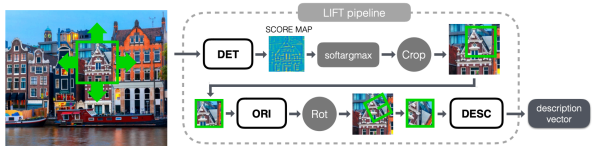
A Brief History



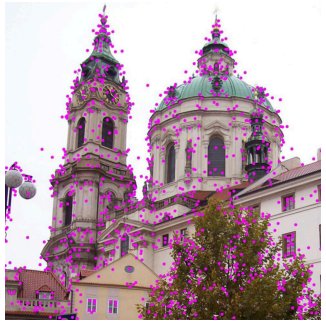
Sparse Engineered: SIFT

Sparse Learned: LIFT

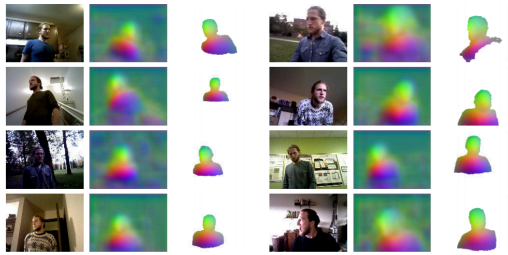




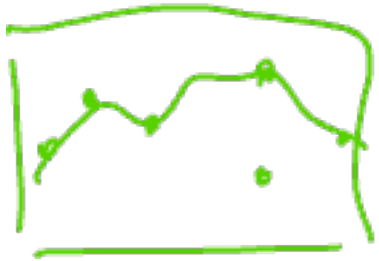
A Brief History



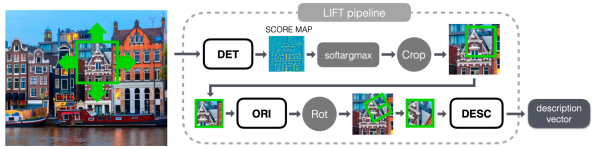
Sparse Engineered: SIFT



Dense Learned



Sparse Learned: LIFT

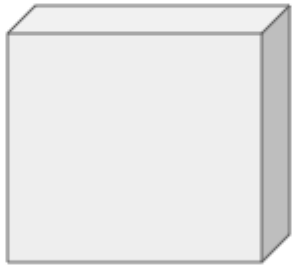


Why Dense?

Bachrach et. al.

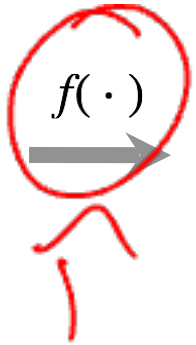
Dense Descriptors

Input is an RGB image

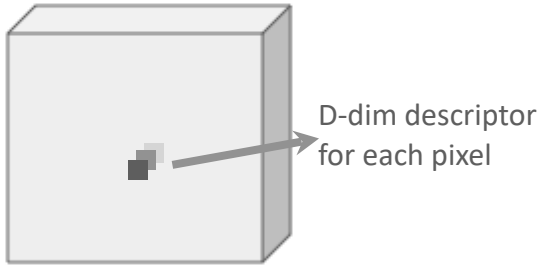


$\mathbb{R}^{W \times H \times 3}$

W width
H height
RGB → 3



Output

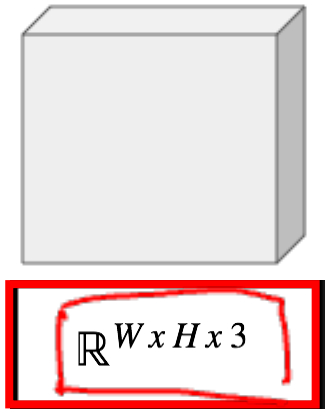


$\mathbb{R}^{W \times H \times D}$

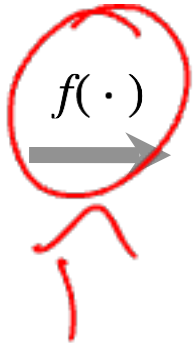
D = Dimension of Descriptor / Feature Vector

Dense Descriptors

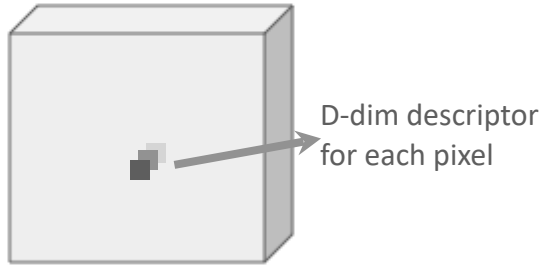
Input is an RGB image



W width
H height
RGB → 3



Output

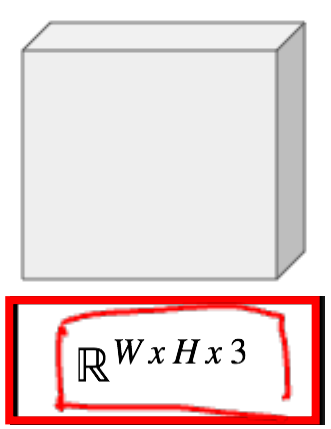


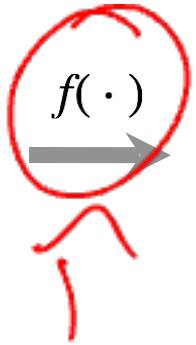
$\mathbb{R}^{W \times H \times D}$

D = Dimension of Descriptor / Feature Vector

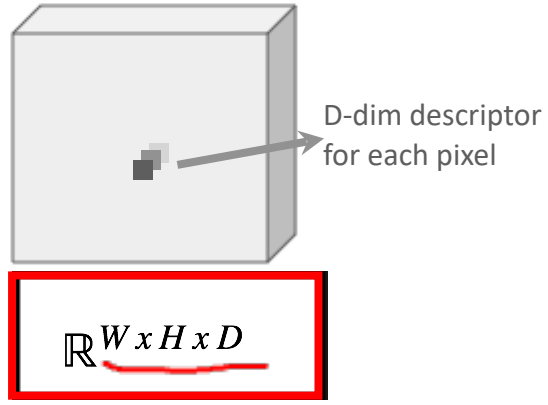
Dense Descriptors

Input is an RGB image





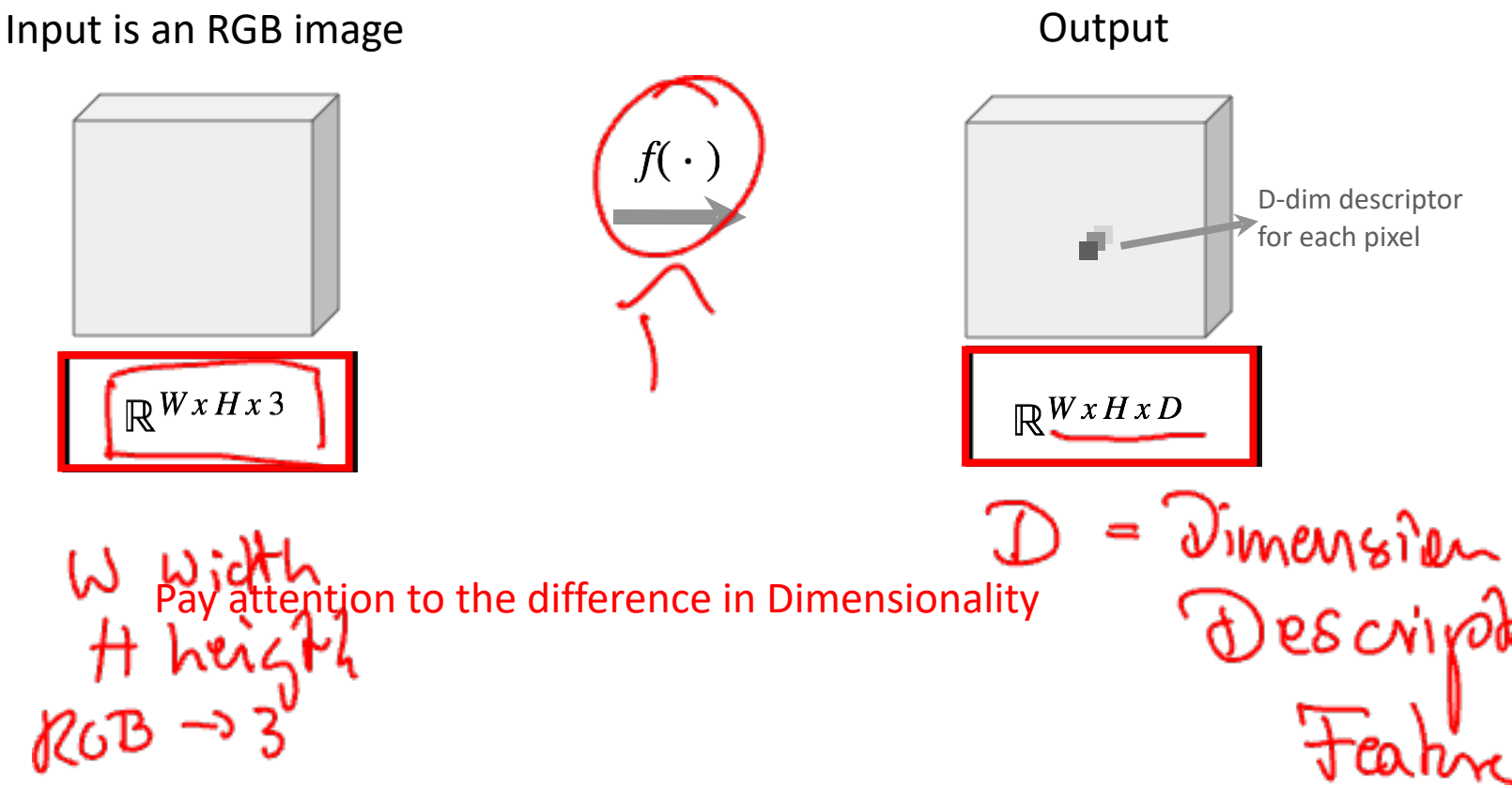
Output



W width
H height
RGB → 3

D = Dimension of Descriptor / Feature Vector

Dense Descriptors



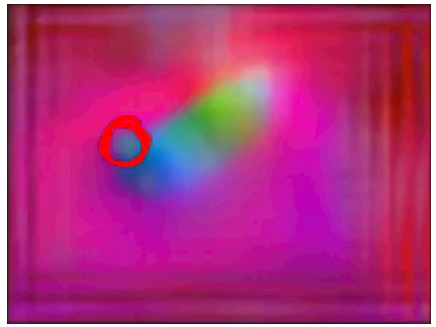
Dense Descriptors

Input is an RGB image

$\mathbb{R}^{W \times H \times 3}$

$f(\cdot)$

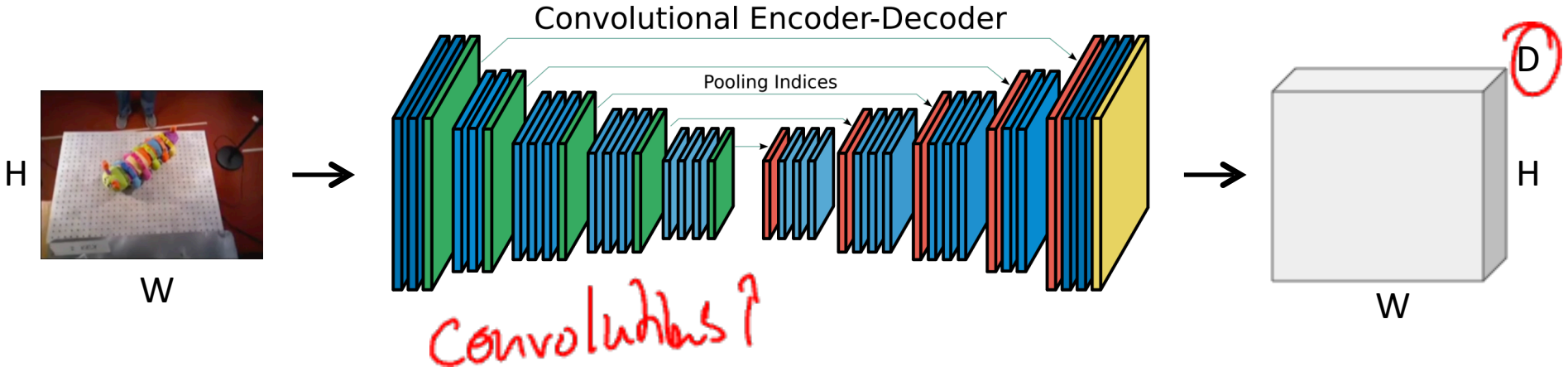
Output



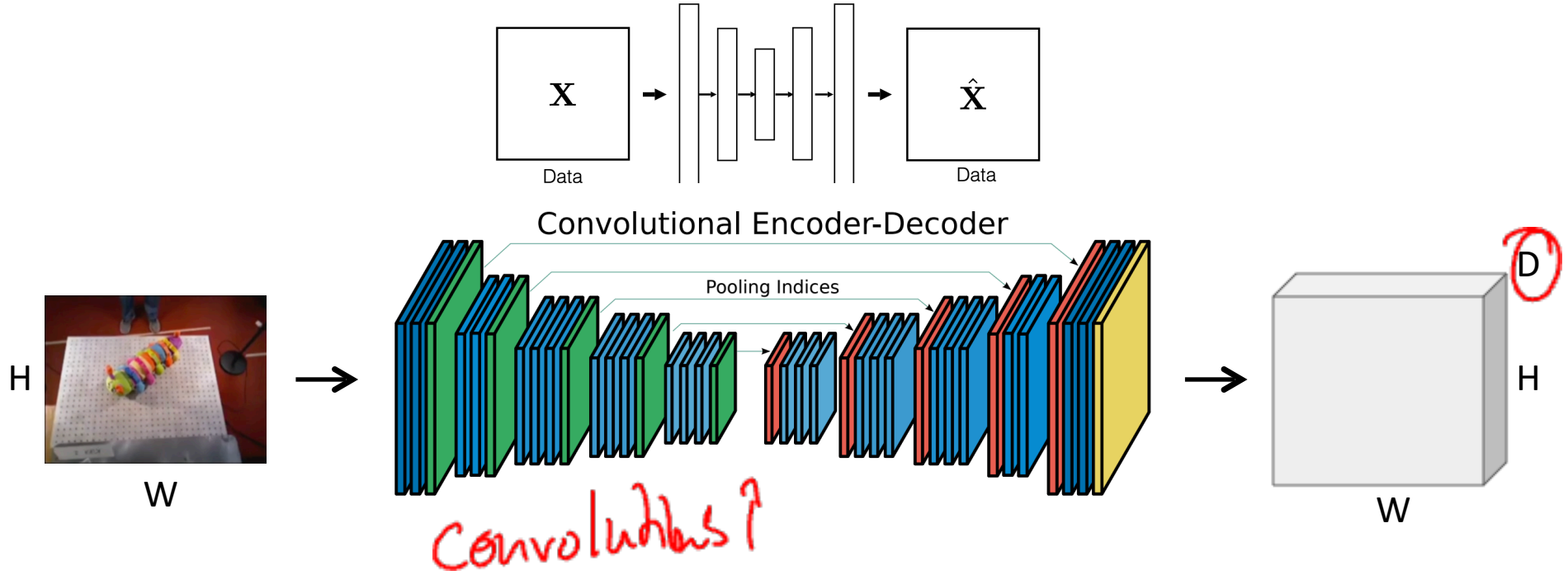
$\mathbb{R}^{W \times H \times D}$

D → PCA compress to 3 dim
remap to RGB

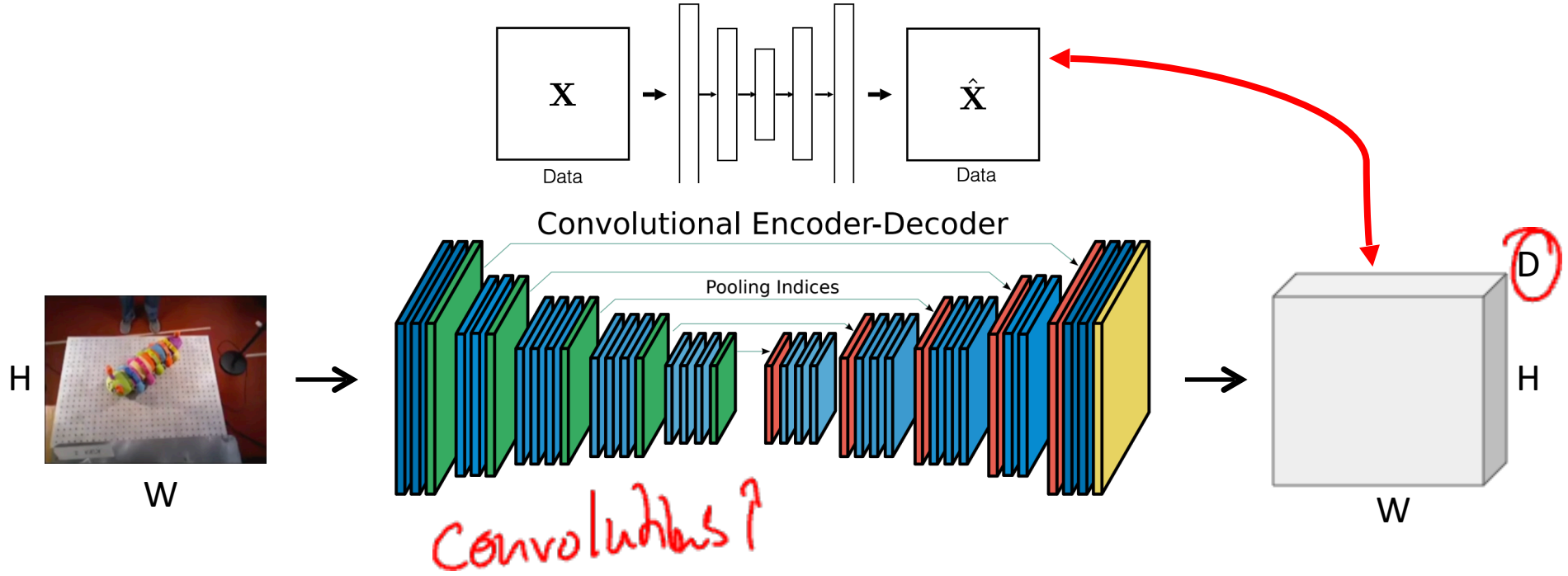
Network Architecture



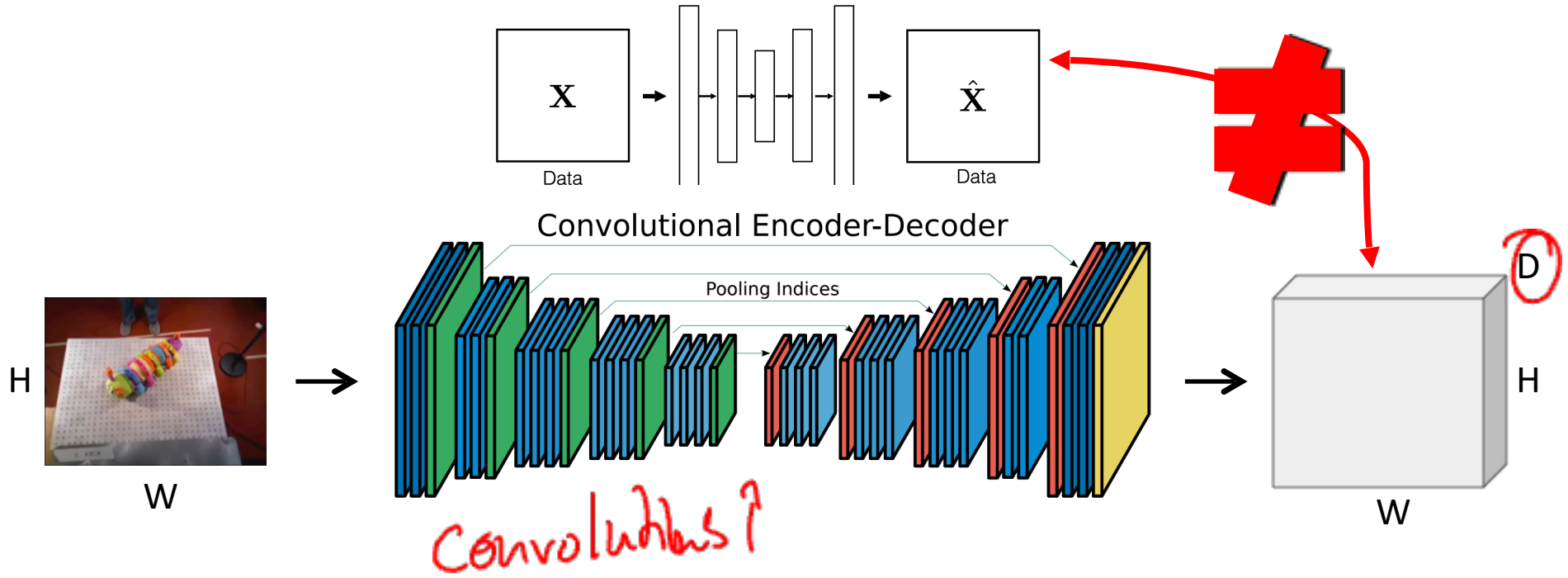
Network Architecture



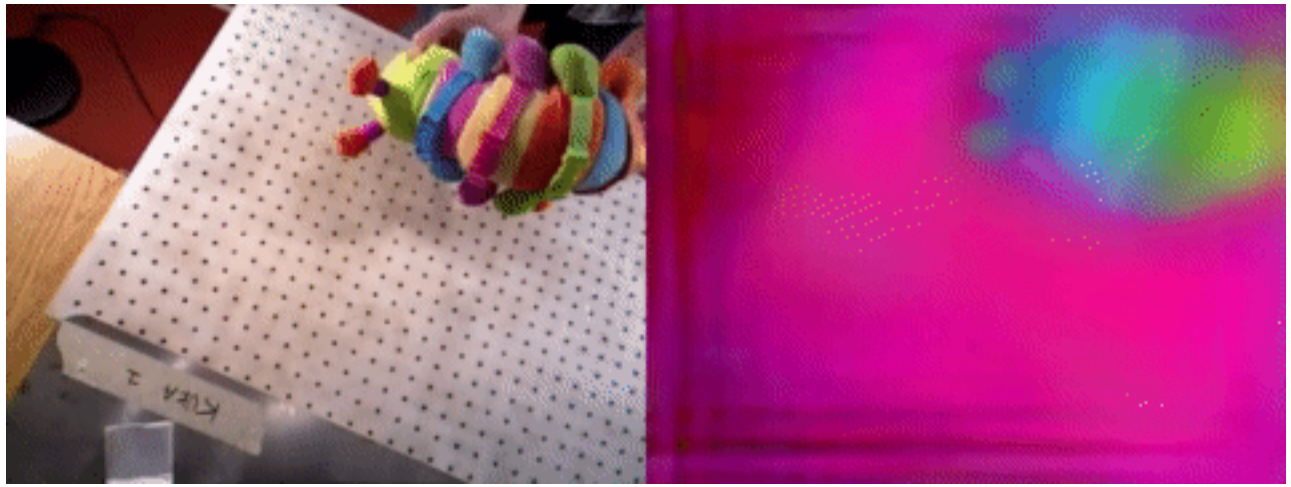
Network Architecture



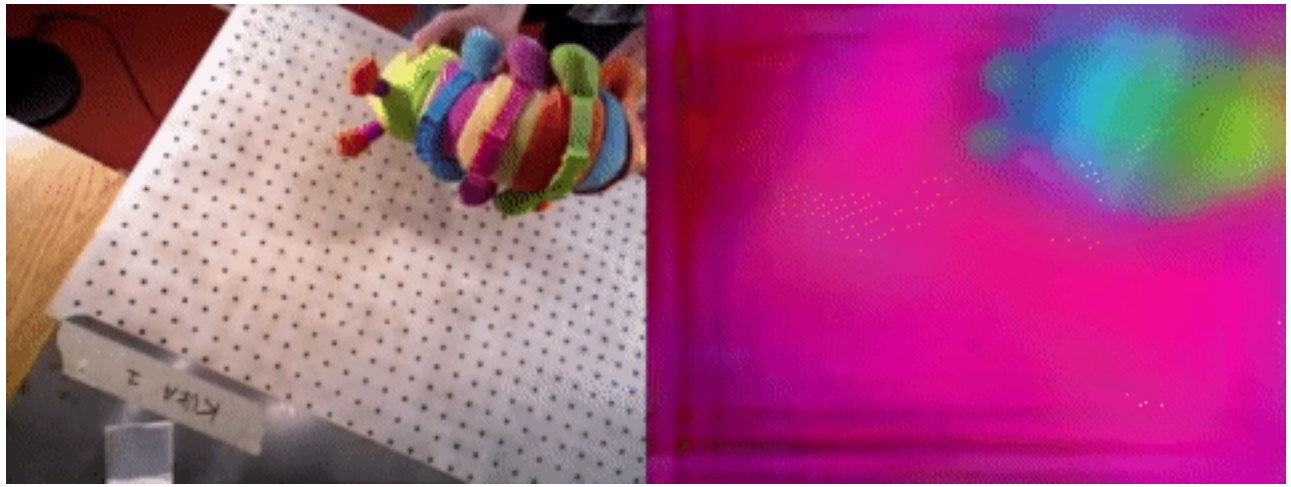
Network Architecture



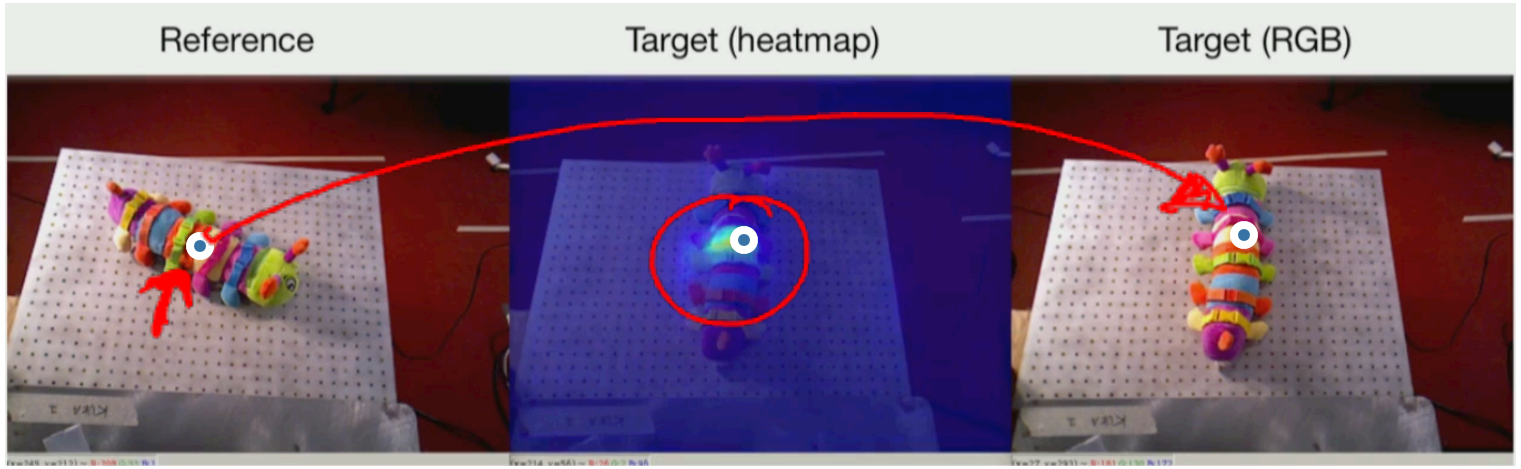
Single Object



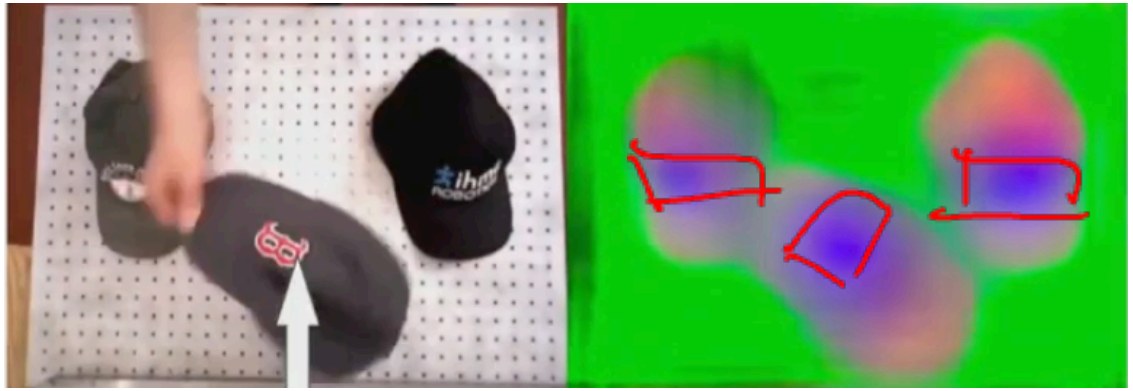
Single Object



Learned Dense Correspondences



Class consistent descriptors



Hough transform

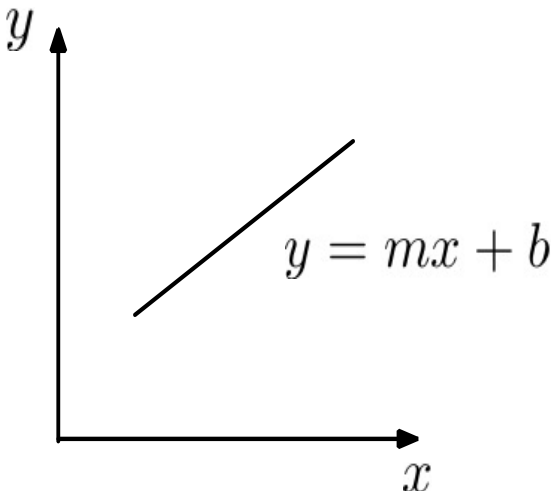
- Key idea: each point votes for a *set* of plausible line parameters

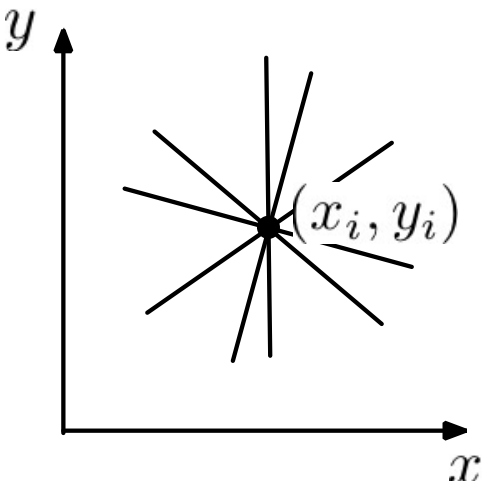
- A line has two parameters: (m, b)

- Given a point $(x_i,\ y_i)$, the lines that could pass through this point are all (m, b) satisfying

$$y_i = mx_i + b, \quad \text{or } b = -mx_i + y_i$$

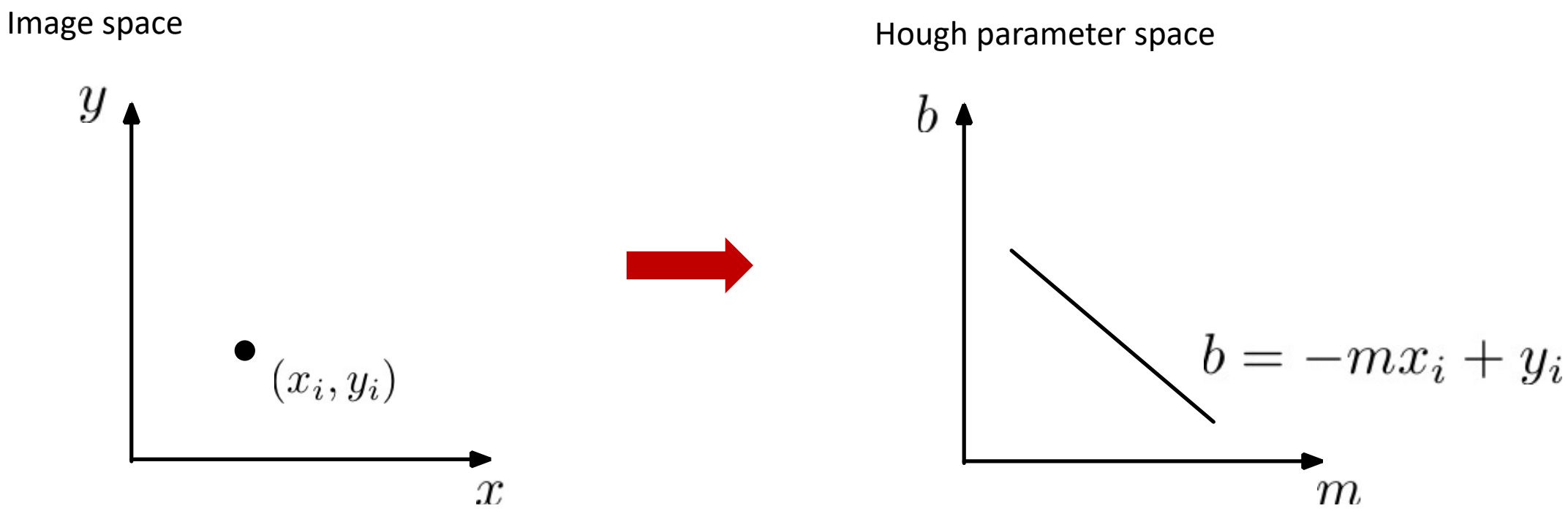
slope y intersection





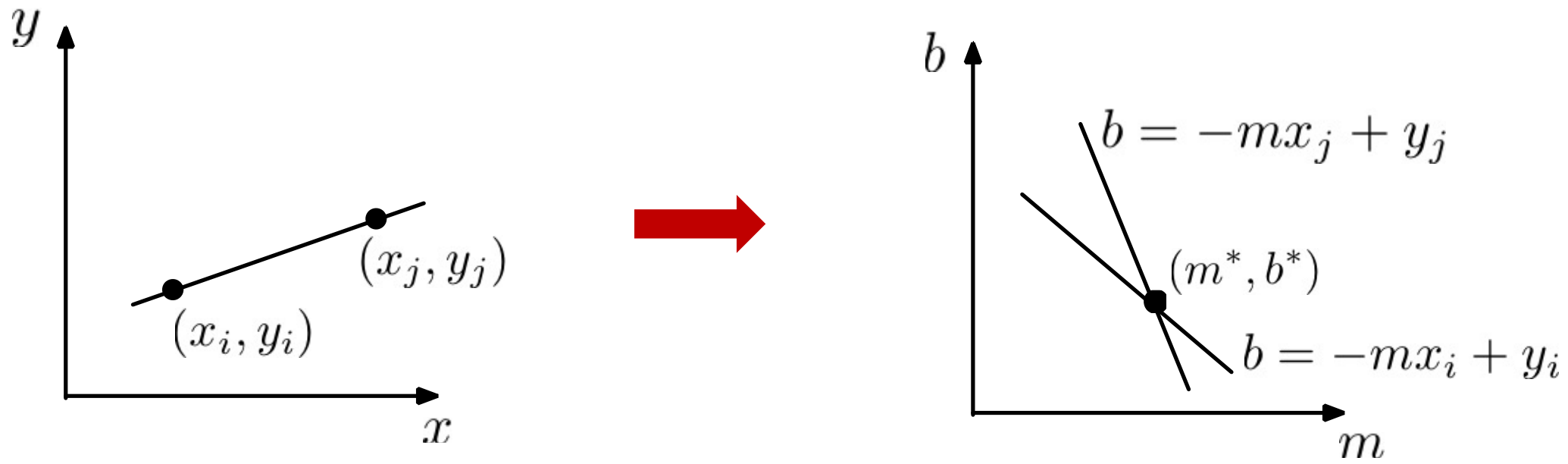
Hough transform

- A point in image space maps into a line in *Hough space*



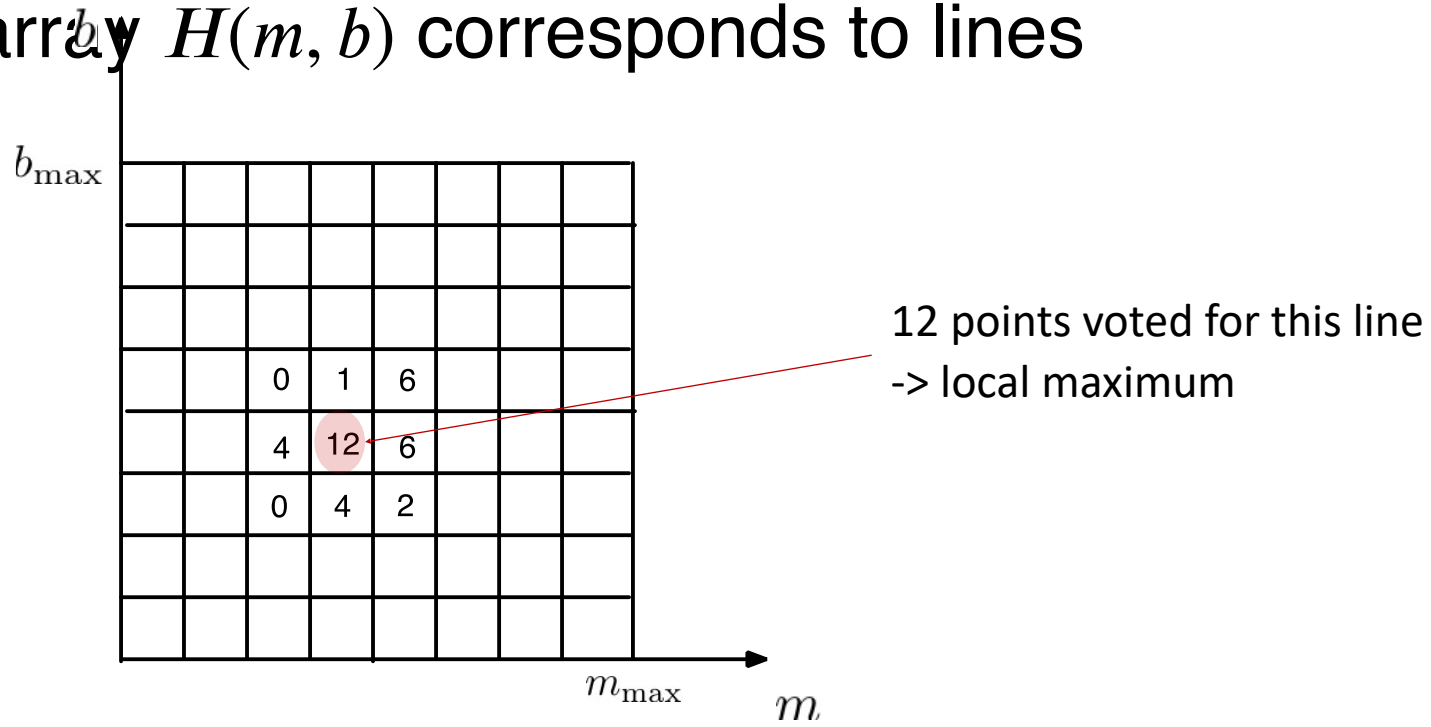
Hough transform

- Key fact: all points on a line in image space yield lines in parameter space which intersect at a *common point*, (m^*, b^*)



Hough transform algorithm

1. Initialize an accumulator array $H(m, b)$ to zero
2. For each point $(x_i,\ y_i)$, increment all cells that satisfy $b = -x_i m + y_i$
3. Local maxima in array $H(m, b)$ corresponds to lines

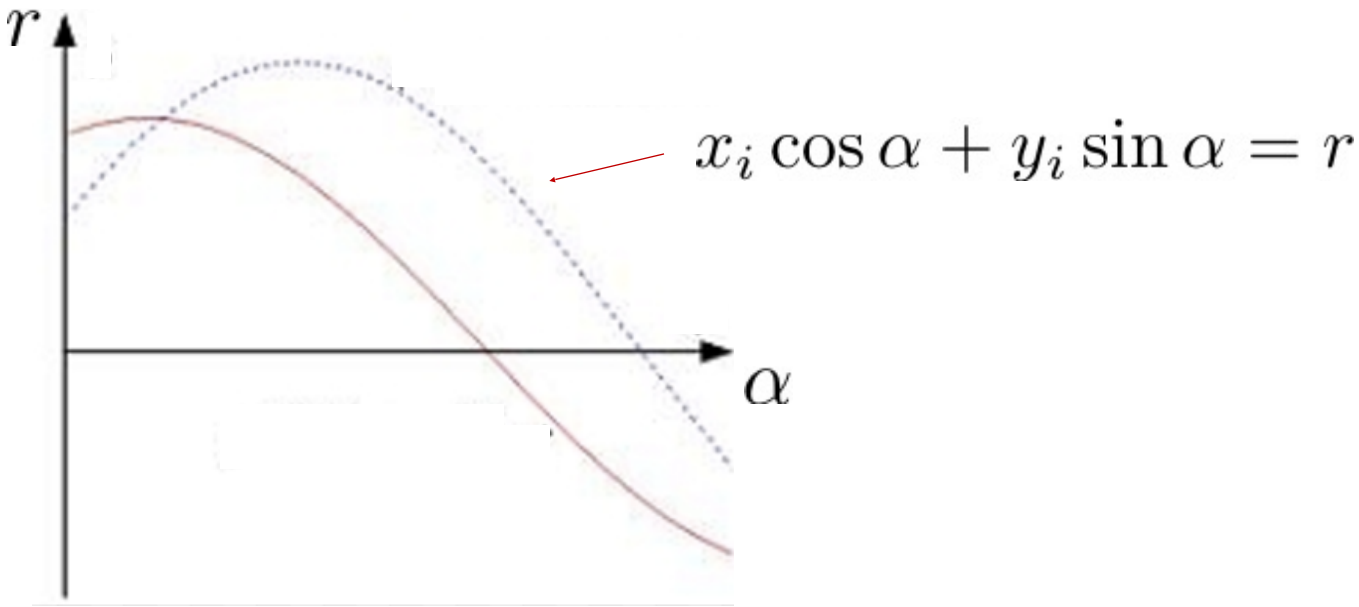


Hough transform algorithm: polar coordinate representation

- Equation of a line in polar coordinates

$$x \cos \alpha + y \sin \alpha = r$$

- The parameter space transform of a point is a sinusoidal curve



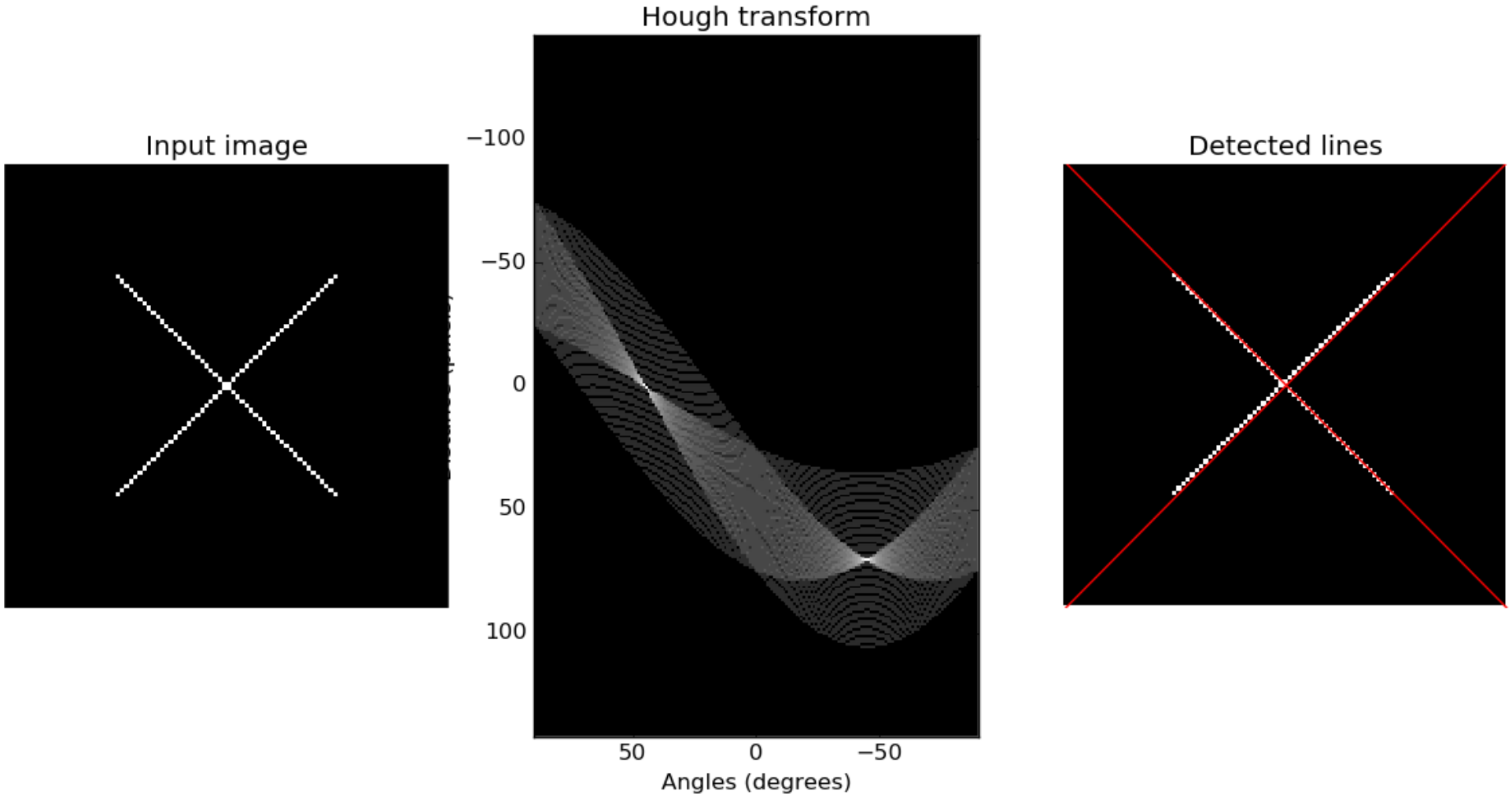
$$x_i \cos \alpha + y_i \sin \alpha = r$$

- Avoids infinite slope
- Constant resolution

Hough transform algorithm, revised

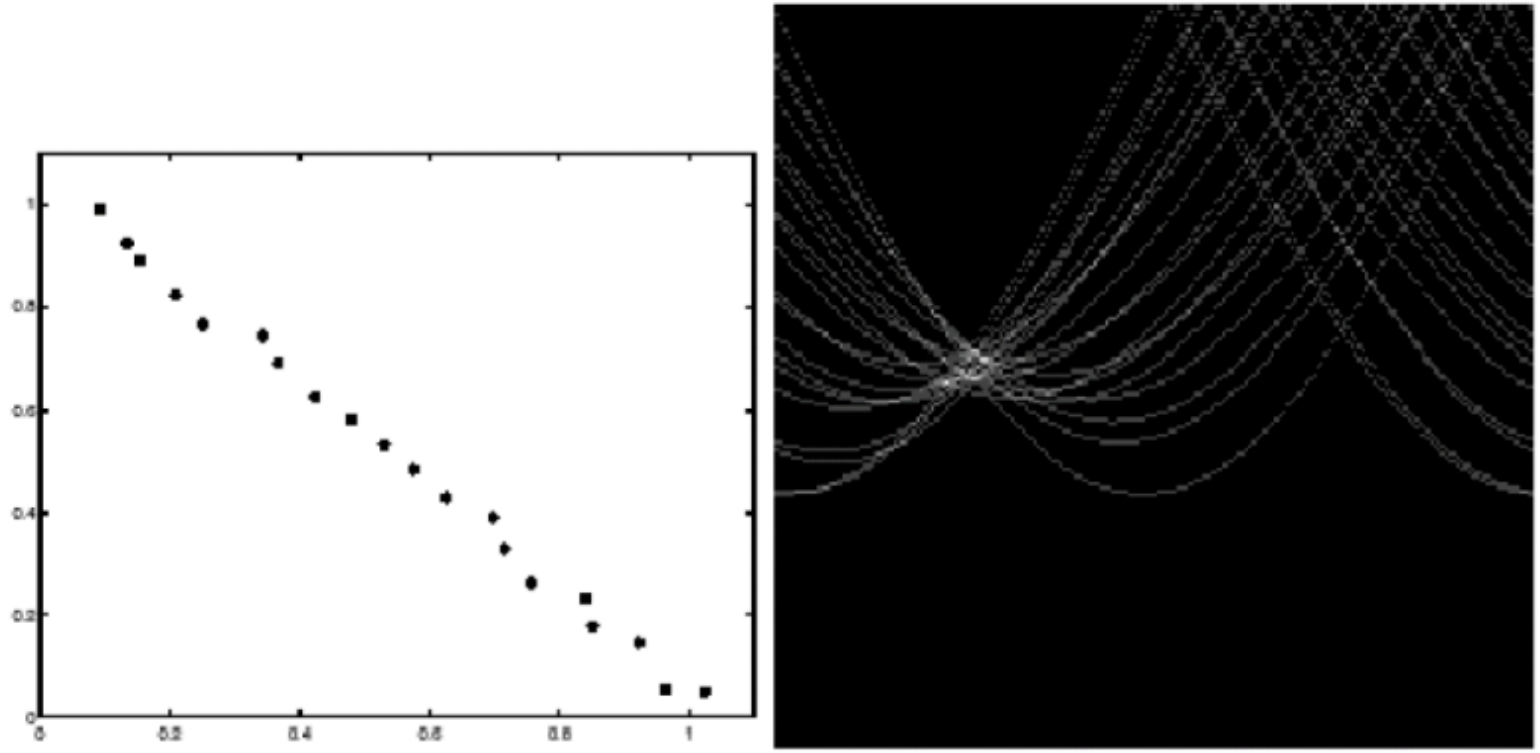
Data: Set S containing N points
Result: Line fitting the points in S
Initialize $n_\alpha \times n_r$ accumulator H with zeros;
foreach $(x_i, y_i) \in S$ **do**
  foreach $\alpha \in \{\alpha_1, \ldots, \alpha_{n_\alpha}\}$ **do**
    compute $r = x_i \cos\alpha + y_i \sin\alpha$;
    $H[\alpha, r] \leftarrow H[\alpha, r] + 1$;
  end
end
Choose (α^*, r^*) that corresponds to largest count in H;
Return line defined by (α^*, r^*)

Hough transform: example



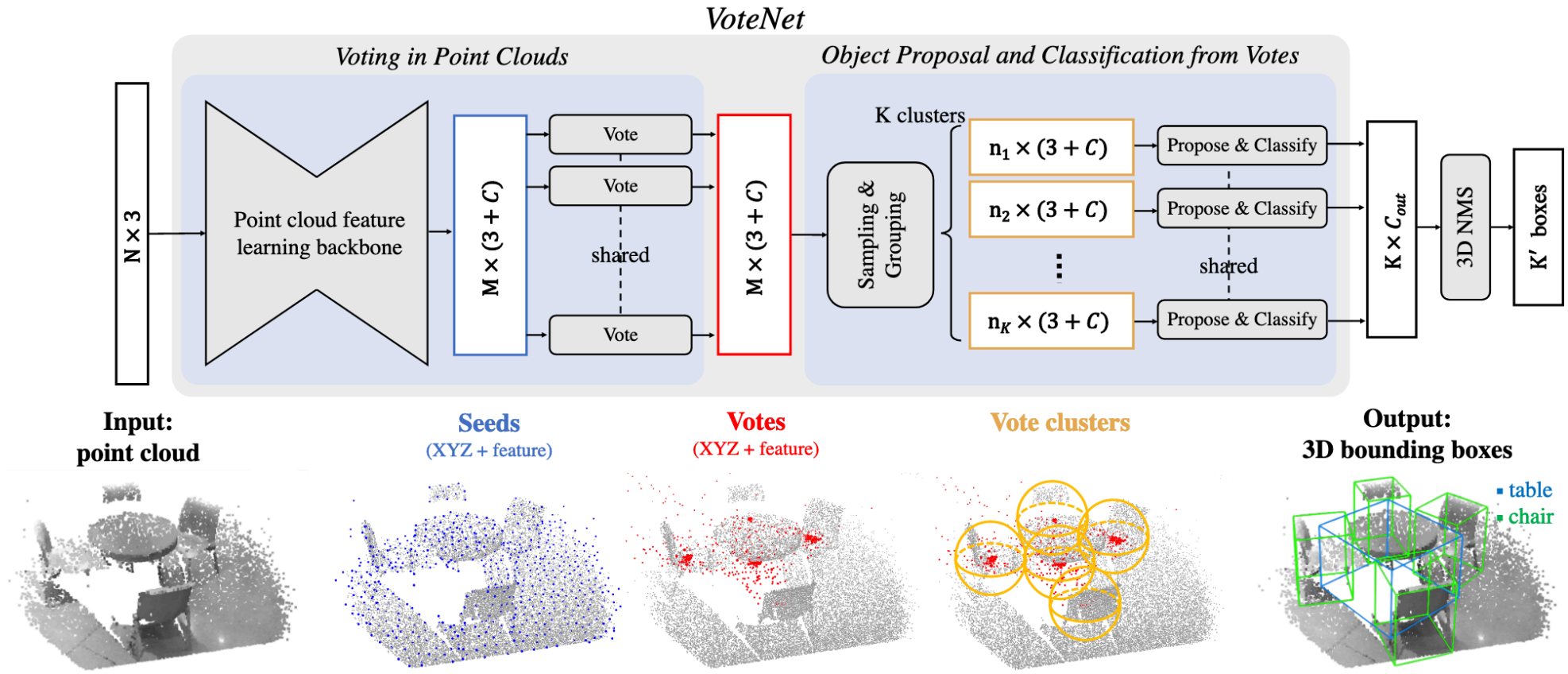
Hough transform: example

- With noise, peaks may be hard to detect



Deep Hough Voting for 3D Object Detection in Point Clouds

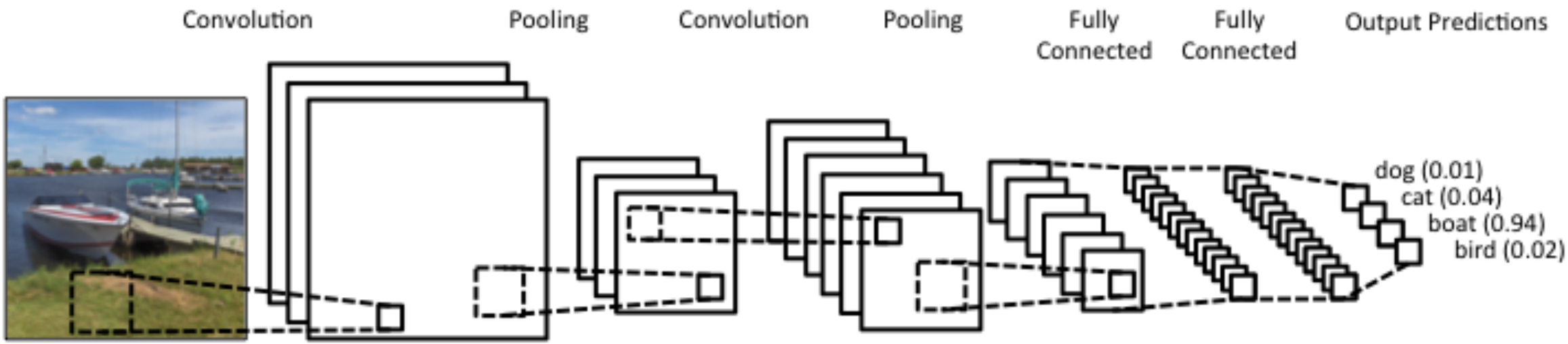
Charles R. Qi[1] Or Litany[1] Kaiming He[1] Leonidas J. Guibas[1,2]
[1]Facebook AI Research [2]Stanford University



Object recognition

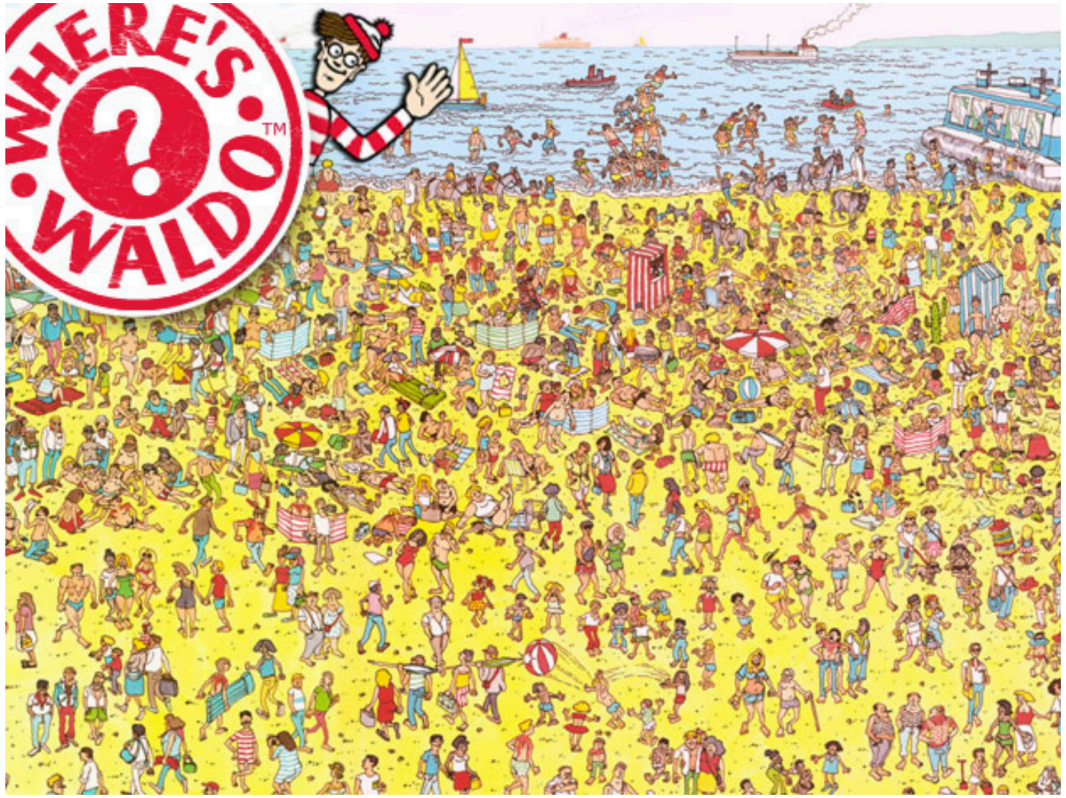
- Object recognition: capability of naming discrete objects in the world
- Why is it hard? Many reasons, including:
  1. Real world is made of a jumble of objects, which all occlude one another and appear in different poses
  2. There is a lot of variability intrinsic within each class (e.g., dogs)
- In this class, we will look at three methods:
  1. Template matching
  2. Bag of visual words
  3. Neural network methods (treated as a black box, take AA274B for details)

Standard paradigm - CNNs for recognition



Template matching

- How can we find Waldo?



Source: Sanja Fidler

Template matching

- Slide and compare!

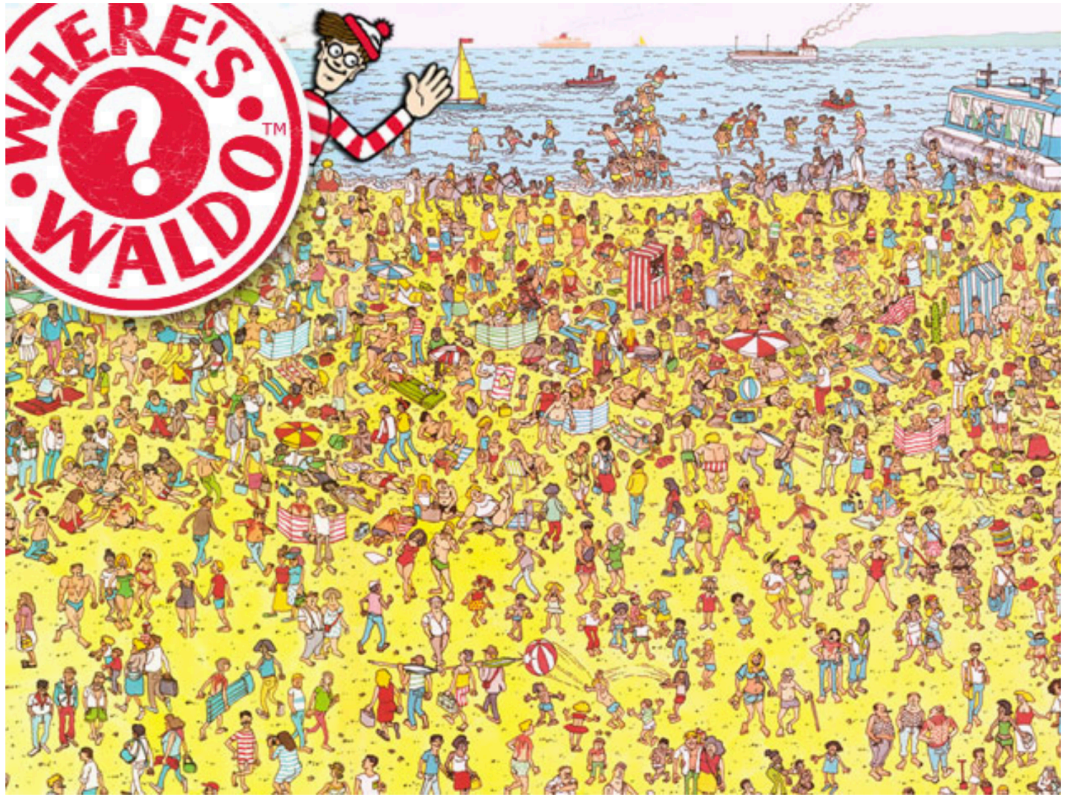


Image I

Filter F

Source: Sanja Fidler

Template matching

- Slide and compare!

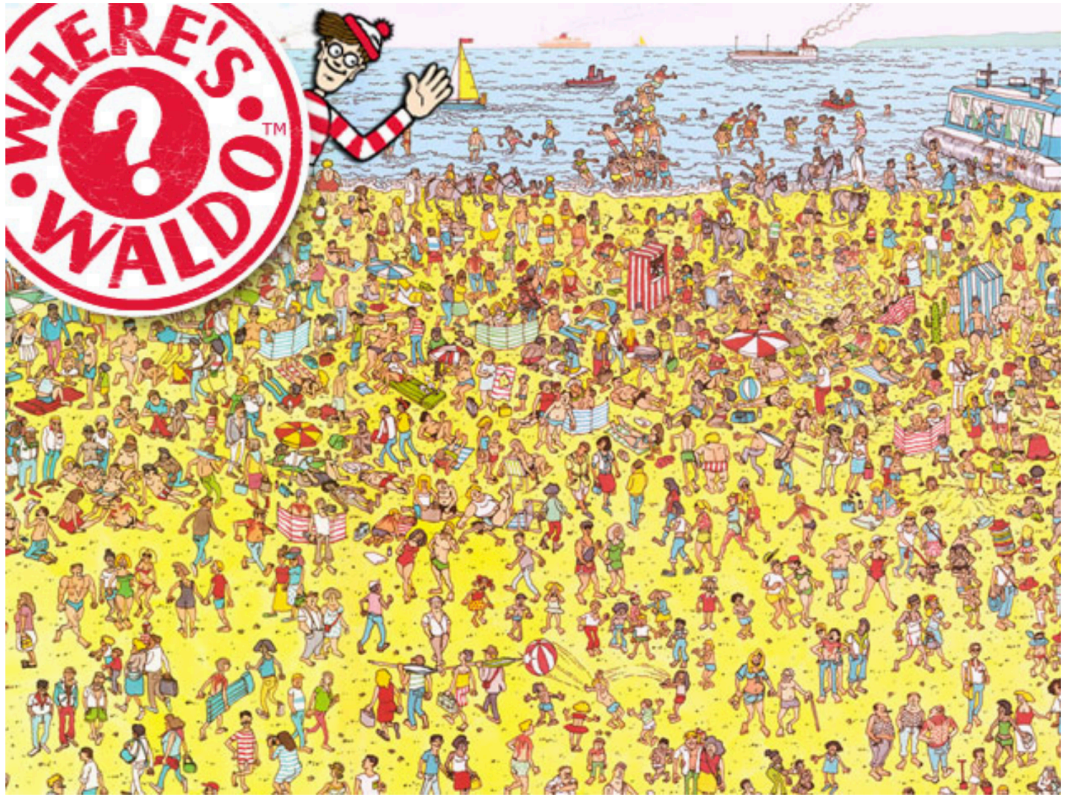
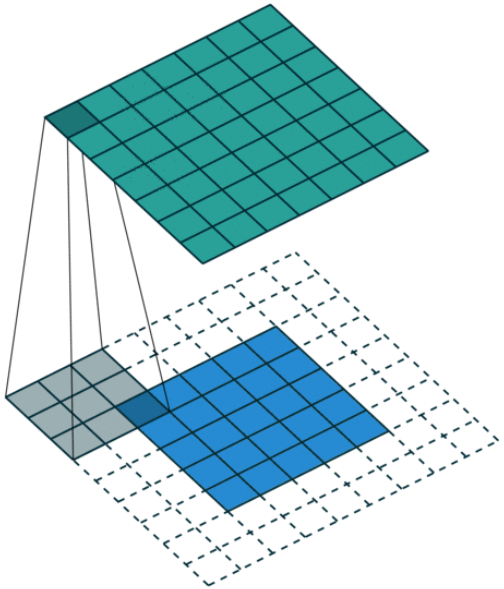


Image I



Filter F

Source: Sanja Fidler

Template matching

- In practice, remember correlation:

$$I'(x,y) = F \circ I = \sum_{i=-N}^{N} \sum_{j=-M}^{M} F(i,j) I(x+i, y+j)$$

- One can equivalently write: $I'(x,y) = \mathbf{f}^{\mathrm{T}} \cdot \mathbf{t}_{\mathrm{ij}}$

Vector representation of filter

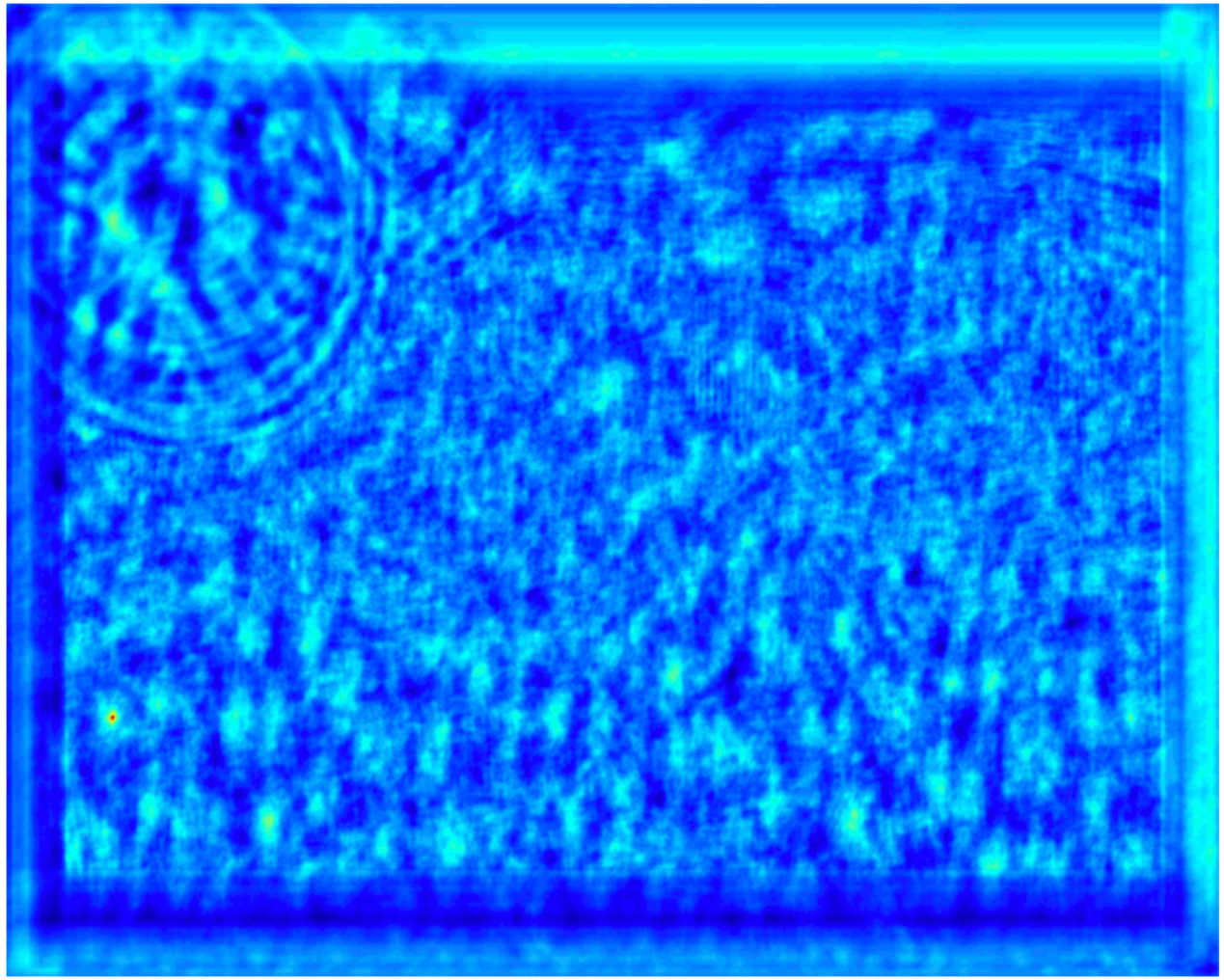
Vector representation of neighborhood patch

- To ensure that perfect matching yields one, we consider *normalized* correlation, that is

$$I'(x,y) = \frac{\mathbf{f}^{\mathrm{T}} \cdot \mathbf{t}_{\mathrm{ij}}}{\|\mathbf{f}\| \|\mathbf{t}_{\mathrm{ij}}\|}$$

Template matching

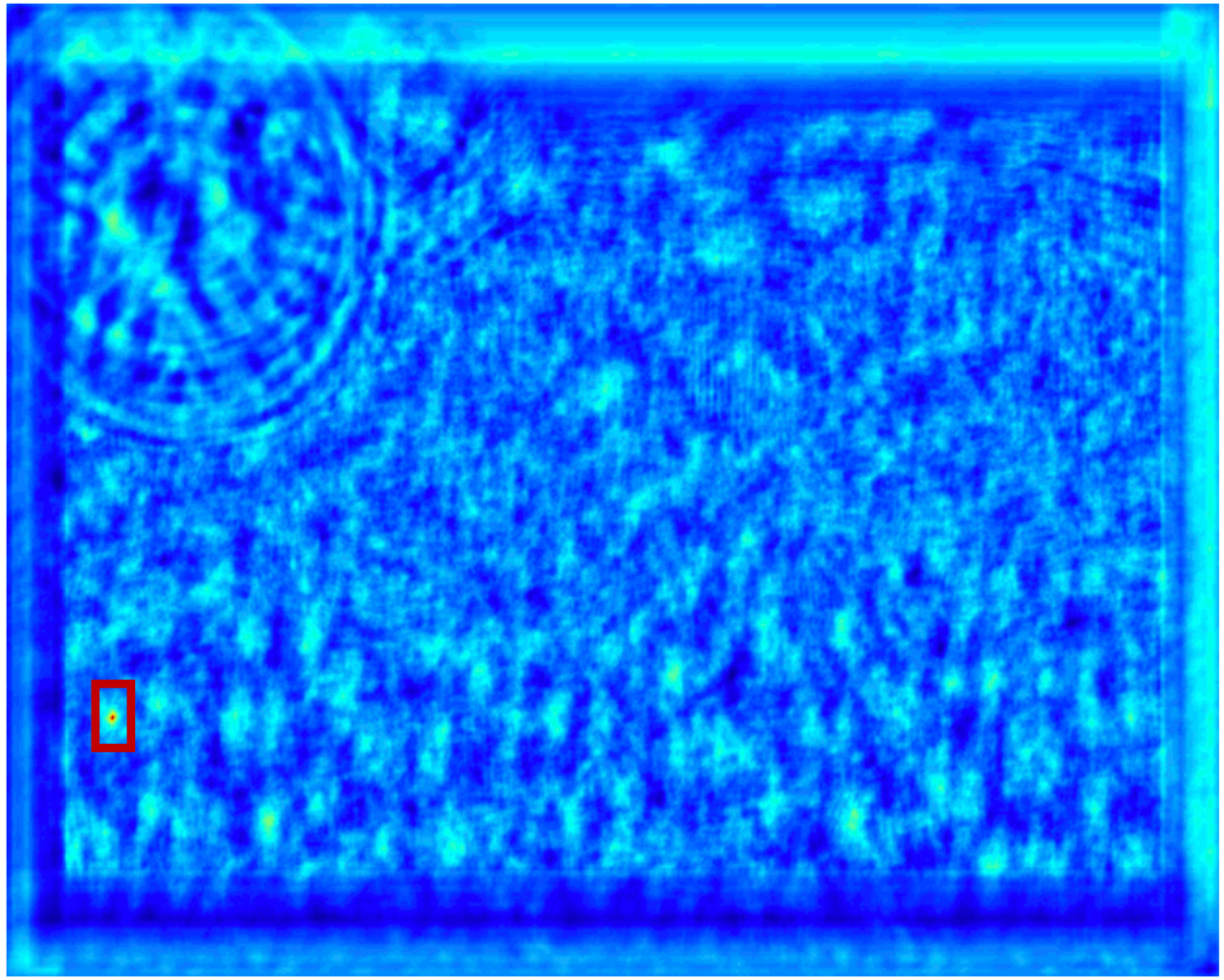
Result:



Source: Sanja Fidler

Template matching

Result:



Source: Sanja Fidler

Correlation versus Convolution

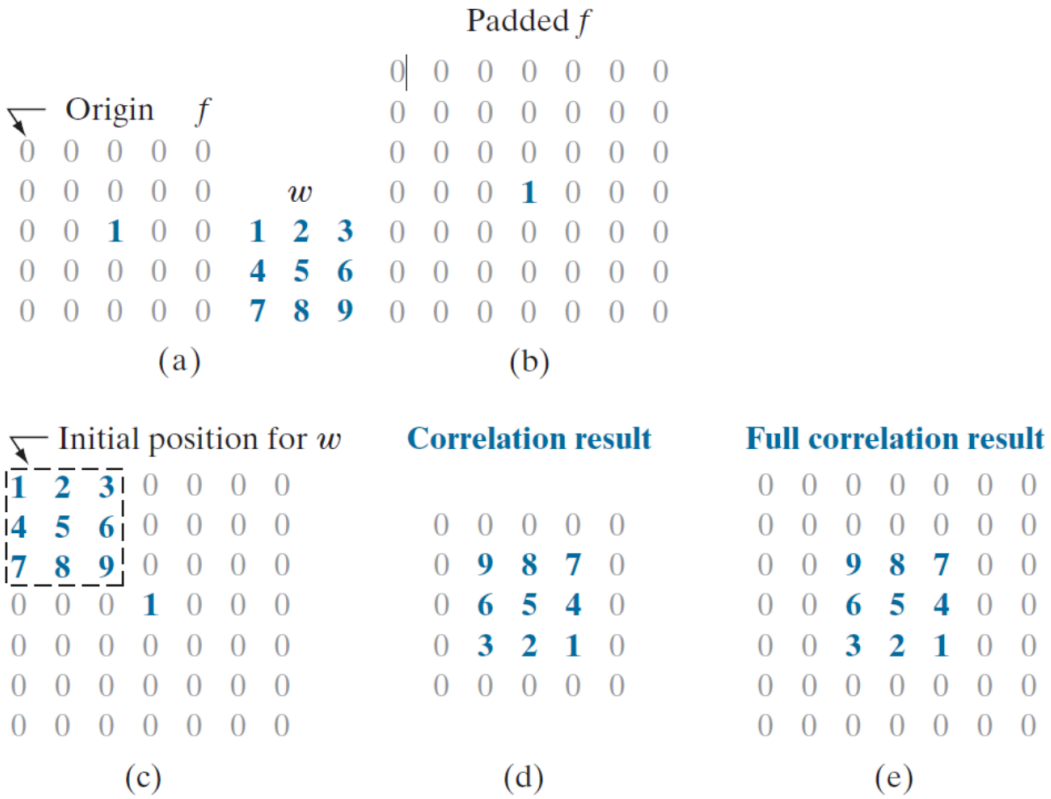


Figure 5. The complete correlation operation

Convolution is Associative.
This allows to combine smoothing and edge detection into one filter.

Visualization adapted from https://towardsdatascience.com/convolution-vs-correlation-af868b6b4fb5

Correlation versus Convolution

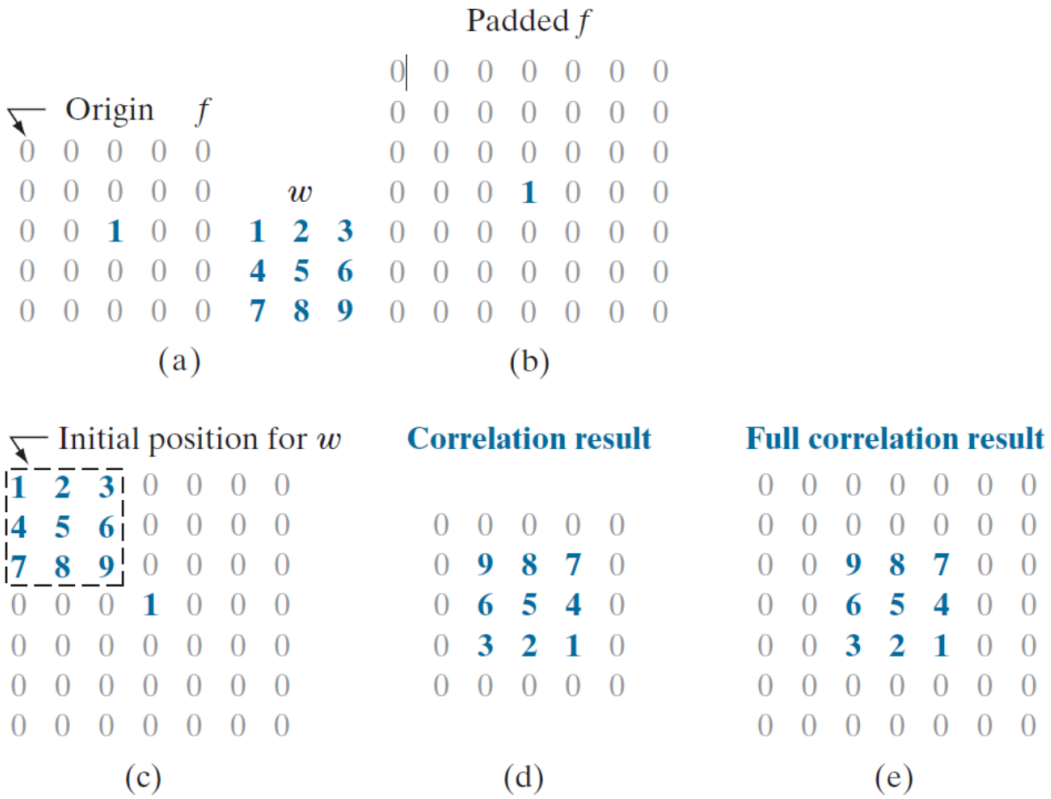


Figure 5. The complete correlation operation

Convolution is Associative.
This allows to combine smoothing and edge detection into one filter.

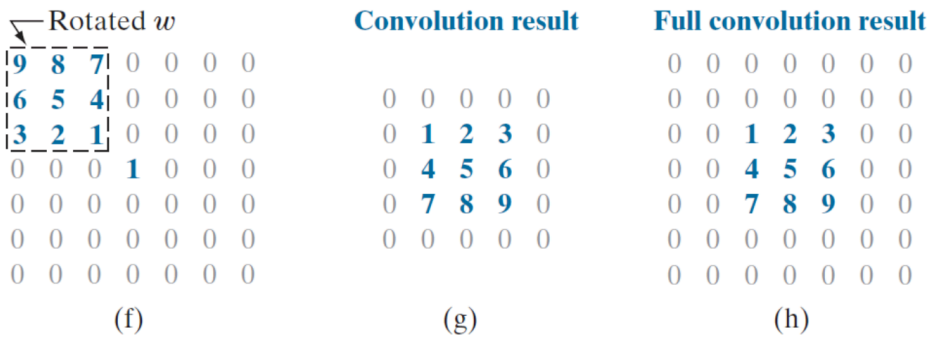


Figure 6. Applying the convolutional operation on Image b in Figure 5.

Visualization adapted from https://towardsdatascience.com/convolution-vs-correlation-af868b6b4fb5

Template matching

- Problem: what if the object in the image is much larger or much smaller than our template?
- Solution: re-scale the image multiple times and do correlation on every size!
- This leads to the idea of *image pyramids*

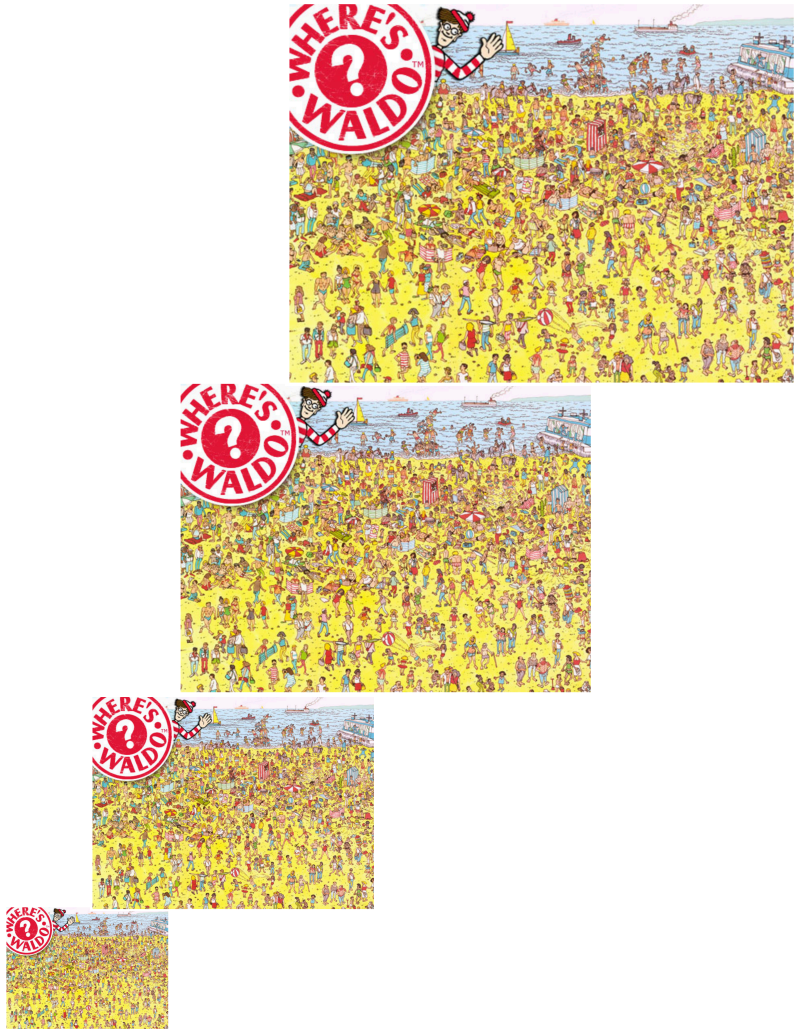


Image pyramids: scaling down

- Naïve solution: keep only some rows and columns
- E.g.: keep every other column to reduce image by 1/2 in width direction

Source:
Sanja Fidler

Image pyramids: scaling down

- Naïve solution: keep only some rows and columns
- E.g.: keep every other column to reduce image by 1/2 in width direction

Source:
Sanja Fidler

Image pyramids: scaling down

- Solution: blur the image via Gaussian, *then* subsample
- Intuition: remove high frequency content in the image

Source:
Sanja Fidler

Image pyramids: scaling down

- Solution: blur the image via Gaussian, *then* subsample
- Intuition: remove high frequency content in the image

Source:
Sanja Fidler

Image pyramids: scaling down

- Solution: blur the image via Gaussian, *then* subsample
- Intuition: remove high frequency content in the image

Source:
Sanja Fidler

Image pyramids

- A sequence of images created with Gaussian blurring and down-sampling is called a Gaussian pyramid
- The other step is to perform up-sampling (nearest neighbor, bilinear, bicubic, etc.